# AUTOMATED ENTAILMENT PROVING IN SEPARATION LOGIC

**TA QUANG TRUNG**

(B.Sc – Hanoi University of Science and Technology)

**A THESIS SUBMITTED**
**FOR THE DEGREE OF DOCTOR OF PHILOSOPHY**

**DEPARTMENT OF COMPUTER SCIENCE**
**NATIONAL UNIVERSITY OF SINGAPORE**

**2018**

# Declaration of Authorship

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis. This thesis has also not been submitted for any degree in any university previously.

Ta Quang Trung

Singapore, January 16th, 2018

*For my beloved family...*

# Acknowledgements

Having been traveling through a memorable Ph.D. journey with many ups and downs, lessons learned and lessons forgotten, I feel deeply indebted to my wonderful supervisors Prof. Khoo Siau Cheng and Prof. Chin Wei Ngan. Thank you so much for being my great teachers and my great friends, who always keep my feet on the ground so that I can move forward. Without your enthusiastic support and guidance, I do not know where my journey would end up to.

My study could not be possible without the support from The Government of Singapore and National University of Singapore (NUS). Thank you for generously granting me the research scholarship, and providing such an excellent studying, working and living environment. This dissertation could not be complete without the evaluation of my thesis examiners, Prof. Joxan Jaffar and Prof. David Rosenblum from School of Computing, NUS, and Prof. Alwen Tiu from the Australian National University. Thank you for the careful reading and the constructive comments and suggestions.

Looking back, I am grateful for having opportunities to learn from inspirational teachers. I thank Prof. Trần Đình Khang and Dr. Trần Đức Khánh, from Hanoi University of Science and Technology, for initiating my research interests in logic and formal methods. I thank Prof. Olivier Danvy, from Aarhus University and Yale-NUS college, and Dr. Alexander Kreuzer, from NUS Mathematics Department, for delightful courses about advanced topics in programming languages, communication skills in computer science, and fundamental backgrounds in mathematical logic. These courses had blown fresh winds into my perspective about conducting scientific research.

In this exciting journey, I feel privileged for having chances to discuss, collaborate, and learn from many excellent researchers. I thank Prof. Ooi Beng Chin, Prof. Makoto Tatsuta, Prof. Luke Ong, Dr. Jiang Lingxiao, Dr. James Brotherston, and Dr. Aquinas Hobor for all the insightful discussions about research in general, and separation logic, program verification, and program analysis in particular. I thank Dr. Cristina Cifuentes, Dr. Paddy Krishnan, Mr. Nathan Keynes, Dr. François Gauthier, and other colleagues at Oracle Labs

Australia for all the inspiring research discussions during my time as an intern student there. I have learned a lot about the impacts and the challenges of applying academic research to solve real-world problems.

My Ph.D. journey could not be that enjoyable, had I not meet my wonderful friends. I thank Lê Tôn Chánh and Andreea Costea for all the supporting discussions, sharing and collaborations. I thank Teodora Băluță, Cristian Gherghina, Narcisa Milea, Yamilet Serrano Llerena, Behnaz Hassanshahi, Rasool Maghareh, Aishwarya Sivaraman, Benedict Lee, Faisal Al Ameen, Adi Yoga, Cai Zhuohong, Zuo Zhiqiang, Nguyễn Minh Huy, Nguyễn Quang Huy, Nguyễn Hữu Thiên Tân, Trần Lâm Ẩn, Lương Bá Linh, Nguyễn Trường Khanh, Nguyễn Lê Trúc, Phạm Văn Thuận, Hứa Bỉnh Sơn, Nguyễn Mạnh Dũng, Nguyễn Thanh Toàn, Nguyễn Đức Thân, and many other great friends in Singapore for all the cheerful moments during those years.

Last but not least, thousands of words cannot express how grateful I am for being the son of my beloved family. I thank my late father, Tạ Quang Chất, for teaching me to be strong. Dad, I miss you every day. I am indebted to my mother, Tạ Phương Lân, for her unlimited patience and unconditional love. Mom, this is the biggest gift of my life. I thank my grandparents, my younger sister, Tạ Thị Phương Thủy, and her little family, and many other members of my big family for their endless care and supports. You all have shaped me who I am today.

# Abstract

Separation Logic has been actively developed in the last two decades as a promising formalism to model and reason about the memory-safety of imperative programs. It provides an expressive model of the programs' states by combining *spatial operations* which describe the separation of the memory, and *inductive heap predicates* which depict the shapes of recursive data structures such as linked lists, trees, and graphs. However, this expressiveness also leads to challenges in proving separation logic entailments, which are verification conditions introduced in the program reasoning.

In this thesis, we consider the problem of proving separation logic entailments in a fragment of *symbolic-heap separation logic* with *user-defined inductive heap predicates*. This is one of the most popular separation logic fragment, which has been widely studied and utilized in both academic and industrial research. We propose comprehensive solutions to automatically prove the entailments by utilizing proof techniques based on *mathematical induction*. To summarize, this thesis makes the following contributions.

Firstly, we present a mutual induction proof system by proposing a novel induction principle on the size of the heap model satisfying the goal entailment. This principle allows entailments which are already derived in an on-going proof to be used as hypotheses to prove future entailments which will be introduced within the same proof. Since the applicable hypotheses are not limited to only the induction hypotheses but also other derived entailments, our presented technique increases the success chance of proving entailments by induction.

Secondly, we introduce a lemma synthesis framework which can automatically synthesize lemmas to serve as eureka steps to assist in proving entailments. More specifically, the framework can discover necessary lemma templates having similar heap structure with the goal entailment, and utilize structural induction proof with constraint solving to refine the templates and obtain the desired lemmas. Our framework can synthesize general and modular lemmas to support proving sophisticated entailments, which cannot be handled by other existing provers.

Finally, we develop a learning from failure framework which can systematically analyze and learn from the failure of the induction hypothesis applications to remedy the unsuccessful proofs. In particular, the framework can identify non-unifiable constants, variables or pure formulas in an unproductive induction hypothesis, and introduce inference rules to generalize or weaken the induction hypothesis, or to execute a case analysis so that the corresponding induction hypothesis can be applied. With these mechanisms, our learning from failure framework can increase the chance of successfully applying an induction hypothesis, which makes the induction proof more effective.

We have implemented all of these proof techniques in a prover named **Songbird** and have experimented it with various literature benchmarks from the separation logic competition SL-COMP 2014, as well as synthetic benchmarks containing challenging entailments. To date, our prototype prover **Songbird** can prove most of the entailments and outperforms state-of-the-art separation logic provers in all experimented benchmarks. The entailment proving techniques proposed in this thesis have opened up new opportunities for Separation Logic to verify more real-world programs automatically.

# Contents

**Declaration of Authorship** **i**
**Acknowledgements** **v**
**Abstract** **vii**
**Contents** **ix**
**List of Publications** **xiii**
**List of Figures** **xv**
**List of Tables** **xvii**
**List of Abbreviations** **xix**
**1 Introduction** **1**
1.1 "Move Fast and Break Things!" . . . . . 1
1.2 Formal Verification Using Separation Logic. . . . . 3
1.3 The Entailment Proving Problem . . . . . 6
1.4 Contributions of this Thesis. . . . . 8
**2 Background** **11**
2.1 Symbolic-heap Separation Logic . . . . . 11
2.1.1 Syntax of Formulas. . . . . 12
2.1.2 Semantics of Formulas. . . . . 14
2.1.3 Separation Logic Entailments. . . . . 15
2.1.4 Substitution. . . . . 16
2.2 Mathematical Induction . . . . . 16
2.2.1 Peano Induction Scheme. . . . . 17
2.2.2 Noetherian Induction Scheme. . . . . 17
2.2.3 Applying Mathematical Induction to Prove Entailments. . . . . 18
**3 Literature Review** **19**
3.1 Separation Logic with Separating Conjunction . . . . . 20
3.2 Separation Logic with both Separating Conjunction and Separating Implication . . . . . 24
**4 The Mutual Induction Proof System** **25**
4.1 Overview . . . . . 25
4.2 Motivating Example . . . . . 26

4.3 The Symbolic-Heap Separation Logic Fragment . . . . . 29
4.4 The Mutual Induction Principle for Entailment Proof . . . . . 30
4.5 The Mutual Induction Proof System . . . . . 32
4.5.1 Logical Rules . . . . . 33
4.5.2 Induction Rules . . . . . 34
4.5.3 The Proof Search Procedure . . . . . 37
4.5.4 Soundness . . . . . 39
4.6 Implementation . . . . . 39
4.7 Experiment . . . . . 43
4.8 Related Work . . . . . 48
4.9 Discussion . . . . . 48
4.10 Summary . . . . . 50
**5 The Lemma Synthesis Framework 51**
5.1 Overview . . . . . 51
5.2 Motivating Example . . . . . 52
5.3 Theoretical Background . . . . . 56
5.3.1 Symbolic-Heap Separation Logic with Unknown Relations . . . . . 56
5.3.2 Structural Induction Proof for Separation Logic Entailments . . . . 59
5.4 The Structural Induction Proof System . . . . . 60
5.4.1 Inference Rules for Standard Entailments . . . . . 61
5.4.2 Synthesis Rules for Unknown Entailments . . . . . 65
5.4.3 The Proof Search Procedure . . . . . 66
5.4.4 Soundness of the Proof System . . . . . 69
5.5 The Lemma Synthesis Framework . . . . . 70
5.5.1 The Lemma Synthesis Procedure . . . . . 71
5.5.2 Discovering Lemma Templates . . . . . 72
5.5.3 Refining Lemma Templates' Antecedents . . . . . 73
5.5.4 Refining Lemma Templates' Consequents . . . . . 77
5.5.4.1 Preprocessing Lemma Templates . . . . . 78
5.5.4.2 Fine-Tuning Lemma Templates' Consequents . . . . . 81
5.5.5 Inferring Pure Invariants of Inductive Heap Predicates . . . . . 83
5.5.6 Solving Assumption Constraints with Farkas' Lemma . . . . . 84
5.5.7 Soundness of the Lemma Synthesis Framework . . . . . 86
5.6 Experiments . . . . . 87
5.7 Related Work . . . . . 92
5.8 Summary . . . . . 93
**6 The Failure Learning Framework 95**
6.1 Overview . . . . . 95
6.2 Motivation . . . . . 96

6.2.1 Weakening an Induction Hypothesis . . . . . 97
6.2.2 Generalizing an Induction Hypothesis . . . . . 98
6.2.3 Performing a Case Analysis on an Equality Constraint to Apply an Induction Hypothesis . . . . . 100
6.2.4 Performing a Case Analysis on a Linear Arithmetic Constraint to Apply an Induction Hypothesis . . . . . 102
6.3 Background . . . . . 103
6.3.1 Symbolic-Heap Separation Logic . . . . . 103
6.3.2 Structural Induction Principle for Separation Logic Entailment Proof 104
6.4 The Standard Structural Induction Proof System . . . . . 105
6.4.1 Logical Rules . . . . . 106
6.4.2 Induction Rules . . . . . 107
6.4.3 The Standard Proof Search Procedure . . . . . 108
6.5 How Can an Induction Proof Fail? . . . . . 109
6.5.1 The Failure of the Induction Hypothesis's Spatial Unification . . . 110
6.5.1.1 The Non-Unifiability of Constants and Variables . . . . . 111
6.5.1.2 The Unification Algorithm for Spatial Formulas . . . . . 112
6.5.2 The Failure of the Induction Hypothesis's Pure Unification . . . . . 115
6.5.2.1 The Non-Unifiability of Pure Formula . . . . . 116
6.5.2.2 The Unification Algorithm for Pure Formulas . . . . . 117
6.6 The Failure Learning Framework . . . . . 118
6.6.1 Learning from Failure Rules . . . . . 119
6.6.2 The Learning From Failure Proof Search Procedure . . . . . 119
6.6.3 Soundness . . . . . 123
6.7 Experiments . . . . . 123
6.7.1 Task 1: Proving Validity of Entailments . . . . . 124
6.7.2 Task 2: Proving Unsatisfiability of Formulas . . . . . 126
6.8 Related Work . . . . . 127
6.9 Summary . . . . . 129
**7 Future Work 131**
7.1 Separation Logic with More Expressive Properties . . . . . 131
7.2 Bi-Abductive Reasoning in Separation Logic . . . . . 132
7.3 Separation Logic with Magic Wand . . . . . 133
**8 Conclusion 135**
**A The soundness of the Mutual Induction Proof System 137**
A.1 Inference Rules . . . . . 137
A.2 The Proof System . . . . . 141
**B The soundness of the Lemma Synthesis Framework 143**
B.1 Inference Rules . . . . . 143

B.2 Synthesis Rules . . . . . 148
B.3 Proofs of Propositions 5.1, 5.2, 5.3 . . . . . 149
**C Soundness of the Failure Learning Framework** **153**
C.1 Logical and Induction Rules . . . . . 153
C.2 Failure Learning Rules . . . . . 154
C.3 The Proof System . . . . . 155
**Bibliography** **157**

# List of Publications

**Peer-reviewed conference papers:**

1. *Automated Mutual Explicit Induction Proof in Separation Logic*
   Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
   *The 21$^{st}$ International Symposium on Formal Methods (FM), 2016*

2. *Automated Lemma Synthesis in Symbolic-Heap Separation Logic*
   Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
   *The 45$^{th}$ ACM Symposium on Principles of Programming Languages (POPL), 2018.*

**Peer-reviewed journal paper:**

3. *Automated Mutual Induction Proof in Separation Logic*
   Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
   *Submitted to Journal of Formal Aspects of Computing (FAOC), August 2017.*

**Ongoing work:**

4. *Automated Induction Proof in Separation Logic: Learning From Failure*
   Quang-Trung Ta, Ton Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin
   *Ongoing work, January 2018.*

# List of Figures


1.1 Inductive heap predicate representing a linked list segment. . . . . 4
1.2 A procedure that returns the length of a linked list . . . . . 5
1.3 Frame rule . . . . . 5
1.4 Symbolically verifying the procedure `length` . . . . . 7

2.1 Syntax of $\mathsf{SL}_{\mathsf{ID}}$ assertions. . . . . 13
2.2 Semantics of $\mathsf{SL}_{\mathsf{ID}}$ assertions. . . . . 14
2.3 Peano induction principle . . . . . 17
2.4 Noetherian induction principle . . . . . 17

4.1 A linked-list traversal algorithm with random jump . . . . . 26
4.2 Proof tree of $E_2$, using induction hypothesis $E$ . . . . . 28
4.3 Ordinary proof tree of $E_3$, using induction hypothesis $E$ . . . . . 28
4.4 New proof tree of $E_3$, using hypotheses $E_2$ and $E_4$ . . . . . 28
4.5 Syntax of $\mathsf{SL}_{\mathsf{ID}}$ assertions. . . . . 29
4.6 Semantics of $\mathsf{SL}_{\mathsf{ID}}$ assertions. . . . . 30
4.7 Logical rules. . . . . 33
4.8 Induction rules. . . . . 34
4.9 Applying hypothesis . . . . . 35
4.10 The mutual induction proof search procedure . . . . . 38
4.11 The procedure that finds unification rules. . . . . 42
4.12 The procedure that compares inference rules. . . . . 43

5.1 Proof tree of the motivating lemma $E_1$ . . . . . 54
5.2 Partial induction proof tree of the inductive lemma $L_1$ . . . . . 55
5.3 Direct proof tree of the trivial lemma $L'_3$ . . . . . 56
5.4 Syntax of formulas in $\mathsf{SL}^{\mathsf{U}}{}_{\mathsf{ID}}$ . . . . . 57
5.5 Semantics of formulas in $\mathsf{SL}^{\mathsf{U}}{}_{\mathsf{ID}}$ . . . . . 58
5.6 Logical rules . . . . . 62
5.7 Induction rules . . . . . 63
5.8 Lemma application rules . . . . . 64
5.9 Synthesis rules . . . . . 65
5.10 The main proof search procedure. . . . . 67

5.11 The main lemma synthesis procedure . . . . . 71
5.12 Refining a lemma template's antecedent . . . . . 74
5.13 A possible proof tree of the unknown entailment $E_{u_1}$ . . . . . 76
5.14 Refining a lemma template's consequent . . . . . 78
5.15 Preprocess a lemma template's antecedent . . . . . 79
5.16 A possible proof tree of the unknown entailment $E_{u_2}$. . . . . 80
5.17 Fine-tuning the consequent of a lemma template . . . . . 81
5.18 A partial proof tree of the unknown entailment $E_{u_3}$. . . . . 82

6.1 Induction proof tree of $E_1$. . . . . 98
6.2 Induction proof tree of $E_2$. . . . . 100
6.3 Induction proof tree of $E_3$ . . . . . 101
6.4 Induction proof tree of $E_4$. . . . . 102
6.5 Syntax of Symbolic-heap Separation Logic formulas . . . . . 103
6.6 Semantics of Symbolic-heap Separation Logic formulas . . . . . 104
6.7 Logical rules . . . . . 106
6.8 Induction rules . . . . . 107
6.9 The standard proof search procedure . . . . . 108
6.10 The unification algorithm for spatial formulas. . . . . 113
6.11 The unification algorithm for expressions. . . . . 115
6.12 The unification algorithm for pure formulas. . . . . 118
6.13 Learning from failure rules. . . . . 119
6.14 The learning-from-failure proof search procedure . . . . . 120
6.15 Find inference rules . . . . . 122

A.1 Original proof tree $\mathcal{T}$ . . . . . 142
A.2 Transformed tree $\mathcal{T}'$ . . . . . 142

# List of Tables

4.1 Evaluation on the benchmarks of SL-COMP 2014 . . . . . 44
4.2 Comparison between **Songbird**$_{\mathbf{MI}}$ and other provers. . . . . 45
4.3 Evaluation on the synthetic benchmark slrd_ind . . . . . 47

5.1 Evaluation on the literature entailment benchmarks. . . . . 88
5.2 Statistics on the lemma synthesis and lemma application of the prover **Songbird**$_{\mathbf{LS}}$ on the literature benchmarks. . . . . 89
5.3 Evaluation on the synthetic benchmark slrd_lm. . . . . 90
5.4 Statistics on the lemma synthesis and lemma application of the prover **Songbird**$_{\mathbf{LS}}$ on the synthetic benchmark slrd_lm. . . . . 91

6.1 Evaluation on the benchmarks of SL-COMP 2014. . . . . 124
6.2 Evaluation on the synthetic benchmarks slrd_ind and slrd_lm. . . . . 125
6.3 Evaluation on the synthetic benchmark slrd_unsat. . . . . 126

# List of Abbreviations

| | |
|---|---|
| ID | **In**d**u**ction |
| IH | **I**nduction **H**ypothesis |
| IHs | **I**nduction **H**ypotheses |
| SL | **S**eparation **L**ogic |
| $SL_{ID}$ | **S**eparation **L**ogic with **In**d**u**ctive Heap Predicates |
| $SL^{U}_{ID}$ | **S**eparation **L**ogic with **U**nknown Relations and **In**d**u**ctive Heap Predicates |
| SL-COMP | **S**eparation **L**ogic **Comp**etition |
| SMT | **S**atisfiability **M**odulo **T**heories |
| FV | **F**ree **V**ariables |
| GCD | **G**reatest **C**ommon **D**ivisor |
| ll | **l**inked **l**ist |
| ls | **l**inked list **s**egment |
| dll | **d**oubly **l**inked **l**ist |
| tseg | **t**ree **seg**ment |
| rev | **rev**erse |
| arith | **arith**metic |
| MI | **M**utual **I**nduction |
| LS | **L**emma **S**ynthesis |
| LFF | **L**earning **F**rom **F**ailure |

# Chapter 1

# Introduction

"Moving fast with software verification."

– Calcagno et al.[1]

## 1.1 "Move Fast and Break Things!"

– Mark Zuckerberg[2]

The title of this section is based on a motto which has led the giant social network Facebook to its enormous success. However, speaking in a recent Facebook developer conference F8[3] in 2014, Mark Zuckerberg, the CEO of Facebook, had changed his mind: *"We used to have this famous mantra [of move fast and break things]. The idea here is that, as developers, moving quickly is so important that we were even willing to tolerate a few bugs in order to do it. What we realized over time is that it was not helping us to move faster because we had to slow down to fix these bugs and it was not improving our speed."* [Mashable, 2014]

In fact, software bugs have been creating consequences which are worse than one can imagine. In a study commissioned in 2002 by the National Institute of Standards and Technology, Department of Commerce, United States of America (U.S.), they were estimated to cost the U.S. economy about US$ 59.5 billion, annually [Research Triangle

[1] Researchers at the Facebook corporation.
[2] The CEO of the Facebook corporation.
[3] The "F8" name comes from Facebook's tradition of 8-hour hackathon in its early days.

Institute, 2002]. This cost was equivalent to 0.54% of the overall GDP (Gross Domestic Product) of the U.S. in 2002, which was about US$ 10,980 billion [World Bank, 2017].

Even extreme consequences had happened due to the existence of bugs in mission-critical software systems. For instance, the failure of a U.S. Patriot Missile system in Saudi Arab in 1991 had resulted in the destruction of a U.S. Army barrack, killed 28 soldiers and injured around 100 others. It was investigated later that a software rounding error occurred in the Patriot system's computer. This error led to the incorrect calculation of time, causing the Patriot system's failure to intercept an incoming Iraqi Scud missile, which later fatally struck the entire army barrack [United State General Accounting Office, 1992].

Another example is the explosion of an unmanned Ariane 5 rocket, launched by the European Space Agency in 1996. The rocket exploded just 40 seconds after its lift-off from Kourou, French Guiana, creating damage of about US$ 500 million. The cause was due to an overflow error occurring in a guidance computer when it tried to convert the rocket's sideways velocity from 64-bit to a 16-bit format. This error led to the shutdown of the guidance computer. When the accident happened, the rocket's control was passed to a backup guidance system, which also failed since it was running the same conversion algorithm [European Space Agency, 1996].

Given the negative consequences created by software bugs, computer scientists nowadays do agree that *it is preferable to verify that computer programs do not go wrong, before running them* [Cousot et al., 2010]. The common agreement has also been shared in the industry, where technology giants such as Microsoft, Google, Facebook, Oracle, IBM, NASA have been setting up research labs focusing on *detecting and preventing software bugs* to improve the software quality.

To address the above concern about software bugs, *formal verification* has been actively developed as a useful and promising methodology to ensure the correctness of computer programs [Cousot et al., 2010]. This methodology provides mathematically sound techniques that can guarantee full coverage of all program behaviors, thus is able to check if a program behaves correctly according to its desired specifications. Practical use of

formal verification has been described in a recent research paper, entitled *"Moving fast with software verification!"* [Calcagno et al., 2015], which appeared in NASA International Symposium on Formal Methods in 2015.

## 1.2 Formal Verification Using Separation Logic.

*"Separation logic is the twenty-first-century variant of Hoare logic that permits verification of pointer-manipulating programs."*

– Appel et al.[4]

Among various software verification techniques, *separation logic* appears as a promising formalism to verify programs that manipulate data structures stored in the computer memory [Reynolds, 2000; O'Hearn et al., 2001]. Typical examples of these programs are mission-critical systems such as navigating systems of aircraft and spaceships, or nuclear reactor safety systems. Other examples include kernels, drivers, system code libraries and applications of modern operating systems, such as Windows, Unix, Linux, MacOS, Android, and iOS.

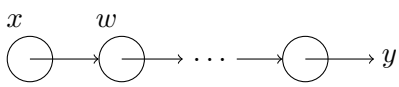
In essence, separation logic describes a *program state* by a combination of a *store* and a *heap*, which correspond to the state of statically allocated variables and dynamically-allocated data structures in common programming languages such as C/C++ and Java. Besides standard logical operators, it also provides *spatial operators* $\mathsf{emp}, \mapsto, *, \mathrel{-\!\!*}$ to model content and separation of computer memory. More specifically, the *empty heap* predicate $\mathsf{emp}$ asserts that the heap is undefined for all memory addresses. The *singleton heap* predicate $e \mapsto e_1, \ldots, e_n$ indicates that the heap is defined at exactly one location and maps a given memory address $e$ to a tuple of values $e_1, \ldots, e_n$. The *separating conjunction* operator $P * Q$ states that two predicates $P$ and $Q$ hold for separate portions of the memory. Finally, the *separating implication* operator $P \mathrel{-\!\!*} Q$ specifies that if a memory portion is extended with a disjoint part satisfied by $P$, then the extended portion is satisfied by $Q$.

[4]Researchers at Princeton University, Portland State University, National University of Singapore, Université de Rennes 1 and INRIA Paris-Rocquencourt.

Also, separation logic supports the use of *inductive heap predicates*, which can be recursively defined, to concisely and expressively model complex data structures such as variants of linked lists, trees, and graphs. For example, the inductive heap predicate $\mathsf{ls}(x, y, n)$ in Figure 1.1 (a) represents a segment of a linked list data structure containing $n$ elements, where the first element is pointed to by $x$, and the last element has a pointer out to $y$. Each element in the list is described by a singleton heap predicate $u{\mapsto}v$, where $u$ is the element's address, and $v$ is the element's content pointing to the next element in the list. Moreover, $\mathsf{ls}(x, y)$ is defined by two definition cases: the base case $\mathsf{emp} \wedge x{=}y$ states that the list segment does not contain any element; and the inductive case $\exists w.(x{\mapsto}w * \mathsf{ls}(w, y, n{-}1))$ indicates that this segment comprises of an element $x{\mapsto}w$ and a sub-segment $\mathsf{ls}(w, y, n{-}1)$. The detailed illustration of the linked list segment $\mathsf{ls}(x, y, n)$ is provided in Figure 1.1 (b).

$$\mathsf{ls}(x, y, n) \stackrel{\text{def}}{=} (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \vee \exists w.(x{\mapsto}w * \mathsf{ls}(w, y, n{-}1))$$

(a)




(b)

FIGURE 1.1: Inductive heap predicate representing a linked list segment.

Furthermore, separation logic extends Hoare logic [Hoare, 1969] to reason about behaviors of computer programs. It abstracts the execution of a program $\mathsf{C}$ by a *Hoare triple* $\{P\}\ \mathsf{C}\ \{Q\}$, where $P$ and $Q$ are logical formulas representing *pre-condition* and *post-condition* of $\mathsf{C}$. The Hoare triple $\{P\}\ \mathsf{C}\ \{Q\}$ asserts that if the program $\mathsf{C}$ executes in an initial state that satisfies the pre-condition $P$, then when it terminates, the final states satisfy the post-condition $Q$. Furthermore, $\mathsf{C}$ can access only to memory locations which are either stated in the pre-condition $P$ or allocated by $\mathsf{C}$ during its execution.

For instance, the procedure length in Figure 1.2 is specified by a pair of pre-condition and post-condition which are respectively indicated by the keywords `requires` and `ensures`. More specifically, this specification indicates that given an input linked list, which is modeled by $\mathsf{ls}(x, \mathsf{nil}, n)$, the procedure does not modify the linked list, and when it exits, the final result is the length of the linked list $(\mathsf{result}{=}n)$. Here, result is a special keyword

indicating the returned value of the procedure.

```
// the data structure node
struct node {
    struct node * next;
    int value;
}
```

```
// the procedure length
void length(struct node * x)
//@ requires ls(x, nil, n)
//@ ensures ls(x, nil, n) ∧ result=n
{
    if (x == NULL) return 0;
    else {
        struct node * y = x→next;
        int m = length(y);
        return (m + 1);
    }
}
```

FIGURE 1.2: A procedure that returns the length of a linked list

In addition to standard verification rules inherited from Hoare logic, separation logic is also equipped with a special *frame rule*, which enables local reasoning about the programs' execution. Essentially, it asserts that if a program C can safely execute in a given program state $P$, then it can also execute safely in an extended program state $P * R$, given that the additional state $R$ is not affected by the execution of the program C. The frame rule is formally described in Figure 1.3, where $\mathtt{FV}(\mathsf{C})$ and $\mathtt{FV}(R)$ denote free variables appearing in C and $R$.

$$\frac{\{P\}\ \mathsf{C}\ \{Q\}}{\{P * R\}\ \mathsf{C}\ \{Q * R\}}\ \mathtt{FV}(\mathsf{C}) \cap \mathtt{FV}(R) = \varnothing$$

FIGURE 1.3: Frame rule

At the current stage, separation logic has been successfully implemented by both academic verification tools such as **Hip/Sleek** [Chin et al., 2012], **Cyclist** [Brotherston et al., 2014b] and in industrial static analysis tools like **Infer** [Calcagno et al., 2015], and **SLAYer** [Berdine et al., 2011] to verify the memory-safety or find memory bugs in drivers of operating systems, system code libraries, and mobile applications. In essence, these tools verify a computer program by a three-step recipe. Firstly, the program is specified by pairs of pre-conditions and post-conditions, which are separation logic formulas describing the

program' inputs and expected results, or other desired properties like memory safety, thread safety. Moreover, these pre- and post-conditions are also called specifications for short, and they can be manually written by developers or automatically synthesized by analysis tools. Secondly, the program's source code will be statically analyzed against the specifications to derive a set of verification conditions which abstractly represent all possible executions of the program. Finally, the derived verification conditions will be proved by a separation logic theorem prover to determine whether the original program behaves accordingly to its specifications.

## 1.3 The Entailment Proving Problem

The verification conditions generated by static analysis tools using separation logic like **Infer**, **SLAYer**, **Hip/Sleek**, **Cyclist** often appear in the form of *separation logic entailments*. More specifically, these entailments are often obtained when proving pre-conditions of callee functions, proving the post-conditions when exiting a function, or normalizing program states.

We consider the following example when a verification tool symbolically verifies the procedure length against its specification. The analysis is described in details in Figure 1.4, where the program's specifications are indicated by //@, and the program's states after executing each statement are marked by //.

After executing the first return statement, i.e., `return 0`, the obtained program state is: $\mathsf{ls}(x, \mathsf{nil}, n) \wedge x{=}\mathsf{nil} \wedge \mathsf{result}{=}0$ (lines 8, 9). Here, the verification system needs to prove that this program state entails the post-condition. This results in the following entailment, corresponding to the entailment (1) in Figure 1.4.

$$(1): \quad \mathsf{ls}(x, \mathsf{nil}, n) \wedge x{=}\mathsf{nil} \wedge \mathsf{result}{=}0 \vdash \mathsf{ls}(x, \mathsf{nil}, n) \wedge \mathsf{result}{=}n$$

Similarly, the verification systems also need to prove entailments corresponding to the normalization of a program state when the first element is extracted from the linked

```
1:  void length(struct node * x)
2:  //@ requires ls(x, nil, n)
3:  //@ ensures ls(x, nil, n) ∧ result=n
4:  {
5:     // ls(x, nil, n)
6:     if (x == NULL)
7:        // ls(x, nil, n) ∧ x=nil
8:        return 0;
9:        // ls(x, nil, n) ∧ x=nil ∧ result=0                                  (1)
10:    else {
11:       // ls(x, nil, n) ∧ x≠nil
12:       struct node * y = x→next;
13:       // x↦y * ls(y, nil, n−1) ∧ x≠nil                                     (2)
14:       int m = length(y);                                                    (3)
15:       // x↦y * ls(y, nil, n−1) ∧ x≠nil ∧ m=n−1
16:       return (m + 1);
17:       // x↦y * ls(y, nil, n−1) ∧ x≠nil ∧ m=n−1 ∧ result=m+1                (4)
18:    }
19: }
```

(1): prove post-condition: $\mathsf{ls}(x, \mathsf{nil}, n) \wedge x{=}\mathsf{nil} \wedge \mathsf{result}{=}0 \vdash \mathsf{ls}(x, \mathsf{nil}, n) \wedge \mathsf{result}{=}n$

(2): prove normalization: $\mathsf{ls}(x, \mathsf{nil}, n) \wedge x{\neq}\mathsf{nil} \wedge \vdash \exists y.(x{\mapsto}y * \mathsf{ls}(y, \mathsf{nil}, n{-}1) \wedge x{\neq}\mathsf{nil})$

(3): prove pre-condition: $\mathsf{ls}(y, \mathsf{nil}, n{-}1) \vdash \exists n'.\mathsf{ls}(y, \mathsf{nil}, n')$

(4): prove post-condition: $x{\mapsto}y * \mathsf{ls}(y, \mathsf{nil}, n{-}1) \wedge x{\neq}\mathsf{nil} \wedge m{=}n{-}1 \wedge \mathsf{result}{=}m{+}1 \vdash \mathsf{ls}(x, \mathsf{nil}, n) \wedge \mathsf{result}{=}n$

FIGURE 1.4: Symbolically verifying the procedure `length`

list (line 12), the proving of the pre-condition when the function length is invoked to apply to the remaining list (line 14), and the proving the of post-condition at the second return statement, i.e., `return m+1` (line 16). They are respectively the three entailments (2), (3), (4) in Figure 1.4.

$$(2):\ \mathsf{ls}(x, \mathsf{nil}, n) \wedge x{\neq}\mathsf{nil} \wedge \vdash \exists y.(x{\mapsto}y * \mathsf{ls}(y, \mathsf{nil}, n{-}1) \wedge x{\neq}\mathsf{nil}$$

$$(3):\ \mathsf{ls}(y, \mathsf{nil}, n{-}1) \vdash \exists n'.\mathsf{ls}(y, \mathsf{nil}, n')$$

$$(4):\ x{\mapsto}y * \mathsf{ls}(y, \mathsf{nil}, n{-}1) \wedge x{\neq}\mathsf{nil} \wedge m{=}n{-}1 \wedge \mathsf{result}{=}m{+}1 \vdash \mathsf{ls}(x, \mathsf{nil}, n) \wedge \mathsf{result}{=}n$$

There have been numerous efforts to prove separation logic entailments and the most challenging obstacle lies in handling the inductive heap predicates. A popular approach

is to restrict the heap predicates, such as considering only *pre-defined* predicates whose syntax and semantics are defined beforehand, or predicates satisfying the *bounded tree width property*. In essence, the restriction opens ways for various techniques to tackle classes of inductive heap predicates efficiently. However, it also diminishes the natural expressiveness of separation logic. We will carefully survey this approach in Chapter 2. The other strategy to deal with inductive heap predicates is to use techniques based on mathematical induction [Brotherston et al., 2011a; Chu et al., 2015]. This strategy can deal with *general inductive heap predicates* and does not sacrifice the logic's expressiveness. However, techniques following this strategy are often incomplete, due to the incompleteness of mathematical induction in inductive theories [Bundy, 2001]. We will also discuss the detail in Chapter 2.

## 1.4 Contributions of this Thesis.

In this thesis, we consider the problem of *proving separation logic entailments* in a setting of *symbolic-heap separation logic with user-defined inductive heap predicates*. On the one hand, this logic fragment is *simple* since its formula can be separated into a spatial formula describing the shape of data structures allocated in the heap, and a pure formula capturing Boolean constraints over variables allocated in the stack. On the other hand, it is also *flexible* when allowing inductive heap predicates to be arbitrary forms. This logic fragment has been *widely used* in both academic and industrial research [Berdine et al., 2004; Berdine et al., 2005b; Chin et al., 2012; Calcagno et al., 2015; Berdine et al., 2011; Brotherston et al., 2016].

We follow the work [Bundy, 2001] to utilize mathematical induction to reason about the recursive structure of *inductive heap predicates*. Besides, we empower the proof system with comprehensive solutions to enhance the completeness of mathematical induction. Our contributions are stated as follows:

– Firstly, we present a mutual induction proof system by proposing a novel induction

principle on the size of the heap model satisfying the goal entailment. This principle allows entailments which are already derived in an on-going proof can be used as hypotheses to prove future entailments which will be introduced within the same proof. Since the applicable hypotheses are not limited to only the induction hypotheses but also other derived entailments, our presented technique increases the success chance of proving entailments by induction.

– Secondly, we introduce a lemma synthesis framework which can automatically synthesize lemmas to serve as eureka steps to assist in proving entailments. More specifically, the framework can discover necessary lemma templates having similar heap structure with the goal entailment, and utilize structural induction proof with constraint solving to refine the templates and obtain the desired lemmas. Our framework can synthesize general and modular lemmas to support proving sophisticated entailments, which cannot be handled by any existing provers.

– Finally, we develop a learning from failure framework which can systematically analyze and learn from the failure of the induction hypothesis applications to remedy the unsuccessful proofs. In particular, the framework can identify non-unifiable constants, variables or pure formulas in an unproductive induction hypothesis, and introduce inference rules to generalize or weaken the induction hypothesis, or to execute a case analysis so that the corresponding induction hypothesis can be applied. By that mean, this framework can increase the success chance of applying an induction hypothesis, which makes the induction proof more efficient.

We have implemented all of these proof techniques in a prover named **Songbird**, which is available for online download at:

`http://songbird-prover.github.io`

We have experimented **Songbird** with various literature benchmarks from the separation logic competition SL-COMP 2014 [Sighireanu et al., 2016] and synthetic benchmarks containing sophisticated entailments [Ta et al., 2016; Ta et al., 2018]. To date, our prototype prover **Songbird** can prove most of the entailments and outperforms state-of-the-art

separation logic provers in all experimented benchmarks. The entailment proving techniques proposed in this thesis have opened up more opportunities to apply Separation Logic to verify real-world programs.

The rest of this dissertation is structured as follows. We present the background of separation logic and mathematical induction in Chapter 2 and survey related work in Chapter 3. Afterwards, we describe our main contribution in Chapters 4, 5 and 6. We discuss future work in Chapter 7 and finally conclude the thesis in Chapter 8.

# Chapter 2

# Background

In this thesis, we focus on the problem of proving separation logic entailments in the fragment of symbol-heap separation logic with inductive heap predicates. This is one of the most popular separation logic fragments, which is deployed in industrial static analysis tools such as **SLAYer** [Berdine et al., 2011] and **Infer** [Calcagno et al., 2015]. One of the challenges of proving separation logic entailments in this fragment is how to effectively handle inductive heap predicates appears in the entailments' antecedents and consequents. In our work, we propose to prove separation logic entailment in this fragment using mathematical induction. We will present theoretical background of separation logic and mathematical induction in this chapter.

## 2.1 Symbolic-heap Separation Logic

The symbolic-heap separation logic is first proposed in [Berdine et al., 2004; Berdine et al., 2005b]. Each formula in this fragment consists of a *heap* part which symbolically models the memory layout and a *pure* part which captures additional constraints in the layout. This logic fragment is actively studied in both academia and industrial research [Berdine et al., 2004; Berdine et al., 2005b; Nguyen et al., 2007; Chin et al., 2012; Iosif et al., 2013; Brotherston et al., 2016; Ta et al., 2016; Berdine et al., 2011; Calcagno et al., 2015].

We extend this symbolic-heap separation logic fragment with user-defined inductive

heap predicates and linear arithmetic constraints to capture more properties of data structures, such as the number of elements, or constraints about these elements' values. We denote our logic fragment as $\mathsf{SL}_{\mathsf{ID}}$, where the subscript ID indicates the support of **i**n**d**uctive heap predicates. Its syntax and semantics are introduced below.

### 2.1.1 Syntax of Formulas.

Figure 2.1 presents the syntax of formulas in our logic fragment $\mathsf{SL}_{\mathsf{ID}}$. We write $x$ to denote either an *integer variable* representing a numeric property of data structures or a *spatial variable* modeling a memory address. In addition, $c$ represents an *integer constant*, and $e$ is an *integer expression*, which is composed of other expressions by the standard linear arithmetic operators: addition, subtraction, and multiplication by a constant $(+, -, \cdot)$. Moreover, nil is a special *spatial constant* indicating a dangling memory address and $a$ is a *spatial expression* representing a memory address in general. We use $\pi$ to indicate a *pure atomic formula* which can be an equality constraint among spatial expressions $(=, \neq)$ or a linear arithmetic constraint among integer expressions $(=, \neq, <, \leq, >, \geq)$. These pure atomic formulas compose a *pure formula* $\Pi$ via application of standard logical connectives $(\wedge, \vee, \neg, \Rightarrow)$ and standard logical quantifications $(\forall, \exists)$ as in first-order logic.

Moreover, we write $\sigma$ to denote a *spatial atomic formula*, which can be either a predicate emp, a *singleton heap predicate* $x \overset{\iota}{\mapsto} x_1, ..., x_n$ or an *inductive heap predicate* $\mathsf{P}(x_1, ..., x_n)$. More specifically, emp models an *empty* memory, i.e., an unallocated memory. The *singleton heap predicate* $x \overset{\iota}{\mapsto} x_1, ..., x_n$ describes an $n$-field data structure of sort $\iota$, which is located in the memory pointed to by $x$, and has $x_1, ..., x_n$ as values of its fields. The sort $\iota$ represents a unique type of data structure. This sort notation can be omitted when there is only one considered data type in the context, like the entailments in Section 1.3. Lastly, the *inductive heap predicate* $\mathsf{P}(x_1, ..., x_n)$ represents a recursive data structure whose properties are captured by the parameters $x_1, ..., x_n$. We will formally define the inductive heap predicate in Definition 2.1. Moreover, these spatial atomic formulas constitute *spatial formula* $\Sigma$ via the separating conjunction operator $(*)$.

Finally, we write $F$ to represent a *symbolic-heap formula*, whose components are separated by the conjunction connective ($\wedge$) into two parts: a spatial part represented by $\Sigma$ and a pure part indicated by $\Pi$. One of these two parts can be omitted if $\Sigma$ is emp or $\Pi$ is true. We also allow existential quantification ($\exists$) over both spatial and integer variables of the symbolic-heap formula $F$.

$$
\begin{array}{lcl}
e & := & c \mid x \mid -e \mid e_1{+}e_2 \mid e_1{-}e_2 \mid c \cdot e \\
a & := & \mathsf{nil} \mid x \\
\pi & := & \mathsf{true} \mid \mathsf{false} \mid a_1{=}a_2 \mid a_1{\neq}a_2 \mid e_1{=}e_2 \mid e_1{\neq}e_2 \mid e_1{>}e_2 \mid e_1{\geq}e_2 \mid e_1{<}e_2 \mid e_1{\leq}e_2 \\
\sigma & := & \mathsf{emp} \mid x \overset{\iota}{\mapsto} x_1, ..., x_n \mid \mathsf{P}(x_1, ..., x_n) \\
\Pi & := & \pi \mid \neg\Pi \mid \Pi_1 \wedge \Pi_2 \mid \Pi_1 \vee \Pi_2 \mid \Pi_1 \Rightarrow \Pi_2 \mid \forall x.\Pi \mid \exists x.\Pi \\
\Sigma & := & \sigma \mid \Sigma_1 * \Sigma_2 \\
F & := & \Sigma \mid \Pi \mid \Sigma \wedge \Pi \mid \exists x.\, F
\end{array}
$$

FIGURE 2.1: Syntax of $\mathsf{SL}_{\mathsf{ID}}$ assertions.

**Definition 2.1 (Inductive heap predicates)** *A system of $k$ inductive heap predicates $\Pi$ of arity $n_i$, with $i = 1, ..., k$, is defined as:*

$$\left\{ \mathsf{P}_i(x_1^i, ..., x_{n_i}^i) \ \overset{\text{def}}{=} \ F_1^i(x_1^i, ..., x_{n_i}^i) \ \vee \ \ldots \ \vee \ F_{m_i}^i(x_1^i, ..., x_{n_i}^i) \right\}_{i=1}^{k}$$

*where $F_j^i(x_1^i, ..., x_{n_i}^i)$ is called a* definition case *of $\mathsf{P}_i(x_1^i, ..., x_{n_i}^i)$, and this relation is denoted by $F_j^i(x_1^i, ..., x_{n_i}^i) \overset{\text{def}}{\Rightarrow} \mathsf{P}_i(x_1^i, ..., x_{n_i}^i)$. Moreover, $F_j^i(x_1^i, ..., x_{n_i}^i)$ is a* base case, *if it does not contain any predicates mutually defined with $\mathsf{P}_i(x_1^i, ..., x_{n_i}^i)$; otherwise, it is an* inductive case.

**Example 2.1 (Linked list segment)** *The predicate $\mathsf{ls}(x, y)$ in Figure 1.1 is an example of inductive heap predicates. It is self-recursively defined, i.e., its definition is only built up from heap predicates of the same symbol, and not from any other inductive heap predicate symbols.*

**Example 2.2 (Linked list segment of even and odd length)** *The two inductive heap predicates $\mathsf{lsOdd}(x, y)$ and $\mathsf{lsEven}(x, y)$ below were introduced in [Brotherston et al., 2011a] to model segments of linked list data structures which respectively contain an* even *and an* odd *number of elements.*

$$\begin{array}{lcl} \mathsf{IsOdd}(x,y) & \stackrel{\text{def}}{=} & x{\mapsto}y \ \vee \ \exists u.(x{\mapsto}u * \mathsf{IsEven}(u,y)) \\ \mathsf{IsEven}(x,y) & \stackrel{\text{def}}{=} & \exists u.(x{\mapsto}u * \mathsf{IsOdd}(u,y)) \end{array}$$

*They are* mutually recursively defined *since the predicate symbol* IsOdd *appears in the recursive definition of* $\mathsf{IsEven}(x,y)$ *and vice versa.*

### 2.1.2 Semantics of Formulas.

Figure 2.2 exhibits the semantics of formulas in our separation logic fragment $\mathsf{SL_{ID}}$. Given a set `Var` of variables, a set `Sort` of sorts, a set `Val` of values and a set `Loc` of memory addresses (`Loc` $\subset$ `Val`), a model $s, h$ of an $\mathsf{SL_{ID}}$ formula consists of: the *stack* model $s$, which is a function $s$: `Var` $\rightarrow$ `Val`, and the *heap* model $h$, which is a partial function $h$: (`Loc` $\times$ `Sort`) $\rightharpoonup$ `Val`$^+$.

| | | |
|---|---|---|
| $s, h \models \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$; $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models \mathsf{emp}$ | iff | $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models x \stackrel{\iota}{\mapsto} x_1, ..., x_n$ | iff | $\mathrm{dom}(h) = \{s(x)\}$ and $h(s(x), \iota) = (s(x_1), ..., s(x_n))$ |
| $s, h \models \mathsf{P}(x_1, ..., x_n)$ | iff | $s, h \models F_i(x_1, ..., x_n)$, with $F_i(x_1, ..., x_n) \stackrel{\text{def}}{\Rightarrow} \mathsf{P}(x_1, ..., x_n)$ |
| $s, h \models \Sigma_1 * \Sigma_2$ | iff | $\exists h_1, h_2 : h_1 \# h_2$; $h_1 \circ h_2 = h$; $s, h_1 \models \Sigma_1$; $s, h_2 \models \Sigma_2$ |
| $s, h \models \Sigma \wedge \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$; $s, h \models \Sigma$ |
| $s, h \models \exists x.\, F$ | iff | $\exists v \in \mathtt{Val} : [s\vert x{:}v], h \models F$ |

FIGURE 2.2: Semantics of $\mathsf{SL_{ID}}$ assertions.

We write $[\![\Pi]\!]_s$ to denote the valuation of a pure formula $\Pi$ under the stack model $s$. Moreover, $\mathrm{dom}(h)$ denotes the domain of $h$; $h \# h'$ indicates that $h$ and $h'$ have disjoint domains, i.e., $\mathrm{dom}(h) \cap \mathrm{dom}(h') = \varnothing$; $h \circ h'$ is the union of two disjoint heap models $h$ and $h'$; and $[f|x{:}y]$ is a function like $f$ except that it returns $y$ for the input $x$. We follow Reynolds' semantics to consider the *finite* heap model, i.e., $\forall h.\, |h| < +\infty$, where $|h|$ is the domain size of $h$ [Reynolds, 2008].

### 2.1.3 Separation Logic Entailments.

Given the syntax and the semantics of separation logic formulas, we are ready to define separation logic entailments as follow.

**Definition 2.2 (Entailments)** *An entailment between two formulas $F_1$ and $F_2$, which is denoted as $F_1 \vdash F_2$, is said to be* valid, *iff for all model $s, h$, if $s, h$ satisfies $F_1$ then it also satisfies $F_2$. Formally:*

$$F_1 \vdash F_2 \text{ is valid, iff } \forall s, h.(s, h \models F_1 \rightarrow s, h \models F_2).$$

In the above definition, $F_1, F_2$ are respectively called the antecedent and the consequent of the entailment. In the following, we present the new concepts *model* and *counter-model* as well as the *transitivity law* of separation logic entailments.

**Definition 2.3 (Models and counter-models of an entailment)** *Given an entailment $E \triangleq F \vdash G$. A separation logic model $s, h$ is called a* model *of $E$, iff $s, h \models F$ implies $s, h \models G$. On the contrary, $s, h$ is called a* counter-model *of $E$, iff $s, h \models F$ and $s, h \not\models G$.*

We denote $s, h \models (F \vdash G)$, or $s, h \models E$, if $s, h$ is a model of $E$. Similarly, we write $s, h \not\models (F \vdash G)$, or $s, h \not\models E$, if $s, h$ is a counter-model of $E$. Given a list of $n$ entailments $E_1, ..., E_n$, we write $s, h \models E_1, ..., E_n$ if $s, h$ is a model of *all* entailments $E_1, ..., E_n$, and $s, h \not\models E_1, ..., E_n$ if $s, h$ is a counter-model of *at least one* entailment in $E_1, ..., E_n$.

**Theorem 2.1 (Transitivity law of entailments)** *Given two entailments $F_1 \vdash \exists \vec{x}.F_2$ and $F_2 \vdash F_3$, where $\vec{x}$ is a list of variables. If they are valid, then the entailment $F_1 \vdash \exists \vec{x}.F_3$ is also valid.*

*Proof.* Consider an arbitrary model $s, h$ such that $s, h \models F_1$. Since the hypothesis gives that $F_1 \vdash \exists \vec{x}.F_2$ is valid, it follows from Definition 2.2 that $s, h \models \exists \vec{x}.F_2$. By the semantics of formulas, there is an evaluation mapping the variables in $\vec{x}$ to theirs values $\vec{v}$ to obtain an extended stack model $s' = [s|\vec{x}{:}\vec{v}]$ such that $s', h \models F_2$. Since $F_2 \vdash F_3$ is valid, by Definition 2.2, $s', h \models F_3$. We have shown that $s' = [s|\vec{x}{:}\vec{v}]$, it follows the semantics of formulas again that $s, h \models \exists \vec{x}.F_3$. Since $s, h$ is chosen arbitrarily, and $s, h \models F_1$ implies that $s, h \models \exists \vec{x}.F_3$, it follows from Definition 2.2 that $F_1 \vdash \exists \vec{x}.F_3$ is valid. □

### 2.1.4 Substitution.

We write $[e_1/v_1, ..., e_n/v_n]$ to denote a *simultaneous substitution* and $F[e_1/v_1, ..., e_n/v_n]$ denotes a formula that is obtained from $F$ by simultaneously replacing all occurrences of $v_1$ by $e_1$, $v_2$ by $e_2$, ..., $v_n$ by $e_n$. The simultaneous substitution has the following properties.

**Proposition 2.1 (Substitution law for formulas [Reynolds, 2008])** *Given a formula $F$ and a substitution $\theta = [e_1/v_1, ..., e_n/v_n]$. Let $s, h$ be a separation logic model, where $\mathrm{dom}(s)$ contains $(\mathtt{FV}(F) \setminus \{ v_1, ..., v_n \}) \cup \mathtt{FV}(e_1) \cup ... \cup \mathtt{FV}(e_n)$, and let $\hat{s} = [s \mid v_1 : [\![e_1]\!]_s \mid ... \mid v_n : [\![e_n]\!]_s]$. Then,*

$$s, h \models F\theta, \textit{ iff } \hat{s}, h \models F$$

**Theorem 2.2 (Substitution law for entailments)** *Given an entailment $F_1 \vdash F_2$ and a substitution $\theta$. If $F_1 \vdash F_2$ is valid, then $F_1\theta \vdash F_2\theta$ is also valid.*

*Proof.* Suppose that $\theta = [e_1/v_1, ..., e_n/v_n]$. Consider an arbitrary model $s, h$ of $F_1\theta$, i.e., $s, h \models F_1\theta$. Let $\hat{s} = [s \mid v_1 : [\![e_1]\!]_s \mid ... \mid v_n : [\![e_n]\!]_s]$. Then by Proposition 2.1, $s, h \models F_1\theta$ implies that $\hat{s}, h \models F_1$. In addition, the theorem's hypothesis provides that $F_1 \vdash F_2$ is valid, therefore $\hat{s}, h \models F_2$. It is implied by Proposition 2.1 again that $s, h \models F_2\theta$. Given the hypothesis that $F_1 \vdash F_2$ is valid, we have shown that if $s, h$ is a model of $F_1\theta$, then it is also a model of $F_2\theta$. Since $s, h$ is chosen arbitrarily, by Definition 2.2, the entailment $F_1\theta \vdash F_2\theta$ is valid. □

## 2.2 Mathematical Induction

Mathematical induction is a popular theorem proving technique for reasoning about objects, events or procedures containing repetition. These include mathematical objects like the natural number or recursive data structures such as lists or trees. Many properties of such objects can be proved by using mathematical induction [Bundy, 2001].

In a simple setting, an induction proof consists of one or more *base cases*, and one or more *inductive cases*. In a base case, the conclusion is proved for a particular value; in a

inductive case, the conclusion is proved for a latter value under the assumption that it is true for one or more previous values.

### 2.2.1 Peano Induction Scheme.

A classic example of mathematical induction is Peano induction principle. This principle states that for a given conjecture $\mathcal{P}$ on natural number, if $\mathcal{P}(0)$ holds, and for all natural number $n$, $\mathcal{P}(n)$ holds implies that $\mathcal{P}(n{+}1)$, then $\mathcal{P}$ holds for all natural number $n$. It is formally presented in Figure 2.3, where $nat$ represents the natural number, and $s(n)$ is the successor function, i.e., $s(n) = n{+}1$.

$$\frac{\mathcal{P}(0) \qquad \forall n : nat.\ \mathcal{P}(n) \Rightarrow \mathcal{P}(s(n))}{\forall n : nat.\ \mathcal{P}(n)}$$

FIGURE 2.3: Peano induction principle

The Peano induction is subsumed by a single, general schema of *Noetherian induction, a.k.a. well-founded induction*, which is used to prove properties of recursive objects in general. The Noetherian in is exhaustively described in [Bundy, 2001]. We briefly summarize it as follows.

### 2.2.2 Noetherian Induction Scheme.

Given a conjecture $\mathcal{P}(\alpha)$, with $\alpha$ is a structure of type $\tau$. The general schema of Noetherian induction states that for an arbitrary structure $\alpha$, if $\mathcal{P}$ holds for all sub-structure $\beta$ of $\alpha$ implies that $\mathcal{P}$ also holds for $\alpha$, then $\mathcal{P}$ holds for all structure $\alpha$. Formally:

$$\frac{\forall \alpha : \tau.\ (\forall \beta : \tau.\ \beta \prec_\tau \alpha \rightarrow \mathcal{P}(\beta)) \rightarrow \mathcal{P}(\alpha))}{\forall \alpha : \tau.\ \mathcal{P}(\alpha)}$$

FIGURE 2.4: Noetherian induction principle

where $\prec_\tau$ is a well-founded relation on $\tau$, i.e., there is no infinite descending chain, like $\ldots \prec_\tau \alpha_n \prec_\tau \ldots \prec_\tau \alpha_2 \prec_\tau \alpha_1$. Noetherian induction can be applied for arbitrary type

$\tau$, such as data structures or control flow. However, success in proving a conjecture by induction is highly dependent on the choice of the induction variable $\alpha$ and the well-founded relation $\prec_\tau$.

### 2.2.3 Applying Mathematical Induction to Prove Entailments.

In this work, we utilize mathematical induction to prove separation logic entailments by reasoning about the recursive structure of inductive heap predicates or the size of heap model of formulas. These correspond to the two induction principles: structural induction and mutual induction. We will explain them in details in Chapters 4, 5 and 6.

# Chapter 3

# Literature Review

There have been various works studying the problem of proving separation logic entailments. Among these works, researchers have been proved that in general, the entailment proving problem is *undecidable* [Calcagno et al., 2001; Brotherston et al., 2010; Antonopoulos et al., 2014]. This undecidability is due to the combination of the separating conjunction operator ($*$), the separating implication operator ($-\!\!*$) and inductive heap predicates.

Currently, most of the efforts have been focusing on proving entailments in the setting of separation logic supporting the separating conjunction operator ($*$) and inductive heap predicates. The authors either restrict the syntax and the semantics of inductive heap predicates to obtain decidability or propose incomplete methods to handle user-defined inductive heap predicates. We will survey these approaches in Section 3.1. Additional related works discussion will also be described in Chapters 4, 5 and 6.

There have also been attempts to tackle the entailment proving problem in separation logic fragments which support both the separating conjunction ($*$) and the separating implication ($-\!\!*$) operators. We will briefly discuss them in Section 3.2. Note that we do not focus on these logic fragments in this thesis.

## 3.1 Separation Logic with Separating Conjunction

We have discussed earlier that existing entailment proving techniques either *restrict the syntax and the semantics* of heap predicates to obtain decidability or handle inductive heap predicates by *incomplete methods*.

In the first approach, the authors of [Berdine et al., 2004; Berdine et al., 2005b; Cook et al., 2011; Pérez et al., 2011; Pérez et al., 2013; Enea et al., 2014; Piskac et al., 2013; Piskac et al., 2014; Bozga et al., 2008; Bozga et al., 2010] *pre-define* both the syntax and the semantics of heap predicates to model specific kinds of the linked list and the tree data structures. Also, researchers in [Iosif et al., 2013; Iosif et al., 2014; Piskac et al., 2013; Piskac et al., 2014; Enea et al., 2014; Enea et al., 2015] consider a class of heap predicates satisfying certain properties such as the *compositionality* or the *bounded tree width* properties. These restrictions enable the authors to invent efficient techniques handling inductive heap predicates when proving entailments.

The second approach is practiced in [Chin et al., 2012; Nguyen et al., 2008; Brotherston et al., 2011a; Brotherston et al., 2014a; Madhusudan et al., 2012; Qiu et al., 2013; Chu et al., 2015]. These works aim to prove entailments by utilizing proof systems based on sequent calculus, cyclic proof, induction proofs. However, these proof techniques are often incomplete, due to the failure of cut elimination in inductive theories [Bundy, 2001].

We will elaborate all the aforementioned works as follows.

**Symbolic-heap separation logic** in [Berdine et al., 2004; Berdine et al., 2005b]. These are the first works studying the entailment proving problem in separation logic. Here, the authors proposed the *symbolic-heap separation logic*, whose formulas containing a heap part and a pure part as described earlier in Chapter 2. More specifically, this logic fragment contains only two inductive heap predicates, which are *pre-defined* to model the *singly linked list* and the *binary tree* data structures. The authors proposed a *sound and complete* proof system based on sequent calculus, which contains a special rule to unroll (unfold) heap predicates by its definitions, which permits the proof system to reason

about the predicates' sub-structures. The proof system is proved to have a time complexity of Co-NP. It was also implemented in a well-known verification system, named *Smallfoot* [Berdine et al., 2005a].

**Separation logic with linked list** in [Cook et al., 2011]. This work also aims to prove entailments in a separation logic fragment with a *pre-defined* heap predicate modeling the singly linked list. However, the authors of this work indirectly translate separation logic formulas into formulas in graph theory and therefore transform the entailment proving into the graph homomorphism checking. Due to the linear structure the linked list, their graph representation enjoys several special structural properties so that the homomorphism checking problem can even be solved in polynomial time. This technique was deployed in a theorem prover named SeLoger [Haase et al., 2013].

**Separation logic with linked list** in [Pérez et al., 2011; Pérez et al., 2013]. The *pre-defined linked list* is also considered in these works. However, their authors took two different approaches in proving entailments. In the first work [Pérez et al., 2011], they proposed a proof-based approach to reason about equality constraints in the entailments, using a superposition calculus. In the other work [Pérez et al., 2013], they proved the entailments by a model-based approach. Specifically, given an entailment, this approach first constructs formula representing the separation and the well-formedness conditions of heap predicates. Then, it invokes an SMT solver to check whether a counter-example can be constructed to satisfy the entailment's antecedent but not the consequent, and decide the entailment's validity. These two techniques are both sound and complete, although their approaches are different. They were developed in a prover called *Asterix*.

**Separation logic with linked list and quantification** in [Bozga et al., 2008]. The authors of this work not only considered the *pre-defined linked list* but also extended the logic fragment to support both universal and existential quantification. Their technique is based on a translation of existentially and universally quantified separation logic formulas into finite structures called *symbolic graph representations (SGR)*. Given a goal entailment, formulas corresponding to its antecedent and consequent will be translated into a set of

SGRs. Thereafter, the entailment checking problem can be transformed into the emptiness checking for a *difference SGR*, which is introduced when subtracting the antecedent's SGR from the consequent's SGR.

**Separation logic with linked list and tree** in [Piskac et al., 2013; Piskac et al., 2014]. These works extend separation logic to support the *pre-defined heap predicates* modeling not only the linked list but also the *tree* data structures. Also, their approach to prove entailments is by translating separation logic formulas into a theory of graph and set, called *Graph Reachability and Stratified Sets (GRASS)*. This translation can be done in linear time. Consequently, the validity of an entailment is decided by invoking SMT solvers to check for implication between formulas corresponding to its antecedent and consequent in the GRASS theory. This translation technique enables the authors to develop a sound and complete proof system to prove separation logic entailments, which was implemented in a verification system called *GRASShopper*.

**Separation logic with various linked lists** in [Enea et al., 2014]. In contrast with all above works which study only the *pre-defined* heap predicates, the authors of this work considered a *less-restricted* fragment of separation logic with *various kind of predicates* modeling the *linked list data structures*, such as singly linked list, doubly linked list, skip lists. Entailments in this fragment can be proved by translating their antecedents and consequents to the theory of tree automata and checking for the membership of the automata corresponding to the entailments' antecedents and consequents. However, the developed entailment checking procedure is sound but incomplete, due to the broadly considered class of inductive heap predicates. It was deployed in a prover called *Spen*.

**Separation logic with user-defined inductive heap predicates** in [Nguyen et al., 2007; Chin et al., 2012]. These works consider a general fragment of separation logic with *user-defined inductive heap predicates*. Their authors developed a sequent-based proof system, which is equipped with two special rules: the *matching* and the *unfolding* rules. Specifically, the *matching* rule removes identical heap predicates from the goal entailments, and the *unfolding* rule replaces inductive heap predicates by their definitions to derive new entailments. These rules *partially* enable the proof system to deal with the general inductive

heap predicates when proving the entailments. This proof system was implemented in a prover called *SLEEK*. However, the proof system is incomplete and semi-automated since *user-provided* lemmas might be required to reason about the inductive heap predicates.

**Separation logic with general inductive heap predicates** in [Madhusudan et al., 2012; Qiu et al., 2013]. Similar to the above works, the authors of these works also considered the separation logic fragment with *general inductive heap predicates*. They developed a proof system based on natural deduction to prove entailments, using inference rules similar to the *matching* and *unfolding* rules mentioned earlier. This natural proof system was developed in a verification system called *Dryad*. Nonetheless, the proof system is also incomplete and semi-automated, where *axioms*, i.e., *user-provided*, are also required to assist in proving the entailments.

**Cyclic proof in separation logic with general inductive heap predicates** in [Brotherston et al., 2011a]. The authors of this work proposed a general proof technique called the *cyclic proof*, which is to mathematical induction and can be used to reason about the recursive structures of general inductive heap predicates. In essence, cyclic proof allows a proof tree to contain cycles and can be perceived as an infinite derivation tree. The discovery of such proof cycles can be considered as equivalent to the application of induction hypotheses in mathematical induction. In other words, this discovery leads to the introduction of an induction hypothesis, allow the induction hypotheses to be implicitly identified. However, such proof cycles also prevent the cyclic proof from checking for the soundness condition locally. Instead, the soundness proof will be checked globally in the entire of a proof tree: all infinite paths in the proof tree must have infinitely progressing traces. This condition guarantees the well-foundedness of all reasoning in the proof tree. Moreover, the cyclic proof was implemented in a prover called *Cyclist*.

**Induction proof for data structures in imperative programs** in [Chu et al., 2015]. This work considers the problem of proving entailments obtained during the verification of imperative programs. The authors of this work utilized a formalism similar to separation logic, which also contains general inductive heap predicates modeling the data structures. Also, they proposed a proof technique based on mathematical induction to prove

the entailments containing such general predicates. However, different from the cyclic proof, the induction proof requires induction hypotheses to be explicitly identified via applications of the induction rules. Nonetheless, soundness condition of this proof can be checked locally in such induction rule applications.

## 3.2 Separation Logic with both Separating Conjunction and Separating Implication

Little research has been conducted in the separation logic fragment which supports both the separating conjunction ($*$) and separating implication ($-\!\!*$), i.e., the magic wand. We briefly summary them in the following. Note that in this thesis, we do not focus on handling the separating implication operator ($-\!\!*$).

**Separation logic with the magic wand** in [Lee et al., 2014]. This work aims to prove entailments in a fragment of separation logic which supports both the separating conjunction and the separating implication operators. The authors developed a *sound and complete* sequent calculus-based proof system and propose a set of inference rules designated to handle the magic wand operator. The key contribution of this work lies in a proof search procedure which effectively applies the inference rules to prove entailments. However, the considered separation logic fragment is simple: *it does not support any inductive heap predicates*.

**Separation logic with the magic wand and linked list** in [Hou et al., 2015]. The authors of this work considered a broader fragment of separation logic with magic wand: their logic fragment also supports *pre-defined inductive heap predicates* modeling the linked list data structures. Different from the previous approach, the proof technique proposed here is based on a *labeled* sequent calculus to supports reasoning about the complicated structure of the *pre-defined linked list*. Their proof system was proved *sound* and *complete*. It was implemented in a theorem prover called *Separata*.

# Chapter 4

# The Mutual Induction Proof System

## 4.1 Overview

In this chapter, we present a mutual induction proof system to prove separation logic entailments. Our proof system is an instance of the well-founded induction, a.k.a., Noetherian induction. More specifically, we propose a novel induction principle based on a well-founded relation of separation logic models. We implement this principle explicitly as inference rules so that it can be easily integrated into a deductive proof system. Our induction principle has an advantage in allowing a goal entailment and other entailments derived during a proof search can be used as hypotheses to mutually prove each other. This feature increases the success chance of proving the goal entailment. We have implemented our *mutual induction proof* technique in a prototype prover and evaluated it on two entailment benchmarks collected from the literature as well as a hand-crafted benchmark. The experimental results are promising since our prover can prove most of the *valid* entailments in these benchmarks, and achieves a better performance than other state-of-the-art separation logic provers. In summary, our work makes the following contributions:

– We define a well-founded relation on separation logic models and use it to construct a novel mutual induction principle to prove separation logic entailments.

– We develop a deductive system for proving entailments based on the proposed mutual induction principle. Our system can prove entailments more effectively since it allows entailments derived in a proof tree can be used as hypotheses to prove other entailments derived in the same proof.

– We implement a prototype prover, named **Songbird$_{\mathbf{MI}}$**, and experiment on it with two entailment benchmarks from a separation logic competition SL-COMP 2014 as well as a handcrafted benchmark. Our experiment shows a promising result where **Songbird$_{\mathbf{MI}}$** can prove most of the entailments in these benchmarks. The prover **Songbird$_{\mathbf{MI}}$** is available online at:

http://songbird-prover.github.io/mutual-induction/ .

## 4.2 Motivating Example

We consider the procedure `traverse` in Figure 4.1, which is written in a C-like language to traverse a linked list data structure whose elements are of type `node`. More specifically, this procedure visits the linked list's elements by randomly jumping either one or two steps at a time.

```
struct node {struct node * next; };

void traverse(struct node * x) {
    if (x == NULL)
        return;
    bool jump = random();
    if (jump && x→next != NULL)
        traverse(x→next→next);
    else traverse(x→next); }
```

FIGURE 4.1: A linked-list traversal algorithm with random jump

In order to verify the memory safety of this procedure, an automated verification tool first needs to formulate the shape of the memory manipulated by `traverse`. Suppose

the initially discovered shape is represented by an *inductive heap predicate* $\mathsf{tmp}(x)$ which is defined as follows:

$$\begin{array}{rcl} \mathsf{tmp}(x) & \stackrel{\text{def}}{=} & \mathsf{emp} \\ & & \vee\ \exists u.(x{\mapsto}u * \mathsf{tmp}(u)) \\ & & \vee\ \exists u, v.(x{\mapsto}u * u{\mapsto}v * \mathsf{tmp}(v)) \end{array}$$

Intuitively, $\mathsf{tmp}(x)$ covers three possible cases of the shape, which can be an empty memory $\mathsf{emp}$ (when `x == NULL`), or can be recursively expanded by a single data structure $x{\mapsto}u$ (when `traverse` jumps one step), or can be recursively expanded by two structures $x{\mapsto}u$ and $u{\mapsto}v$ (when `traverse` jumps two steps). Here, $x{\mapsto}u$ and $u{\mapsto}v$ are two separation logic predicates modeling the data structure `node`.

Since the derived shape $\mathsf{tmp}(x)$ is anomalous, the automated verification tool may want to examine if this shape is actually a linked <u>l</u>ist <u>s</u>egment, which can be modeled by the following predicate:

$$\begin{array}{rcl} \mathsf{ls}(x, y) & \stackrel{\text{def}}{=} & (\mathsf{emp} \wedge x{=}y) \\ & & \vee\ \exists w.(x{\mapsto}w * \mathsf{ls}(w, y)) \end{array}$$

This can be done by checking the validity of the following entailment:

$$E \triangleq \mathsf{tmp}(x) \vdash \exists y.\, \mathsf{ls}(x, y)$$

The entailment $E$ is said to be *valid* if all memory models that satisfy $\mathsf{tmp}(x)$ also satisfy $\exists y.\, \mathsf{ls}(x, y)$. Since this entailment contains the inductive heap predicate $\mathsf{tmp}(x)$ in its antecedent, a natural approach to prove it is to perform mathematical induction on $\mathsf{tmp}(x)$. In particular, the entailment $E$ is first recorded as an induction hypothesis (IH), and then the predicate $\mathsf{tmp}(x)$ is analyzed in each case of its definition, via a method called *unfolding*, to derive new entailments $E_1, E_2$ and $E_3$ as follows.

$$\begin{array}{rcl} E_1 & \triangleq & \mathsf{emp} \vdash \exists y.\, \mathsf{ls}(x, y) \\ E_2 & \triangleq & x{\mapsto}u * \mathsf{tmp}(u) \vdash \exists y.\, \mathsf{ls}(x, y) \\ E_3 & \triangleq & x{\mapsto}u * u{\mapsto}v * \mathsf{tmp}(v) \vdash \exists y.\, \mathsf{ls}(x, y) \end{array}$$

The entailment $E_1$ can be easily proved by unfolding the predicate $\mathsf{ls}(x, y)$ by its first definition case to obtain a valid entailment $\mathsf{emp} \vdash \exists y.(\mathsf{emp} \wedge x = y)$. On the contrary, $E_2$ can be proved using the induction hypothesis $E$. Its proof tree is depicted in Figure 4.2.

$$
\begin{array}{rl}
\dfrac{}{\mathit{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)} & (\vdash_{\Pi}) \text{: Valid, proved by external provers, e.g. Z3} \\[2ex]
\dfrac{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)}{\mathit{true} \vdash \exists y, w.\, (u{=}w \wedge t{=}y)} & (*\mathsf{P}) \text{: Match and remove } \mathsf{ls}(u,t) \text{ and } \mathsf{ls}(w,y) \\[2ex]
\dfrac{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))}{\mathsf{ls}(u,t) \vdash \exists y, w.\, (\mathsf{ls}(w,y) \wedge u{=}w)} & (*{\mapsto}) \text{: Match and remove } x{\mapsto}u \text{ and } x{\mapsto}w \\[2ex]
\dfrac{(E_4)\ x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y.\, \mathsf{ls}(x,y)}{x{\mapsto}u * \mathsf{ls}(u,t) \vdash \exists y, w.\, (x{\mapsto}w * \mathsf{ls}(w,y))} & (\mathsf{P}_{\mathrm{R}}) \text{: Unfold } \mathsf{ls}(x,y) \text{ by its inductive case} \\[2ex]
\dfrac{}{(E_2)\ x{\mapsto}u * \mathsf{tmp}(u) \vdash \exists y.\, \mathsf{ls}(x,y)} & (\mathsf{IH}) \text{: Apply IH } E \text{ with } [u/x]\text{, rename } y \text{ to a fresh var } t
\end{array}
$$

FIGURE 4.2: Proof tree of $E_2$, using induction hypothesis $E$

We can also prove $E_3$ by the same method, i.e., applying the IH $E$, and its proof tree is shown in Figure 4.3.

$$
\begin{array}{rl}
\dfrac{}{\mathit{true} \vdash \exists y, z, w.\, (u{=}z \wedge v{=}w \wedge t{=}y)} & (\vdash_{\Pi}) \text{: Valid, proved by external prover, e.g. Z3} \\[2ex]
\dfrac{\mathsf{ls}(v,t) \vdash \exists y, z, w.\, (\mathsf{ls}(w,y) \wedge u{=}z \wedge v{=}w)}{\mathit{true} \vdash \exists y, z, w.\, (u{=}z \wedge v{=}w \wedge t{=}y)} & (*\mathsf{P}) \text{: Match and remove } \mathsf{ls}(v,t) \text{ and } \mathsf{ls}(w,y) \\[2ex]
\dfrac{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z, w.\, (z{\mapsto}w * \mathsf{ls}(w,y) \wedge u{=}z)}{\mathsf{ls}(v,t) \vdash \exists y, z, w.\, (\mathsf{ls}(w,y) \wedge u{=}z \wedge v{=}w)} & (*{\mapsto}) \text{: Match and remove } u{\mapsto}v \text{ and } z{\mapsto}w \\[2ex]
\dfrac{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z.\, (\mathsf{ls}(z,y) \wedge u{=}z)}{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z, w.\, (z{\mapsto}w * \mathsf{ls}(w,y) \wedge u{=}z)} & (\mathsf{P}_{\mathrm{R}}) \text{: Unfolding } \mathsf{ls}(z,y) \text{ by inductive case} \\[2ex]
\dfrac{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z.\, (x{\mapsto}z * \mathsf{ls}(z,y))}{u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z.\, (\mathsf{ls}(z,y) \wedge u{=}z)} & (*{\mapsto}) \text{: Remove data nodes } x{\mapsto}u \text{ and } x{\mapsto}z \\[2ex]
\dfrac{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y.\, \mathsf{ls}(x,y)}{x{\mapsto}u * u{\mapsto}v * \mathsf{ls}(v,t) \vdash \exists y, z.\, (x{\mapsto}z * \mathsf{ls}(z,y))} & (\mathsf{P}_{\mathrm{R}}) \text{: Unfold } \mathsf{ls}(x,y) \text{ by inductive case} \\[2ex]
\dfrac{}{(E_3)\ x{\mapsto}u * u{\mapsto}v * \mathsf{tmp}(v) \vdash \exists y.\, \mathsf{ls}(x,y)} & (\mathsf{IH}) \text{: Apply IH } E \text{ with } [v/x]\text{, rename } y \text{ to } t
\end{array}
$$

FIGURE 4.3: Ordinary proof tree of $E_3$, using induction hypothesis $E$

Using a different strategy, we observe that once $E_2$ is proved, entailments derived during its proof, i.e., $E_2$ and $E_4$, can be used as hypotheses to prove $E_3$. In this case, the new proof of $E_3$, which is demonstrated in Figure 4.4, is even simpler than its original proof shown in Figure 4.3. The proving process in this new strategy, therefore, is more efficient.

$$
\begin{array}{rl}
\dfrac{}{\mathit{true} \vdash \exists y.\, y = z} & (\vdash_{\Pi}) \text{: Valid, proved by external provers, e.g., Z3} \\[2ex]
\dfrac{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)}{\mathit{true} \vdash \exists y.\, y = z} & (*\mathsf{P}) \text{: Match and remove } \mathsf{ls}(x,z) \text{ and } \mathsf{ls}(x,y) \\[2ex]
\dfrac{x{\mapsto}u * \mathsf{ls}(u,r) \vdash \exists y.\, \mathsf{ls}(x,y)}{\mathsf{ls}(x,z) \vdash \exists y.\, \mathsf{ls}(x,y)} & (\mathsf{IH}) \text{: Apply } E_4 \text{ with } [r/t]\text{, rename } y \text{ to } z \\[2ex]
\dfrac{}{(E_3)\ x{\mapsto}u * u{\mapsto}v * \mathsf{tmp}(v) \vdash \exists y.\, \mathsf{ls}(x,y)} & (\mathsf{IH}) \text{: Apply } E_2 \text{ with } [u/x, v/u]\text{, rename } y \text{ to } r
\end{array}
$$

FIGURE 4.4: New proof tree of $E_3$, using hypotheses $E_2$ and $E_4$

In the new proof tree, the entailment $E_4$ can be directly used as a hypothesis to prove other entailments since it is already proved *valid*: its proof tree does not rely on any external hypotheses (Figure 4.2). However, when applying $E_2$ to prove $E_3$ (Figure 4.4),

it is not straightforward to conclude about $E_3$'s validity since the $E_2$'s validity is still *unknown*. This is because the proof of $E_2$ uses $E$ as a hypothesis (Figure 4.2). However, $E$'s validity also relies on the validity of both $E_2$ and $E_3$. Therefore, $E$, $E_2$ and $E_3$ jointly form a *mutual induction* proof, in which they are used to prove each other. We will discuss in details this proof technique in Section 4.4.

## 4.3 The Symbolic-Heap Separation Logic Fragment

Our mutual induction framework is developed based on the symbolic-heap separation logic fragment $\mathsf{SL}_{\mathsf{ID}}$. We briefly recall its syntax and semantics as follows. More details can be found in Chapter 2.

$$
\begin{array}{lcl}
e & := & c \mid x \mid -e \mid e_1{+}e_2 \mid e_1{-}e_2 \mid c\,e \\
a & := & \mathsf{nil} \mid x \\
\pi & := & \mathsf{true} \mid \mathsf{false} \mid a_1{=}a_2 \mid a_1{\neq}a_2 \mid e_1{=}e_2 \mid e_1{\neq}e_2 \mid e_1{>}e_2 \mid e_1{\geq}e_2 \mid e_1{<}e_2 \mid e_1{\leq}e_2 \\
\sigma & := & \mathsf{emp} \mid x \overset{\iota}{\mapsto} x_1, ..., x_n \mid \mathsf{P}(x_1, ..., x_n) \\
\Pi & := & \pi \mid \neg\Pi \mid \Pi_1 \wedge \Pi_2 \mid \Pi_1 \vee \Pi_2 \mid \Pi_1 \Rightarrow \Pi_2 \mid \forall x.\Pi \mid \exists x.\Pi \\
\Sigma & := & \sigma \mid \Sigma_1 * \Sigma_2 \\
F & := & \Sigma \mid \Pi \mid \Sigma \wedge \Pi \mid \exists x.\, F
\end{array}
$$

FIGURE 4.5: Syntax of $\mathsf{SL}_{\mathsf{ID}}$ assertions.

**Syntax.** The syntax of $\mathsf{SL}_{\mathsf{ID}}$ is described in Figure 4.5. We write $c$ to represent integer constant; $x$ to denote variables; and $e$, $a$ are respectively integer expression, spatial expression. These expressions compose the pure formula $\pi$ via applications of standard equality and arithmetic binary relation. In addition, $\mathsf{emp}$, $x \overset{\iota}{\mapsto} x_1,...,x_n$ and $\mathsf{P}(x_1,...,x_n)$ are respectively the *empty* memory predicate, the singleton heap predicate, and the inductive heap predicate. They are also called atomic spatial formula $\sigma$. These atomic

pure and spatial formulas are used to construct pure formula $\Pi$, spatial formula $\Sigma$, and symbolic-heap formula $F$.

**Semantics.** The semantics of $\mathsf{SL_{ID}}$ assertions are given in Figure 4.6. Given a set $\texttt{Var}$ of variables, $\texttt{Sort}$ of sorts, $\texttt{Val}$ of values, $\texttt{Loc}$ of memory addresses ($\texttt{Loc} \subset \texttt{Val}$), a model of a formula consists of: a *stack* model $s$, which is a function $s$: $\texttt{Var} \to \texttt{Val}$, and a *heap* model $h$ which is a partial function $h$: $(\texttt{Loc} \times \texttt{Sort}) \rightharpoonup \texttt{Val}^+$.

| | | |
|---|---|---|
| $s, h \models \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$; $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models \mathsf{emp}$ | iff | $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models x \overset{\iota}{\mapsto} x_1, ..., x_n$ | iff | $\mathrm{dom}(h) = \{s(x)\}$ and $h(s(x), \iota) = (s(x_1), ..., s(x_n))$ |
| $s, h \models \mathsf{P}(x_1, ..., x_n)$ | iff | $s, h \models F_i(x_1, ..., x_n)$, with $F_i(x_1, ..., x_n) \overset{\text{def}}{\Rightarrow} \mathsf{P}(x_1, ..., x_n)$ |
| $s, h \models \Sigma_1 * \Sigma_2$ | iff | $\exists h_1, h_2 : h_1 \# h_2$; $h_1 \circ h_2 = h$; $s, h_1 \models \Sigma_1$ and $s, h_2 \models \Sigma_2$ |
| $s, h \models \Sigma \wedge \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$; $s, h \models \Sigma$ |
| $s, h \models \exists x.\, F$ | iff | $\exists v \in \texttt{Val} : [s\|x{:}v], h \models F$ |

FIGURE 4.6: Semantics of $\mathsf{SL_{ID}}$ assertions.

## 4.4 The Mutual Induction Principle for Entailment Proof

We observe that an entailment $E$ is said to be *valid* if $s, h \models E$ for all model $s, h$. In Reynolds' semantics, the heap domain of any separation model $s, h$ is finite, i.e., $|h| < +\infty$ [Reynolds, 2008]. This property inspires us to define a relation among separation logic models by comparing the size of their heap domains. We call this relation as the *model order*, and it is a *well-founded* relation.

To prove a separation logic entailment by Noetherian induction based on this model order, we will show that for an arbitrary separation logic model, if all the smaller models satisfying the entailment implies that the given model also satisfies it, then the entailment

is satisfied by all separation logic models and is valid. We formally describe the model order and the induction principle as follows.

**Definition 4.1 (Model order)** *The* model order *is a binary relation between separation logic models. It is denoted by $\prec$, and is defined as: $s_1, h_1 \prec s_2, h_2$, iff $|h_1| < |h_2|$.*

**Theorem 4.1 (Well-founded relation)** *The model order $\prec$ is a well-founded relation.*

*Proof.* By contradiction, suppose that $\prec$ were not well-founded, then there would exist an infinite descending chain: $... \prec s_n, h_n \prec ... \prec s_1, h_1$. It follows that there would exist an infinite descending chain: $... < |h_n| < ... < |h_1|$. This is impossible since domain size of heap model is finite, i.e., $|h_1|, ..., |h_n|, ... \in \mathbb{N}$. □

**Theorem 4.2 (Induction principle)** *Given an entailment $E$. For all model $s, h$, if all smaller models $s', h'$ satisfy $E$ implies that $s, h$ also satisfies $E$, then $E$ is valid. Formally:*

$$\frac{\forall s, h.\ (\forall s', h'.\ s', h' \prec s, h \rightarrow s', h' \models E) \rightarrow s, h \models E}{\forall s, h.\ s, h \models E}$$

Since our induction principle is constructed on the separation logic model order, an induction hypothesis can be used in the proof of any entailment whenever the decreasing condition on model order is satisfied. This flexibility allows us to extend the aforementioned principle to support *mutual induction*, in which multiple entailments can participate in an induction proof, and each of them can be used as a hypothesis to prove the other. In the following, we will introduce our *mutual induction principle*. Note that the induction principle in Theorem 4.2 is an instance of this principle when only one entailment takes part in the induction proof.

**Theorem 4.3 (Mutual induction principle)** *Given $n$ entailments $E_1, ..., E_n$. For all model $s, h$, if all smaller models $s', h'$ satisfy $E_1, ..., E_n$ implies that $s, h$ also satisfies $E_1, ..., E_n$, then all the entailments $E_1, ..., E_n$ are valid. Formally:*

$$\frac{\forall s, h.\ (\forall s', h'.\ s', h' \prec s, h \rightarrow s', h' \models E_1, ..., E_n) \rightarrow s, h \models E_1, ..., E_n}{\forall s, h.\ s, h \models E_1, ..., E_n}$$

*Proof.* By contradiction, assume that some of $E_1, ..., E_n$ were invalid. Then, there would exist a counter-model $s, h$ of $E_1, ..., E_n$. Since $\prec$ is a well-founded relation, assuming that $s_1, h_1$ is the *least* counter-model, that is $s_1, h_1 \not\models E_1, ..., E_n$, and, $s'_1, h'_1 \models E_1, ..., E_n$ for all $s'_1, h'_1 \prec s_1, h_1$. Sinc this theorem's hypothesis gives $\forall s, h.\ (\forall s', h'.\ s', h' \prec s, h \rightarrow s', h' \models E_1, ..., E_n) \rightarrow s, h \models E_1, ..., E_n$, we have $s_1, h_1 \models E_1, ..., E_n$. This contradicts with the assumption that $s_1, h_1$ is a counter-model. □

## 4.5 The Mutual Induction Proof System

In this section, we formally describe a proof system, which implements the mutual induction principle to prove separation logic entailments. Our proof system comprises logical rules in Subsection 4.5.1, which deal with the logical structure of the entailments, and induction rules in Subsection 4.5.2, which record and apply induction hypotheses. We also introduce a general proof search procedure in Subsection 4.5.3 and discuss its soundness in Subsection 4.5.4.

Each inference rule in our proof system has zero or more premises, one conclusion and possibly one side condition. Each of the premises or the conclusion is described in the same form of a triple $\mathcal{H},\ \rho,\ F_1 \vdash F_2$. More specifically, $F_1 \vdash F_2$ is an entailment. $\mathcal{H}$ is a set of entailments with their validity statuses. These entailments are recorded during a proof search and can be used as hypotheses to prove the entailment $F_1 \vdash F_2$. Lastly, $\rho$ is a proof trace capturing a chronological list of inference rules applied by the proof search procedure to reach $F_1 \vdash F_2$.

We also call the entailment in each rule's conclusion as the goal entailment. The rules with empty premise are called axiom rules. A proof trace $\rho$ containing $n$ rules $R_1, \ldots, R_n$, with $n \geq 0$, is represented by $[(R_1), \ldots, (R_n)]$, where the head $(R_1)$ of $\rho$ is the latest rule applied by the proof search procedure. We utilize basic operations to manipulate proof traces, which include proof trace insertion: $(R) :: \rho$, membership checking: $(R) \in \rho$, and proof trace concatenation: $\rho_1 \mathbin{@} \rho_2$.

$$\vdash_{\Pi} \frac{}{\mathcal{H},\ \rho,\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2 \qquad \bot^1_{\mathrm{L}} \frac{}{\mathcal{H},\ \rho,\ F_1 \wedge \Pi \vdash F_2}\ \Pi \rightarrow \mathsf{false}$$

$$=_{\mathrm{L}} \frac{\mathcal{H},\ \rho',\ F_1[u/v] \vdash F_2[u/v]}{\mathcal{H},\ \rho,\ F_1 \wedge u{=}v \vdash F_2} \qquad \bot^2_{\mathrm{L}} \frac{}{\mathcal{H},\ \rho,\ F_1 * u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{w} \vdash F_2}$$

$$=_{\mathrm{R}} \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.F_2}{\mathcal{H},\ \rho,\ F_1 \vdash \exists \vec{x}.(F_2 \wedge u{=}u)} \qquad \exists_{\mathrm{L}} \frac{\mathcal{H},\ \rho',\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \rho,\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2)$$

$$\mathsf{E}_{\mathrm{L}} \frac{\mathcal{H},\ \rho',\ F_1 \vdash F_2}{\mathcal{H},\ \rho,\ F_1 * \mathsf{emp} \vdash F_2} \qquad *\!\mapsto \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})}{\mathcal{H},\ \rho,\ F_1 * u \overset{\iota}{\mapsto} \vec{v} \vdash \exists \vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})}\ u, \vec{v} \# \vec{x}$$

$$\mathsf{E}_{\mathrm{R}} \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.F_2}{\mathcal{H},\ \rho,\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{emp})} \qquad *\mathsf{P} \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.(F_2 \wedge \vec{u}{=}\vec{v})}{\mathcal{H},\ \rho,\ F_1 * \mathsf{P}(\vec{u}) \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{v}))}\ \vec{u} \# \vec{x}$$

$$\mathsf{P}_{\mathrm{R}} \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists \vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))}{\mathcal{H},\ \rho,\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{u}))}\ F_i^{\mathsf{P}}(\vec{u}) \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u})$$

(If $\rho$ is the trace in a rule $R$'s conclusion, then $\rho' \triangleq (R) :: \rho$ is the trace in $R$'s premises.)

FIGURE 4.7: Logical rules.

### 4.5.1 Logical Rules

Figure 4.7 presents logical rules of our proof system to deal with the logical structure of separation logic entailments. For brevity, we write $\vec{x}$ to indicate a list of variables $x_1, \ldots, x_n$, and $|\vec{x}|$ means the number of variables in $\vec{x}$. We denote the symbolic-heap formula $\exists \vec{x}.(\Sigma \wedge \Pi)$ as $F$, where the variable list $\vec{x}$ can be empty. When $F \equiv \Sigma \wedge \Pi$, we write $F * \Sigma'$ to denote $\Sigma * \Sigma' \wedge \Pi$ and $F \wedge \Pi'$ to denote $\Sigma \wedge \Pi \wedge \Pi'$. When $\vec{u}$ and $\vec{v}$ are two variable lists of the same length, i.e., $\vec{u} = u_1, \ldots, u_n$ and $\vec{v} = v_1, \ldots, v_n$, we write $\vec{u}{=}\vec{v}$ to indicate $(u_1{=}v_1) \wedge \ldots \wedge (u_n{=}v_n)$. We also write $\vec{x} \# \vec{y}$ to denote $\vec{x}$ and $\vec{y}$ are disjoint, i.e., $\nexists u.(u \in \vec{x} \wedge u \in \vec{y})$, and use $\mathtt{FV}(F)$ to denote the list of all free variables of the formula $F$. Moreover, $F[e/x]$ is a formula obtained from $F$ by substituting the expression $e$ for all occurrences of the free variable $x$ in $F$.

The set of logical rules are explained in details as follows:

- **Axiom rules.** The rule $\vdash_{\Pi}$ proves a pure entailment $\Pi_1 \vdash \Pi_2$ by invoking off-the-shelf provers such as Z3 [Moura et al., 2008] to check the implication $\Pi_1 \Rightarrow \Pi_2$ in its side

condition. The two rules $\bot^1_{\mathrm{L}}$ and $\bot^2_{\mathrm{L}}$ decide an entailment *vacuously* valid if its antecedent is unsatisfiable, i.e., the antecedent contains a contradiction (as indicated in the rule $\bot^1_{\mathrm{L}}$'s side condition $\Pi \Rightarrow \mathsf{false}$) or separating singleton heap predicates having the same memory address ($u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{w}$, which violates the separating condition of separation logic).

- **Normalization rules.** These rules simplify their goal entailments by either eliminating existentially quantified variables ($\exists_{\mathrm{L}}$), or removing equalities ($=_{\mathrm{L}}, =_{\mathrm{R}}$) or the empty heap predicate $\mathsf{emp}$ ($\mathsf{E}_{\mathrm{L}}, \mathsf{E}_{\mathrm{R}}$) from antecedents or consequents of the entailments.
- **Matching rules.** The two rules $*\!\mapsto$ and $*\mathsf{P}$ match and remove *identical* spatial atoms from two sides of their goal entailments. More specifically, the matching condition, i.e., the identical condition, is guaranteed by adding equality constraints of these spatial atoms' arguments into the consequents of the derived entailments. These constraints need be proved later during the proof derivation.
- **Unfolding rule.** The rule $\mathsf{P}_{\mathrm{R}}$ derives new entailments by unfolding a heap predicate in the consequent of a goal entailment by its inductive definition. The original goal entailment is valid if at least one of the derived entailments is valid. On the other hand, heap predicates in the entailment's antecedent will be handled by the induction rule ID in the next subsection.

### 4.5.2 Induction Rules

$$\mathsf{ID}\ \frac{\mathcal{H}',\ (\mathsf{ID}) :: \rho,\ F_1 * F_1^{\mathsf{P}}(\vec{u}) \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ (\mathsf{ID}) :: \rho,\ F_1 * F_m^{\mathsf{P}}(\vec{u}) \vdash F_2}{\mathcal{H},\ \rho,\ F_1 * \mathsf{P}(\vec{u}) \vdash F_2}\ \dagger$$

$$\dagger:\ \mathcal{H}' \triangleq \mathcal{H}' \cup \{(F_1 * \mathsf{P}(\vec{u}) \vdash F_2, \mathbf{?})\}, \text{ and } \mathsf{P}(\vec{u}) \overset{\text{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u})$$

$$\mathsf{IH}\ \frac{\mathcal{H} \cup \{(H, status)\},\ (\mathsf{IH}) :: \rho,\ F_4\theta * \Sigma' \wedge \Pi_1 \vdash F_2}{\mathcal{H} \cup \{(H \triangleq \Sigma_3 \wedge \Pi_3 \vdash F_4, status)\},\ \rho,\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \exists \theta, \Sigma'.(\Sigma_1 \cong \Sigma_3\theta * \Sigma' \wedge \Pi_1 \rightarrow \Pi_3\theta);\ \dagger$$

$$\begin{aligned}\dagger:\ (status = \checkmark)\ &\vee\ \exists\, \iota, u, \vec{v}, \Sigma''.(\Sigma' \cong u \overset{\iota}{\mapsto} \vec{v} * \Sigma'') \\ &\vee\ \exists\, \rho_1, \rho_2.(\rho = \rho_1 @ [(*\!\mapsto)] @ \rho_2 \wedge (\mathsf{ID}) \notin \rho_1 \wedge (\mathsf{ID}) \in \rho_2)\end{aligned}$$

FIGURE 4.8: Induction rules.

Figure 4.8 presents two induction rules ID and IH, which are the main rules implementing our mutual induction principle. We will carefully explain them below.

The rule ID records its goal entailment as an induction hypothesis $H$, and unfolds an inductive heap predicate in the antecedent of $H$ to derive new entailments. When $H$ is inserted into the hypothesis vault $\mathcal{H}$, its status is initially assigned to **?** (*unknown*), indicating that its validity is not known at the moment. Later, the status of $H$ will be updated to ✓ (*valid*) once the proof search procedure is able to prove it valid. Generally, an entailment $E$ can be concluded valid if the proof search procedure is able to derive a proof tree $\mathcal{T}$, where (i) every leaf of $\mathcal{T}$ is empty via applications of axiom rules, and (ii) all hypotheses used by the rule IH must be derived within $\mathcal{T}$.

The rule IH applies an appropriate hypothesis $H \triangleq \Sigma_3 \wedge \Pi_3 \vdash F_4$ to prove its goal entailment $E \triangleq \Sigma_1 \wedge \Pi_1 \vdash F_2$. It firstly unifies the antecedents of $H$ and $E$ by a substitution $\theta$, i.e., there exists a spatial assertion $\Sigma'$ such that $\Sigma_1 \cong \Sigma_3\theta * \Sigma'$ and $\Pi_1 \Rightarrow \Pi_3\theta$. If such $\theta$ and $\Sigma'$ exist, we can weaken the antecedent of $E$ as follows $(\Sigma_1 \wedge \Pi_1) \vdash (\Sigma_3\theta * \Sigma' \wedge \Pi_3\theta \wedge \Pi_1) \vdash (F_4\theta * \Sigma' \wedge \Pi_1)$. Here, we apply Theorem 2.2 to obtain $\Sigma_3\theta \wedge \Pi_3\theta \vdash F_4\theta$ from the hypothesis $H$. The proof system then derives a new sub-goal entailment $F_4\theta * \Sigma' \wedge \Pi_1 \vdash F_2$ as shown in the premise of the rule IH.

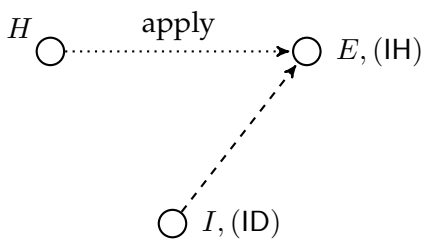



FIGURE 4.9: Applying hypothesis

The side condition $\dagger_{(\mathsf{IH})}$ of the rule IH ensures the decreasing condition of our mutual induction principle. In particular, suppose that the proof search procedure applies a hypothesis $H$ in $\mathcal{H}$ to prove an entailment $E$ by using the rule IH. If the status of $H$ is ✓, which is denoted by the first condition in $\dagger_{(\mathsf{IH})}$, then $H$ is already proved to be valid; thus it can be surely used to prove other entailments. Otherwise, the status of $H$ is **?**, and $H$

may participate in a mutual induction proof with an entailment $I$ in the proof path of $E$. We illustrate this induction hypothesis application in Figure 4.9, where the entailment $I$ has been recorded earlier as an induction hypothesis by an application of the rule ID.

In the latter case, our mutual induction principle requires the decrease of the model size when applying the hypothesis $H$ to prove the entailment $I$. We will show that this decreasing condition holds if at least one of the last two conditions of $\dagger_{(\mathsf{IH})}$ is satisfied. These two conditions are presented as follows:

(i) $\exists \iota, u, \vec{v}, \Sigma''.(\Sigma' \ \cong \ u \overset{\iota}{\mapsto} \vec{v} * \Sigma'')$, which indicates that the left-over heap part $\Sigma'$ after unifying antecedent of $H$ into that of $E$ contains at least one singleton heap predicate, or

(ii) $\exists \rho_1, \rho_2.(\rho = \rho_1 @ [(*\mapsto)] @ \rho_2 \ \wedge \ (\mathsf{ID}) \notin \rho_1 \ \wedge \ (\mathsf{ID}) \in \rho_2)$, which requires that there is a removal step of a singleton heap predicate, which is applied by the rule $*\mapsto$ in a proof path from the most recent induction step ID to the current hypothesis application IH.

Now, to prove the decreasing condition when applying the hypothesis $H$ to prove the entailment $I$, we will consider an arbitrary model $s, h$ of $I$. During the derivation path from $I$ to $E$, the model $s, h$ is transformed into a corresponding model $s_e, h_e$ of $E$. We always have $|h_e| \leq |h|$ since applications of all inference rules never increase heap model size of corresponding entailments. Moreover, when applying $H$ to prove $E$, the model $s', h'$ of $H$, which corresponds to $s_e, h_e$ of $E$, satisfies $|h'| \leq |h_e|$, due to the unification step in rule IH. We consider two following cases. If the condition (i) is satisfied, then the size of heap model of the left-over part $\Sigma'$ is at least 1 since $\Sigma'$ contains a singleton heap predicate. As a result, $|h'| < |h_e|$ and it follows that $|h'| < |h|$. If the condition (ii) is satisfied, then $|h_e| < |h|$ since there is a singleton heap predicate, whose heap model size is 1, is removed when deriving $I$ to $E$. This implies that $|h'| < |h|$. In summary, we obtain that $|h'| < |h|$ for both the two cases; thus, $s', h' \prec s, h$. This concludes our explanation about the rule IH.

### 4.5.3 The Proof Search Procedure

Figure 4.10 presents our proof search procedure Prove, which is designed in a self-recursive manner. Its inputs consist of a set of hypotheses $\mathcal{H}$, a proof trace $\rho$, and an entailment $F \vdash G$. These inputs correspond to the components of the conclusions of all inference rules. When Prove is invoked in the first time to prove a goal entailment, the induction hypothesis set $\mathcal{H}$ and the proof trace $\rho$ are initially assigned to an empty ($\varnothing$ and $[\,]$). Their values will be accumulated later during the proof derivation.

To prove the input entailment $F \vdash G$, the procedure Prove firstly discovers among all inference rules presented in Figures 4.7 and 4.8, a set of suitable rules $\mathcal{R}$ whose conclusion can be unified with the entailment $F \vdash G$. This inference rule discovery is done by invoking an auxiliary procedure FindRules at line 1. If no suitable rule is found, this procedure immediately returns `Unkn`, indicating that it is unable to prove the entailment (line 2). Otherwise, it subsequently processes each discovered rule R in $\mathcal{R}$ and either (i) returns `Valid` to announce a valid result, if an axiom rule is selected (line 4), or (ii) recursively searches for proofs of new entailments derived in the premises of R (lines 9–16). In the latter case, the procedure switches to apply the next discovered rule if one of the currently derived entailments is not proved (line 13), or returns `Valid` if all of them are proved by the current rule R (line 16). Finally, it simply returns `Unkn` when it cannot prove the goal entailment with all selected rules R (line 17).

**Procedure** Prove($\mathcal{H}, \rho, F \vdash G$)

**Input:** $\mathcal{H}$, $\rho$ and $F \vdash G$ are respectively a set of hypotheses, a proof trace, and a goal entailment.

**Output:** Validity result (`Valid` or `Unkn`), a set of derived entailments with their validity statuses, and a set of hypotheses used in the proof of $F \vdash G$.

1: $\mathcal{R} \leftarrow$ FindRules($\mathcal{H}, \rho, F \vdash G$)
2: **if** $\mathcal{R} = \varnothing$ **then return** (`Unkn`, $\varnothing, \varnothing$) *// no rule is selected*
3: **for each** R **in** $\mathcal{R}$ **do**
4: **if** R $\in \{ \vdash_{\Pi}, \bot^1_{\mathrm{L}}, \bot^2_{\mathrm{L}} \}$ **then return** (`Valid`, $\varnothing, \varnothing$) *// R is an axiom rule*
5: $res \leftarrow$ `Valid`
6: $\mathcal{H}_{\text{used}} \leftarrow \varnothing$
7: $\mathcal{H}_{\text{derived}} \leftarrow \varnothing$
8: **if** R = IH with hypothesis $E$ **then** $\mathcal{H}_{\text{used}} \leftarrow \mathcal{H}_{\text{used}} \cup \{E\}$
9: **for each** $(\mathcal{H}_i, \rho_i, F_i \vdash G_i)$ **in** Premises(R) **do**
10: $res, \mathcal{H}_{\text{derived}}, \mathcal{H}'_{\text{used}} \leftarrow$ Prove($\mathcal{H}_i \oplus \mathcal{H}_{\text{derived}}, \rho_i, F_i \vdash G_i$)
11: **if** $res =$ `Unkn` **then break**
12: $\mathcal{H}_{\text{used}} \leftarrow \mathcal{H}_{\text{used}} \cup \mathcal{H}'_{\text{used}}$
13: **if** $res =$ `Unkn` **then continue**
14: **else if** $\mathcal{H}_{\text{used}} \subseteq (\mathcal{H}_{\text{derived}} \cup \{F \vdash G\})$ **then** $\mathcal{H}_{\text{derived}} \leftarrow \mathcal{H}_{\text{derived}} \oplus \{(F \vdash G, \checkmark)\}$
15: **else** $\mathcal{H}_{\text{derived}} \leftarrow \mathcal{H}_{\text{derived}} \oplus \{(F \vdash G, ?)\}$
16: **return** (`Valid`, $\mathcal{H}_{\text{derived}}, \mathcal{H}_{\text{used}}$) *// all derived premises are proved*
17: **return** (`Unkn`, $\varnothing, \varnothing$) *// all rules fail to prove $F \vdash G$*

FIGURE 4.10: The mutual induction proof search procedure

The procedure Prove uses a local variable $\mathcal{H}_{\text{used}}$ to store all hypotheses used during the proof search. $\mathcal{H}_{\text{used}}$ is updated when the rule IH is applied (line 8) or after the procedure finishes proving a derived entailment (lines 10 and 12). Prove also uses another variable $\mathcal{H}_{\text{derived}}$ to capture all entailments derived during the proof search and their validity statuses. The condition at line 14 checks if all hypotheses used to prove the entailment $F \vdash G$ are only introduced during the entailment's proof. If this condition is satisfied, then $F \vdash G$ is updated with a *valid status* ✓ (line 14). Otherwise, the entailment may participate in a mutual induction proof, thus its status is assigned to *unknown* **?** (line 15).

At line 10, the procedure Prove uses not only the hypothesis set $\mathcal{H}_i$, introduced by the selected inference rule but also the set $\mathcal{H}_{\text{derived}}$ containing entailments derived during

the proof search to prove a new goal entailment $F_i \vdash G_i$. This hypothesis application reflects our mutual induction principle which allows a goal entailment and other derived entailments to be used as hypotheses to prove each other. Lastly, we use the *union and update* operator $\oplus$ in the algorithm to insert new entailments and their statuses into a set of hypotheses or to update existing entailments with their new statuses.

### 4.5.4 Soundness

We claim that our mutual induction proof system is sound, that is, if the proof search procedure is able to prove an entailment, then it is indeed valid. We formally restate this soundness property in Theorem 4.4.

**Theorem 4.4 (Soundness)** *Given an entailment $E$, if the proof search procedure* Prove *returns* `Valid` *when proving $E$, then $E$ is indeed valid.*

To prove the above soundness property, we first prove that all inference rules are sound, i.e., for each rule, if all entailments in its premises are valid, and its side condition, if any, holds, then the goal entailment in its conclusion is also valid. Afterward, we also need to argue that when the proof search procedure applies these inference rules to prove an entailment, the corresponding proof tree reflects accurately our mutual induction principle. The detailed soundness proof can be referred to in Appendix A.

## 4.6 Implementation

We have implemented the mutual induction proof system in a prototype prover named **Songbird**$_{\mathbf{MI}}$. It is written using the OCaml programming language and is built on top of the existing theorem prover **Songbird**[1], and the name **Songbird**$_{\mathbf{MI}}$ stands for "**Songbird** + **M**utual **I**ninduction. The prover **Songbird**$_{\mathbf{MI}}$ is available online for download at:

`http://songbird-prover.github.io/mutual-induction/`

[1]The inductive prover **Songbird**: `http://songbird-prover.github.io`

In this section, we will focus on discussing the heuristics we apply to make the proof search in **Songbird$_{\mathbf{MI}}$** more efficient. These details have not been addressed in the description of the general proof search procedure Prove in the previous section.

We recall that for a given goal entailment, the procedure Prove first finds all applicable inference rules whose conclusions can unify with the goal entailment. Thereafter, it sequentially applies each of the discovered rules until either one of them succeeds to prove the entailment (Prove returns `Valid`), or all of them fail (Prove returns `Unkn`). In our actual implementation, we aim to minimize the effort spent on these tasks. Firstly, we design a heuristic to determine only the necessary inference rules, but not all possible rules. Secondly, we reorder all discovered inference rules by their priorities, i.e., the chance of successfully proving the goal entailment when applying these rules, so that the proof search procedure can selectively apply these rules. The details are as follows.

To minimize the effort spent on finding inference rules, we rank all rules by their importance to decide which one should be explored first. We observe that the axiom rules ($\vdash_{\mathrm{II}}, \bot^1_{\mathrm{L}}$ or $\bot^2_{\mathrm{L}}$) are the most important rules, since they do not introduce any new entailments and directly conclude about their goal entailments' validity. Moreover, once the proof search procedure is able to find an axiom rule, it can immediately apply the rule without the need to search for other rules. These rules, therefore, will be explored first by our proof system. The normalization rules ($\exists_{\mathrm{L}}, =_{\mathrm{L}}, =_{\mathrm{R}}, \mathsf{E}_{\mathrm{L}}$ or $\mathsf{E}_{\mathrm{R}}$) are the next important rules, since they remove surplus constraints from their goal entailments and make them more concise and uniform. This normalization enables the proof search procedure to explore other future inference rules more easily. Therefore, if the proof search procedure is able to find a normalization rule, it also needs to apply this rule early.

We rank the remaining rules ($*\!\!\mapsto, *\mathsf{P}, \mathsf{P}_{\mathrm{R}}, \mathsf{ID}$ and $\mathsf{IH}$) as the last important rules since they all change the heap structures of their goal entailments. In general, we consider these rules to be equally important. Hence, for a given entailment, if the proof search procedure Prove is not able to utilize any axiom rule or normalization rule, it will need to find all possible instances of the rules $*\!\!\mapsto, *\mathsf{P}, \mathsf{P}_{\mathrm{R}}, \mathsf{ID}$ and $\mathsf{IH}$. Then, depending on the effect that each of the discovered rules produces to the goal entailment, we rank them

against the others. We consider the following typical scenarios:

- If Prove can discover an instance of the rule $*\mapsto$ which removes two singleton heap predicates $x\mapsto\vec{y}$ and $x\mapsto\vec{z}$, each from the two sides of the entailment, then this rule is the most important in comparison with other discovered rules. This is because the two singleton heaps have the same memory address $x$, and are likely to be identical.
- If Prove can identify an instance of the rule $*\mathsf{P}$ to remove two inductive heap predicates $\mathsf{P}(\vec{x_1}, y, \vec{z_1})$ and $\mathsf{P}(\vec{x_2}, y, \vec{z_2})$, where $|\vec{x_1}| = |\vec{x_2}|$ and $|\vec{z_1}| = |\vec{z_2}|$, then this rule is the second most important since these two predicates have the same argument $y$ at the same position, hence, they can be identical.
- If Prove can find an instance of the rules $\mathsf{P_R}, \mathsf{ID}, \mathsf{IH}$ which can be applied to produce potentially identical singleton heap predicates (having the same memory address) or inductive heap predicates (having at least one similar argument), then this rule is also important since it may introduce future opportunities to apply the two rules $*\mapsto$ or $*\mathsf{P}$ during the proof search.

The above observations lead us to design the FindRules procedure to discover applicable inference rules for a given goal entailment. This procedure FindRules is invoked by the proof search procedure Prove, as discussed in Subsection 4.5.3. We formally describe FindRules and its heuristic in Figure 4.11. Given an input entailment $F \vdash G$, this procedure sequentially checks if an axiom rule or a normalization rule can be applied for the goal entailment, via the invocation of an auxiliary procedure CanUnifyRule (lines 18 – 19). If such rule exists, it immediately returns the unified instance of that rule, via the application of another auxiliary procedure CreateRuleInstance (lines 20 – 21). If FindRules is not able to discover any axiom or normalization rules, it will find all other remaining inference rules (lines 22 – 26). The discovered rules will be accumulated into a set $\mathcal{R}$ as shown in line 26.

**Procedure** FindRules($\mathcal{H}, \rho, F \vdash G$)

**Input:** $\mathcal{H}, \rho$, and $F \vdash G$ are respectively a set of hypotheses, a proof trace and a goal entailment.

**Output:** Possible instances of inference rules

```
18: for each R in { ⊢_Π, ⊥_L^1, ⊥_L^2, ∃_L, =_L, =_R, E_L, E_R } do      // axiom and normalization rules,
19:     if CanUnifyRule((H, ρ, F ⊢ G), R) then
20:         R ← CreateRuleInstance((H, ρ, F ⊢ G), R)
21:         return {R}                                                  // return the first one.
22: R ← ∅
23: for each R in { *↦, *P, P_R, ID, IH } do                            // other remaining rules,
24:     if CanUnifyRule((H, ρ, F ⊢ G), R) then
25:         R ← CreateRuleInstance((H, ρ, F ⊢ G), R)
26:         R ← R ∪ {R}
27: R ← ReorderRules(R, CompareRule)                                    // reorder them by priority,
28: return R                                                            // and return all
```

FIGURE 4.11: The procedure that finds unification rules.

Thereafter, the procedure FindRules descendingly reorders all discovered rules in $\mathcal{R}$ by their importance (line 27). It utilizes a pair-wise comparison procedure CompareRule to assist in this reordering. The procedure CompareRule is presented in Figure 4.12, where its two inputs $\mathsf{R}_1$ and $\mathsf{R}_2$ are instances of inference rules $*\!\!\mapsto, *\mathsf{P}, \mathsf{P}_\mathsf{R}, \mathsf{ID}$, and IH. Its output can be one of the three values `HIGH`, `LOW`, and `EQUAL`, indicating that the rule $\mathsf{R}_1$ is respectively more, less, or equally important in comparison to $\mathsf{R}_2$. We implement the heuristics discussed earlier to decide the importance of a rule. In particular, there are three corresponding cases:

- If either $\mathsf{R}_1$ or $\mathsf{R}_2$ is an instance of the two rules $*\!\!\mapsto$ or $*\mathsf{P}$ and they match two potentially identical singleton heap predicates of the same memory address or inductive heap predicates having at least one similar argument, which is checked by an application of the procedure MatchPotentiallyIdenticalPred, then the corresponding rule is more important than the other (lines 31, 33, 38).

- If either $\mathsf{R}_1$ or $\mathsf{R}_2$ is an instance of the rules $\mathsf{P}_\mathsf{R}, \mathsf{ID}$ or IH, which can produce two potentially identical heap predicates (checked by ProducePotentiallyIdenticalPred), then the corresponding is also more important than the other (lines 35, 40, 41).

– If none of the above conditions satisfy, then CompareRule is unable to decide the priority between $R_1$ and $R_2$. In this case, the procedure CompareRule simply returns `EQUAL` to indicate that these two rules are equally important (line 43).

**Procedure** CompareRule($R_1, R_2$)

**Input:** $R_1$ and $R_2$ are two instances of the following rules $*\mapsto, *P, P_R, ID, IH$

**Output:** Priority of the rule $R_1$ in comparison with $R_2$, which can be: `HIGH`, `EQUAL`, `LOW`

```
29: if R1 ∈ { *↦, *P } then                                   // Compare *↦, *P with other rules
30:     if MatchPotentiallyIdenticalPred(R1) then
31:         return HIGH
32:     if R2 ∈ { *↦, *P } and MatchPotentiallyIdenticalPred(R2) then
33:         return LOW
34:     if R2 ∈ { PR, ID, IH } and ProducePotentiallyIdenticalPred(R2) then
35:         return LOW
36: if R1 ∈ { PR, ID, IH } then                               // Comparing PR, ID, IH with other rules
37:     if R2 ∈ { *↦, *P } and MatchPotentiallyIdenticalPred(R2) then
38:         return LOW
39:     if ProducePotentiallyIdenticalPred(R1) then
40:         return HIGH
41:     if R2 ∈ { PR, ID, IH } and ProducePotentiallyIdenticalPred(R2) then
42:         return LOW
43: return EQUAL
```

FIGURE 4.12: The procedure that compares inference rules.

## 4.7 Experiment

In the previous sections, we have discussed the mutual induction proof system and the prototype prover **Songbird$_{\mathbf{MI}}$**. To evaluate the effectiveness of our prover, we experiment it on two entailment benchmarks from the separation logic literature as well as a handcrafted benchmark. We also compare the experimental result of **Songbird$_{\mathbf{MI}}$** with the results of other state-of-the-art separation logic provers. Our evaluation was performed on an Ubuntu 14.04 LTS machine with CPU Intel® E5-2620 (2.4GHz) and RAM 64GB. We will describe the details as follows.

The first experiment was conducted on two entailment benchmarks sll_entl and slrd_entl from a separation logic competition named SL-COMP in 2014 [Sighireanu et al., 2016]. We consider only *valid entailments* in these benchmarks since our prover **Songbird$_{\mathbf{MI}}$** cannot disprove an entailment, i.e., it cannot conclude if the entailment is invalid. We compare the performance of **Songbird$_{\mathbf{MI}}$** with other participating provers of the SL-COMP 2014. They include **Slide** [Iosif et al., 2014], **Sleek** [Chin et al., 2012], **Spen**[Enea et al., 2014], and **Cyclist**[Brotherston et al., 2012]. On the other hand, we are also unable to make a direct comparison with the induction-based proof technique presented in [Chu et al., 2015] as their prover was not available by the time we conduct the experiment.

| **Benchmark** | | | **Proved entailments** | | | | | **Total proving time (s)** | | | | | **Average time (s)** | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Category | | #ent | Sld | Slk | Spn | Ccl | **Smi** | Sld | Slk | Spn | Ccl | **Smi** | Sld | Slk | Spn | Ccl | **Smi** |
| sll_entl | bolognesa | 57 | 0 | 0 | **57** | 0 | **57** | – | – | 17.4 | – | 140.3 | – | – | 0.31 | – | 2.46 |
| | clones | 60 | 0 | **60** | **60** | **60** | **60** | – | 1.7 | 2.2 | 0.2 | 3.9 | – | 0.03 | 0.04 | 0.00 | 0.07 |
| | smallfoot | 55 | 0 | 51 | **55** | **55** | **55** | – | 1.5 | 1.6 | 6.8 | 3.5 | – | 0.03 | 0.03 | 0.12 | 0.06 |
| slrd_entl | singly-ll | 64 | 11 | 48 | 3 | **63** | **63** | 0.3 | 2.0 | 0.1 | 1.0 | 1.8 | 0.03 | 0.04 | 0.03 | 0.02 | 0.03 |
| | doubly-ll | 37 | 13 | 18 | 9 | **29** | 25 | 20.7 | 1.2 | 0.3 | 68.5 | 0.8 | 1.59 | 0.06 | 0.03 | 2.36 | 0.03 |
| | nested-ll | 11 | 0 | 5 | **11** | 7 | **11** | – | 0.8 | 0.3 | 9.3 | 0.4 | – | 0.16 | 0.03 | 1.33 | 0.04 |
| | skip-list | 13 | 0 | 4 | **13** | 5 | **13** | – | 0.4 | 0.9 | 0.2 | 1.2 | – | 0.09 | 0.07 | 0.04 | 0.09 |
| | tree | 26 | 13 | 14 | 0 | 23 | **24** | 60.5 | 0.9 | – | 12.9 | 56.5 | 4.66 | 0.07 | – | 0.56 | 2.35 |
| Total | | 323 | 37 | 200 | 189 | 242 | **308** | 81.5 | 8.5 | 22.8 | 99.0 | 208.6 | 2.20 | 0.04 | 0.11 | 0.41 | 0.68 |

(Sld, Slk, Spn, Ccl and Smi respectively stands for **Slide**, **Sleek**, **Spen**, **Cyclist** and **Songbird$_{\mathbf{MI}}$**)

TABLE 4.1: Evaluation on the benchmarks of SL-COMP 2014

Entailments in the benchmark sll_entl relate only singly linked lists and they are categorized into three groups: bolognesa, clones and smallfoot, which are named after their original benchmarks. On the other hand, entailments in the benchmark slrd_entl contain inductive heap predicates modeling more diverse data structures. We category this benchmarks based on their predicate types, such as singly linked lists (singly-ll), doubly linked lists (doubly-ll), nested lists (nested-ll), skip lists (skip-list) and trees (tree). In Figure 4.1, we report the number of entailments successfully proved by a prover in each category, with a timeout of 120 seconds for proving an entailment, as well as the total and

average time spent by each prover to prove these entailments. To ensure a fair comparison, we calculate only the running time that each prover spends on entailments that it can successfully prove. We do not consider time spent on unsuccessful entailments since a prover possibly passes a timeout of 120 seconds when it fails to prove an entailment.

In each category, the total number of entailments is shown in the column **#ent**, and the best results among the examined provers are highlighted in bold. Figure 4.1 shows that **Songbird$_{\mathbf{MI}}$** can prove more entailments than all the other tools. Particularly, **Songbird$_{\mathbf{MI}}$** can prove 308/323 entailments (95.4%) with an average running time of 0.68 seconds per each proved entailment. The second best prover **Cyclist** can prove only 242/323 entailments (74.9%), which is less than **Songbird$_{\mathbf{MI}}$** 66/323 entailments (20.4%). **Cyclist** is averagely faster than **Songbird$_{\mathbf{MI}}$** when it spends about 0.41 seconds per each proved entailment. The two provers **Sleek** and **Spen** are slightly behind **Cyclist** when they can respectively prove 200/323 (61.9%) and 189/323 (58.5%) entailments. The last prover **Slide** can prove only 37/323 entailments (11.5%). Figure 4.1 also shows that **Songbird$_{\mathbf{MI}}$** outperforms other provers in almost all categories, except for doubly-ll. In the doubly-ll category, we are behind solely **Cyclist**, the second best prover, by only 4/37 entailments in this category. **Songbird$_{\mathbf{MI}}$** cannot prove 12/37 entailments in this doubly-ll category since they contain sophisticated heap or pure formulas. Our technique may require more effective generalization to handle these unproved entailments.

| **Songbird$_{\mathbf{MI}}$** | $\checkmark_{sb}\,\times_{o}$ | $\times_{sb}\,\checkmark_{o}$ | $\checkmark_{sb}\,\checkmark_{o}$ | $\times_{sb}\,\times_{o}$ |
|---|---|---|---|---|
| **Cyclist** | 71 | 5 | 237 | 10 |
| **Spen** | 104 | 4 | 204 | 11 |
| **Sleek** | 109 | 1 | 199 | 14 |
| **Slide** | 281 | 10 | 27 | 5 |

TABLE 4.2: Comparison between **Songbird$_{\mathbf{MI}}$** and other provers.

In Figure 4.2, we make a detailed comparison among **Songbird$_{\mathbf{MI}}$** and other provers. Specifically, the first column ($\checkmark_{sb}\,\times_{o}$) shows the number of entailments that **Songbird$_{\mathbf{MI}}$**

can prove valid whereas the others cannot. The second column ($\checkmark_{sb}$ $\checkmark_{o}$) reports the number of entailments that can be proved by other tools, but not by **Songbird$_{\mathbf{MI}}$**. The last two columns list the number of entailments that both **Songbird$_{\mathbf{MI}}$** and others can ($\checkmark_{sb}$ $\checkmark_{o}$) or cannot ($\times_{sb}$ $\times_{o}$) prove. There are 71 (resp. 104, 109 and 281) entailments that can be proved by our tool, but *not* by **Cyclist** (resp. **Spen**, **Sleek**, and **Spen**). Furthermore, **Songbird$_{\mathbf{MI}}$** can prove almost all entailments that can be proved by other provers. There is only 1 (resp. 4, 5 and 10) over totally 323 entailments that can be proved by **Sleek** (resp. **Spen**, **Cyclist**, and **Slide**) but not by **Songbird$_{\mathbf{MI}}$**. This result is encouraging, thanks to the proposed mutual induction proof technique.

Secondly, we would like to highlight the efficiency of *mutual induction* in our proof technique via a comparison between **Songbird$_{\mathbf{MI}}$** and its variant **Songbird$_{\mathbf{SI}}$**, which exploits only induction hypotheses found within a *single* proof path. This mimics the structural induction technique which explores induction hypotheses in the same proof path. For this purpose, we designed an entailment benchmark, namely slrd_ind, whose entailments are more complex than those in the benchmarks sll_entl and slrd_entl. For example, the slrd_ind benchmark contains an entailment:

$$\mathsf{lsEven}(x, y) * y{\mapsto}z * \mathsf{lsEven}(z, t) \vdash \exists u.\, \mathsf{lsEven}(x, u) * u{\mapsto}t$$

Here, the predicate $\mathsf{lsEven}(x, y)$ denoting list segments with even length. The above entailment was inspired by the entailment $\mathsf{lsEven}(x, y) * \mathsf{lsEven}(y, z) \vdash \mathsf{lsEven}(x, z)$ in the problem 11.tst.smt2 of slrd_entl, contributed by team **Cyclist**. The benchmark slrd_ind is available at: http://songbird-prover.github.io/mutual-induction/

Entailments in this benchmark are also categorized by their predicate types, including regular singly linked lists (ll/ll2), linked lists with even or odd length (ll-even/odd) and linked list segments which are left- or right-recursively defined (ll-left/right). In addition, entailments in the misc category involve all aforementioned linked list predicates.

As shown in Figure 4.3, **Songbird$_{\mathbf{SI}}$** is able to prove 102/116 entailments (87.9%), slightly behind the second best prover **Cyclist**, which can prove 104/116 entailments (89.7%). We do not list other provers **Sleek**, **Spen** and **Slide** in this experiment since they cannot prove

any entailment in the benchmark slrd_ind. On the other hand, **Songbird$_{\mathbf{MI}}$**, with full capability of mutual induction, can prove all 116 entailments in slrd_ind (100%). In addition, **Songbird$_{\mathbf{MI}}$** spends averagely only 0.10 seconds to prove an entailment. This running time is faster than the second best prover **Cyclist**, which takes averagely 1.83 seconds to prove each entailment. The running time of **Songbird$_{\mathbf{MI}}$** is only slightly slower than its variant **Songbird$_{\mathbf{SI}}$**, which needs averagely 0.07 seconds for each proved entailment (0.03 seconds faster than **Songbird$_{\mathbf{MI}}$**). A possible reason for the speedy performance of **Songbird$_{\mathbf{SI}}$** is that it only searches for applications of induction hypotheses recorded in a single proof path but not in an entire proof tree as by **Songbird$_{\mathbf{MI}}$**. Consequently, **Songbird$_{\mathbf{SI}}$** spends less time on discovering such induction hypothesis applications. However, this feature also prevents **Songbird$_{\mathbf{SI}}$** from utilizing potential hypotheses derived in other paths of the proof tree. In fact, **Songbird$_{\mathbf{SI}}$** cannot prove 14/116 entailments in the benchmark slrd_ind. On the other hand, by exploiting hypotheses derived in the entire proof tree, **Songbird$_{\mathbf{MI}}$** can prove all 116/116 entailments. This experimental result is encouraging as it shows the usefulness and essentials of our mutual induction proof technique in proving sophisticated separation logic entailments.

| **Benchmark** | | | **Proved entailments** | | | **Total time (s)** | | | **Average time (s)** | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Category | | #ent | Ccl | Ssi | Smi | Ccl | Ssi | Smi | Ccl | Ssi | Smi |
| slrd_ind | ll/ll2 | 44 | 33 | 30 | **44** | 37.8 | 3.3 | 7.8 | 1.15 | 0.11 | 0.18 |
| | ll-even/odd | 20 | **20** | **20** | **20** | 17.3 | 0.9 | 1.0 | 0.87 | 0.05 | 0.05 |
| | ll-left/right | 20 | **20** | **20** | **20** | 13.6 | 0.7 | 0.8 | 0.68 | 0.03 | 0.04 |
| | misc | 32 | 31 | **32** | **32** | 121.2 | 2.4 | 2.5 | 3.91 | 0.07 | 0.08 |
| Total | | 116 | 104 | 102 | **116** | 189.9 | 7.2 | 12.1 | 1.83 | 0.07 | 0.10 |

(Ccl, Ssi and Smi respectively stands for **Cyclist**, **Songbird$_{\mathbf{SI}}$** and **Songbird$_{\mathbf{MI}}$**)

TABLE 4.3: Evaluation on the synthetic benchmark slrd_ind

## 4.8 Related Work

We recall our literature review about the entailment proving problem in Chapter 3. There have been two works which apply induction-based proof techniques to prove separation logic entailments: [Brotherston et al., 2011a; Chu et al., 2015]. In the first work [Brotherston et al., 2011a], the authors proposed the *cyclic proof*, which allows proof trees to contain cycles, and can be perceived as infinite derivation trees. Furthermore, during the proof derivation, induction hypotheses are not explicitly identified via applications of induction rules; instead, they are implicitly obtained via the discovery of valid cycle proofs. Consequently, a soundness condition needs to be checked globally on proof trees. On the other hand, the authors of the second work [Chu et al., 2015] apply *structural induction* on inductive heap predicates for proving separation logic entailments. During the proof search, this technique dynamically uses derived entailments as induction hypotheses. When applying induction hypotheses, it performs a local check to ensure that predicates in the target entailments are substructures of predicates in the entailments captured as hypotheses. This dynamicity in hypothesis generation enables multiple induction hypotheses within a single proof path to be exploited; however, it does not admit hypotheses obtained from different proof paths.

The technique we propose in this chapter differs from both the two aforementioned works that not only it allows soundness condition to be checked locally at inference rule level, but it also supports mutual induction where entailments from different proof paths can be used as hypotheses to prove each other.

## 4.9 Discussion

In this section, we would like to discuss in detail the limitation of our mutual induction proof system. There are two limitations as below.

Firstly, our mutual induction proof system can prove that an entailment is valid, but

it cannot conclude if the entailment is invalid. More general, our proof system is an incomplete system. That is, for a given entailment, it cannot always decide whether the entailment is valid or not. This drawback is generally a disadvantage of induction-based proof techniques, such as cyclic proof [Brotherston et al., 2011a] and structural induction proof [Chu et al., 2015]. Theoretically, this incompleteness problem is due to the failure of cut-elimination in inductive theories, where the theory of separation logic with inductive heap predicates is an instance. Since cut rule cannot be eliminated, every formula or valid entailment can be a potential cut point, hence the search space of an inductive prover can grow up exponentially. This search space explosion might prevent the inductive prover from proving an entailment. The incompleteness issue of mathematical induction is also carefully addressed in the work of Alan Bundy [Bundy, 2001]. Interested readers are invited to refer to it for more details.

Secondly, our proof system currently does not perform necessary generalization of induction hypotheses. We recall that when proving an entailment, the proof system directly records it as an induction hypothesis and unfolds an appropriate inductive heap predicate to derive new entailments. Nevertheless, this induction hypothesis might be too sophisticated to be efficiently handled by an inductive prover. For example, it might contain numerous inductive heap predicates, or complex pure constraints, which overwhelm the inductive hypothesis application step. In fact, such sophisticated entailments appear in the benchmark slrd_entl of SL-COMP 2014. For example, each entailment in the test cases *dll-spaghetti.smt2*, *dll-spaghetti-existential.smt2*, *dll2-spaghetti.smt2* and *dll2-spaghetti-existential.smt2* in this benchmark contains 42 inductive heap predicates of the two predicate symbols DLL_plus and DLL_plus_rev which model doubly linked list data structures. These entailments currently cannot be proved by either our prover **Songbird$_{\mathbf{MI}}$** or the prover **Cyclist** within the timeout of 120 seconds. On the other hand, we believe that **Songbird$_{\mathbf{MI}}$** would be able to prove these entailments, if it could discover more general hypotheses, such as $\mathsf{DLL_plus}(x, y, z, t) \vdash \mathsf{DLL_plus_rev}(x, y, z, t)$ or $\mathsf{DLL_plus_rev}(x, y, z, t) \vdash \mathsf{DLL_plus}(x, y, z, t)$, which shows that the two inductive heap predicates $\mathsf{DLL_plus}(x, y, z, t)$ and $\mathsf{DLL_plus_rev}(x, y, z, t)$ are essentially equivalent.

## 4.10 Summary

We have proposed a mutual induction principle and developed a proof system to automatically prove separation logic entailments. In particular, we have shown that mathematical induction can perform well on the size of heap model of separation logic entailments. Our approach allows a goal entailment, and other entailments derived during the proof search can be used as hypotheses to prove each other mutually. This mutual hypothesis application can theoretically lead to the discovery of more succinct proof trees. We also demonstrate the effectiveness of our induction proof by experimenting on three entailment benchmarks, in which our prototype prover $\mathbf{Songbird_{MI}}$ outperforms the state-of-the-art separation logic provers.

# Chapter 5

# The Lemma Synthesis Framework

## 5.1 Overview

In this chapter, we propose a novel *lemma synthesis framework*, which automatically discovers lemmas to serve as eureka steps in the entailment proofs. Mathematical induction and template-based constraint solving are two pillars of our framework. To derive the supporting lemmas for a given entailment, the framework firstly identifies several possible lemma templates from the entailment's heap structure, then sets up unknown relations among the variables of each template and *conducts structural induction proof* to generate constraints about these relations, and finally *solves these constraints* to find out the unknown relations' actual definitions, thus discovers the lemmas. We have integrated this framework in a prototype separation logic prover and experiment it on various entailment benchmarks. The experimental results show that our lemma-synthesis-assisted prover can prove many entailments while the existing techniques cannot. In summary, the contributions of our lemma synthesis framework are as follows:

- We propose a novel framework to automatically synthesize lemmas serving as eureka steps to assist in proving entailments in a fragment of symbolic-heap separation logic with inductive heap predicates.
- We integrate the lemma synthesis framework into an entailment proof system, allowing lemmas to be flexibly discovered *on-the-fly* to assist in proving entailment.

– We implement a prototype prover, named **Songbird$_{\mathbf{LS}}$**, and experiment it with numerous entailments from various benchmarks. The result is promising since our prover can prove most of the valid entailments in these benchmarks and outperforms all existing tools. The prover **Songbird$_{\mathbf{LS}}$** is available online at:

http://songbird-prover.github.io/lemma-synthesis/ .

## 5.2 Motivating Example

We present a running example to illustrate our work and result. Here, we consider an inductive heap predicate dll and its variant dllrev, both modeling a doubly linked list data structure with a length property.

$$\begin{array}{rcl} \mathsf{dll}(hd, pr, tl, nt, len) & \stackrel{\text{def}}{=} & hd{\mapsto}pr, nt \wedge hd{=}tl \wedge len{=}1 \\ & & \vee\ \exists u.(hd{\mapsto}pr, u * \mathsf{dll}(u, hd, tl, nt, len{-}1)) \\ \mathsf{dllrev}(hd, pr, tl, nt, len) & \stackrel{\text{def}}{=} & hd{\mapsto}pr, nt \wedge hd{=}tl \wedge len{=}1 \\ & & \vee\ \exists u.(\mathsf{dllrev}(hd, pr, u, tl, len{-}1) * tl{\mapsto}u, nt) \end{array}$$

Each element in the linked list is modeled by a singleton heap predicate $x{\mapsto}pr, nt$, where $x$ is its memory address, $pr$ and $nt$ respectively point to the *previous* and the *next* element in the list. Moreover, both $\mathsf{dll}(hd, pr, tl, nt, len)$ and $\mathsf{dllrev}(hd, pr, tl, nt, len)$ denote a non-empty doubly linked list from the first element pointed-to by $hd$ (head) to the last element pointed-to by $tl$ (tail). The *previous* and the *next* element of the entire list are respectively pointed-to by $pr$ and $nt$, and $len$ denotes the linked list's length. The only difference between the two predicates is that dll is recursively defined from the linked list's head to the tail, whereas dllrev is defined in the reversed direction.

Suppose that when verifying a program manipulating the doubly linked list data structure, a verifier needs to prove an entailment relating to an extraction of the last element from a concatenation of two linked lists with certain constraints on their lengths, such as the following entailment $E_1$.

$$E_1 \triangleq \mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$$

Unfortunately, the existing techniques could not prove $E_1$. Specifically, the predicate-restriction approaches either consider only singly linked list [Berdine et al., 2005b; Bozga et al., 2010; Cook et al., 2011; Pérez et al., 2011; Pérez et al., 2013], or do not handle linear arithmetic constraints [Iosif et al., 2013; Iosif et al., 2014; Enea et al., 2014], or require a pre-defined translation of heap predicates into other theories [Piskac et al., 2013; Piskac et al., 2014]. Moreover, the non-induction-based approaches require users to provide supporting lemmas [Chin et al., 2012]. Finally, the induction-based techniques [Brotherston et al., 2011a; Chu et al., 2015] fail to prove $E_1$ because this entailment is *not general enough* to be an effective induction hypothesis, due to its *specific constant values*.

We briefly explain the *failure of induction proof* on $E_1$ as follows. Typically, induction is performed on an inductive heap predicate in $E_1$'s antecedent. Suppose the chosen predicate is $\mathsf{dllrev}(x, y, u, v, n)$; $E_1$ is then recorded as an induction hypothesis $H$, whose variables are renamed to avoid confusion:

$$H \triangleq \mathsf{dllrev}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, 200) \wedge m{\geq}100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, m{+}199) * c{\mapsto}k, d)$$

Subsequently, the predicate $\mathsf{dllrev}(x, y, u, v, n)$ of $E_1$ is unfolded by its recursive definition to derive new sub-goal entailments. We consider an interesting case when the below entailment $E_1'$ is obtained from an inductive case unfolding of $\mathsf{dllrev}(x, y, u, v, n)$:

$$E_1' \triangleq \mathsf{dllrev}(x, y, w, u, n{-}1) * u{\mapsto}w, v * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$$

When applying the induction hypothesis $H$ to prove $E_1'$, the predicates of the same symbols (dllrev or dll) in the antecedents of $H$ and $E_1'$ need to be unified by a substitution. However, such substitution does not exist since $q$ is mapped to $u$ when unifying $\mathsf{dllrev}(a, b, p, q, m)$ vs. $\mathsf{dllrev}(x, y, w, u, n{-}1)$, whereas $q$ is mapped to the *different* variable $v$ when unifying $\mathsf{dll}(q, p, c, d, 200)$ vs. $\mathsf{dll}(v, u, z, t, 200)$.

Alternatively, we can weaken the spatial formula $u{\mapsto}w, v * \mathsf{dll}(v, u, z, t, 200)$ of $E_1'$'s antecedent into $\mathsf{dll}(u, w, z, t, 201)$, w.r.t. the definition of dll, to derive the entailment $E_1''$:

$$E_1'' \triangleq \mathsf{dllrev}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$$

Again, no substitution can unify the antecedents of $H$ and $E_1''$. For example, the substitution $\theta_1 \triangleq [x/a, y/b, w/p, u/q, n{-}1/m]$ might be able to unify $\mathsf{dllrev}(a, b, p, q, m)$ with $\mathsf{dllrev}(x, y, w, u, n{-}1)$, that is, $\mathsf{dllrev}(a, b, p, q, m)\theta_1 \equiv \mathsf{dllrev}(x, y, w, u, n{-}1)$. However, the constraint $n{\geq}100$ in the antecedent of $E_1''$ *cannot prove* that $n{-}1{\geq}100$, which is obtained from applying $\theta_1$ on the constraint $m{\geq}100$ of $H$. In addition, the substitution $\theta_2 \triangleq [u/q, w/p, z/c, t/d]$ could not unify $\mathsf{dll}(q, p, c, d, 200)$ with $\mathsf{dll}(u, w, z, t, 201)$ since the two constants 201 and 200 are *non-unifiable*. In short, the above inability in unifying heap predicates makes induction proof fail on $E_1$.

Nevertheless, the entailment $E_1$ is provable if necessary lemmas can be discovered. For instance, the following lemmas $L_1$, $L_2$, and $L_3$ can be introduced to assist in proving $E_1$. More specifically, $L_1$ *converts* a linked list modeled by the predicate symbol dllrev into a linked list modeled by the variant predicate dll; $L_2$ *combines* two linked lists modeled by dll into a new one, which is similar in spirit to the "composition lemma" introduced by [Enea et al., 2015]; lastly, $L_3$ *splits* a linked list modeled by dll into two parts including a new linked list and a singleton heap

$$
\begin{array}{lcl}
L_1 & \triangleq & \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m) \\
L_2 & \triangleq & \mathsf{dll}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, l) \vdash \mathsf{dll}(a, b, c, d, m + l) \\
L_3 & \triangleq & \mathsf{dll}(a, b, c, d, m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a, b, w, c, m{-}1) * c{\mapsto}w, d)
\end{array}
$$

$$
\dfrac{\dfrac{\dfrac{\dfrac{}{n{\geq}100 \vdash (x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200)}\vdash_{\Pi}}{E_4 \triangleq \mathsf{dll}(x, y, z, t, n{+}200) \wedge n{\geq}100 \vdash \mathsf{dll}(x, y, z, t, n{+}200)}*\mathsf{P}}{E_3 \triangleq \mathsf{dll}(x, y, z, t, n{+}200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)}\mathsf{LM_R}^3}{E_2 \triangleq \mathsf{dll}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)}\mathsf{LM_L}^2}{E_1 \triangleq \mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)}\mathsf{LM_L}^1
$$

where $\mathsf{LM_L}^1$ applies $L_1$ with $\theta_1 \equiv [x/a, y/b, u/c, v/d, n/m]$,
$\mathsf{LM_L}^2$ applies $L_2$ with $\theta_2 \equiv [x/a, y/b, u/p, v/q, n/m, z/c, t/d, 200/l]$,
$\mathsf{LM_R}^3$ applies $L_3$ with $\theta_3 \equiv [x/a, y/b, z/c, t/d, r/w, n{+}200/m]$

FIGURE 5.1: Proof tree of the motivating lemma $E_1$

The three lemmas $L_1$, $L_2$, and $L_3$ can be used to prove $E_1$ as shown in Figure 5.1. They are successively utilized by the lemma application rules $\mathsf{LM_L}, \mathsf{LM_R}$ to finally derive a

new entailment $E_4$, which can be easily proved by standard inference rules in separation logic. We will explain the details of these lemma application rules $\mathsf{LM}_{\mathrm{L}}, \mathsf{LM}_{\mathrm{R}}$ and other inference rules in Section 5.4.1.

In the proof tree in Figure 5.1, the entailment $E_1$ can only be concluded valid if all the lemmas $L_1$, $L_2$, and $L_3$ are also valid. Furthermore, this proof tree is not constructed by any induction proofs. Instead, induction is performed in the proof of each lemma. We illustrate a partial induction proof tree of $L_1$ in Figure 5.2, where an induction hypothesis is recorded by the induction rule ID and is later utilized by the induction hypothesis application rule IH. These three lemmas are called *inductive lemmas* since induction proof are required to prove their validity. In general, the inductive lemmas help to modularize the proof of a complex entailment like $E_1$: induction is not directly performed to prove $E_1$; it is alternatively utilized to prove simpler supporting lemmas such as $L_1$, $L_2$, and $L_3$. This modularity, therefore, increases the success chance of proving complex entailments.

$$
\dfrac{\dfrac{\dots}{a{\mapsto}b, d \wedge a{=}c \wedge m{=}1 \vdash \mathsf{dll}(a,b,c,d,m)}\,\mathsf{P_R} \quad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{true \vdash \exists t.(v{=}t \wedge a{=}a \wedge c{=}c \wedge d{=}d \wedge m{-}1{=}m{-}1)}\,\vdash_{\Pi}}{\mathsf{dll}(v,a,c,d,m{-}1) \vdash \exists t.(\mathsf{dll}(t,a,c,d,m{-}1) \wedge v{=}t)}\,{*}\mathsf{P}}{\mathsf{dll}(v,a,u,c,m{-}2) * c{\mapsto}u, d \vdash \exists t.(\mathsf{dll}(t,a,c,d,m{-}1) \wedge v{=}t)}\,\mathsf{IH}^4}{a{\mapsto}b, v * \mathsf{dll}(v,a,u,c,m{-}2) * c{\mapsto}u, d \vdash \exists t.(a{\mapsto}b, t * \mathsf{dll}(t,a,c,d,m{-}1))}\,{*}{\mapsto}}{a{\mapsto}b, v * \mathsf{dll}(v,a,u,c,m{-}2) * c{\mapsto}u, d \vdash \mathsf{dll}(a,b,c,d,m)}\,\mathsf{P_R}}{\mathsf{dll}(a,b,u,c,m{-}1) * c{\mapsto}u, d \vdash \mathsf{dll}(a,b,c,d,m)}\,\mathsf{ID}^3}{\mathsf{dllrev}(a,b,u,c,m{-}1) * c{\mapsto}u, d \vdash \mathsf{dll}(a,b,c,d,m)}\,\mathsf{IH}^2}{L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)}\,\mathsf{ID}^1
$$

where $\mathsf{ID}^1$ records $H_1 \triangleq \mathsf{dllrev}(a',b',c',d',m') \vdash \mathsf{dll}(a',b',c',d',m')$,
$\mathsf{ID}^3$ records $H_2 \triangleq \mathsf{dll}(a',b',u',c',m'{-}1){*}c'{\mapsto}u', d' \vdash \mathsf{dll}(a',b',c',d',m')$
$\mathsf{IH}^2$ applies $H_1$ with $\theta_1 \equiv [a/a', b/b', u/c', c/d', m{-}1/m']$,
$\mathsf{IH}^4$ applies $H_2$ with $\theta_2 \equiv [v/a', a/b', u/u', c/c', d/d', m{-}1/m']$.

FIGURE 5.2: Partial induction proof tree of the inductive lemma $L_1$

On the other hand, we are *not* interested in *trivial lemmas*, which can be proved *directly without using* any induction hypotheses: the rule IH is not used in their proofs. For example, $L_3' \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(a{\mapsto}b, w * \mathsf{dll}(w,a,c,d,m{-}1))$ is a trivial lemma, whose direct proof tree is presented in Figure 5.3. Obviously, if a trivial lemma can be applied to prove an entailment, then the same sequence of inference rules utilized in the

lemma's proof can also be directly applied to the goal entailment. For this reason, the *trivial lemmas* do not help to modularize the induction proof.

$$\dfrac{\dfrac{}{a{\mapsto}b, d \wedge a{=}c \wedge m{=}1 \wedge m{\geq}2 \vdash \exists w.(a{\mapsto}b, w * \mathsf{dll}(w,a,c,d,m{-}1))}\ \bot^2_{\mathrm{L}} \qquad \dfrac{\dfrac{\dfrac{}{m{\geq}2 \vdash \exists w.(u{=}w \wedge a{=}a \wedge c{=}c \wedge d{=}d \wedge m{-}1{=}m{-}1)}\ \vdash_{\Pi}}{\mathsf{dll}(u,a,c,d,m{-}1) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(w,a,c,d,m{-}1) \wedge u{=}w)}\ *\mathsf{P}}{a{\mapsto}b, u * \mathsf{dll}(u,a,c,d,m{-}1) \wedge m{\geq}2 \vdash \exists w.(a{\mapsto}b, w * \mathsf{dll}(w,a,c,d,m{-}1))}\ *{\mapsto}}{L'_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(a{\mapsto}b, w * \mathsf{dll}(w,a,c,d,m{-}1))}\ \mathsf{ID}$$

FIGURE 5.3: Direct proof tree of the trivial lemma $L'_3$

In this work, we propose a novel framework to synthesize *inductive lemmas*, such as $L_1$, $L_2$, and $L_3$, to assist in proving separation logic entailments. For a given goal entailment $E$, we first identify all possible lemma templates, which essentially are entailments constructed from heap predicate symbols appearing in $E$. The templates will be refined with more Boolean constraints on their variables until *valid* inductive lemmas are discovered. We will explain the details in Section 5.5.

## 5.3 Theoretical Background

Our lemma synthesis framework is developed to assist in proving entailments in the fragment *of symbolic-heap separation logic* with inductive heap predicates and linear arithmetic, which has been introduced earlier in Chapter 2. It is also extended with *unknown relations* to represent Boolean constraints of the desired lemmas. We will present the related background of the entailment proof in the following subsections.

### 5.3.1 Symbolic-Heap Separation Logic with Unknown Relations

**Syntax.** We denote our *symbolic-heap separation logic* fragment with *inductive heap predicates* and *unknown relations* as $\mathsf{SL}^{\mathsf{U}}_{\mathsf{ID}}$ and present its syntax in Figure 5.4. Here, $\sigma$ denotes

a *spatial atom*, which is either (i) a predicate emp modeling an *empty* memory, (ii) a *singleton* heap predicate $x \overset{\iota}{\mapsto} x_1, \ldots, x_n$ describing an $n$-field data structure of sort $\iota$ in the memory, pointed-to by $x$, and having $x_1, \ldots, x_n$ as values of its fields, or (iii) an *inductive* heap predicate $\mathsf{P}(x_1, \ldots, x_n)$ modeling a recursively defined data structure. These spatial atoms constitute a *spatial formula* $\Sigma$ via the separating conjunction operator $*$. On the other hand, $\pi$ denotes a *pure atom* comprising equality constraints among spatial expressions and linear arithmetic constraints among integer expressions. These pure atoms compose a *pure formula* $\Pi$ using standard first-order logic connectives and quantifiers. Moreover, $\Pi$ may contain an *unknown relation* $U$ (Definition 5.1). We will utilize the unknown relation in various phases of the lemma synthesis.

$$
\begin{array}{lcl}
e & ::= & c \mid x \mid -e \mid e_1 + e_2 \mid e_1 - e_2 \mid c \cdot e \\
a & ::= & \mathsf{nil} \mid x \\
\pi & ::= & \mathsf{true} \mid \mathsf{false} \mid a_1{=}a_2 \mid a_1{\neq}a_2 \mid e_1{=}e_2 \mid e_1{\neq}e_2 \mid e_1{>}e_2 \mid e_1{\geq}e_2 \mid e_1{<}e_2 \mid e_1{\leq}e_2 \\
\sigma & ::= & \mathsf{emp} \mid x \overset{\iota}{\mapsto} x_1, \ldots, x_n \mid \mathsf{P}(x_1, \ldots, x_n) \\
\Pi & ::= & \pi \mid U(x_1, \ldots, x_n) \mid \neg\Pi \mid \Pi_1 \wedge \Pi_2 \mid \Pi_1 \vee \Pi_2 \mid \Pi_1 \rightarrow \Pi_2 \mid \forall x.\Pi \mid \exists x.\Pi \\
\Sigma & ::= & \sigma \mid \Sigma_1 * \Sigma_2 \\
F & ::= & \Sigma \mid \Pi \mid \Sigma \wedge \Pi \mid \exists x.F
\end{array}
$$

FIGURE 5.4: Syntax of formulas in $\mathsf{SL}^{\mathsf{U}}{}_{\mathsf{ID}}$

**Definition 5.1 (Unknown relation)** *An unknown relation $U(u_1, \ldots, u_n)$ is an $n$-ary pure predicate in first-order logic, whose definition is undefined.*

**Semantics.** Figure 5.5 exhibits the semantics of formulas in $\mathsf{SL}^{\mathsf{U}}{}_{\mathsf{ID}}$. Given a set `Var` of variables, `Sort` of sorts, `Val` of values, `Loc` of memory addresses (`Loc` $\subset$ `Val`), a model of a formula consists of: a *stack* model $s$, which is a function $s$: `Var` $\rightarrow$ `Val`, and a *heap* model $h$, which is a partial function $h$: (`Loc` $\times$ `Sort`) $\rightharpoonup$ `Val`$^+$. We write $[\![\Pi]\!]_s$ to denote the valuation of a pure formula $\Pi$ under the stack model $s$. To define the semantics of the inductive heap predicates, we follow the standard *least fixed point semantics* by interpreting an inductive predicate symbol $\mathsf{P}$ as the least prefixed point $[\![\mathsf{P}]\!]$ of a monotone operator

constructed from its inductive definition. The construction is standard and can be found in many places, such as [Brotherston et al., 2011b].

| | | |
|---|---|---|
| $s, h \models \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$ and $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models \mathsf{emp}$ | iff | $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models x \overset{\iota}{\mapsto} x_1, ..., x_n$ | iff | $\mathrm{dom}(h) = \{s(x)\}$ and $h(s(x), \iota) = (s(x_1), ..., s(x_n))$ |
| $s, h \models \mathsf{P}(x_1, ..., x_n)$ | iff | $(h, [\![x_1]\!]_s, ..., [\![x_n]\!]_s) \in [\![\mathsf{P}]\!]$ |
| $s, h \models \Sigma_1 * \Sigma_2$ | iff | $\exists h_1, h_2 : h_1 \# h_2; h_1 \circ h_2 = h; s, h_1 \models \Sigma_1$ and $s, h_2 \models \Sigma_2$ |
| $s, h \models \Sigma \wedge \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$ and $s, h \models \Sigma$ |
| $s, h \models \exists x.\, F$ | iff | $\exists v \in \mathtt{Val} : [s \,\vert\, x{:}v], h \models F$ |

FIGURE 5.5: Semantics of formulas in $\mathsf{SL}^{\mathsf{U}}_{\mathsf{ID}}$

**Unknown Entailments and Unknown Assumptions**. Recall that we propose to synthesize lemmas by firstly discovering essential lemma templates and then refining them. The template refinement is conducted in 3 steps: (i) creating *unknown entailments*, which contain *unknown relations* representing the lemmas' desired pure constraints, (ii) proving the entailments by induction and collecting *unknown assumptions* about the relations, and (iii) solving the assumptions to discover the lemmas. We formally define the *unknown entailments* and the *unknown assumptions* as follows.

**Definition 5.2 (Unknown entailment)** *An entailment $F_1 \vdash F_2$ is called an* unknown entailment *if the antecedent $F_1$ or the consequent $F_2$ contains an unknown relation $U(\vec{x})$.*

**Definition 5.3 (Unknown assumption)** *A pure implication $\Pi_1 \rightarrow \Pi_2$ is called an* unknown assumption *of the unknown relation $U(\vec{x})$ if $U(\vec{x})$ occurs in at least one of the two pure formulas $\Pi_1$ and $\Pi_2$.*

### 5.3.2 Structural Induction Proof for Separation Logic Entailments

We prove entailments by applying Noetherian induction (Chapter 2) on the recursive structure of inductive heap predicates. The *substructural relation* between two inductive heap predicates is formally defined in the following Definition 5.4.

**Definition 5.4 (Substructural relation)** *A heap predicate* $\mathsf{P}_1(\vec{v})$ *is said to be a* substructure *of* $\mathsf{P}_2(\vec{u})$*, denoted by* $\mathsf{P}_1(\vec{v}) \prec \mathsf{P}_2(\vec{u})$*, if* $\mathsf{P}_1(\vec{v})$ *occurs in a formula obtained from* directly unfolding $\mathsf{P}_2(\vec{u})$ *or from* unfolding any substructure *of* $\mathsf{P}_2(\vec{u})$*. These conditions are formally stated as follows:*

*(1)* $\exists \vec{w}, \Sigma, \Pi, F(\vec{u}).\ (F(\vec{u}) \cong \exists \vec{w}.(\mathsf{P}_1(\vec{v}) * \Sigma \wedge \Pi) \quad \wedge \quad F(\vec{u}) \overset{\text{def}}{\Rightarrow} \mathsf{P}_2(\vec{u}))$

*(2)* $\exists \vec{w}, \Sigma, \Pi, F(\vec{t}), \mathsf{P}'(\vec{t}).\ (F(\vec{t}) \cong \exists \vec{w}.(\mathsf{P}_1(\vec{v}) * \Sigma \wedge \Pi) \quad \wedge \quad F(\vec{t}) \overset{\text{def}}{\Rightarrow} \mathsf{P}'(\vec{t}) \quad \wedge \quad \mathsf{P}'(\vec{t}) \prec \mathsf{P}_2(\vec{u}))$

In the above definition, $\mathsf{P}_1$ and $\mathsf{P}_2$ can be the same or different predicate symbols. The latter happens when they are mutually recursively defined, such as listE and listO in Example 2.2.

**Theorem 5.1 (Well-foundedness)** *Given an inductive heap predicate* $\mathsf{P}_1(\vec{u}_1)$*. If it is satisfiable, i.e.,* $\mathsf{P}_1(\vec{u}_1) \not\equiv \mathsf{false}$*, then under the least fixed point semantics of inductive heap predicates, there is no infinite chain like* $\ldots \prec \mathsf{P}_n(\vec{u}_n) \prec \ldots \prec \mathsf{P}_1(\vec{u}_1)$*.*

*Proof.* Suppose that there exists an infinite chain $\ldots \prec \mathsf{P}_n(\vec{u}_n) \prec \ldots \prec \mathsf{P}_1(\vec{u}_1)$. For all $i \geq 1$, we can always insert all predicates derived when unfolding $\mathsf{P}_i(\vec{u}_i)$ to obtain $\mathsf{P}_{i+1}(\vec{u}_{i+1})$ into the current chain. Therefore, w.l.o.g., we can assume that $\mathsf{P}_{i+1}(\vec{u}_{i+1})$ is obtained from directly unfolding $\mathsf{P}_i(\vec{u}_i)$, for all $i \geq 1$. Hence, $\ldots \prec \mathsf{P}_n(\vec{u}_n) \prec \ldots \prec \mathsf{P}_1(\vec{u}_1)$ is also the infinite unfolding chain of $\mathsf{P}_1(\vec{u}_1)$. In the *least fixed point semantics*, if a predicate is unfolded infinitely, it can be evaluated to false. Therefore, $\mathsf{P}_1(\vec{u}_1) \equiv \mathsf{false}$. This contradicts with the theorem's hypothesis that $\mathsf{P}_1(\vec{u}_1)$ is satisfiable. □

**Structural Induction.** We apply the *substructural relation* $\prec$ to propose a *structural induction principle* for the entailment proof. Consider an entailment $E$ whose antecedent contains an inductive predicate $\mathsf{P}(\vec{u})$, that is, $E \triangleq F_1 * \mathsf{P}(\vec{u}) \vdash F_2$, for some formulas

$F_1, F_2$. We write $E(\mathsf{P}(\vec{u}))$ to parameterize $E$ by $\mathsf{P}(\vec{u})$; and $E(\mathsf{P}(\vec{v}))$ is an entailment obtained from $E(\mathsf{P}(\vec{u}))$ by replacing $\mathsf{P}(\vec{u})$ by $\mathsf{P}(\vec{v})$ and respectively replacing variables in $\vec{u}$ by variables in $\vec{v}$. Moreover, we write $\models E(\mathsf{P}(\vec{u}))$ to denote that $E$ holds for $\mathsf{P}(\vec{u})$. The structural induction principle is formally stated as follows:

**Theorem 5.2 (Structural induction)** *The entailment $E(\mathsf{P}(\vec{u})) \triangleq F_1 * \mathsf{P}(\vec{u}) \vdash F_2$ is valid, if for all predicate $\mathsf{P}(\vec{u})$, the fact that $E$ holds for all sub-structure predicates $\mathsf{P}(\vec{v})$ of $\mathsf{P}(\vec{u})$ implies that $E$ also holds for $\mathsf{P}(\vec{u})$. Formally:*

$$\frac{\forall \mathsf{P}(\vec{u}).\ (\forall \mathsf{P}(\vec{v}) \prec \mathsf{P}(\vec{u}).\ \models E(\mathsf{P}(\vec{v}))) \ \rightarrow\ \models E(\mathsf{P}(\vec{u}))}{\forall \mathsf{P}(\vec{u}).\ \models E(\mathsf{P}(\vec{u}))}$$

*Proof.* We consider two scenarios w.r.t. $\mathsf{P}(\vec{u})$ in the entailment $E(\mathsf{P}(\vec{u})) \triangleq F_1 * \mathsf{P}(\vec{u}) \vdash F_2$ as follows. (1) If $\mathsf{P}(\vec{u}) \equiv \mathsf{false}$. Then $F_1 * \mathsf{P}(\vec{u}) \equiv \mathsf{false}$, therefore $E(\mathsf{P}(\vec{u}))$ is valid. (2) If $\mathsf{P}(\vec{u}) \not\equiv \mathsf{false}$. Then by Theorem 5.1, the substructural relation $\prec$, which applies to $\mathsf{P}(\vec{u})$, is *well-founded*. In this scenario, the above induction principle is an instance of the Noetherian induction [Bundy, 2001] where the *substructural relation* $\prec$ is chosen as the *well-founded relation*. Therefore, the correctness of this theorem is automatically implied by the soundness of the Noetherian induction principle. □

## 5.4 The Structural Induction Proof System

In this section, we present a structural induction proof system for separation logic entailments. Initially, this system aims to prove *normal entailments* using a set of inference rules (Section 5.4.1). These rules include the two lemma application rules which apply synthesized lemmas to assist in proving entailments (Figure 5.8). We use the same proof system to reason about *unknown entailments*, introduced in the lemma synthesis, using a set of synthesis rules (Section 5.4.2). The proof system also includes a proof search procedure which selectively applies the aforementioned rules to prove a goal entailment (Section 5.4.3). We will explain the details in the following sections.

### 5.4.1 Inference Rules for Standard Entailments

Each inference rule of our proof system contains zero or more premises, a conclusion, and possibly a side condition. The premises and the conclusion are in the form of $\mathcal{H},\ \mathcal{L},\ F_1 \vdash F_2$, where $\mathcal{H}$ and $\mathcal{L}$ is respectively a set of induction hypotheses and a set of valid lemmas, and $F_1 \vdash F_2$ is the (sub-)goal entailment. An inference rule can be interpreted as follows: *if all entailments in its premises are valid, and its side condition (if present) is satisfied, then its goal entailment is also valid.*

For brevity, we write $F$ to denote a symbolic-heap formula $\exists\vec{x}.(\Sigma \wedge \Pi)$, where $\vec{x}$ is a list of quantified variables (possibly empty). Furthermore, we define $F * \Sigma' \triangleq \exists\vec{x}.(\Sigma * \Sigma' \wedge \Pi)$ and $F \wedge \Pi' \triangleq \exists\vec{x}.(\Sigma \wedge \Pi \wedge \Pi')$, given that $\mathtt{FV}(\Sigma') \cap \vec{x} = \varnothing$ and $\mathtt{FV}(\Pi') \cap \vec{x} = \varnothing$. We also write $\vec{u} = \vec{v}$ to denote $(u_1 = v_1) \wedge \ldots \wedge (u_n = v_n)$, given that $\vec{u} \triangleq u_1, \ldots, u_n$ and $\vec{v} \triangleq v_1, \ldots, v_n$ are two variable lists of the same size. Finally, $\vec{u} \# \vec{v}$ indicates that the two lists $\vec{u}$ and $\vec{v}$ are disjoint, i.e., $\nexists w.(w \in \vec{u} \wedge w \in \vec{v})$.

The set of inference rules comprises logical rules (Figure 5.6) reasoning about the logical structure of the entailments, induction rules (Figure 5.7) proving the entailments by structural induction, and lemma application rules (Figure 5.8) applying supporting lemmas to assist in proving the entailments.

**Logical Rules.** The set of logical rules in Figure 5.6 consists of:

- *Axiom rules* $\vdash_{\Pi}, \bot^1_{\mathrm{L}}, \bot^2_{\mathrm{L}}$. These rules prove pure entailments by invoking an off-the-shelf prover such as Z3 [Moura et al., 2008] (as in the rule $\vdash_{\Pi}$), or prove entailments whose antecedents are inconsistent, i.e., they contain overlaid singleton heaps ($u \overset{\iota_1}{\mapsto} \vec{v}$ and $u \overset{\iota_2}{\mapsto} \vec{t}$ in the rule $\bot^1_{\mathrm{L}}$) or contradictions ($\Pi_1 \rightarrow \mathsf{false}$ in the rule $\bot^2_{\mathrm{L}}$).
- *Normalization rules* $\exists_{\mathrm{L}}, \exists_{\mathrm{R}}, =_{\mathrm{L}}, \mathsf{E}_{\mathrm{L}}, \mathsf{E}_{\mathrm{R}}$. These rules simplify the goal entailment by either eliminating existentially quantified variables ($\exists_{\mathrm{L}}, \exists_{\mathrm{R}}$), or removing equalities ($=_{\mathrm{L}}$) or empty heap predicates ($\mathsf{E}_{\mathrm{L}}, \mathsf{E}_{\mathrm{R}}$) from the entailment.
- *Case analysis rule* CA. This rule performs a case analysis on a pure condition $\Pi$ by deriving two sub-goal entailments whose antecedents respectively contain $\Pi$ and $\neg\Pi$.

The underlying principle of this rule is known as the *law of excluded middle* [Whitehead et al., 1912].

– *Unfolding rule* $\mathsf{P}_{\mathrm{R}}$. This rule derives a new entailment by unfolding a heap predicate in the goal entailment's consequent by its inductive definition.

– *Matching rules* $*\!\!\mapsto, *\mathsf{P}$. These rules remove *identical* singleton heap predicates ($*\!\!\mapsto$) or inductive heap predicates ($*\mathsf{P}$) from two sides of the goal entailments. Here, we ensure that these predicates are identical by adding equality constraints about their parameters into the derived entailments' consequents.

$$\vdash_{\Pi} \frac{}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2 \qquad \bot^1_{\mathrm{L}} \frac{}{\mathcal{H},\ \mathcal{L},\ F_1 * u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{t} \vdash F_2}$$

$$=_{\mathrm{L}} \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/v] \vdash F_2[u/v]}{\mathcal{H},\ \mathcal{L},\ F_1 \wedge u{=}v \vdash F_2} \qquad \bot^2_{\mathrm{L}} \frac{}{\mathcal{H},\ \mathcal{L},\ F_1 \wedge \Pi_1 \vdash F_2}\ \Pi_1 \rightarrow \mathsf{false}$$

$$\exists_{\mathrm{L}} \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2) \qquad \mathsf{CA} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \wedge \Pi \vdash F_2 \qquad \mathcal{H},\ \mathcal{L},\ F_1 \wedge \neg\Pi \vdash F_2}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash F_2}$$

$$\exists_{\mathrm{R}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.F_2[u/v]}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}, v.(F_2 \wedge u{=}v)} \qquad \mathsf{P}_{\mathrm{R}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{u}))}\ F_i^{\mathsf{P}}(\vec{u}) \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u})$$

$$\mathsf{E}_{\mathrm{L}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ F_1 * \mathsf{emp} \vdash F_2} \qquad *\!\!\mapsto \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})}{\mathcal{H},\ \mathcal{L},\ F_1 * u \overset{\iota}{\mapsto} \vec{v} \vdash \exists \vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})}\ u, \vec{v} \# \vec{x}$$

$$\mathsf{E}_{\mathrm{R}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.F_2}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{emp})} \qquad *\mathsf{P} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 \wedge \vec{u}{=}\vec{v})}{\mathcal{H},\ \mathcal{L},\ F_1 * \mathsf{P}(\vec{u}) \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{v}))}\ \vec{u} \# \vec{x}$$

FIGURE 5.6: Logical rules

**Induction Rules.** The structural induction principle is integrated into our proof system via the two induction rules ID and IH (Figure 5.7). These rules are explained in details as follows:

– *Rule* ID. This rule performs the structural induction proof on a chosen heap predicate $\mathsf{P}(\vec{u})$ in the antecedent of the goal entailment $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$. In particular,

the predicate $\mathsf{P}(\vec{u})$ is unfolded by its inductive definition $\mathsf{P}(\vec{u}) \stackrel{\text{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee ... \vee F_m^{\mathsf{P}}(\vec{u})$ to derive new sub-goal entailments as shown in this rule's premises. Moreover, the goal entailment is also recorded as an induction hypothesis in the set $\mathcal{H}'$ so that it can be utilized later to prove the sub-goal entailments.

$$\mathsf{ID}\ \frac{\mathcal{H}',\ \mathcal{L},\ \Sigma_1 * F_1^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2 \quad \ldots \quad \mathcal{H}',\ \mathcal{L},\ \Sigma_1 * F_m^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger : \mathsf{P}(\vec{u}) \stackrel{\text{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee ... \vee F_m^{\mathsf{P}}(\vec{u})$ and $\mathcal{H}' \triangleq \mathcal{H} \cup \{\, H \,\}$,

where $H$ is obtained by freshly renaming all variables of $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$

$$\mathsf{IH}\ \frac{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\ \mathcal{L},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\ \mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v}); \dagger$$

$\dagger : \exists \theta, \Sigma. (\Sigma_1 * \mathsf{P}(\vec{u}) \ \cong\ \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta)$

FIGURE 5.7: Induction rules

– *Rule* IH. This rule applies an appropriate recorded induction hypothesis $H \triangleq \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4$ to prove the goal entailment $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$. It first checks the well-founded condition $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v})$, i.e. $\mathsf{P}(\vec{u})$ is a substructure of $\mathsf{P}(\vec{v})$, which is required by the structural induction principle. In practice, this condition can be easily examined by labeling each inductive heap predicate $\mathsf{Q}(\vec{x})$ in a proof tree with a set of its ancestor predicates, which are consecutively unfolded to derive $\mathsf{Q}(\vec{x})$. By that mean, $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v})$ iff $\mathsf{P}(\vec{v})$ appears on the label of $\mathsf{P}(\vec{u})$. Afterwards, the induction hypothesis application is performed in two steps:

+ *Unification step:* unify the antecedents of both the goal entailment and the induction hypothesis by syntactically finding a substitution $\theta$ and a spatial formula $\Sigma$ such that $\Sigma_1 * \mathsf{P}(\vec{u}) \cong \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma$ and $\Pi_1 \rightarrow \Pi_3\theta$. If these conditions hold, then it is certain that the entailment $E \triangleq \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1$ is valid.

+ *Proving step:* if such $\theta$ and $\Sigma$ exist, then derive a new sub-goal entailment $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$. We will explain why this sub-goal entailment is derived. The induction hypothesis $\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4$ implies that $H\theta \triangleq \Sigma_3\theta * \mathsf{P}(\vec{v})\theta \wedge \Pi_3\theta \vdash F_4\theta$ is also valid,

by Theorem 2.2. From $E$ and $H\theta$, we then have a derivation chain: $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma \wedge \Pi_1$. Therefore, if the sub-goal entailment $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$ can be proved, then the goal entailment holds. Here, we propagate the pure condition $\Pi_1$ through the chain as we want the antecedent of the sub-goal entailment to be the strongest in order to prove $F_2$.

**Lemma Application Rules.** The two rules $\mathsf{LM}_\mathrm{L}, \mathsf{LM}_\mathrm{R}$ in Figure 5.8 derive a new sub-goal entailment by applying a lemma on the goal entailment's antecedent or consequent. In particular:

- The rule $\mathsf{LM}_\mathrm{L}$ applies a lemma on the goal entailment's antecedent. It is similar to the induction application rule IH, except that we do not need to check the well-founded condition of the structural induction proof since the applied lemma is already proved valid.
- The rule $\mathsf{LM}_\mathrm{R}$ applies a lemma on the goal entailment's consequent. It also needs to perform a unification step: finding a substitution $\theta$ and a formula $\Sigma$ so that the heap parts of the goal entailment's and the lemma's consequents are unified, i.e., $\Sigma_4\theta * \Sigma \cong \Sigma_2$. If this step succeeds, then $E_1 \triangleq \Sigma_4\theta * \Sigma \wedge \Pi_2 \vdash \Sigma_2 \wedge \Pi_2$ is valid. The lemma $F_3 \vdash \exists\vec{w}.(\Sigma_4 \wedge \Pi_4)$ implies that $F_3 \vdash \exists\vec{w}.\Sigma_4$ is valid. Hence, $F_3\theta \vdash \exists\vec{w}\theta.\Sigma_4\theta$ is valid by Theorem 2.2, following that $F_3\theta * \Sigma \wedge \Pi_2 \vdash \exists\vec{w}\theta.(\Sigma_4\theta * \Sigma \wedge \Pi_2)$ is also valid. By the rule's side condition $\vec{w}\theta \subseteq \vec{x}$, we have another valid entailment $E_2 \triangleq F_3\theta * \Sigma \wedge \Pi_2 \vdash \exists\vec{x}.(\Sigma_4\theta * \Sigma \wedge \Pi_2)$. Therefore, by proving the entailment $F_1 \vdash \exists\vec{x}.(F_3\theta * \Sigma \wedge \Pi_2)$ in this rule's premise and applying Theorem 2.1 twice sequentially on $E_2$ and then $E_1$, we can conclude that the goal entailment $F_1 \vdash \exists\vec{x}.(\Sigma_2 \wedge \Pi_2)$ is also valid.

$$\mathsf{LM}_\mathrm{L}\ \frac{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash F_4\},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash F_4\},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \exists\theta, \Sigma.\ (\Sigma_1 \cong \Sigma_3\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta)$$

$$\mathsf{LM}_\mathrm{R}\ \frac{\mathcal{H},\ \mathcal{L} \cup \{F_3 \vdash \exists\vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists\vec{x}.(F_3\theta * \Sigma \wedge \Pi_2)}{\mathcal{H},\ \mathcal{L} \cup \{F_3 \vdash \exists\vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists\vec{x}.(\Sigma_2 \wedge \Pi_2)}\ \exists\theta, \Sigma.(\Sigma_4\theta * \Sigma \cong \Sigma_2) \wedge (\vec{w}\theta \subseteq \vec{x})$$

FIGURE 5.8: Lemma application rules

### 5.4.2 Synthesis Rules for Unknown Entailments

$$\mathsf{U}^1_{\Pi}\ \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_2\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2} \qquad \mathsf{U}^1_{\Sigma}\ \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2}\ \Sigma_1 \neq \mathsf{emp}$$

$$\mathsf{U}^2_{\Pi}\ \frac{\mathcal{A} \triangleq \{\Pi_1 \rightarrow \exists \vec{w}.(\Pi_2 \wedge U(\vec{x}))\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \vdash \exists \vec{w}.(\Pi_2 \wedge U(\vec{x}))} \qquad \mathsf{U}^2_{\Sigma}\ \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \wedge U(\vec{x}) \vdash \exists \vec{w}.(\Sigma_2 \wedge \Pi_2)}\ \Sigma_2 \neq \mathsf{emp}$$

$$\mathsf{U}_{\mathrm{IH}}\ \frac{\mathcal{H}',\ \mathcal{L},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2 \qquad \mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow (\Pi_3 \wedge U(\vec{y}))\theta\}}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash F_4\},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \wedge U(\vec{x}) \vdash F_2}\ \dagger$$

$\dagger:\ \mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v});\ \exists \theta, \Sigma.\ (\Sigma_1 * \mathsf{P}(\vec{u}) \cong \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma)$
and $\mathcal{H}' = \mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash F_4\}$

FIGURE 5.9: Synthesis rules

Figure 5.9 presents synthesis rules, which deal with unknown entailments introduced in the lemma synthesis. These rules share the similar structure to the inference rules, except that each of them also contains a special premise indicating an assumption set $\mathcal{A}$ of the unknown relations. We will describe in details the synthesis rules as follows:

– *Axiom synthesis rules* $\mathsf{U}^1_{\Pi}, \mathsf{U}^2_{\Pi}, \mathsf{U}^1_{\Sigma}, \mathsf{U}^2_{\Sigma}$. These rules conclude the validity of their unknown goal entailments under the assumption sets $\mathcal{A}$ in the rules' premises. The rule $\mathsf{U}^1_{\Pi}$ and $\mathsf{U}^2_{\Pi}$ handle pure entailments with unknown relations either in the antecedents or the consequents. The rule $\mathsf{U}^1_{\Sigma}$ and $\mathsf{U}^2_{\Sigma}$ deal with unknown entailments whose antecedents contain non-empty spatial formulas while the consequents are pure formulas or vice versa. In both cases, the antecedents must be inconsistent to make the goal entailments valid. Here, we only create assumptions on the pure parts, i.e., $\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}$, since inconsistency in their heap parts, if any, can be detected earlier by the rule $\perp^1_{\mathrm{L}}$. Also, we do not consider the case that the unknown relations only appear in the consequents since these relations cannot make the antecedents inconsistent.

– *Induction hypothesis synthesis rule* $\mathsf{U}_{\mathrm{IH}}$. This rule applies an induction hypothesis to prove a derived unknown entailment. The induction hypothesis also contains unknown relations since it is recorded earlier from an unknown goal entailment. The rule $\mathsf{U}_{\mathrm{IH}}$ is similar to the normal induction hypothesis application rule IH, except that it does not contain a side condition like $\Pi_1 \wedge U(\vec{x}) \rightarrow (\Pi_3 \wedge U(\vec{y}))\theta$, due to the appearance of the unknown relation $U$. Instead, this condition will be registered in the unknown assumption set $\mathcal{A}$ in the premises of $\mathsf{U}_{\mathrm{IH}}$.

### 5.4.3 The Proof Search Procedure

Figure 5.10 presents the core proof search procedure Prove of our proof system. Its first three inputs include an induction hypothesis set $\mathcal{H}$, a valid lemma set $\mathcal{L}$, and a goal entailment $F_1 \vdash F_2$, which correlate to an inference step of the proof system. We also use an additional input $mode$ to control when new lemmas can be synthesized (if $mode =$ SynLM) or strictly not (if $mode =$ NoSyn). The procedure Prove returns the validity of the goal entailment, a set of new lemmas synthesized during the proof, and a set of assumptions to make the entailment valid.

There are two main contexts where this procedure is initially invoked:

– In the *entailment proving* phase, Prove is invoked to prove a *normal* entailment $F_1 \vdash F_2$, which does not contain any unknown relation, with the initial setting $\mathcal{H} = \varnothing$, $\mathcal{L} = \varnothing$, and $mode =$ SynLM.

– In the *lemma synthesis* phase, Prove is invoked either to prove an *unknown* entailment $F_1 \vdash F_2$ related to a lemma template or to verify whether a discovered lemma is an inductive lemma. In the first case, Prove will establish sufficient assumptions about unknown relations appearing in the entailment so that the entailment can become valid. Moreover, Prove is invoked with $mode =$ NoSyn in both the two cases to avoid entering into nested lemma synthesis phases.

**Procedure** $\mathsf{Prove}(\mathcal{H}, \mathcal{L}, F_1 \vdash F_2, mode)$

**Input:** $F_1 \vdash F_2, \mathcal{H}, \mathcal{L}$ is respectively a goal entailment, an induction hypothesis set, a valid lemma set, and $mode$ controls whether new lemmas will be synthesized (SynLM) or strictly not (NoSyn)

**Output:** The goal entailment's validity ($\mathsf{Valid}\langle\xi\rangle$ or $\mathsf{Unkn}$, where $\xi$ is the witness valid proof tree), a set of new synthesized lemmas, and a set of unknown assumptions

1: $\mathcal{R} \leftarrow \{\, \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \mathcal{L}, F_1 \vdash F_2)) \mid \mathsf{R} \in \mathcal{R} \,\}$ *// Find applicable rules from $\mathcal{R}$*
2: $\mathcal{L}_{\text{syn}} \leftarrow \varnothing$ *// Initialize the new synthesized lemma set*
3: **if** $(mode = \mathsf{SynLM})$ **and** $\mathsf{NeedLemmas}(F_1 \vdash F_2, \mathcal{R})$ **then**
4: $\quad \mathcal{L}_{\text{syn}} \leftarrow \mathsf{SynthesizeLemma}(\mathcal{L},\ F_1 \vdash F_2)$ *// Synthesize new lemmas*
5: $\quad \mathcal{R} \leftarrow \mathcal{R} \cup \{\, \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \mathcal{L}_{\text{syn}}, F_1 \vdash F_2)) \mid \mathsf{R} \in \{\mathsf{LM_L}, \mathsf{LM_R}\} \,\}$
6: **for each** $\bar{\mathsf{R}}$ **in** $\mathcal{R}$ **do**
7: $\quad \mathcal{A} \leftarrow \varnothing$ *// Initialize the assumption set*
8: $\quad$ **if** $\mathsf{IsSynthesisRule}(\bar{\mathsf{R}})$ **then** $\mathcal{A} \leftarrow \mathsf{Assumptions}(\bar{\mathsf{R}})$ *// $\bar{\mathsf{R}} \in \{\, \mathsf{U}^1_\Pi, \mathsf{U}^2_\Pi, \mathsf{U}^1_\Sigma, \mathsf{U}^2_\Sigma, \mathsf{U}_{IH} \,\}$*
9: $\quad$ **if** $\mathsf{IsAxiomRule}(\bar{\mathsf{R}})$ **then** *// $\bar{\mathsf{R}} \in \{\, \vdash_\Pi, \bot^1_L, \bot^2_L, \mathsf{U}^1_\Pi, \mathsf{U}^2_\Pi, \mathsf{U}^1_\Sigma, \mathsf{U}^2_\Sigma \,\}$*
10: $\quad\quad \xi \leftarrow \mathsf{CreateWitnessProofTree}(F_1 \vdash F_2, \bar{\mathsf{R}}, \varnothing)$
11: $\quad\quad$ **return** $(\mathsf{Valid}\langle\xi\rangle, \mathcal{L}_{\text{syn}}, \mathcal{A})$
12: $\quad (\mathcal{H}_i, \mathcal{L}_i, F_{1i} \vdash F_{2i})_{i=1\ldots n} \leftarrow \mathsf{Premises}(\bar{\mathsf{R}})$
13: $\quad (r_i, \mathcal{L}^i_{\text{syn}}, \mathcal{A}_i)_{i=1\ldots n} \leftarrow \mathsf{Prove}(\mathcal{H}_i,\ \mathcal{L} \cup \mathcal{L}_{\text{syn}} \cup \mathcal{L}_i,\ F_{1i} \vdash F_{2i}, mode)_{i=1\ldots n}$
14: $\quad$ **if** $r_i = \mathsf{Valid}\langle\xi_i\rangle$ **for all** $i = 1 \ldots n$ **then**
15: $\quad\quad \xi \leftarrow \mathsf{CreateWitnessProofTree}(F_1 \vdash F_2, \bar{\mathsf{R}}, \{\, \xi_1, \ldots, \xi_n \,\})$
16: $\quad\quad$ **return** $(\mathsf{Valid}\langle\xi\rangle,\ \mathcal{L}_{\text{syn}} \cup \mathcal{L}^1_{\text{syn}} \cup \ldots \cup \mathcal{L}^n_{\text{syn}},\ \mathcal{A} \cup \mathcal{A}_1 \cup \ldots \cup \mathcal{A}_n)$
17: **return** $(\mathsf{Unkn}, \varnothing, \varnothing)$ *// All rules fail to prove $F_1 \vdash F_2$*

FIGURE 5.10: The main proof search procedure.

Given the goal entailment $F_1 \vdash F_2$, the procedure Prove first finds from all rules $\mathcal{R}$ (Figures 5.6, 5.7, 5.8, and 5.9) a set of potential inference and synthesis rules $\mathcal{R}$, which can be unified with $F_1 \vdash F_2$ (line 1). When the lemma synthesis mode is enabled ($mode = \mathsf{SynLM}$), it invokes a subroutine NeedLemmas to examine the selected rules $\mathcal{R}$ and the goal entailment $F_1 \vdash F_2$ to decide whether it really needs to synthesize new lemmas (line 3). Note that the input valid lemma set $\mathcal{L}$ is also exploited to make the lemma synthesis more effective (line 4). The newly synthesized lemmas, if any, will be utilized to discover new lemma application rules (line 5). Thereafter, Prove successively applies each rule $\bar{\mathsf{R}} \in \mathcal{R}$ to derive new sub-goal entailments, as in the premises of $\bar{\mathsf{R}}$, and recursively

searches for their proofs (lines 6 – 16). It returns the *valid* result ($\mathsf{Valid}\langle\xi\rangle$, where $\xi$ is the witness proof tree) if the selected rule $\bar{\mathsf{R}}$ does not introduce any new sub-goals (lines 9 – 11), or all derived sub-goals are successfully proved (lines 12 – 16). In essence, the proof tree $\xi$ is composed of a root (the goal entailment $F_1 \vdash F_2$), a label (the applied rule $\bar{\mathsf{R}}$), and sub-trees, if any, corresponding to the sub-goal entailments' proofs (lines 10, 15). Its form is intuitively similar to the proof trees depicted in Figures 5.1, 5.2, and 5.3. On the other hand, Prove announces the *unknown* result ($\mathsf{Unkn}$) when all selected rules in $\mathcal{R}$ fail to prove the goal entailment (line 17). Besides, Prove also returns a set of newly synthesized lemmas and a set of unknown assumptions. These lemmas are discovered when Prove is invoked in the *entailment proving* phase. The unknown assumptions are collected by the synthesis rules (line 8) when Prove is executed in the *lemma synthesis* phase.

Details of the lemma synthesis will be presented in Section 5.5. In the following, we will explain when Prove decides to synthesize new lemmas (line 3). The procedure NeedLemmas returns *true* when all the following conditions are satisfied:

- $F_1 \vdash F_2$ *is not* an unknown entailment, which implies that lemmas are possibly needed.
- $\mathcal{R}$ *does not contain* any axiom or normalization ($\vdash_{\Pi}, \bot^1_{\mathrm{L}}, \bot^2_{\mathrm{L}}, \exists_{\mathrm{L}}, \exists_{\mathrm{R}}, =_{\mathrm{L}}, \mathsf{E}_{\mathrm{L}}, \mathsf{E}_{\mathrm{R}}$), matching rules of identical heap predicates ($*\!\!\mapsto, *\mathsf{P}$), or unfolding rules that introduce identical predicates ($\mathsf{ID}, \mathsf{P}_{\mathrm{R}}$). This condition indicates that all rules in $\mathcal{R}$ cannot make any immediate proof progress.
- $\mathcal{R}$ *does not have* any induction hypothesis or lemma application rules ($\mathsf{IH}, \mathsf{LM}_{\mathrm{L}}, \mathsf{LM}_{\mathrm{R}}$), or any case analysis rule ($\mathsf{CA}$) that potentially leads to the application of an induction hypothesis or a lemma. This condition conveys that existing induction hypotheses and lemmas are inapplicable
- $F_1 \vdash F_2$ *is not* a good induction hypothesis candidate, which indicates that the induction hypothesis recorded from $F_1 \vdash F_2$ cannot be used to prove other derived entailments.

While the first three conditions can be checked syntactically on $F_1 \vdash F_2$ and $\mathcal{R}$, the last condition can be tested by a trial and error method. Specifically, $F_1 \vdash F_2$ will firstly be recorded as a temporary induction hypothesis candidate, and then each inductive heap

predicate in $F_1$ will be consecutively unfolded to search for an application of the recorded induction hypothesis via the rule IH. If no induction hypothesis application can be found, then $F_1 \vdash F_2$ is evidently not a good candidate.

### 5.4.4 Soundness of the Proof System

When the proof search procedure Prove is invoked to prove a normal goal entailment $E$, which does not contain any unknown relation, the initial setting $\mathcal{H} = \varnothing$, $\mathcal{L} = \varnothing$, $mode =$ SynLM indicates that no induction hypothesis or lemma is provided beforehand, and the proof system can synthesize new lemmas to assist in proving $E$. When synthesizing the new supporting lemmas, the proof system can be utilized to prove an *unknown entailment* related to a lemma template or to verify a discovered lemma, which is a *normal entailment* not containing any unknown relation. In addition, the proof system is also invoked to prove sub-goal entailments, which are *normal entailments* derived from $E$.

In the following, we present Propositions 5.1, 5.2, and 5.3, which respectively specify the soundness of our proof system in the three typical scenarios: (i) proving an unknown entailment with the lemma-synthesis-disabled mode, (ii) verifying a discovered lemma, i.e., proving a normal entailment with the lemma-synthesis-disabled mode, or (iii) proving a normal entailment with the lemma-synthesis-enabled mode. Finally, we describe the overall soundness of the proof system in Theorem 5.3 when Prove is invoked with the initial setting $\mathcal{H} = \varnothing$, $\mathcal{L} = \varnothing$, $mode =$ SynLM. Note that Propositions 5.1 and 5.2 are relevant to the lemma synthesis in Section 5.5. Proposition 5.3 directly relates to the overall soundness in Theorem 5.3.

**Proposition 5.1 (Proof of an unknown entailment)** *Given an unknown entailment $E$. If the procedure* Prove *returns* Valid$\langle_\rangle$ *and generates an assumption set $\mathcal{A}$ when proving $E$ in the lemma-synthesis-disabled mode* (NoSyn)*, using an empty induction hypothesis set* ($\mathcal{H}{=}\varnothing$) *and a valid lemma set $\mathcal{L}$, then $E$ is semantically valid, given that all assumptions in $\mathcal{A}$ are valid.*

**Proposition 5.2 (Proof of a normal entailment when the lemma synthesis is disabled)** *Given a normal entailment $E$ which does not contain any unknown relation. If the procedure*

Prove *returns* Valid$\langle_\rangle$ *when proving* $E$ *in the lemma-synthesis-disabled mode* (NoSyn)*, using an empty induction hypothesis set* ($\mathcal{H}{=}\varnothing$) *and a valid lemma set* $\mathcal{L}$ *as its inputs, then* $E$ *is semantically valid.*

**Proposition 5.3 (Proof of a normal entailment when the lemma synthesis is enabled)** *Given a normal entailment* $E$ *which does not contain any unknown relation. If the procedure* Prove *returns* Valid$\langle_\rangle$ *and synthesizes a set of lemmas* $\mathcal{L}_{\text{syn}}$ *when proving* $E$ *in the lemma-synthesis-enabled mode* (SynLM)*, using an empty induction hypothesis set* ($\mathcal{H}{=}\varnothing$) *and a valid lemma set* $\mathcal{L}$ *as its inputs, then the entailment* $E$ *and all lemmas in* $\mathcal{L}_{\text{syn}}$ *are semantically valid.*

*Proof (Propositions 5.1, 5.2, 5.3).* We first need to show that all inference rules and synthesis rules are sound, and the proof system in the NoSyn mode is also sound. Based on that, we can prove Propositions 5.1 and 5.2. The proof of Proposition 5.3 will be argued based on the lemma synthesis's soundness in Section 5.5. The detailed proofs will be presented in Appendix B. □

**Theorem 5.3 (The overall soundness of the proof system)** *Given a normal entailment* $E$ *which does not contain any unknown relation, if the procedure* Prove *returns* Valid$\langle_\rangle$ *when proving* $E$ *in the initial context that there is no induction hypothesis or lemma provided and the lemma synthesis is enabled* ($\mathcal{H}{=}\varnothing, \mathcal{L}{=}\varnothing, mode{=}$SynLM)*, then* $E$ *is semantically valid.*

*Proof.* Since $E$ does not contain any unknown relation, both the input induction hypothesis set and lemma set are empty ($\mathcal{H}{=}\varnothing, \mathcal{L}{=}\varnothing$), and Prove is invoked in the SynLM mode, it follows from Proposition 5.3 that if Prove returns Valid$\langle_\rangle$ when proving $E$, then $E$ is semantically valid. □

## 5.5 The Lemma Synthesis Framework

We are now ready to describe our lemma synthesis framework. It consists of a main synthesis procedure, presented in Subsection 5.5.1, and auxiliary subroutines, described in Subsections 5.5.2, 5.5.3, and 5.5.4.

### 5.5.1 The Lemma Synthesis Procedure

Figure 5.11 presents the main lemma synthesis procedure (SynthesizeLemma). Its inputs include a goal entailment $F_1 \vdash F_2$ which needs to be proved by new lemmas, and a set of previously synthesized lemmas $\mathcal{L}$ which will be exploited to make the current synthesis more effective. The procedure SynthesizeLemma first identifies a set of desired lemma templates based on the entailment's heap structure, via the invocation of the procedure CreateTemplate (line 18). These lemma templates are of the form $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$, in which $\Sigma_1$ and $\Sigma_2$ are spatial formulas constructed from heap predicate symbols appearing in $F_1$ and $F_2$, respectively, and $\vec{v}$ are all free variables in $\Sigma_2$. By this construction, each synthesized lemma will share a similar heap structure with the goal entailment, hence they can be unified by the lemma application rules $\mathsf{LM}_{\mathrm{L}}$ and $\mathsf{LM}_{\mathrm{R}}$.

**Procedure** SynthesizeLemma($\mathcal{L}, F_1 \vdash F_2$)

**Input:** $F_1 \vdash F_2$ is the goal entailment, $\mathcal{L}$ is a valid lemma set

**Output:** A set of new synthesized lemmas

18: **for each** $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$ **in** CreateTemplate($\mathcal{L}, F_1 \vdash F_2$) **do**
19: $\quad\mathcal{L}_1 \leftarrow$ RefineAnte($\mathcal{L},\ \Sigma_1 \wedge \mathsf{true} \vdash \Sigma_2$)
20: $\quad\mathcal{L}_2 \leftarrow$ RefineConseq($\mathcal{L},\ \Sigma_1 \vdash \exists\vec{v}.\Sigma_2$)
21: $\quad$**if** $(\mathcal{L}_1 \cup \mathcal{L}_2) \neq \varnothing$ **then return** $(\mathcal{L}_1 \cup \mathcal{L}_2)$
22: **return** $\varnothing$

FIGURE 5.11: The main lemma synthesis procedure

We will formally define the lemma templates in Subsection 5.5.2. In our implementation, CreateTemplate returns a list of possible lemma templates, which are sorted in the ascending order of their simplicity, i.e., templates containing less spatial atoms are on the top of the list. Moreover, any template sharing the same heap structure with a previously synthesized lemma in $\mathcal{L}$ will not be considered.

The framework then successively refines each potential lemma template by continuously discovering and adding in *pure constraints* of its variables until valid *inductive lemmas* are found. For a given lemma template, the refinement is performed by a 3-step recipe:

1. Establishing an unknown relation representing a desired constraint inside the template and creating an unknown entailment.

2. Proving the unknown entailment by structural induction and collecting assumptions about the unknown relation.

3. Solving the assumptions to find out the actual definition of the unknown relation, thus discovering the desired inductive lemma.

There are two possible places to refine a lemma template: on its antecedent (line 19) and its consequent (line 20). We aim to synthesize a lemma which has an *as weak as possible* antecedent or an *as strong as possible* consequent. We will elaborate the details in Subsections 5.5.3 and 5.5.4. Our framework *immediately* returns a non-empty set of synthesized lemmas (line 21) once it successfully refines a template. Otherwise, it returns an empty set ($\varnothing$) indicating that no lemma can be synthesized (line 22).

### 5.5.2 Discovering Lemma Templates

The lemma templates for a given goal entailment can be discovered from the entailment's heap structure. In the followings, we present the formal definition of the lemma templates and also illustrate by examples how to create them.

**Definition 5.5 (Lemma template)** *A lemma template for a goal entailment $F_1 \vdash F_2$ is an entailment of the form $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$, where:*

1. *$\Sigma_1$ and $\Sigma_2$ are spatial formulas containing at least one inductive heap predicate.*
2. *Heap predicate symbols in $\Sigma_1$ and $\Sigma_2$ are sub-multisets of those in $F_1$ and $F_2$.*
3. *Variables in $\Sigma_1$ and $\Sigma_2$ are separately named (no variables appear twice) and $\vec{v} \equiv \mathtt{FV}(\Sigma_2)$.*

The condition (1) limits the templates for only inductive lemmas. The condition (2) guarantees that the synthesized lemmas are *unifiable* with the goal entailment via applications of the rules $\mathsf{LM}_{\mathrm{L}}$ and $\mathsf{LM}_{\mathrm{R}}$. Moreover, the condition (3) ensures that each template is as general as possible so that desirable pure constraints can be subsequently discovered in

the next phases.

For instance, given the motivating entailment $E_1$ in Section 5.2:

$$E_1 \triangleq \mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$$

The following lemma templates can be created, where the template $T_1$ is used to synthesize the lemma $L_1 \triangleq \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$.

$$T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$$
$$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9{\mapsto}x_{10}, x_{11})$$
$$\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9{\mapsto}x_{10}, x_{11})$$
$$\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) * \mathsf{dll}(x_5, x_6, x_7, x_8, n_2) \vdash \exists x_9, x_{10}, x_{11}, x_{12}, n_3.\mathsf{dll}(x_9, x_{10}, x_{11}, x_{12}, n_3)$$

Similarly, there can be several possible lemma templates relating to the entailments $E_2$ and $E_3$ in Figure 5.1. Among them, the templates $T_2$ and $T_3$ below are used to synthesize the lemmas $L_2$ and $L_3$:

$$T_2 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) * \mathsf{dll}(x_5, x_6, x_7, x_8, n_2) \vdash \exists x_9, x_{10}, x_{11}, x_{12}, n_3.\mathsf{dll}(x_9, x_{10}, x_{11}, x_{12}, n_3)$$
$$T_3 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9{\mapsto}x_{10}, x_{11})$$

### 5.5.3 Refining Lemma Templates' Antecedents

Figure 5.12 presents the antecedent refinement procedure (RefineAnte), which aims to strengthen the antecedent of a lemma template $\Sigma_1 \wedge \Pi_1 \vdash \Sigma_2$ with pure constraints of all its variables. Note that when RefineAnte is invoked in the first time by SynthesizeLemma (Figure 5.11, line 19), $\Pi_1$ is set to true and the existential quantification $\exists\vec{v}$ in the original template $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$ is removed since the antecedent will be strengthened with constraints of all variables in the template.

**Procedure** RefineAnte($\mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash \Sigma_2$)

**Input:** $\Sigma_1 \wedge \Pi_1 \vdash \Sigma_2$ is a lemma template, $\mathcal{L}$ is a valid lemma set

**Output:** a set of at most one synthesized lemma

```
23: u⃗ ← FreeVars(Σ1, Σ2, Π1)
24: U(u⃗) ← CreateUnknownRelation(u⃗)
25: r, _, A ← Prove(∅, L, Σ1 ∧ Π1 ∧ U(u⃗) ⊢ Σ2, NoSyn)
26: if r = Valid⟨_⟩ then
27:     for each A′ in SuperSet(A) do
28:         Ū(u⃗) ← Solve(A′, U(u⃗))
29:         if Σ1 ∧ Π1 ∧ Ū(u⃗) ≢ false then
30:             L ← (Σ1 ∧ Π1 ∧ Ū(u⃗) ⊢ Σ2)
31:             r′, _, _ ← Prove(∅, L, L, NoSyn)          // Verify ...
32:             if r′ = Valid⟨ξ⟩ and IH ∈ ξ then          // and return ...
33:                 return { L }                          // the first inductive lemma,
34:             else return RefineAnte(L, L)              // or continue refining.
35: return ∅                                              // No lemmas are found.
```

FIGURE 5.12: Refining a lemma template's antecedent

Initially, RefineAnte creates an unknown entailment $\Sigma_1 \wedge \Pi_1 \wedge U(\vec{u}) \vdash \Sigma_2$, where $U(\vec{u})$ is an unknown relation over all variables $\vec{u}$ in $\Sigma_1, \Sigma_2, \Pi_1$ (lines 23, 24). Then, it proves the entailment by induction and collects a set $\mathcal{A}$ of unknown assumptions, if any, about $U(\vec{u})$ (line 25). In this proof derivation (also in other invocations of Prove in the lemma synthesis), we prevent the proof system from synthesizing new lemmas (by passing NoSyn) to avoid initiating a new lemma synthesis phase inside the current synthesis, i.e., to prohibit the nested lemma synthesis. In addition, the assumption set $\mathcal{A}$ will be examined, via the procedure Solve, to discover a solution of $U(\vec{u})$ (lines 27, 28). We implement in Solve a constraint solving technique using Farkas' lemma [Schrijver, 1986; Colón et al., 2003]. This technique assigns $U(\vec{u})$ to a predefined linear arithmetic formula with unknown coefficients and applies Farkas' lemma to transform $\mathcal{A}$ into a set of constraints involving only the unknown coefficients. Then, it utilizes an off-the-shelf prover such as Z3 [Moura et al., 2008] to find a concrete model of the coefficients, thus obtains the actual definition $\overline{U}(\vec{u})$ of $U(\vec{u})$. We will describe this technique in Subsection 5.5.6.

Furthermore, we aim to find a *non-spurious* solution $\overline{U}(\vec{u})$ which does not refute the antecedent, i.e., $\Sigma_1 \wedge \Pi_1 \wedge \overline{U}(\vec{u}) \not\equiv \mathsf{false}$, to avoid creating a *useless* lemma: $\mathsf{false} \vdash \Sigma_2$ (line 29). To examine this refutation, we follow the literature [Brotherston et al., 2014a; Le et al., 2016] to implement an unsatisfiability checking algorithm, which over-approximates a symbolic-heap formula to a pure formula and invokes the off-the-shelf prover to check the pure formula's unsatisfiability, thus concludes about the unsatisfiability of the original formula.

In general, discovering a *non-spurious* solution $\overline{U}(\vec{u})$ is challenging, because:

- The assumption set $\mathcal{A}$ can be complicated since the parameters $\vec{u}$ of the unknown relation $U(\vec{u})$ are all variables in the lemma template. This complexity can easily overwhelm the underlying prover when finding the model of the corresponding unknown coefficients.
- The discovered proof tree of the unknown entailment might not be similar to the actual proof tree of the desired lemma, due to the occurrence of the unknown relation. Therefore, the set $\mathcal{A}$ might contain *noise* assumptions of $U(\vec{u})$, which results in a spurious solution. Nonetheless, a part of the expected solution can still be discovered from a subset of $\mathcal{A}$, which corresponds to the common part of the discovered and the desired proof trees.

The above challenges inspire us to design an *exhaustive approach* to solve $\mathcal{A}$ (line 27). In particular, RefineAnte first solves the entire set $\mathcal{A}$ (the first element in $\mathsf{SuperSet}(\mathcal{A})$) to find a *complete* solution, which satisfies *all* assumptions in $\mathcal{A}$. If such solution is *not* available, RefineAnte iteratively examines each subset of $\mathcal{A}$ (among the remaining elements in $\mathsf{SuperSet}(\mathcal{A})$) to discover a *partial* solution, which satisfies *some* assumptions in $\mathcal{A}$.

The discovered solution (complete or partial) will be verified whether it can form a valid inductive lemma. In particular, the proof system will be invoked to prove the candidate lemma $L$ (line 31). The verification is successful when $L$ is proved valid, and its witness proof tree $\xi$ contains an induction hypothesis application (labeled by IH) (line 32). The latter condition ensures that the returned lemma is actually an inductive lemma.

We also follow an *incremental approach* to refine the lemma template's antecedent. That is, the antecedent will be strengthened with the discovered solution $\overline{U}(\vec{u})$ to derive a new template. The new template will be refined again until the first valid lemma is discovered (lines 32 – 34). Note that this refinement stops at the first solution to ensure that the discovered antecedent is as weak as possible. Finally, RefineAnte returns $\varnothing$ if no valid inductive lemma can be discovered (line 35).

For example, given the template:

$$T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2),$$

RefineAnte creates an unknown relation $U(x_1, ..., x_8, n_1, n_2)$ and introduces the entailment $E_{u_1}$:

$$E_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge U(x_1, ..., x_8, n_1, n_2) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathcal{A}_1 \triangleq \{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1\}}{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,\mathsf{U}^1_{\Pi}}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_5{\mapsto}x_6, x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,{*}{\mapsto}}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)}\,\mathsf{P_R} \quad \dfrac{\dfrac{\dfrac{\mathcal{A}_2 \triangleq \{U(\vec{x},\vec{n}) \rightarrow \mathsf{false}\}}{x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash a_5{=}x_5 \wedge a_6{=}x_6 \wedge a_7{=}x_7 \wedge a_8{=}x_8 \wedge b_2{=}n_2}\,\mathsf{U}^1_{\Sigma}}{\mathsf{dll}(a_5, a_6, a_7, a_8, b_2) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)}\,{*}\mathsf{P} \quad \mathcal{A}_3 \triangleq \{U(\vec{x},\vec{n}) \rightarrow U(\vec{a},\vec{b})\theta\}}{\mathsf{dllrev}(x_1, x_2, u, x_3, n_1{-}1) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)}\,\mathsf{U_{IH}}}{E_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)}\,\mathsf{ID}$$

where $U(\vec{x},\vec{n}) \equiv U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$;

$U(\vec{a},\vec{b}) \equiv U(a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, b_1, b_2)$;

ID records $H \triangleq \mathsf{dllrev}(a_1, a_2, a_3, a_4, b_1) \wedge U(\vec{a},\vec{b}) \vdash \mathsf{dll}(a_5, a_6, a_7, a_8, b_2)$;

$\mathsf{U_{IH}}$ applies $H$ with $\theta = [x_1/a_1, x_2/a_2, u/a_3, x_3/a_4, n_1{-}1/b_1]$.

FIGURE 5.13: A possible proof tree of the unknown entailment $E_{u_1}$

Figure 5.13 presents a possible proof tree of the unknown entailment $E_{u_1}$. From this proof, we can obtain a set $\mathcal{A} = \mathcal{A}_1 \cup \mathcal{A}_2 \cup \mathcal{A}_3$ of three unknown assumptions about the relation $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$:

(1) $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
$x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$

(2) $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow \mathsf{false}$

(3) $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

Our framework first attempts to solve the full assumption set $\mathcal{A}$. Unfortunately, there is only a *spurious solution* $\overline{U}(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv \mathsf{false}$, since the assumption (2) is too strong. It then tries to find another solution by *partially solving* the set $\mathcal{A}$. In this case, it can discover the following partial solution $\overline{U}$ when solving a subset containing only the assumption (1):

$$\overline{U}(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$$

Since the above is only a partial solution, the framework needs to verify whether it can construct a *valid inductive lemma*. Indeed, $\overline{U}$ can be used to derive the entailment $\overline{E}_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2 \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$, which can be simplified to become:

$$\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \mathsf{dll}(x_1, x_2, x_3, x_4, n_1)$$

The above entailment can be equivalently transformed into the motivating lemma $L_1$ by a renaming on its variables.

### 5.5.4 Refining Lemma Templates' Consequents

The consequent refinement is presented in the procedure RefineConseq (Figure 5.14). Unlike the antecedent refinement, this refinement is not straightforward by simply adding pure constraints into the template's consequent, since the existing template's antecedent might not be strong enough to prove any formulas derived from the consequent. For example, all entailments derived from adding pure constraints to only the consequent of the lemma template $T_3$ are invalid, because when $n_1{=}1$, the list $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1)$ has the length of 1, thus cannot be split into a singleton heap $x_9{\mapsto}x_{10}, x_{11}$ and a list $\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$, whose length is at least 1, by the definition of dll.

$$T_3 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9{\mapsto}x_{10}, x_{11})$$

**Procedure** RefineConseq($\mathcal{L},\ \Sigma_1 \vdash \exists\vec{v}.\Sigma_2$)

**Input:** $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$ is a lemma template, $\mathcal{L}$ is a set of valid lemmas

**Output:** a set of at most one synthesized lemma

36: $\mathcal{T} \leftarrow$ Preprocess($\mathcal{L},\ \Sigma_1 \vdash \exists\vec{v}.\Sigma_2$)
37: **if** $\mathcal{T} = \{\ \Sigma_1 \wedge \Pi_1 \vdash \exists\vec{v}.\Sigma_2\ \}$ **then**
38: **return** FineTuneConseq($\mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash \exists\vec{v}.(\Sigma_2 \wedge \mathsf{true})$)
39: **return** $\varnothing$

FIGURE 5.14: Refining a lemma template's consequent

To overcome this problem, we decompose the consequent refinement into two phases, *preprocessing* and *fine-tuning*. For an input lemma template $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$, in the first phase, the framework infers a pure constraint $\Pi_1$ so that the entailment $\Sigma_1 \wedge \Pi_1 \vdash \exists\vec{v}.\Sigma_2$ is valid (lines 36, 37). In the second phase, it incrementally strengthens the consequent of the derived entailment until discovers a valid inductive lemma (line 38). Note that, we retain the existential quantification in this consequent, and any constraints added into the consequent will be bound by this quantification. These two phases will be elaborated in details as follows.

#### 5.5.4.1 Preprocessing Lemma Templates

Figure 5.15 presents our antecedent preprocessing procedure (Preprocess). Similar to the antecedent refinement in Section 5.5.3, this procedure strengthens the antecedent of the template $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$ with a *non-spurious* condition $\Pi_1$ to make the lemma template valid. We also prevent the framework from entering nested lemma synthesis phases by invoking Prove in the NoSyn mode (line 43).

However, this preprocessing step differs from the antecedent refinement (Section 5.5.3) when it creates an unknown entailment by introducing the unknown relation $U(\vec{u})$ on only the antecedent's free variables $\vec{u}$ (lines 40, 41). We also equip the consequent $\exists\vec{v}.\Sigma_2$ of the unknown entailment with a conjunction $\Pi_{inv}$ of its inductive heap predicates' pure

invariants (lines 42, 43). These invariants are pure formulas representing Boolean constraints of the predicates' variables. We will briefly describe the invariant construction in Section 5.5.5. This construction is also well-studied in separation logic literature [Chin et al., 2012; Brotherston et al., 2014a; Le et al., 2016].

**Procedure** Preprocess($\mathcal{L}$, $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$)

**Input:** $\Sigma_1 \vdash \exists\vec{v}.\Sigma_2$ is a lemma template, $\mathcal{L}$ is a valid lemma set

**Output:** a set of at most one refined template

40: $\vec{u} \leftarrow \mathsf{FreeVars}(\Sigma_1)$
41: $U(\vec{u}) \leftarrow \mathsf{CreateUnknownRelation}(\vec{u})$
42: $\Pi_{inv} \leftarrow \bigwedge_{\mathsf{P}(\vec{x}) \in \Sigma_2} \mathsf{Invariant}(\mathsf{P}(\vec{x}))$
43: $r, _, \mathcal{A} \leftarrow \mathsf{Prove}(\varnothing,\ \mathcal{L},\ \Sigma_1 \wedge U(\vec{u}) \vdash \exists\vec{v}.(\Sigma_2 \wedge \Pi_{inv}),\ \mathsf{NoSyn})$
44: **if** $r = \mathsf{Valid}\langle_\rangle$ **then**
45: $\quad \overline{U}(\vec{u}) \leftarrow \mathsf{Solve}(\mathcal{A},\ U(\vec{u}))$
46: $\quad$ **if** $\Sigma_1 \wedge \overline{U}(\vec{u}) \not\equiv \mathsf{false}$ **then return** $\{\ \Sigma_1 \wedge \overline{U}(\vec{u}) \vdash \exists\vec{v}.\Sigma_2\ \}$
47: **return** $\varnothing$ // *No refined templates are found.*

FIGURE 5.15: Preprocess a lemma template's antecedent

In theory, equipping additional pure invariants of inductive heap predicates *does not* weaken or strengthen the lemma template's consequent, i.e., $\Sigma_2 \equiv \Sigma_2 \wedge \Pi_{inv}$. In our approach, Preprocess solves the entire assumption constraint set $\mathcal{A}$ at once (line 45), and not incrementally as in the antecedent refinement (Section 5.5.3). Therefore, the additional pure invariant $\Pi_{inv}$ is useful for Preprocess to solve the entire set $\mathcal{A}$ more precisely and effectively.

For example, given the lemma template $T_3$, the procedure Preprocess sets up an unknown relation $U(x_1, x_2, x_3, x_4, n_1)$ in the template's antecedent, and introduces the invariant $n_2 \geq 1$ of $\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$ in the template's consequent to create the following unknown entailment $E_{u_2}$.

$$E_{u_2} \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge U(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge n_2 \geq 1)$$

$$
\dfrac{
\dfrac{
\dfrac{\mathcal{A}_1 \triangleq \{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \rightarrow \mathsf{false}\}}{
\begin{array}{l}
x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash \\
\quad \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) \wedge \\
\qquad x_1{=}x_9 \wedge x_2{=}x_{10} \wedge x_4{=}x_{11} \wedge n_2{\geq}1)
\end{array}
}\,\mathsf{U}^2_{\Sigma}
}{
\begin{array}{l}
x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash \\
\quad \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * \\
\qquad x_9{\mapsto}x_{10}, x_{11} \wedge n_2{\geq}1)
\end{array}
}\,{*}{\mapsto}
\quad
\dfrac{
\dfrac{
\begin{array}{l}
\mathcal{A}_2 \triangleq \{\, U(\vec{x},\vec{n}) \rightarrow \exists x_{5,\ldots,11}, n_2.(x_1{=}x_9 \wedge \\
\qquad x_2{=}x_{10} \wedge u{=}x_{11} \wedge u{=}x_5 \wedge x_1{=}x_6 \wedge \\
\qquad x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{-}1{=}n_2 \wedge n_2{\geq}1)\}
\end{array}
}{
\begin{array}{l}
U(\vec{x},\vec{n}) \vdash \exists x_{5,\ldots,11}, n_2.(x_1{=}x_9 \wedge \\
\qquad x_2{=}x_{10} \wedge u{=}x_{11} \wedge u{=}x_5 \wedge x_1{=}x_6 \wedge \\
\qquad x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{-}1{=}n_2 \wedge n_2{\geq}1)
\end{array}
}\,\mathsf{U}^1_{\Pi}
}{
\begin{array}{l}
\mathsf{dll}(u,x_1,x_3,x_4,n_1{-}1) \wedge U(\vec{x},\vec{n}) \vdash \\
\quad \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) \wedge \\
\qquad x_1{=}x_9 \wedge x_2{=}x_{10} \wedge u{=}x_{11} \wedge n_2{\geq}1)
\end{array}
}\,{*}\mathsf{P}
}{
\begin{array}{l}
x_1{\mapsto}x_2, u * \mathsf{dll}(u,x_1,x_3,x_4,n_1{-}1) \wedge U(\vec{x},\vec{n}) \vdash \\
\quad \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * \\
\qquad x_9{\mapsto}x_{10}, x_{11} \wedge n_2{\geq}1)
\end{array}
}\,{*}{\mapsto}
}{
E_{u_2} \triangleq \mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \wedge U(\vec{x},\vec{n}) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9{\mapsto}x_{10}, x_{11} \wedge n_2{\geq}1)
}\,\mathsf{ID}
$$

(where $U(\vec{x},\vec{n}) \triangleq U(x_1,x_2,x_3,x_4,n_1)$)

FIGURE 5.16: A possible proof tree of the unknown entailment $E_{u_2}$.

This entailment will be proved by induction to collect constraints about the unknown relation $U$. We present its detailed proof in Figure 5.16. Observe that the assumption constraint $U(x_1,x_2,x_3,x_4,n_1) \rightarrow \exists x_5,\ldots,x_{11},n_2.(x_1{=}x_9 \wedge x_2{=}x_{10} \wedge u{=}x_{11} \wedge u{=}x_5 \wedge x_1{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{-}1{=}n_2 \wedge n_2{\geq}1)$ in $\mathcal{A}_2$ can be simplified by eliminating existentially quantified variables to become $U(x_1,x_2,x_3,x_4,n_1) \rightarrow n_1{\geq}2$. Therefore, we can obtain a set $\mathcal{A} = \mathcal{A}_1 \cup \mathcal{A}_2$ containing the following unknown assumptions:

(1) $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1,x_2,x_3,x_4,n_1) \rightarrow \mathsf{false}$
(2) $U(x_1,x_2,x_3,x_4,n_1) \rightarrow n_1{\geq}2$

The procedure Preprocess can solve the *full assumption constraint set* $\mathcal{A}$ to discover a potential solution $\overline{U}(x_1,x_2,x_3,x_4,n_1) \equiv n_1{\geq}2$, which allows $T_3$ to be refined to a new lemma template $T_3'$:

$$T_3' \triangleq \mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \wedge n_1{\geq}2 \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9{\mapsto}x_{10}, x_{11})$$

#### 5.5.4.2 Fine-Tuning Lemma Templates' Consequents

This step aims to refine further the consequent of the lemma template discovered in the preprocessing phase. The refinement is performed by the procedure FineTuneConseq (Figure 5.17), whose input is a lemma template of the form $\Sigma_1 \wedge \Pi_1 \vdash \exists \vec{v}.(\Sigma_2 \wedge \Pi_2)$. When FineTuneConseq is invoked for the first time $\Pi_2$ is set to true (Figure 5.14, line 38).

Initially, FineTuneConseq establishes an unknown relation $U(\vec{u})$ on all variables in the templates and creates an unknown entailment $\Sigma_1 \wedge \Pi_1 \vdash \exists \vec{v}.(\Sigma_2 \wedge \Pi_2 \wedge U(\vec{u}))$ (lines 48, 49). Then, it proves the unknown entailment by induction to collect a set $\mathcal{A}$ of unknown assumptions (line 50). Similarly to the antecedent refinement (Section 5.5.3), $\mathcal{A}$ will be *exhaustively solved*: the entire set and its subsets will be examined until a feasible solution is found to obtain a valid inductive lemma (lines 52 – 59).

**Procedure** FineTuneConseq($\mathcal{L}$, $\Sigma_1 \wedge \Pi_1 \vdash \exists \vec{v}.(\Sigma_2 \wedge \Pi_2)$)

**Input:** $\Sigma_1 \wedge \Pi_1 \vdash \exists \vec{v}.(\Sigma_2 \wedge \Pi_2)$ is a lemma template, $\mathcal{L}$ is a valid lemma set

**Output:** a set of at most one synthesized lemma

48: $\vec{u} \leftarrow$ FreeVars$(\Sigma_1, \Sigma_2, \Pi_1, \Pi_2)$
49: $U(\vec{u}) \leftarrow$ CreateUnknownRelation$(\vec{u})$
50: $r$, _, $\mathcal{A} \leftarrow$ Prove$(\varnothing,\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \vdash \exists \vec{v}.(\Sigma_2 \wedge \Pi_2 \wedge U(\vec{u}))$, NoSyn)
51: **if** $r =$ Valid$\langle$_$\rangle$ **then**
52: **for each** $\mathcal{A}'$ **in** SuperSet$(\mathcal{A})$ **do**
53: $\overline{U}(\vec{u}) \leftarrow$ Solve$(\mathcal{A}',\ U(\vec{u}))$
54: $L \leftarrow (\Sigma_1 \wedge \Pi_1 \vdash \exists \vec{v}.(\Sigma_2 \wedge \Pi_2 \wedge \overline{U}(\vec{u}))$
55: $r'$, _, _ $\leftarrow$ Prove$(\varnothing,\ \mathcal{L},\ L,$ NoSyn$)$ // *Verify ...*
56: **if** $r' =$ Valid$\langle\xi\rangle$ **and** IH $\in \xi$ **then** // *the inductive lemma, and ...*
57: $\mathcal{L}_{\text{syn}} \leftarrow$ FineTuneConseq$(\mathcal{L},\ L)$ // *strengthen its consequent.*
58: **if** $\mathcal{L}_{\text{syn}} = \varnothing$ **then return** $\{\, L \,\}$
59: **else return** $\mathcal{L}_{\text{syn}}$
60: **return** $\varnothing$ // *No lemmas are found.*

FIGURE 5.17: Fine-tuning the consequent of a lemma template

We also prevent the proof system from entering nested lemma synthesis phases, controlled by the argument NoSyn, when Prove is invoked to collect the unknown assumption

set $\mathcal{A}$ (line 50) or to verify the inductive lemma (line 55). Witness proof tree of a candidate lemma, if having, is examined for the occurrence of the induction hypothesis application rule (IH), to determine if this candidate is a valid inductive lemma (line 56).

However, unlike the antecedent refinement, FineTuneConseq keeps refining the template until its consequent cannot be refined further (lines 56, 59). This repetition ensures that the returned lemma's consequent, if discovered, is as strong as possible.

For example, given the preprocessed template $T_3'$, FineTuneConseq creates an unknown relation $U(x_1, \ldots, x_{11}, n_1, n_2)$, or $U(\vec{x}, \vec{n})$ for short, involving all variables in $T_3'$ and constructs the unknown entailment $E_{u_3}$. This entailment will be proved again to collect constraints on $U(\vec{x}, \vec{n})$.

$$E_{u_3} \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, \ldots, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}))$$

$$\dfrac{\dfrac{\mathcal{A}_1}{\vdots} \quad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\mathcal{A}_3 \triangleq \{n_1 \geq 2 \wedge r = x_3 \wedge n_1 - 2 = 1 \rightarrow \exists x_{5,\ldots,11}, n_2, v.(U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v \wedge v = x_7 \wedge r = x_8 \wedge r = x_9 \wedge u = x_{10} \wedge x_4 = x_{11} \wedge n_2 - 1 = 1)\}}{n_1 \geq 2 \wedge r = x_3 \wedge n_1 - 2 = 1 \vdash \exists x_{5,\ldots,11}, n_2, v.(U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v \wedge v = x_7 \wedge r = x_8 \wedge r = x_9 \wedge u = x_{10} \wedge x_4 = x_{11} \wedge n_2 - 1 = 1)}\,\mathsf{U}^1_{\Pi}}{r \mapsto u, x_4 \wedge n_1 \geq 2 \wedge r = x_3 \wedge n_1 - 2 = 1 \vdash \exists x_{5,\ldots,11}, n_2, v.(x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v \wedge v = x_7 \wedge r = x_8 \wedge n_2 - 1 = 1)}\,*\!\mapsto \quad \dfrac{\mathcal{A}_4}{\vdots}}{\mathsf{dll}(r, u, x_3, x_4, n_1 - 2) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2, v.(x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v \wedge v = x_7 \wedge r = x_8 \wedge n_2 - 1 = 1)}\,\mathsf{ID}}{u \mapsto x_1, r * \mathsf{dll}(r, u, x_3, x_4, n_1 - 2) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2, v.(v \mapsto x_5, x_8 * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v \wedge v = x_7 \wedge n_2 - 1 = 1)}\,*\!\mapsto}{\mathsf{dll}(u, x_1, x_3, x_4, n_1 - 1) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2, v.(v \mapsto x_5, x_8 * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v \wedge v = x_7 \wedge n_2 - 1 = 1)}\,\mathsf{ID}}{\mathsf{dll}(u, x_1, x_3, x_4, n_1 - 1) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2, v.(\mathsf{dll}(v, x_5, x_7, x_8, n_2 - 1) * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}) \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge u = v)}\,\mathsf{P_R}}{x_1 \mapsto x_2, u * \mathsf{dll}(u, x_1, x_3, x_4, n_1 - 1) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2, v.(x_5 \mapsto x_6, v * \mathsf{dll}(v, x_5, x_7, x_8, n_2 - 1) * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}))}\,*\!\mapsto}{x_1 \mapsto x_2, u * \mathsf{dll}(u, x_1, x_3, x_4, n_1 - 1) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}))}\,\mathsf{P_R}}{E_{u_3} \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge U(\vec{x}, \vec{n}))}\,\mathsf{ID}$$

(where $U(\vec{x}, \vec{n}) \triangleq U(x_1, ..., x_{11}, n_1, n_2)$)

FIGURE 5.18: A partial proof tree of the unknown entailment $E_{u_3}$.

We present a partial proof tree of $E_{u_3}$ in Figure 5.18, where FineTuneConseq is able to collect a set of unknown assumptions $\mathcal{A} = \mathcal{A}_1 \cup \mathcal{A}_2 \cup \mathcal{A}_3 \cup \mathcal{A}_4$. More specifically, $\mathcal{A}_3 \triangleq \{n_1{\geq}2 \wedge r{=}x_3 \wedge n_1{-}2{=}1 \vdash \exists x_{5,\ldots,11}, n_2, v.\,(U(\vec{x},\vec{n}) \wedge x_1{=}x_5 \wedge x_2{=}x_6 \wedge u{=}v \wedge v{=}x_7 \wedge r{=}x_8 \wedge r{=}x_9 \wedge u{=}x_{10} \wedge x_4{=}x_{11} \wedge n_2{-}1{=}1)\}$ is an assumption subset derived from a potential proof path of an inductive lemma, and $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_4$ relate to other proof paths. The set $\mathcal{A}$ can be partially solved by considering only $\mathcal{A}_3$. Here, a possible solution of $\mathcal{A}_3$ is $\overline{U}_p(x_1, ..., x_{11}, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_8 \wedge x_4{=}x_{11} \wedge x_7{=}x_{10} \wedge x_8{=}x_9$. This solution can be substituted back to $E_{u_3}$ to derive the refined lemma template $T_3''$.

$$T_3'' \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1{\geq}2 \vdash \exists x_7, n_2.(\mathsf{dll}(x_1, x_2, x_7, x_3, n_2) * x_3{\mapsto}x_7, x_4)$$

Then, FineTuneConseq constructs another unknown entailment $U'(x_1, x_2, x_3, x_4, x_7, n_1, n_2)$ on all variables of $T_3''$ and creates a new unknown entailment, which will be proved again to find a solution $\overline{U}'_p(x_1, x_2, x_3, x_4, x_7, n_1, n_2) \equiv n_1{=}n_2{+}1$. This solution helps to refine the template $T_3''$ to obtain an inductive lemma $\overline{T}_3 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1{\geq}2 \vdash \exists x_7.(\mathsf{dll}(x_1, x_2, x_7, x_3, n_1{-}1) * x_3{\mapsto}x_7, x_4)$, which can be equivalently transformed to the motivating lemma $L_3$ in Section 5.2.

### 5.5.5 Inferring Pure Invariants of Inductive Heap Predicates

We present in this subsection the construction of inductive heap predicates' invariants, which are mainly utilized to preprocess the lemma templates' antecedents (Section 5.5.4). Our construction is inspired from separation logic literature [Chin et al., 2012; Brotherston et al., 2014a; Le et al., 2016].

**Definition 5.6 (Invariant of inductive heap predicates)** *Given an inductive heap predicate* $\mathsf{P}(\vec{x})$*, a pure formula* $\mathsf{Invariant}(\mathsf{P}(\vec{x}))$ *is called an invariant of* $\mathsf{P}(\vec{x})$ *iff* $s, h \models \mathsf{P}(\vec{x})$ *implies that* $s \models \mathsf{Invariant}(\mathsf{P}(\vec{x}))$*, for all model* $s, h$*. Formally,* $\forall s, h.\,(s, h \models \mathsf{P}(\vec{x}) \rightarrow s \models \mathsf{Invariant}(\mathsf{P}(\vec{x})))$.

**Constructing Pure Invariants**. We also exploit a template-based approach to discover the pure invariants. For a system of $k$ inductive heap predicates $\mathsf{P}_1(\vec{x}_1), \ldots, \mathsf{P}_k(\vec{x}_k)$, we first create $k$ unknown relations $U_{\mathsf{P}_1}(\vec{x}_1), \ldots, U_{\mathsf{P}_k}(\vec{x}_k)$ to respectively represent their desired

invariants. Then, we follow the predicates' definitions to establish constraints about the unknown relations.

In particular, consider each definition case $F^i_j(\vec{x_i})$ of a predicate $\mathsf{P}_i(\vec{x_i})$, which is of the form $F^i_j(\vec{x_i}) \triangleq \exists\vec{u}.(\Sigma^i_j \wedge \Pi^i_j)$. Suppose that $\mathsf{Q}_1(\vec{u}_1), ..., \mathsf{Q}_n(\vec{u}_n)$ are *all* inductive heap predicates in $\Sigma^i_j$, where $\mathsf{Q}_1, ..., \mathsf{Q}_n \in \{\, \mathsf{P}_1, ..., \mathsf{P}_k \,\}$, we create the following unknown assumption:

$$U_{\mathsf{Q}_1}(\vec{u}_1) \wedge \ldots \wedge U_{\mathsf{Q}_n}(\vec{u}_n) \wedge \Pi^i_j \quad \rightarrow \quad U_{\mathsf{P}_i}(\vec{x_i})$$

Thereafter, the system of all unknown assumptions can be solved by the constraint solving technique based on Farkas' lemma (Section 5.5.6) to discover the actual invariants of $\mathsf{P}_1(\vec{x_1}), \ldots, \mathsf{P}_k(\vec{x_k})$.

For example, given the predicate $\mathsf{dll}(hd, pr, tl, nt, len)$ in Section 5.2, whose definition is

$$\begin{array}{lcl} \mathsf{dll}(hd, pr, tl, nt, len) & \stackrel{\text{def}}{=} & (hd{\mapsto}pr, nt \wedge hd{=}tl \wedge len{=}1) \\ & & \vee\ \exists u.(hd{\mapsto}pr, u * \mathsf{dll}(u, hd, tl, nt, len{-}1)) \end{array}$$

We create an unknown relation $U_{\mathsf{dll}}(hd, pr, tl, nt, len)$ representing its invariant and follow the definition of $\mathsf{dll}(hd, pr, tl, nt, len)$ to establish the following constraints:

$$\begin{array}{ll} (1) & hd{=}tl \wedge len{=}1 \rightarrow U_{\mathsf{dll}}(hd, pr, tl, nt, len) \\ (2) & U_{\mathsf{dll}}(u, hd, tl, nt, len{-}1) \rightarrow U_{\mathsf{dll}}(hd, pr, tl, nt, len) \end{array}$$

We then solve these constraints to obtain the solution $\overline{U}_{\mathsf{dll}}(hd, pr, tl, nt, len) \equiv len \geq 1$, which is also the invariant of $\mathsf{dll}(hd, pr, tl, nt, len)$.

### 5.5.6 Solving Assumption Constraints with Farkas' Lemma

In this subsection, we describe the underlying constraint solving technique based on Farkas' lemma. This technique is implemented in the procedure Solve, which is frequently invoked in the lemma synthesis to solve an unknown assumption set (Sections 5.5.3, 5.5.4, and 5.5.5). We will formally restate Farkas' lemma and explain how it is applied in constraint solving.

**Farkas' Lemma** [Schrijver, 1986]. Given a conjunction of linear constraints $\bigwedge_{j=1}^{m} \sum_{i=1}^{n} a_{ij}x_i + b_j \geq 0$, which is satisfiable, and a linear constraint $\sum_{i=1}^{n} c_i x_i + \gamma \geq 0$, we have:

$$\forall x_1 \ldots x_n.\ \Big( \bigwedge_{j=1}^{m} \sum_{i=1}^{n} a_{ij}x_i + b_j \geq 0 \Big) \rightarrow \sum_{i=1}^{n} c_i x_i + \gamma \geq 0$$

iff

$$\exists \lambda_1 \ldots \lambda_m \geq 0.\ \Big( \bigwedge_{i=1}^{n} c_i = \sum_{j=1}^{m} \lambda_j a_{ij} \wedge \sum_{j=1}^{m} \lambda_j b_j \leq \gamma \Big)$$

**Solving Constraints.** Given an assumption set $\mathcal{A}$ of constraints over an unknown relation $U(x_1, ..., x_m, u_1, ..., u_n)$, where $x_1, \ldots, x_m$ are spatial variables and $u_1, \ldots, u_n$ are integer variables, the procedure Solve aims to find an actual definition $\overline{U}$ of $U$ in the following template, where $c_{ij}, d_{ij}$ are unknown coefficients, $M, N$ are pre-specified numbers of conjuncts.

$$U(x_1, ..., x_m, u_1, ..., u_n) \triangleq \big( \bigwedge_{j=1}^{M} \sum_{i=1}^{m} c_{ij}x_i + c_{0j} \geq 0 \big) \ \wedge\ \big( \bigwedge_{j=1}^{N} \sum_{i=1}^{n} d_{ij}u_i + d_{0j} \geq 0 \big)$$

Recall that our lemma synthesis framework can incrementally discover the final solution $\overline{U}$ in several passes. Hence, we can set $M, N$ small, e.g., $1 \leq M, N \leq 3$, or $M=0, N \leq 6$ (no spatial constraints), or $M \leq 6, N=0$ (no arithmetic constraints) to make the constraint solving more effective. We restrict the coefficients $c_{ij}$ to represent equality or disequality constraints of spatial variables, e.g., $x_k = x_l$, $x_k \neq x_l$, $x_k = 0$, and $x_k \neq 0$, where $0$ denotes nil. The constraint $x_k = x_l$ can be encoded as $x_k - x_l \geq 0 \wedge -x_k + x_l \geq 0$, or the encoding of $x_k \neq x_l$ is $x_k - x_l - 1 \geq 0$. Therefore, it requires that $-1 \leq c_{ij} \leq 1$ for $i > 0$, and $c_{0j} = 0$ (for equalities) or $c_{0j} = -1$ (for disequalities). We also add the restrictions $-1 \leq \sum_{i=1}^{m} c_{ij} \leq 1$ and $1 \leq \sum_{i=1}^{m} |c_{ij}| \leq 2$ to ensure that the spatial constraints involve at most two variables.

In summary, the assumpts set $\mathcal{A}$ of constraints about the unknown relation $U$ can be solved in three steps:

- Normalize assumptions in the set $\mathcal{A}$ into the form of Horn clause, and substitute all occurrences of $U$ in the normalized assumptions by its template to obtain a set of normalized constraints.
- Apply Farkas' lemma to eliminate universal quantification to obtain new constraints

with only existential quantification over the unknown coefficients $c_{ij}, d_{ij}$, and the factors $\lambda_j$.

– Solve the new constraints by an off-the-shelf prover, such as Z3 [Moura et al., 2008], to find the concrete values of the unknown coefficients, thus discover the actual definition of $U$.

### 5.5.7 Soundness of the Lemma Synthesis Framework

We claim that our lemma synthesis framework is sound, that is, all lemmas synthesized by the framework are semantically valid. We formally state the soundness in the following Theorem 5.4.

**Theorem 5.4 (Soundness of the lemma synthesis)** *Given a normal goal entailment $E$ which does not contain any unknown relation and a set of valid input lemma $\mathcal{L}$, if the lemma synthesis procedure* SynthesizeLemma *returns a set of lemmas $\mathcal{L}_{\text{syn}}$, then all lemmas in $\mathcal{L}_{\text{syn}}$ are semantically valid.*

*Proof.* Figure 5.11 shows that all lemmas returned by SynthesizeLemma are directly discovered by RefineAnte and RefineConseq. In addition, all lemmas returned by RefineConseq are realized by FineTuneConseq (Figure 5.14, line 38). Moreover, all lemmas discovered by RefineAnte and FineTuneConseq are verified by Prove in a setting that disables the lemma synthesis (NoSyn) and utilizes the valid lemma set $\mathcal{L}$ (Figure 5.12, line 31 and Figure 5.17, line 55). It follows from Proposition 5.2 that if Prove returns Valid$\langle_\rangle$ when verifying a lemma, then the lemma is semantically valid. Consequently, all lemmas returned by SynthesizeLemma are semantically valid. □

## 5.6 Experiments

We have implemented the lemma synthesis framework into a prototype prover, named **Songbird$_{\mathbf{LS}}$** and have conducted two experiments to evaluate its ability in proving entailments. Note that **Songbird$_{\mathbf{LS}}$** is built on top of our existing prover **Songbird**[1], and the name **Songbird$_{\mathbf{LS}}$** stands for "**Songbird** + **L**emma **S**ynthesis". The prover **Songbird$_{\mathbf{LS}}$** and the experiment details are available online at:

https://songbird-prover.github.io/lemma-synthesis .

The first experiment was done on literature benchmarks, which include sll_entl, slrd_entl from the separation logic competition SL-COMP 2014 [Sighireanu et al., 2016], and slrd_ind by [Ta et al., 2016]. These benchmarks focus on representing the data structures' shapes, hence, their pure formulas solely contain equality constraints among spatial variables. Therefore, we decided to compose a richer entailment benchmark slrd_lm, which captures also arithmetic constraints of the data structures' size and content, and then performed the second experiment. Moreover, we only considered entailments that are marked as *valid*. Our experiments were conducted on an Ubuntu 14.04 LTS machine with CPU Intel® Core™ i7-6700 (3.4GHz) and RAM 16GB, with a *timeout of 180 seconds* per entailment. We compared **Songbird$_{\mathbf{LS}}$** against state-of-the-art separation logic provers, including **Slide** [Iosif et al., 2013], **Sleek** [Chin et al., 2012], **Spen** [Enea et al., 2014], **Cyclist** [Brotherston et al., 2011a] and **Songbird$_{\mathbf{MI}}$** [Ta et al., 2016]. These provers were configured to run with a timeout of 180 seconds for each entailment.

We present the first experiment in Tables 5.1 and 5.2. The benchmark sll_entl relates to only singly linked list and is categorized by its original sub-benchmarks. The two benchmarks slrd_entl and slrd_ind are classified based on the related data structures, which are variants of linked lists and trees. In Table 5.1, we report in each category the number of *entailments successfully proved* by each prover, where the best results are in **bold** (the total number of entailments is shown in the column **#Ent**). We also report the *total* and the *average time* in seconds of each prover for all *proved entailments*. To be fair, time spent on

[1]The inductive prover **Songbird**: http://songbird-prover.github.io

*unproved entailments* is *not considered* since a prover might spend up to the timeout of 180 seconds for each such entailment. We also provide the statistics of how lemmas are synthesized and applied by **Songbird$_{\mathbf{LS}}$** in Table 5.2. The numbers of lemmas synthesized and used are presented in the column **#Lm**, where $x[y]$ means that there are totally $x$ lemmas synthesized, and only $y$ of them are successfully used. The *total* and the *average time* spent on synthesizing all lemmas are displayed in the columns **T** and **A**. The synthesis *overhead* is shown in the column **O**, which is the percentage (%) of the total synthesizing time over the total proving time. We also classify the synthesized lemmas into three groups of *conversion*, *split*, and *combination* lemmas (**#Cv**, **#Sp**, and **#Cb**), in a spirit similar to the lemmas $L_1, L_2$ and $L_3$ in Section 5.2. The number $x(y)$ in each group indicates that there are $x$ lemmas applied, and they are repeatedly applied $y$ times.

| **Benchmark** | | | **Proved Entailments** | | | | | | **Total Proving Time (s)** | | | | | | **Average Time (s)** | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Category | | #Ent | Sld | Slk | Spn | Ccl | Smi | **Sls** | Sld | Slk | Spn | Ccl | Smi | **Sls** | Sld | Slk | Spn | Ccl | Smi | **Sls** |
| sll_entl | bolognesa | 57 | 0 | 0 | **57** | 0 | **57** | **57** | 0.0 | 0.0 | 23.6 | 0.0 | 140.3 | 18.5 | – | – | 0.41 | – | 2.46 | 0.32 |
| | clones | 60 | 0 | **60** | **60** | **60** | **60** | **60** | 0.0 | 3.7 | 3.5 | 0.5 | 3.9 | 0.7 | – | 0.06 | 0.01 | 0.01 | 0.07 | 0.01 |
| | smallfoot | 55 | 0 | **55** | **55** | **55** | **55** | **55** | 0.0 | 2.7 | 2.5 | 11.8 | 3.5 | 4.2 | – | 0.05 | 0.08 | 0.22 | 0.06 | 0.08 |
| slrd_entl | singly-ll | 64 | 12 | 48 | 3 | 63 | **64** | **64** | 1.0 | 6.3 | 0.1 | 2.1 | 8.3 | 1.7 | 0.08 | 0.13 | 0.04 | 0.03 | 0.13 | 0.03 |
| | doubly-ll | 37 | 14 | 17 | 9 | 29 | 25 | **35** | 38.3 | 3.1 | 0.4 | 112.5 | 11.3 | 91.9 | 2.74 | 0.18 | 0.04 | 3.88 | 0.45 | 2.63 |
| | nested-ll | 11 | 0 | 5 | **11** | 7 | **11** | **11** | 0.0 | 2.2 | 0.5 | 16.7 | 2.4 | 0.4 | – | 0.44 | 0.04 | 2.38 | 0.21 | 0.04 |
| | skip-list | 13 | 0 | 4 | **13** | 5 | **13** | **13** | 0.0 | 1.1 | 1.1 | 0.6 | 8.1 | 1.3 | – | 0.27 | 0.08 | 0.11 | 0.63 | 0.10 |
| | tree | 26 | 12 | 14 | 0 | 23 | 23 | **25** | 110.0 | 2.9 | 0.0 | 58.8 | 11.5 | 72.4 | 9.16 | 0.21 | – | 2.55 | 0.50 | 2.89 |
| slrd_ind | ll/ll2 | 24 | 0 | 0 | 0 | **24** | **24** | **24** | 0.0 | 0.0 | 0.0 | 60.2 | 11.9 | 78.5 | – | – | – | 2.51 | 0.50 | 3.27 |
| | ll-even/odd | 20 | 0 | 0 | 0 | **20** | **20** | **20** | 0.0 | 0.0 | 0.0 | 34.6 | 50.3 | 1.3 | – | – | – | 1.73 | 2.52 | 0.07 |
| | ll-left/right | 20 | 0 | 0 | 0 | **20** | **20** | **20** | 0.0 | 0.0 | 0.0 | 19.5 | 10.6 | 1.3 | – | – | – | 0.97 | 0.53 | 0.06 |
| | misc. | 32 | 0 | 0 | 0 | 31 | **32** | **32** | 0.0 | 0.0 | 0.0 | 254.2 | 55.0 | 19.5 | – | – | – | 8.20 | 1.72 | 0.61 |
| Total | | 419 | 38 | 203 | 208 | 337 | 404 | **416** | 149.2 | 22.0 | 31.6 | 571.5 | 317.2 | 291.8 | 3.93 | 0.11 | 0.15 | 1.70 | 0.79 | 0.70 |

(Sld, Slk, Spn, Ccl, Smi and Sls respectively stands for
**Slide**, **Sleek**, **Spen**, **Cyclist**, **Songbird$_{\mathrm{MI}}$** and **Songbird$_{\mathrm{LS}}$**)

TABLE 5.1: Evaluation on the literature entailment benchmarks.

| Benchmark | | | Lemma Synthesis Stats | | | | Lemma Application Stats | | |
|---|---|---|---|---|---|---|---|---|---|
| Category | | #Ent | #Lm | T (s) | A (s) | O (%) | #Cv | #Sp | #Cb |
| sll_entl | bolognesa | 57 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | clones | 60 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | smallfoot | 55 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| slrd_entl | singly-ll | 64 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | doubly-ll | 37 | 18 [10] | 51.2 | 2.84 | 55.7 | 8 (68) | 0 | 2 (2) |
| | nested-ll | 11 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | skip-list | 13 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | tree | 26 | 0 | 62.0 | – | 85.7 | 0 | 0 | 0 |
| slrd_ind | ll/ll2 | 24 | 14 [10] | 12.6 | 0.90 | 16.1 | 4 (4) | 0 | 6 (11) |
| | ll-even/odd | 20 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | ll-left/right | 20 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| | misc. | 32 | 2 [2] | 1.3 | 0.65 | 6.7 | 2 (3) | 0 | 0 |
| Total | | 419 | 34 [22] | 127.2 | 3.74 | 43.6 | 14 (75) | 0 | 8 (13) |

TABLE 5.2: Statistics on the lemma synthesis and lemma application of the prover **Songbird$_{\mathbf{LS}}$** on the literature benchmarks.

Table 5.1 shows that **Songbird$_{\mathbf{LS}}$** can prove *most* of the entailments in existing benchmarks (414/418 $\approx$ 99%) on an average of 0.54 seconds per entailment, which is 1.46 times faster than the second best prover **Songbird$_{\mathbf{MI}}$**. Moreover, **Songbird$_{\mathbf{LS}}$** outperforms the third best prover **Cyclist** in both the number of proved entailments (more than 78 entailments) and the average proving time (3.15 times faster). Other provers like **Sleek**, **Spen**, and **Slide** can prove no more than 50% of the entailments that **Songbird$_{\mathbf{LS}}$** can prove. In addition, **Songbird$_{\mathbf{LS}}$** achieves the best results in *all* categories, which demonstrates the effectiveness of our lemma synthesis technique. Regarding the 4 entailments that **Songbird$_{\mathbf{LS}}$** cannot prove, they are related to the doubly linked list or the tree data structures (doubly-ll/tree), where the needed lemmas are too complicated to be synthesized in a timeout of 180 seconds.

In the first experiment, the lemma synthesis is only triggered in some categories with an overhead of 29.4%. Although this overhead is considerable, it does not overwhelm the overall proving process since much of the proving time is saved via the lemma application. The lemma synthesis's *efficacy*, determined by the ratio of the number of lemmas applied to the number of lemmas synthesized, is about 65% (22 lemmas used in the total

of 34 lemmas synthesized). More interestingly, these 22 lemmas were applied totally 88 times. This fact implies that a lemma can be (re)used multiple times. In this experiment, the conversion lemmas are applied more often than other lemmas (75/88 times).

In the second experiment, we apply the best three provers **Cyclist**, **Songbird$_{\mathbf{MI}}$**, and **Songbird$_{\mathbf{LS}}$**, on the synthetic benchmark slrd_lm. This benchmark was constructed by enhancing the existing inductive heap predicates with richer *numeric* properties of the data structures' size and content. These new predicates enable more challenging entailments to be designed, such as the motivating entailment $E_1$ from Section 5.2.

$$E_1 \triangleq \mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$$

| **Benchmark** slrd_lm | | **Proved Entails** | | | **Proving Time (s)** | | | **Average Time (s)** | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Category | #Ent | Ccl | Smi | **Sls** | Ccl | Smi | **Sls** | Ccl | Smi | **Sls** |
| ll/rev | 20 | 17 | 18 | **20** | 446.5 | 87.1 | 205.0 | 26.27 | 4.84 | 10.25 |
| ll-even/odd | 17 | **17** | **17** | **17** | 3.5 | 0.6 | 0.4 | 0.21 | 0.03 | 0.02 |
| ll/rev+arith$^\bullet$ | 72 | 0 | 10 | **72** | 0.0 | 0.4 | 203.2 | – | 0.04 | 2.82 |
| ll-sorted$^\bullet$ | 25 | 0 | 3 | **19** | 0.0 | 0.1 | 8.7 | – | 0.03 | 0.46 |
| dll/rev/null | 22 | **22** | **22** | **22** | 8.4 | 17.6 | 149.8 | 0.38 | 0.80 | 6.81 |
| dll/rev/null+arith$^\bullet$ | 94 | 0 | 17 | **94** | 0.0 | 27.6 | 2480.6 | – | 1.63 | 26.39 |
| ll/dll-mixed | 5 | **5** | **5** | **5** | 0.4 | 0.1 | 0.2 | 0.07 | 0.03 | 0.03 |
| ll/dll-mixed+arith$^\bullet$ | 15 | 0 | 5 | **15** | 0.0 | 0.4 | 328.3 | – | 0.08 | 21.89 |
| tree/tseg | 18 | 7 | 7 | **18** | 1.3 | 0.3 | 135.0 | 0.18 | 0.04 | 7.50 |
| tree/tseg+arith$^\bullet$ | 12 | 0 | 3 | **12** | 0.0 | 3.5 | 513.7 | – | 1.15 | 42.81 |
| Total | 300 | 68 | 107 | **294** | 460.0 | 137.7 | 4024.8 | 6.76 | 1.29 | 13.69 |

(Ccl, Smi, and Sls respectively stands for **Cyclist**, **Songbird$_{\mathbf{MI}}$** and **Songbird$_{\mathbf{LS}}$**)

TABLE 5.3: Evaluation on the synthetic benchmark slrd_lm.

Details of the second experiment are presented in Tables 5.3 and 5.4. Entailments in the benchmark slrd_lm are categorized based on the participating heap predicates and their numeric properties. To minimize the table's size, we group various heap predicates related to the same data structure in one category. For example, the category dll/rev/null contains entailments about doubly linked lists, including normal lists (dll), reversed lists

(dllrev), or null-terminated lists (dllnull). Entailments in the category ll/dll-mixed involve both the singly and the doubly linked lists.

Table 5.3 shows that **Songbird$_{\mathbf{LS}}$** outperforms other provers in *all* categories. It can prove 98% of the benchmark problems (294/300 entailments), which is 2.7 times better than the second best prover **Songbird$_{\mathbf{MI}}$** (107/300 entailments). More impressively, in the 5 categories with arithmetic constraints, marked by •, **Songbird$_{\mathbf{LS}}$** can prove more than 5.5 times the number of entailments that **Songbird$_{\mathbf{MI}}$** can (212 vs. 38 entailments). On the other hand, **Cyclist** performs poorly in this experiment because it does not support numeric properties yet. However, among 82 non-arithmetic entailments, **Cyclist** can prove 68 entailments (about 80%), whereas **Songbird$_{\mathbf{LS}}$** can prove all of them.

| **Benchmark** slrd_lm | | **Lemma Synthesis** | | | | **Lemma Application** | | |
|---|---|---|---|---|---|---|---|---|
| Category | #Ent | #Lm | T (s) | A (s) | O (%) | #Cv | #Sp | #Cb |
| ll/rev | 20 | 28 [20] | 8.1 | 0.29 | 3.9 | 18 (25) | 0 | 2 (2) |
| ll-even/odd | 17 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| ll/rev+arith• | 72 | 81 [81] | 50.9 | 0.63 | 25.1 | 0 | 81 (143) | 0 |
| ll-sorted• | 25 | 14 [14] | 4.9 | 0.35 | 55.8 | 0 | 0 | 14 (30) |
| dll/rev/null | 22 | 12 [7] | 59.1 | 4.92 | 39.5 | 7 (8) | 0 | 0 |
| dll/rev/null+arith• | 94 | 236 [158] | 1902.7 | 8.06 | 76.7 | 74 (74) | 69 (103) | 15 (15) |
| ll/dll-mixed | 5 | 0 | 0.0 | – | 0.0 | 0 | 0 | 0 |
| ll/dll-mixed+arith• | 15 | 12 [10] | 227.2 | 18.93 | 69.2 | 8 (8) | 2 (2) | 0 |
| tree/tseg | 18 | 18 [12] | 3.8 | 0.21 | 2.8 | 8 (11) | 0 | 4 (5) |
| tree/tseg+arith• | 12 | 20 [16] | 211.2 | 10.56 | 41.1 | 8 (8) | 4 (5) | 4 (4) |
| Total | 300 | 421 [318] | 2467.7 | 5.86 | 61.3 | 123 (134) | 156 (253) | 39 (56) |

TABLE 5.4: Statistics on the lemma synthesis and lemma application of the prover **Songbird$_{\mathbf{LS}}$** on the synthetic benchmark slrd_lm.

In the second experiment, the lemma synthesis overhead is higher (61.3%) since there are many lemmas synthesized (421), as indicated in Table 5.4. However, the overall efficacy is also improved (75.5%) when 318 synthesized lemmas are actually used to prove goal entailments. It is worth noticing that 100% lemmas synthesized in the two arithmetic categories ll/rev+arith and ll-sorted are utilized; this shows the usefulness of our proposed

framework in proving sophisticated entailments. In this experiment, the *split* lemmas are synthesized (49.1% of the total synthesized lemmas) and prominently used (57% of the total number of lemma application). This interesting fact shows that the slrd_lm benchmark, though handcrafted, was well designed to complement the existing benchmarks.

## 5.7 Related Work

We have surveyed earlier in Chapter 3 that proof technique based on cyclic proof and mathematical induction, such as [Brotherston et al., 2011a] and [Chu et al., 2015], are capable of dealing with *user-defined inductive heap predicates*. However, it is well-known that induction proof is an incomplete technique and its success often depends on clever eureka steps, like discovering missing lemmas [Bundy, 2001].

To the best of our knowledge, there have been two approaches aiming to automatically discover the supporting lemmas. The first approach is the *mutual induction* proof presented in [Ta et al., 2016]. This work speculates lemmas from *all entailments* which are already derived in an on-going induction proof to assist in proving future entailments introduced within the same proof. This speculation provides more lemma/induction hypothesis candidates than the cyclic-based [Brotherston et al., 2011a] and induction-based [Chu et al., 2015] techniques. Consequently, it can increase the chance of successfully proving the entailments. However, the mutual induction proof *cannot handle* sophisticated entailments, such as $E_1$ in Section 5.2. All entailments derived from $E_1$ may contain specific constraints and cannot be applied to prove other derived entailments.

The second approach is the *lemma generation* presented in [Enea et al., 2015]. This work considers an interesting class of inductive heap predicates satisfying the notions of *compositionality* and *completion*. Under these properties, a class of lemmas can be enumerated beforehand, either to *convert* inductive predicates *of the same arity* or to *combine* two inductive predicates. However, this technique *cannot generate* lemmas which convert predicates of different arities or combine a singleton heap predicate with an inductive heap

predicate, or split an inductive heap predicate into other predicates. In addition, an inductive heap predicate satisfying the *compositionality* property has exactly one base-case definition, whose heap part is also empty. Moreover, each inductive-case definition must contain a singleton heap predicate whose *root* address is one of the inductive heap predicate's arguments, like the *compositional* predicate $\mathsf{ls}(x, y) \stackrel{\text{def}}{=} (x{=}y) \vee \exists u.(x{\mapsto}u * \mathsf{ls}(u, y))$. This technique, therefore, cannot generate lemmas for predicates with non-empty base cases, e.g., dll and dllrev in Section 5.2, or lemmas for the predicates defined in a *reverse-fashion*, like the *non-compositional* predicate $\mathsf{lsrev}(x, y) \stackrel{\text{def}}{=} (x{=}y) \vee \exists u.(\mathsf{lsrev}(x, u) * u{\mapsto}y)$. These *reverse-fashion* predicates are prevalent in SL-COMP 2014's benchmarks, such as RList, ListO, DLL_plus, DLL_plus_rev, DLL_plus_mid.

## 5.8 Summary

We have proposed a novel framework for synthesizing lemmas to assist in proving separation logic entailments in a fragment of symbolic-heap separation logic with inductive predicate and linear arithmetic. Our framework is general enough to synthesize a wide range of lemmas for user-defined inductive heap predicates. Experimentally, we have shown that our lemma synthesis framework can significantly improve the completeness of the induction proof in separation logic. In particular, the prototype prover **Songbird$_{\mathbf{LS}}$** is able to prove many entailments that cannot be proved using standard induction proof.

# Chapter 6

# The Failure Learning Framework

## 6.1 Overview

In this chapter, we study the application of mathematical induction in proving both the *validity of separation logic entailments* and the *unsatisfiability of separation logic formulas*. More specifically, the validity of an entailment $F_1 \vdash F_2$ can be *directly* examined by an induction proof, whereas the unsatisfiability of a formula $F$ can be *indirectly* inspected by proving the entailment $F \vdash \mathsf{false}$. Each formula in our considered logic fragment consists of two parts: a *spatial formula* modeling the memory shape of a program's state, and a *pure formula* representing Boolean constraints of the program's variables. When applying an induction hypothesis (IH) $F_3 \vdash F_4$ to prove a goal entailment like $F_1 \vdash F_2$ or $F \vdash \mathsf{false}$, an inductive prover needs to perform two unification steps:

- *Spatial unification*: find a substitution $\theta$ which unifies the spatial formulas of the IH's antecedent ($F_3$) and of the goal entailment's antecedent ($F_1$ or $F$).
- *Pure unification*: examine whether the pure formula of the goal entailment's antecedent ($F_1$ or $F$) can prove the pure formula obtained by from applying the substitution $\theta$ on the IH's antecedent ($F_3\theta$).

If one of the above steps *fails*, the current IH *cannot* be applied to prove the goal entailment. Here, we propose to *learn* from these failures to improve the induction proofs. In the *spatial unification* case, we identify *non-unifiable* variables or constants of the IH, and

then execute a back-jump to the proof derivation step where this IH was recorded to *generalize* these variables or constants by fresh variables to introduce a new IH. In the *pure unification* case, we determine the *unprovable constraint* of the IH, and then perform a *case-analyis* on this constraint so that the current IH can be applied, or back-jump and *weaken* the unprovable constraint to introduce a new IH. We show that these *generalization*, *case analysis*, and *weakening* can increase the success chance of the induction proofs.

In summary, our work makes the following *contributions*:

- We propose systematic strategies to *learn* from the failures of the induction hypothesis applications to introduce new IH by performing a weakening or generalization, or to execute a *case analysis* to enhance the chance of successfully applying the current IH.
- We develop a novel *learning-from-failure* framework by integrating these strategies into a structural induction proof system for separation logic entailments.
- We implement the framework into a prototype prover, named **Songbird$_{\textbf{LFF}}$**, and experiment it on various separation logic benchmarks. The experimental result is promising since our prover can prove more entailments and formulas than existing provers. The prover **Songbird$_{\textbf{LFF}}$** is available online at:

  http://songbird-prover.github.io/learning-from-failure/ .

## 6.2 Motivation

We present several running examples to illustrate how to learn from failure to prove separation logic entailments and formulas. We consider two inductive heap predicates $\mathsf{ls}(x, y, n)$ and $\mathsf{lsrev}(x, y, n)$ which both model a segment of the linked list data structure of length $n$, whose first and last element is respectively pointed-to by $x$ and $y$. Their only difference is that $\mathsf{lsrev}(x, y, n)$ is defined in a reverse-fashion, in comparison to $\mathsf{ls}(x, y, n)$.

$$
\begin{array}{lcl}
\mathsf{ls}(x, y, n) & \stackrel{\text{def}}{=} & (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \\
 & & \vee\ \exists u.\ (x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) \wedge n{\geq}1) \\
\mathsf{lsrev}(x, y, n) & \stackrel{\text{def}}{=} & (\mathsf{emp} \wedge x{=}y \wedge n{=}0) \\
 & & \vee\ \exists u.\ (\mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y \wedge n{\geq}1)
\end{array}
$$

### 6.2.1 Weakening an Induction Hypothesis

Suppose that a verification system needs to prove an entailment $E_1$ describing the concatenation of a linked list segment (length $n{\geq}100$) with a singleton heap.

$$E_1 \triangleq \mathsf{ls}(x, y, n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1)$$

However, $E_1$ cannot be proved by traditionally *cyclic-based* and *induction-based* approaches [Brotherston et al., 2011a; Chu et al., 2015; Ta et al., 2016]. We briefly explain their failures as follow. When proving $E_1$ by induction, an inductive prover will first *implicitly* or *explicitly* record this entailment as the following IH $H_1$, whose variables are renamed to avoid confusion. Then, the predicate $\mathsf{ls}(x, y, n)$ in $E_1$'s antecedent will be unfolded by its inductive definition to derive new sub-goal entailments. We consider the following entailment $E_1'$ which is derived from an inductive-case unfolding:

$$\begin{aligned} H_1 &\triangleq \mathsf{ls}(a, b, k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a, c, k{+}1) \\ E_1' &\triangleq x{\mapsto}u * \mathsf{ls}(u, y, n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x, z, n{+}1) \end{aligned}$$

When applying $H_1$ to prove $E_1'$, two heap formulas $\mathsf{ls}(a, b, k) * b{\mapsto}c$ and $\mathsf{ls}(u, y, n{-}1) * y{\mapsto}z$ in their antecedents can be unified by a substitution $\theta_1 \equiv [u/a, y/b, z/c, n{-}1/k]$. However, the pure constraint $n{\geq}100$ in $E_1'$'s antecedent cannot prove that $n{-}1{\geq}100$, which is derived from applying $\theta_1$ on $k{\geq}100$ of $H_1$. Alternatively, the prover can record $E_1'$ as a new IH $H_1'$ and unfolds inductive heap predicates in $E_1'$ to derive new sub-goal entailments. However, neither $H_1$ nor $H_1'$ can be applied to prove any derived sub-goal entailments, due to the similar failure explained earlier.

**Our proposed solution.** Notice that $H_1$ cannot be applied to prove $E_1'$ because the constraint $n{-}1{\geq}100$, which is derived from $k{\geq}100$ of $H_1$, cannot be proved by $E_1'$'s antecedent. This constraint $k{\geq}100$ relates to $n{\geq}100$ of the original entailment $E_1$. Hence, we propose to weaken the constraint $n{\geq}100$ of $E_1$'s antecedent to derive the following entailment $\overline{E}_1$, which are provable by induction.

$$\overline{E}_1 \triangleq \mathsf{ls}(x, y, n) * y{\mapsto}z \vdash \mathsf{ls}(x, z, n{+}1)$$

We present the proof tree of $E_1$ in Figure 6.1, where the weakening rule $W_\pi$ is applied to

remove the constraint $n{\geq}100$ of $E_1$ to derive $\overline{E}_1$. We will explain in details about $W_\pi$ and other inference rules used in this proof tree in Sections 6.4 and 6.6.

$$
\dfrac{
\dfrac{
\dfrac{
\dfrac{
\dfrac{
\dfrac{}{\mathsf{true} \vdash \exists v.(z{=}v \wedge 0{=}0)}\vdash_{\Pi}
}{\mathsf{true} \vdash \exists v.(\mathsf{ls}(v,z,0) \wedge z{=}v)}\mathsf{P_R}
}{x{\mapsto}z \vdash \exists v.(x{\mapsto}v * \mathsf{ls}(v,z,0))}*{\mapsto}
}{x{\mapsto}z \vdash \mathsf{ls}(x,z,1)}\mathsf{P_R}
}{y{\mapsto}z \wedge x{=}y \wedge n{=}0 \vdash \mathsf{ls}(x,z,n{+}1)}=_{\mathsf{L}}
\quad
\dfrac{
\dfrac{
\dfrac{
\dfrac{
\dfrac{}{n{\geq}1 \vdash \exists r.(u{=}r \wedge n{\geq}0)}\vdash_{\Pi}
}{\mathsf{ls}(u,z,n) \wedge n{\geq}1 \vdash \exists r.(\mathsf{ls}(r,z,n) \wedge u{=}r \wedge n{\geq}0)}*\mathsf{P}
}{\mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge n{\geq}1 \vdash \exists r.(\mathsf{ls}(r,z,n) \wedge u{=}r)}\mathsf{IH}
}{x{\mapsto}u * \mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge n{\geq}1 \vdash \exists r.(x{\mapsto}r * \mathsf{ls}(r,z,n))}*{\mapsto}
}{x{\mapsto}u * \mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge\ n{\geq}1 \vdash \mathsf{ls}(x,z,n{+}1)}\mathsf{P_R}
}{
\dfrac{\overline{E}_1 \triangleq \mathsf{ls}(x,y,n) * y{\mapsto}z \vdash \mathsf{ls}(x,z,n{+}1)}{E_1 \triangleq \mathsf{ls}(x,y,n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x,z,n{+}1)}W_\pi
}\mathsf{ID}
$$

(where ID records $\overline{H}_1 \triangleq \mathsf{ls}(a,b,k) * b{\mapsto}c \vdash \mathsf{ls}(a,c,k{+}1)$,
IH applies $\overline{H}_1$ with $\theta_1 \triangleq [u/a, y/b, z/c, n{-}1/k]$, $W_\pi$ removes $n{\geq}100$.)

FIGURE 6.1: Induction proof tree of $E_1$.

### 6.2.2 Generalizing an Induction Hypothesis

Suppose that the verification system needs to check the satisfiability of a formula $F_1$ below. Intuitively, $F_1$ is unsatisfiable since it contains overlaid memory pointed by $x$: the first element of the linked list segment $\mathsf{ls}(x, \mathsf{nil}, 100)$, and either the first element of $\mathsf{lsrev}(x,y,n)$ (if $x{\neq}y$) or the singleton heap $y{\mapsto}y$ (if $x{=}y$).

$$F_1 \triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x,y,n) * y{\mapsto}y$$

However, existing techniques *cannot* prove that $F_1$ is unsatisfiable. In particular, the *non-induction-based* satisfiability checking [Brotherston et al., 2014a; Gu et al., 2016] and entailment proving [Berdine et al., 2004; Cook et al., 2011; Piskac et al., 2013; Enea et al., 2014; Pérez et al., 2011; Pérez et al., 2013; Iosif et al., 2013] techniques do not support either linear arithmetic or reverse-fashion predicate like lsrev. Hence, they cannot examine the formula $F_1$ or prove the entailment $F_1 \vdash \mathsf{false}$. The *cyclic-based* [Le et al., 2016; Le et al., 2017] and *induction-based* [Brotherston et al., 2011a; Chu et al., 2015; Ta et al., 2016; Ta et al., 2018] techniques also fail to prove the unsatisfiability of $F_1$, due to the specific singleton heap $y{\mapsto}y$. We will explain the failure of induction-based proofs as follows. The failures of the cyclic-based proofs can be described in a similar manner.

To prove that $F_1$ is unsatisfiable, the induction-based approaches [Chu et al., 2015; Ta et al., 2016] need to prove that $F_1 \vdash \mathsf{false}$ like the entailment $E_2$ below. It first records $E_2$ as an IH $H_2$ and unfolds heap predicates in $E_2$'s antecedents to derive new entailments.

$$E_2 \triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}$$
$$H_2 \triangleq \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsrev}(a, b, l) * b{\mapsto}b \vdash \mathsf{false}$$

Suppose $\mathsf{ls}(x, \mathsf{nil}, 100)$ is unfolded to derive new entailments. We consider the following entailment $E_2'$ obtained from an inductive-case unfolding. When applying $H_2$ to prove $E_2'$, the two sub-formulas $\mathsf{ls}(a, \mathsf{nil}, 100)$ of $H_2$ and $\mathsf{ls}(u, \mathsf{nil}, 99)$ of $E_2'$ need to be unified. However, there is no substitution that can map $100$ to $99$.

$$E_2' \triangleq x{\mapsto}u * \mathsf{ls}(u, \mathsf{nil}, 99) * \mathsf{lsrev}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}$$

Alternatively, $\mathsf{lsrev}(x, y, n)$ in $E_2$ can also be unfolded, and we consider the derived entailment $E_2''$ below. When applying $H_2$ to prove $E_2''$, the two sub-formulas $\mathsf{lsrev}(a, b, l) * b{\mapsto}b$ and $\mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y$ also need to be unified. However, there is no valid substitution since $b$ is mapped to different variables $u$ and $y$.

$$E_2'' \triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y * y{\mapsto}y \vdash \mathsf{false}$$

**Our solution.** Notice that $H_2$ *cannot* be applied to prove $E_2'$ and $E_2''$ because $100$ or $b$ in $H_2$ cannot be mapped to $99$ (in $E_2'$) or both $u$ and $y$ (in $E_2''$). Since $100$ and $b$ correspond to $100$ and $y$ in $E_2$, we propose to *generalize* $E_2$ by replacing $100$ or $y$ by fresh variables. For example, the second $y$ in $y{\mapsto}y$ can be renamed by a fresh $z$ to derive the entailment $\overline{E}_2$, which can be easily proved by induction.

$$\overline{E}_2 \triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x, y, n) * y{\mapsto}z \vdash \mathsf{false}$$

We illustrate induction proof tree of $E_2$ in Figure 6.2, where the rule $G_{\mathrm{v}}$ is applied to generalize a variable $y$ of $E_2$ to obtain $\overline{E}_2$. The details about this rule $G_{\mathrm{v}}$ and other inference rules utilized in this proof tree will be explained in Sections 6.4 and 6.6.

$$
\dfrac{\dfrac{\dfrac{\dfrac{}{x{\mapsto}z \wedge n{=}0 \wedge x{=}\mathsf{nil} \wedge 100{=}0 \vdash \mathsf{false}}\perp_{\mathrm{L}}^{1} \quad \dfrac{}{x{\mapsto}u * \mathsf{ls}(u,\mathsf{nil},99) * x{\mapsto}z \wedge n{=}0 \vdash \mathsf{false}}\perp_{\mathrm{L}}^{2}}{\mathsf{ls}(x,\mathsf{nil},100) * x{\mapsto}z \wedge n{=}0 \vdash \mathsf{false}}*\mathsf{P}}{\mathsf{ls}(x,\mathsf{nil},100) * y{\mapsto}z \wedge x{=}y \wedge n{=}0 \vdash \mathsf{false}}=_{\mathrm{L}} \quad \dfrac{\dfrac{}{y{\mapsto}z \wedge \mathsf{false} \vdash \mathsf{false}}\perp_{\mathrm{L}}^{2}}{\mathsf{ls}(x,\mathsf{nil},100) * \mathsf{lsrev}(x,u,n{-}1) * u{\mapsto}y * y{\mapsto}z \vdash \mathsf{false}}\mathsf{IH}}{\overline{E}_2 \triangleq \mathsf{ls}(x,\mathsf{nil},100) * \mathsf{lsrev}(x,y,n) * y{\mapsto}z \vdash \mathsf{false}}\mathsf{ID}}{E_2 \triangleq \mathsf{ls}(x,\mathsf{nil},100) * \mathsf{lsrev}(x,y,n) * y{\mapsto}y^{\dagger} \vdash \mathsf{false}}G_{\mathrm{v}}
$$

(where ID records $H_2 \triangleq \mathsf{ls}(a,\mathsf{nil},100) * \mathsf{lsrev}(a,b,l) * b{\mapsto}c \vdash \mathsf{false}$,
IH applies $H_2$ with $\theta_2 \triangleq [x/a, y/b, u/c, n{-}1/l]$, $G_{\mathrm{v}}$ replaces $y^{\dagger}$ by a fresh variable.)

FIGURE 6.2: Induction proof tree of $E_2$.

### 6.2.3 Performing a Case Analysis on an Equality Constraint to Apply an Induction Hypothesis

We consider the *unsatisfiability* checking of another formula $F_2$, which includes a linked list segment predicate $\mathsf{lsrev}(x,y,n)$ and a singleton heap predicate $x{\mapsto}z$.

$$F_2 \triangleq \mathsf{lsrev}(x,y,n) * x{\mapsto}z \wedge x{\neq}y$$

Intuitively $F_2$ is also unsatisfiable since it variable $x$ points to overlaid memory cells which are the first element of $\mathsf{lsrev}(x,y,n)$ (when $x{\neq}y$) and the singleton heap $x{\mapsto}z$. However, this unsatisfiability *cannot* be concluded by existing satisfiability checking [Brotherston et al., 2014a; Gu et al., 2016; Le et al., 2016; Le et al., 2017] or entailment proving techniques [Berdine et al., 2004; Cook et al., 2011; Piskac et al., 2013; Enea et al., 2014; Pérez et al., 2011; Iosif et al., 2013; Brotherston et al., 2011a; Chu et al., 2015; Ta et al., 2016; Ta et al., 2018], due to the similar reason as explained in the previous section. We will briefly explain the failure of the induction-based approaches [Chu et al., 2015; Ta et al., 2016] as follows.

Similar to the previous section, induction proof will be utilized to prove the entailment $F_2 \vdash \mathsf{false}$ like $E_3$ below. This entailment is first recorded as the IH $H_3$, then the predicate $\mathsf{lsrev}(x,y,n)$ will be unfolded to derive new entailments.

$$
\begin{aligned}
E_3 &\triangleq \mathsf{lsrev}(x,y,n) * x{\mapsto}z \wedge x{\neq}y \vdash \mathsf{false}\\
H_3 &\triangleq \mathsf{lsrev}(a,b,k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false}
\end{aligned}
$$

We consider the following entailment $E'_3$ derived from the inductive case unfolding. When applying $H_3$ to prove $E'_3$, the induction-based prover discovers a substitution $\theta_3 \triangleq [x/a, u/b, z/c, n/k]$ that can unify two spatial formulas $\mathsf{lsrev}(a, b, k) * a \mapsto c$ and $\mathsf{lsrev}(x, u, n) * x \mapsto z$ in $H_3$ and $E'_3$'s antecedents. However, the condition $x \neq u$, which is obtained from applying $\theta_3$ on $a \neq u$ in $H_3$, cannot be proved by $x \neq y \wedge n \geq 1$ in $E'_3$'s antecedent.

$$E'_3 \triangleq \mathsf{lsrev}(x, u, n) * u \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \vdash \mathsf{false}$$

Alternatively, induction can be performed on $\mathsf{lsrev}(x, u, n)$ of $E'_3$ to record new IHs and derive new sub-goal entailments. However, all recorded IHs cannot be applied to prove these entailments, due to the similar failure discussed earlier.

**Our solution.** Notice that when $H_3$ cannot be used to prove $E_3$, a *case analysis* can be performed on the *unprovable* constraint $x \neq u$ to derive two entailments $E'_{31}$ and $E'_{32}$. Here, $H_3$ can be easily applied to prove $E'_{31}$. In addition, $E'_{32}$ can be proved normally using standard inference rules.

$$E'_{31} \triangleq \mathsf{lsrev}(x, u, n) * u \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \wedge x \neq u \vdash \mathsf{false}$$
$$E'_{32} \triangleq \mathsf{lsrev}(x, u, n) * u \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \wedge x = u \vdash \mathsf{false}$$

We present in Figure 6.3 the proof tree of $E_3$ with the case analysis strategy. Note that our solution can also be easily applied by the cyclic-based techniques [Brotherston et al., 2011a; Le et al., 2016; Le et al., 2017] to prove that $F_2$ is unsatisfiable.

$$\dfrac{\dfrac{}{x \mapsto z \wedge x \neq y \wedge x = y \wedge n = 0 \vdash \mathsf{false}}\,\bot^1_{\mathrm{L}} \qquad \dfrac{\dfrac{\dfrac{}{\mathsf{lsrev}(x, x, n-1) * x \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \vdash \mathsf{false}}\,\bot^2_{\mathrm{L}}}{\mathsf{lsrev}(x, u, n-1) * u \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \wedge x = u \vdash \mathsf{false}}\,=_{\mathrm{L}} \quad \dfrac{\dfrac{}{u \mapsto y \wedge x \neq y \wedge x \neq u \wedge n \geq 1 \wedge \mathsf{false} \vdash \mathsf{false}}\,\bot^1_{\mathrm{L}}}{\mathsf{lsrev}(x, u, n-1) * u \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \wedge x \neq u \vdash \mathsf{false}}\,\mathsf{IH}}{\mathsf{lsrev}(x, u, n-1) * u \mapsto y * x \mapsto z \wedge x \neq y \wedge n \geq 1 \vdash \mathsf{false}}\,\mathsf{CA}}{E_3 \triangleq \mathsf{lsrev}(x, y, n) * x \mapsto z \wedge x \neq y \vdash \mathsf{false}}\,\mathsf{ID}$$

(where ID records $H_3 \triangleq \mathsf{lsrev}(a, b, k) * a \mapsto c \wedge a \neq b \vdash \mathsf{false}$,
IH applies $H_3$ with $\theta_3 \triangleq [x/a, u/b, z/c, n-1/k]$, CA is performed on $x \neq u$.)

FIGURE 6.3: Induction proof tree of $E_3$

### 6.2.4 Performing a Case Analysis on a Linear Arithmetic Constraint to Apply an Induction Hypothesis

Now we consider the formula $F_3$, which is a variant of $F_2$, where the non-empty condition is replaced by $n{\geq}1$.

$$F_3 \triangleq \mathsf{lsrev}(x, y, n) * x{\mapsto}z \wedge n{\geq}1$$

This formula is also unsatisfiable since $x$ points to overlaid memory cells in segment $\mathsf{lsrev}(x, y, n)$ and $x{\mapsto}z$. Existing satisfiability checking [Brotherston et al., 2014a; Gu et al., 2016; Le et al., 2016; Le et al., 2017] and entailment proving [Berdine et al., 2004; Cook et al., 2011; Piskac et al., 2013; Enea et al., 2014; Pérez et al., 2011; Iosif et al., 2013; Brotherston et al., 2011a; Chu et al., 2015; Ta et al., 2016; Ta et al., 2018] techniques also fail to prove the unsatisfiability of $F_3$, due to the same reason explained earlier. In our approach, we also utilize structural induction with the case analysis strategy to prove that $F_3 \vdash \mathsf{false}$, like the following entailment $E_4$.

$$E_4 \triangleq \mathsf{lsrev}(x, y, n) * x{\mapsto}z \wedge n{\geq}1 \vdash \mathsf{false}$$

We present the induction proof tree of $E_4$ in Figure 6.4, where the case analysis rule CA is performed on the constraint $n{-}1{\geq}1$, which is learned by analyzing the failure of the previous induction hypothesis applications during the proof search.

$$\dfrac{\dfrac{}{x{\mapsto}z \wedge n{\geq}1 \wedge x{=}y \wedge n{=}0 \vdash \mathsf{false}}\,\bot_{\mathrm{L}}^{1} \qquad \dfrac{\dfrac{\dfrac{}{x{\mapsto}y * x{\mapsto}z \wedge n{=}1 \vdash \mathsf{false}}\,\bot_{\mathrm{L}}^{2}}{u{\mapsto}y * x{\mapsto}z \wedge x{=}u \wedge n{=}1 \vdash \mathsf{false}}\,{=}_{\mathrm{L}} \quad \dfrac{}{\mathsf{lsrev}(x, v, n{-}2) * v{\mapsto}u * u{\mapsto}y * x{\mapsto}z \wedge n{\geq}2 \wedge n{=}1 \vdash \mathsf{false}}\,\bot_{\mathrm{L}}^{1}}{\mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge n{\geq}1 \wedge n{-}1{<}1 \vdash \mathsf{false}}\,\mathsf{ID} \quad \dfrac{\dfrac{}{u{\mapsto}y \wedge n{\geq}2 \wedge \mathsf{false} \vdash \mathsf{false}}\,\bot_{\mathrm{L}}^{1}}{\mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge n{\geq}1 \wedge n{-}1{\geq}1 \vdash \mathsf{false}}\,\mathsf{IH}}{\mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y * x{\mapsto}z \wedge n{\geq}1 \vdash \mathsf{false}}\,\mathsf{CA}}{E_4 \triangleq \mathsf{lsrev}(x, y, n) * x{\mapsto}z \wedge n{\geq}1 \vdash \mathsf{false}}\,\mathsf{ID}^{*}$$

(where $\mathsf{ID}^{*}$ records $H_4 \triangleq \mathsf{lsrev}(a, b, k) * a{\mapsto}c \wedge k{\geq}1 \vdash \mathsf{false}$,
IH applies $H_4$ with $\theta_4 \triangleq [x/a, u/b, z/c, n{-}1/k]$, CA is performed on $n{-}1{\geq}1$.)

FIGURE 6.4: Induction proof tree of $E_4$.

## 6.3 Background

In this section, we briefly recall the background of separation logic and the structural induction proof. More details can be found in Chapters 2 and 5.

### 6.3.1 Symbolic-Heap Separation Logic

$$
\begin{array}{lcl}
e & ::= & k \mid x \mid -e \mid e_1 + e_2 \mid e_1 - e_2 \mid k \cdot e \\
a & ::= & \mathsf{nil} \mid x \\
t & ::= & e \mid a \\
\pi & ::= & \mathsf{true} \mid \mathsf{false} \mid a_1 = a_2 \mid a_1 \neq a_2 \mid e_1 = e_2 \mid e_1 \neq e_2 \mid e_1 > e_2 \mid e_1 \geq e_2 \mid e_1 < e_2 \mid e_1 \leq e_2 \\
\sigma & ::= & \mathsf{emp} \mid x \overset{\iota}{\mapsto} t_1, ..., t_n \mid \mathsf{P}(t_1, ..., t_n) \\
\Pi & ::= & \pi \mid \neg\Pi \mid \Pi_1 \wedge \Pi_2 \mid \Pi_1 \vee \Pi_2 \mid \Pi_1 \rightarrow \Pi_2 \mid \forall x.\Pi \mid \exists x.\Pi \\
\Sigma & ::= & \sigma \mid \Sigma_1 * \Sigma_2 \\
F & ::= & \Sigma \wedge \Pi \mid \exists x.\, F
\end{array}
$$

FIGURE 6.5: Syntax of Symbolic-heap Separation Logic formulas

**Syntax.** Figure 6.5 presents the syntax of formulas, where $x, \vec{x}, k, \mathsf{nil}$ respectively denote a variable, a list of variables, an integer constant, a dangling memory constant. Moreover, $e$ is an integer expression, and $a$ is a spatial expression representing a memory address. We also write $t$ to represent an expression in general (integer or spatial). Moreover, $\mathsf{emp}$ is a predicate modeling an empty memory, and $x \overset{\iota}{\mapsto} t_1, ..., t_n$ is a singleton heap predicate representing a single data structure of sort $\iota$, pointed by $x$, having $n$ fields $t_1, ..., t_n$[1]. The inductive heap predicate $\mathsf{P}(t_1, ..., t_n)$ models a recursive data structure. Finally, $\Pi$ and $\Sigma$ respectively represent a *pure formula* and a *spatial formula*. They are used to compose the *symbolic-heap formula* $F$.

[1]For brevity, we omit $\iota$ when presenting the motivating examples in Section 6.2.

**Semantics.** Figure 6.6 presents the semantics of formulas. Given a set $\texttt{Var}$ of variables, $\texttt{Sort}$ of sorts, $\texttt{Val}$ of values, and $\texttt{Loc} \subset \texttt{Val}$ of memory addresses, a separation logic model consists of a *stack model* $s$: $\texttt{Var} \to \texttt{Val}$ and a *heap model* $h$: $(\texttt{Loc} \times \texttt{Sort}) \rightharpoonup \texttt{Val}^+$. Here, $[\![\Pi]\!]_s$ denotes the valuation of $\Pi$ under the stack model $s$. Moreover, $\mathrm{dom}(h)$ is the domain of $h$; $h \# h'$ indicates that $h$ and $h'$ have disjoint domains; $h \circ h'$ is the union of two disjoint heap models $h, h'$. Finally, $[s|x{:}v]$ extends $s$ with a mapping from $x$ to $v$.

| | | |
|---|---|---|
| $s, h \models \mathsf{emp}$ | iff | $\mathrm{dom}(h) = \varnothing$ |
| $s, h \models x \overset{\iota}{\mapsto} t_1, ..., t_n$ | iff | $\mathrm{dom}(h) = \{s(x)\}$ and $h(s(x), \iota) = (s(t_1), ..., s(t_n))$ |
| $s, h \models \mathsf{P}(t_1, ..., t_n)$ | iff | $s, h \models F$, where $F \overset{\text{def}}{\Rightarrow} \mathsf{P}(t_1, ..., t_n)$ |
| $s, h \models \Sigma_1 * \Sigma_2$ | iff | $\exists h_1, h_2 : h_1 \# h_2;\ h_1 \circ h_2 = h;\ s, h_1 \models \Sigma_1$ and $s, h_2 \models \Sigma_2$ |
| $s, h \models \Sigma \wedge \Pi$ | iff | $[\![\Pi]\!]_s = \mathsf{true}$ and $s, h \models \Sigma$ |
| $s, h \models \exists x.\, F$ | iff | $\exists v \in \texttt{Val} : [s|x{:}v], h \models F$ |

FIGURE 6.6: Semantics of Symbolic-heap Separation Logic formulas

### 6.3.2 Structural Induction Principle for Separation Logic Entailment Proof

We demonstrate our learning-from-failure approach on a standard *structural induction* entailment proof system. In the following, we briefly summarize its fundamental background. The detailed explanation can be found in Chapter 5, Section 5.3.

**Definition 6.1 (Substructural relation [Ta et al., 2018])** *A heap predicate* $\mathsf{P}_1(\vec{v})$ *is said to be a* substructure *of* $\mathsf{P}_2(\vec{u})$*, denoted by* $\mathsf{P}_1(\vec{v}) \prec \mathsf{P}_2(\vec{u})$*, if* $\mathsf{P}_1(\vec{v})$ *occurs in a formula obtained from* directly unfolding $\mathsf{P}_2(\vec{u})$ *or from* unfolding any substructure *of* $\mathsf{P}_2(\vec{u})$*. These conditions are formally stated as follows:*

1. $\exists \vec{w}, \Sigma, \Pi, F(\vec{u}).\ (F(\vec{u}) \cong \exists \vec{w}.(\mathsf{P}_1(\vec{v}) * \Sigma \wedge \Pi)\ \wedge\ F(\vec{u}) \overset{\text{def}}{\Rightarrow} \mathsf{P}_2(\vec{u}))$
2. $\exists \vec{w}, \Sigma, \Pi, F(\vec{t}), \mathsf{P}'(\vec{t}).\ (F(\vec{t}) \cong \exists \vec{w}.(\mathsf{P}_1(\vec{v}) * \Sigma \wedge \Pi)\ \wedge\ F(\vec{t}) \overset{\text{def}}{\Rightarrow} \mathsf{P}'(\vec{t})\ \wedge\ \mathsf{P}'(\vec{t}) \prec \mathsf{P}_2(\vec{u}))$

**Theorem 6.1 (Well-foundedness [Ta et al., 2018])** *Given a satisfiable inductive heap predicate* $\mathsf{P}_1(\vec{u}_1)$*, that is* $\mathsf{P}_1(\vec{u}_1) \not\equiv \mathsf{false}$*. Under the least fixed point semantics of inductive heap predicates, there is no infinite chain like* $... \prec \mathsf{P}_n(\vec{u}_n) \prec ... \prec \mathsf{P}_1(\vec{u}_1)$*.*

**Theorem 6.2 (Structural induction[Ta et al., 2018])** *The entailment* $E \triangleq F_1 * \mathsf{P}(\vec{u}) \vdash F_2$ *is valid, if for all heap predicate* $\mathsf{P}(\vec{u})$*, the fact that* $E$ *holds for all sub-structure predicates* $\mathsf{P}(\vec{v})$ *of* $\mathsf{P}(\vec{u})$ *implies that* $E$ *also holds for* $\mathsf{P}(\vec{u})$*. Formally:*

$$\frac{\forall \mathsf{P}(\vec{u}).\ (\forall \mathsf{P}(\vec{v}) \prec \mathsf{P}(\vec{u}).\ \models E(\mathsf{P}(\vec{v}))) \quad \rightarrow \quad \models E(\mathsf{P}(\vec{u}))}{\forall \mathsf{P}(\vec{u}).\ E(\mathsf{P}(\vec{u}))}$$

## 6.4 The Standard Structural Induction Proof System

In this section, we present a standard structural induction proof system for proving separation logic entailments. It includes a set of logical rules (Subsection 6.4.1) and a set of induction rules (Subsection 6.4.2). This proof system is inspired by separation logic literature [Ta et al., 2016; Ta et al., 2018]. We will utilize it to discuss the induction proof's failure and present our main contribution, a novel proof system with the *learning-from-failure* ability, in the next Section 6.6.

Each inference rule contains zero or more premises, a conclusion, and possibly a side condition. The premises and the conclusion are of the form $\mathcal{H},\ F_1 \vdash F_2$, where $\mathcal{H}$ is a set of IHs and $F_1 \vdash F_2$ is the goal entailment. We write $\mathtt{FV}(F)$ to denote a list of free variables of $F$. Given a symbolic-heap formula $F \triangleq \exists \vec{x}.\ \Sigma \wedge \Pi$, a spatial formula $\Sigma'$ and a pure formula $\Pi'$ such that $\mathtt{FV}(\Sigma') \cap \vec{x} = \varnothing$ and $\mathtt{FV}(\Pi') \cap \vec{x} = \varnothing$, we define $F * \Sigma' \triangleq \exists \vec{x}.\ \Sigma * \Sigma' \wedge \Pi$ and $F \wedge \Pi' \triangleq \exists \vec{x}.\ \Sigma \wedge \Pi \wedge \Pi'$. Moreover $\vec{u} = \vec{v}$ denotes the constraint $(u_1 = v_1) \wedge ... \wedge (u_n = v_n)$, given that $\vec{u} \triangleq u_1, ..., u_n$, $\vec{v} \triangleq v_1, ..., v_n$; and $\vec{x} \# \vec{y}$ indicates that $\vec{x}$ and $\vec{y}$ are disjoint. Finally, $F[e/x]$ is a formula obtained from $F$ by substituting $e$ for all occurrences of the free variable $x$ in $F$.

### 6.4.1 Logical Rules

$$\bot^2_{\mathrm{L}}\ \frac{}{\mathcal{H},\ F_1 * u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{t} \vdash F_2} \qquad \bot^1_{\mathrm{L}}\ \frac{}{\mathcal{H},\ F_1 \wedge \Pi_1 \vdash F_2}\ \Pi_1 \rightarrow \mathsf{false}$$

$$\vdash_{\Pi}\ \frac{}{\mathcal{H},\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2 \qquad =_{\mathrm{L}}\ \frac{\mathcal{H},\ F_1[u/v] \vdash F_2[u/v]}{\mathcal{H},\ F_1 \wedge u{=}v \vdash F_2}$$

$$\exists_{\mathrm{L}}\ \frac{\mathcal{H},\ F_1[u/v] \vdash F_2}{\mathcal{H},\ \exists v.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2) \qquad \exists_{\mathrm{R}}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}.F_2[u/v]}{\mathcal{H},\ F_1 \vdash \exists \vec{x}, v.(F_2 \wedge u{=}v)}$$

$$*{\mapsto}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})}{\mathcal{H},\ F_1 * u \overset{\iota}{\mapsto} \vec{v} \vdash \exists \vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})}\ u, \vec{v} \# \vec{x} \qquad *\mathsf{P}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(F_2 \wedge \vec{u}{=}\vec{v})}{\mathcal{H},\ F_1 * \mathsf{P}(\vec{u}) \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{v}))}\ \vec{u} \# \vec{x}$$

$$\mathsf{P}_{\mathrm{L}}\ \frac{\mathcal{H},\ F_1 * \mathsf{P}(\vec{u}) \vdash F_2}{\mathcal{H},\ F_1 * \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \Sigma_1 \wedge \Pi_1 \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u}) \qquad \mathsf{E}_{\mathrm{L}}\ \frac{\mathcal{H},\ F_1 \vdash F_2}{\mathcal{H},\ F_1 * \mathsf{emp} \vdash F_2} \qquad \mathsf{E}_{\mathrm{R}}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}.F_2}{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{emp})}$$

$$\mathsf{P}_{\mathrm{R}}\ \frac{\mathcal{H},\ F_1 \vdash \exists \vec{x}\vec{y}.(F_2 * \Sigma_2 \wedge \Pi_2)}{\mathcal{H},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{u}))}\ \Sigma_2 \wedge \Pi_2 \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u});\ \vec{y} = \mathtt{FV}(\Sigma_2) \cup \mathtt{FV}(\Pi_2) \setminus \mathtt{FV}(\mathsf{P}(\vec{u}))$$

FIGURE 6.7: Logical rules

The set of logical rules in Figure 6.7 consists of:

- *Axiom rules* $\vdash_{\Pi}, \bot^2_{\mathrm{L}}, \bot^1_{\mathrm{L}}$. These rules prove pure entailments by invoking off-the-shelf provers such as Z3 [Moura et al., 2008] ($\vdash_{\Pi}$), or prove entailments by showing that their antecedents are inconsistent, i.e., they contain overlaid singleton heaps ($\bot^2_{\mathrm{L}}$) or a contradiction ($\bot^1_{\mathrm{L}}$).
- *Normalization rules* $\exists_{\mathrm{L}}, \exists_{\mathrm{R}}, =_{\mathrm{L}}, \mathsf{E}_{\mathrm{L}}, \mathsf{E}_{\mathrm{R}}$. These rules simplify the goal entailment by either eliminating existentially quantified variables ($\exists_{\mathrm{L}}, \exists_{\mathrm{R}}$), or removing equalities ($=_{\mathrm{L}}$) or empty heap predicates ($\mathsf{E}_{\mathrm{L}}, \mathsf{E}_{\mathrm{R}}$) from the entailment's antecedents or consequents.
- *Folding and unfolding rules* $\mathsf{P}_{\mathrm{L}}, \mathsf{P}_{\mathrm{R}}$. These rules derive new entailments by folding a heap predicate's definition case in the goal entailment's antecedent ($\mathsf{P}_{\mathrm{L}}$) or unfolding a heap predicate in the consequent by its definition ($\mathsf{P}_{\mathrm{R}}$).

– *Matching rules* $*\mapsto, *\mathsf{P}$. These rules remove *identical* singleton ($*\mapsto$) or inductive ($*\mathsf{P}$) heap predicates from two sides of the goal entailment.The identicalness of these predicates is ensured by equality constraints added into the derived entailments' consequents.

### 6.4.2 Induction Rules

$$\mathsf{ID}\ \frac{\mathcal{H}',\ \Sigma_1 * F_1^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ \Sigma_1 * F_m^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\ \dagger$$

$\dagger : \mathsf{P}(\vec{u}) \stackrel{\text{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u})$ and $\mathcal{H}' \triangleq \mathcal{H} \cup \{H\}$,
where $H$ is obtained by renaming variables of $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$

$$\mathsf{IH}\ \frac{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\ \mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v}); \dagger$$

$\dagger :\ \exists \theta, \Sigma. ((\mathsf{P}(\vec{u}) \cong \mathsf{P}(\vec{v})\theta)\ \wedge\ (\Sigma_1 \cong \Sigma_3\theta * \Sigma)\ \wedge\ (\Pi_1 \rightarrow \Pi_3\theta))$

FIGURE 6.8: Induction rules

The structural induction principle is implemented into our proof system via the two induction rules ID and IH (Figure 6.8). They are explained in details as follows:

– *Induction rule* ID. This rule performs structural induction on an inductive heap predicate $\mathsf{P}(\vec{u})$ in the goal entailment's antecedent. The goal entailment is first recorded as an IH and is stored in the IHs set $\mathcal{H}'$. Then, the predicate $\mathsf{P}(\vec{u})$ is unfolded by its inductive definition to derive new sub-goal entailments as depicted in this rule's premises.

– *Induction hypothesis application rule* IH. This rule applies an IH $H \triangleq \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4$ to prove a goal entailment $E \triangleq \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$. It first checks whether $\mathsf{P}(\vec{u})$ is a substructure of $\mathsf{P}(\vec{v})$, i.e., $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v})$, according to the structural induction principle. In practice, this structural relation can be examined by maintaining an ancestor record of each predicate, and check if $\mathsf{P}(\vec{v})$ appears in the record of $\mathsf{P}(\vec{u})$. Afterward, this rule performs two steps:

+ *Unification step:* identify a substitution $\theta$ and a spatial formula $\Sigma$ which unify the goal entailment's and the IH' antecedents, that is, $(\mathsf{P}(\vec{u}) \cong \mathsf{P}(\vec{v})\theta) \wedge (\Sigma_1 \cong \Sigma_3\theta * \Sigma)$, i.e., *spatial unification* and $\Pi_1 \rightarrow \Pi_3\theta$, i.e., *pure unification*. If such $\theta, \Sigma$ exist, then $E' \triangleq \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1$ is valid.

+ *Proving step:* derive a sub-goal entailment $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$. This derivation can be explained as follows. By Theorem 2.2, the IH $H$ implies that $H\theta \triangleq \Sigma_3\theta * \mathsf{P}(\vec{v})\theta \wedge \Pi_3\theta \vdash F_4\theta$ is valid. From $E'$ and $H\theta$, we have a derivation chain: $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash \Sigma_3\theta * \mathsf{P}'(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma \wedge \Pi_1$. Therefore, if the sub-goal $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$ can be proved, then the goal entailment $E$ is valid.

### 6.4.3 The Standard Proof Search Procedure

**Procedure** $\mathsf{Prove}(\mathcal{H}, F_1 \vdash F_2)$

**Input:** $\mathcal{H}$ a set of IHs, $F_1 \vdash F_2$ is a goal entailment

**Output:** The validity (`Valid`, `Unkn`) of $F_1 \vdash F_2$

1: $\mathcal{R} \leftarrow \mathsf{FindRules}(\mathcal{H}, F_1 \vdash F_2)$
2: **for each** $R$ **in** $\mathcal{R}$ **do**
3: **if** $R \in \{\vdash_{\Pi}, \bot^2_{\mathrm{L}}, \bot^1_{\mathrm{L}}\}$ **then return** `Valid`
4: $(\mathcal{H}_i, F_{1i} \vdash F_{2i})_{i=1...n} \leftarrow \mathsf{Premises}(R)$
5: $(res_i)_{i=1...n} \leftarrow \mathsf{Prove}(\mathcal{H}_i, F_{1i} \vdash F_{2i})_{i=1...n}$
6: **if** $\forall i{=}1...n.(res_i{=}\texttt{Valid})$ **then return** `Valid`
7: **return** `Unkn`

FIGURE 6.9: The standard proof search procedure

Figure 6.9 introduces the standard proof search procedure Prove, which is implemented in a recursive fashion. Its input includes an IH set $\mathcal{H}$ and an entailment $F_1 \vdash F_2$, which correlate to a conclusion or a premise of an inference rule. Its output is the entailment's validity (`Valid` or `Unkn`). When Prove is invoked for the first time to prove an original goal entailment, $\mathcal{H}$ is set to empty ($\varnothing$).

Given the input entailment $F_1 \vdash F_2$, Prove first finds from all inference rules presented in Figures 6.7 and 6.8 a set of rules $\mathcal{R}$ whose conclusion can be unified with the goal entailment (line 1). It subsequently applies the selected rules in $\mathcal{R}$ to prove $F_1 \vdash F_2$. In particular, Prove immediately returns a valid result (`Valid`), if the selected rule $R$ is an axiom rule, which does not derive any new sub-goal entailment (line 3). Otherwise, it searches for the proof of the sub-goal entailments (lines 4, 5), and returns a valid result (`Valid`) if all of these sub-goals are proved (line 6). The procedure Prove returns an unknown result (`Unkn`) if all rules fail to prove the goal entailment $F_1 \vdash F_2$ (line 7).

## 6.5 How Can an Induction Proof Fail?

We have illustrated in Section 6.2 that traditionally cyclic-based [Brotherston et al., 2011a; Le et al., 2016; Le et al., 2017] and induction-based [Chu et al., 2015; Ta et al., 2016] approaches fail to prove the motivating entailments $E_1, E_2, E_3$. More specifically, these systems could not apply any induction hypotheses to prove the derived sub-goal entailments. In this section, we will formally elaborate the failure of the induction hypothesis application on the structural induction system proposed in Section 6.4. Note that similar explanations can be applied to cyclic-based and induction-based proofs in [Brotherston et al., 2011a; Le et al., 2016; Le et al., 2017; Chu et al., 2015; Ta et al., 2016].

We consider a general case when the following IH $H$ is applied to prove an entailment $E$, which is obtained in a proof derivation that inductively unfolds a predicate $\mathsf{P}(\vec{v})$ in $H$'s antecedent. In addition, the antecedent of $E$ contains a predicate $\mathsf{P}(\vec{u})$, which is a substructure of $\mathsf{P}(\vec{v})$, i.e., $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v})$:

$$H \triangleq \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4 \qquad\qquad E \triangleq \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$$

Then, the induction hypothesis application rule IH needs to perform two unification steps: *spatial unification* and *pure unification*, as described in the rule IH in Section 6.4.2. If one of the above steps fails, the IH $H$ cannot be applied to prove the candidate entailment $E$. We will explain this failure as follows.

### 6.5.1 The Failure of the Induction Hypothesis's Spatial Unification

When unifying the spatial formula $\Sigma_3 * \mathsf{P}(\vec{v})$ of $H$'s antecedent with the spatial formula $\Sigma_1 * \mathsf{P}(\vec{u})$ of $E$'s antecedent, where $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v})$, the rule IH needs to find a substitution $\theta$ and a spatial $\Sigma$ such that $\mathsf{P}(\vec{u}) \cong \mathsf{P}(\vec{v})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$, as stated in IH's side condition. This spatial unification can be performed by two steps:

1. Split $\Sigma_1$ into two components $\Sigma$ and $\Sigma'$, (i.e., $\Sigma_1 \cong \Sigma * \Sigma'$) such that $\Sigma'$ and $\Sigma_3$ have the same multiset of predicate symbols.
2. If step (1) succeeds, then find a substitution $\theta$ which can map $\mathsf{P}(\vec{v})$ and $\Sigma_3$ into $\mathsf{P}(\vec{u})$ and $\Sigma'$, that is, $\mathsf{P}(\vec{v})\theta \cong \mathsf{P}(\vec{u})$ and $\Sigma_3\theta \cong \Sigma'$. If such $\theta$ exist, then clearly $\mathsf{P}(\vec{u}) \cong \mathsf{P}(\vec{v})\theta$ and $\Sigma_1 \cong \Sigma_3\theta * \Sigma$.

If both the above two steps succeed, then the pair $\theta, \Sigma$ is called a *spatial unification* (Definition 6.2), which maps $\Sigma_3 * \mathsf{P}(\vec{v})$ into a sub-formula of $\Sigma_1 * \mathsf{P}(\vec{u})$. However, if one of the two steps above fails, then it is evident that the antecedent of $H$ cannot be unified with the antecedent of $E$.

**Definition 6.2 (Spatial Unification)** *Given two spatial formulas $\Sigma_\mathsf{h}$ and $\Sigma_\mathsf{e}$, then the pair $\theta, \Sigma$ is called a spatial unification that maps $\Sigma_\mathsf{h}$ into a substructure of $\Sigma_\mathsf{e}$ iff $\Sigma_\mathsf{e} \cong \Sigma_\mathsf{h}\theta * \Sigma$*

Here, we consider the failure at step (2), when there is no substitution which can map $\mathsf{P}(\vec{v})$ and $\Sigma_3$ into $\mathsf{P}(\vec{u})$ and $\Sigma'$, respectively. Here, the spatial formulas $\mathsf{P}(\vec{v})$ and $\Sigma_3$ in $H$'s antecedent may contain *non-unifiable* variables or constants which cannot be mapped to expressions of $\mathsf{P}(\vec{u})$ and $\Sigma'$ in $E$'s antecedent. We will formally *define* these non-unifiable variables and constants in Section 6.5.1.1 and present a unification algorithm to *identify* them in 6.5.1.2. These variables and constants can be *generalized* by fresh variables to obtain more potential IHs to increase the success chance of the induction proof.

Note that we do not consider the failure at step (1) because in this case, the antecedent of $H$ might contain a different heap predicate symbol with the antecedent of $E$. Consequently, the *non-unifiable variables* or the *non-unifiable constants* of $H$'s antecedents could *never* be generalized to obtain the potential IH $H'$.

#### 6.5.1.1 The Non-Unifiability of Constants and Variables

As discussed earlier, given that $\mathsf{P}(\vec{u})$ and $\Sigma'$ respectively contains the same multiset of heap predicate symbols with $\mathsf{P}(\vec{v})$ and $\Sigma_3$, the induction hypothesis application rule IH needs to find a substitution $\theta$ such that $\mathsf{P}(\vec{v})\theta \cong \mathsf{P}(\vec{u})$ and $\Sigma_3\theta \cong \Sigma'$. However, such substitution $\theta$ does not exist if $\mathsf{P}(\vec{v})$ and $\Sigma_3$ contains a constant $c$ which can be mapped into a different expression $e$ (Definition 6.4), or a variable $x$ which can be mapped into two different expressions $e_1, e_2$ (Definition 6.3).

**Definition 6.3 (Non-unifiable constant)** *An integer or spatial constant $c$ is said to be* non-unifiable *if it is mapped to a different expression $t$, i.e., $(t{\neq}c)$, by a unification algorithm.*

**Definition 6.4 (Non-unifiable variable)** *A variable $x$ is said to be* non-unifiable *if it is mapped to at least two different expressions $t_1$ and $t_2$, i.e., $t_1{\neq}t_2$, by a unification algorithm.*

For example, in Section 6.2.2, when applying $H_2$ to prove $E_2'$, the constant $100$ is *non-unifiable*, because it is mapped to $99$ when unifying $\mathsf{ls}(a, \mathsf{nil}, 100)$ of $H_2$ and $\mathsf{ls}(u, \mathsf{nil}, 99)$ of $E_2'$. Moreover, when applying $H$ to prove $E_2''$, the variable $b$ is also *non-unifiable* because it is mapped to $u$ when unifying $\mathsf{lsrev}(a, b, l)$ of $H$ and $\mathsf{lsrev}(x, u, n{-}1)$ of $E_2''$, whereas it is also mapped to $y$ while unifying $b{\mapsto}b$ of $H$ with $u{\mapsto}y$ or $y{\mapsto}y$ of $E_2''$.

$$
\begin{aligned}
H_2 &\triangleq \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsrev}(a, b, l) * b{\mapsto}b \vdash \mathsf{false}\\
E_2' &\triangleq x{\mapsto}u * \mathsf{ls}(u, \mathsf{nil}, 99) * \mathsf{lsrev}(x, y, n) * y{\mapsto}y \vdash \mathsf{false}\\
E_2'' &\triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x, u, n{-}1) * u{\mapsto}y * y{\mapsto}y \vdash \mathsf{false}
\end{aligned}
$$

Here, we can generalize the *non-unifiable constant* $100$ or the *non-unifiable variable* $b$ by a fresh variable to obtain are more general IH. For example, we can choose to generalize the third variable $b$ in $H_2$ (indicated by $b^{\dagger}$). Since this variable $b$ corresponds to the third variable $y$ in $E_2$ (indicated by $y^{\ddagger}$), we can replace it by a fresh variable $z$ to obtain the new entailment $\overline{E}_2$, which can be easily proved by induction proof (Section 6.2.2).

$$
\begin{aligned}
H_2 &\triangleq \mathsf{ls}(a, \mathsf{nil}, 100) * \mathsf{lsrev}(a, b, l) * b{\mapsto}b^{\dagger} \vdash \mathsf{false}\\
E_2 &\triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x, y, n) * y{\mapsto}y^{\ddagger} \vdash \mathsf{false}\\
\overline{E}_2 &\triangleq \mathsf{ls}(x, \mathsf{nil}, 100) * \mathsf{lsrev}(x, y, n) * y{\mapsto}z \vdash \mathsf{false}
\end{aligned}
$$

In the next section, we present a unification algorithm that unifies two spatial formulas

and returns a set containing not only *successful* but also *failed* unification with the necessary information to identify the non-unifiable constants or variables. This algorithm will be utilized later in Section 6.6 to apply or to generalize an IH.

#### 6.5.1.2 The Unification Algorithm for Spatial Formulas

In our logic fragment, a spatial expression can be either a constant or a variable, whereas an integer expression (linear arithmetic) can be an addition, subtraction or linear multiplication of other integer expressions. To make the unification algorithm for spatial formulas more efficient, we always keep integer expressions in *canonical form* [Harrison, 2009], where common variables are grouped by a non-zero variable coefficient, and the base coefficient always presents even if it is 0. In addition, when unifying two integer expressions, we recursively unify each variable of one expression with a *sub-expression* of the other expression. The canonical form and sub-expression are respectively defined in Definitions 6.5 and 6.6.

**Definition 6.5 (Canonical integer expression [Harrison, 2009])** *An expression $e$ is said to be in canonical form if $e = k_0 + k_1 \cdot x_1 + ... + k_n \cdot x_n$ where $k_0, k_1, ..., k_n$ are integer constants, $k_0$ is called base coefficient, $k_1, ..., k_n$ are called variable coefficients $(k_1, ..., k_n \neq 0)$, and $x_1, ..., x_n$ are distinct variables.*

**Definition 6.6 (Integer sub-expression)** *Given a canonical integer expression $e = k_0 + k_1 \cdot x_1 + ... + k_n \cdot x_n$, then $e'$ is said to be a sub-expression of $e$ if $e' = k_{i_1} \cdot x_{i_1} + ... + k_{i_j} \cdot x_{i_j}$ or $e' = k_0 + k_{i_1} \cdot x_{i_1} + ... + k_{i_j} \cdot x_{i_j}$, where $j \geq 0$ and $1 \leq i_1, ..., i_j \leq n$ (if $j=0$, then $e'=0$ or $e'=k_0$).*

Figure 6.10 presents our unification algorithm UnifySpatial, which is implemented in a recursive fashion. Its inputs include two spatial formulas $\Sigma_{\mathsf{h}}, \Sigma_{\mathsf{e}}$, which are respectively from an IH's and a goal entailment's antecedents, and a set of existing mappings $\mathrm{M}$. Each mapping $m \in \mathrm{M}$ is a set of elements of the form $\mathtt{Var}\langle v, t_v, pos_v \rangle$ or $\mathtt{Const}\langle c, t_c, pos_c \rangle$ which respectively maps variable $v$ or a constant $c$ to an expression $t_v$ or $t_c$. We also record the position of $v$ and $c$ in the original IH by two data structures $pos_v$ and $pos_c$, which will be

utilized by the proof search procedure to identify and generalize $v$ or $c$ if they are *non-unifiable*. When UnifySpatial is invoked for the first time, the mapping $\mathrm{M}$ is set as empty $\varnothing$. The algorithm will return a set containing both successful unification $\theta, \Sigma$, which satisfies $\Sigma_{\mathsf{h}}\theta * \Sigma \cong \Sigma_{\mathsf{e}}$, and failed mapping $m$, which contains the *non-unifiable* variables or constants.

**Procedure** UnifySpatial$(\Sigma_{\mathsf{h}}, \Sigma_{\mathsf{e}}, \mathrm{M})$

**Input:** $\Sigma_{\mathsf{h}}, \Sigma_{\mathsf{e}}$ are two heap formulas, $\mathrm{M}$ is a set of mappings.

**Output:** A set of successful or failed unifications.

1: $\mathcal{U} \leftarrow \varnothing$
2: **if** $\Sigma_{\mathsf{h}} \not\equiv \mathsf{emp}$ **then**
3: $\quad \sigma_{\mathsf{h}}, \Sigma'_{\mathsf{h}} \leftarrow \mathsf{SplitOneHeapAtom}(\Sigma_{\mathsf{h}})$ // $\sigma_{\mathsf{h}} * \Sigma'_{\mathsf{h}} \equiv \Sigma_{\mathsf{h}}$
4: $\quad \mathcal{A} \leftarrow \{\sigma_{\mathsf{e}} \mid \sigma_{\mathsf{e}} \in \mathsf{HeapAtoms}(\Sigma_{\mathsf{e}}) \wedge \mathsf{SameHeapSymbol}(\sigma_{\mathsf{e}}, \sigma_{\mathsf{h}})\}$
5: $\quad$ **for each** $\sigma_{\mathsf{e}}$ **in** $\mathcal{A}$ **do**
6: $\quad\quad \mathrm{M}' \leftarrow \mathrm{M}$
7: $\quad\quad (t_h^1, ..., t_h^n), (t_e^1, ..., t_e^n) \leftarrow \mathsf{Parameters}(\sigma_{\mathsf{h}}), \mathsf{Parameters}(\sigma_{\mathsf{e}})$
8: $\quad\quad$ **for each** $i$ **in** $1, ..., n$ **do** $\mathrm{M}' \leftarrow \mathsf{UnifyExpr}(t_h^i, t_e^i, \mathrm{M}')$
9: $\quad\quad \Sigma'_{\mathsf{e}} \leftarrow \mathsf{RemoveHeapAtom}(\Sigma_{\mathsf{e}}, \sigma_{\mathsf{e}})$ // $\sigma_{\mathsf{e}} * \Sigma'_{\mathsf{e}} \equiv \Sigma_{\mathsf{e}}$
10: $\quad\quad \mathcal{U} \leftarrow \mathcal{U} \cup \mathsf{UnifySpatial}(\Sigma'_{\mathsf{h}}, \Sigma'_{\mathsf{e}}, \mathrm{M}')$
11: **else** // $\Sigma_{\mathsf{h}} \equiv \mathsf{emp}$
12: $\quad$ **for each** $m$ **in** $\mathrm{M}$ **do**
13: $\quad\quad$ **if** $\exists\, \mathtt{Const}\langle c, t, _\rangle \in m$ **where** $c \neq t$ **then**
14: $\quad\quad\quad \mathcal{U} \leftarrow \mathcal{U} \cup \{\mathtt{Fail}\langle m\rangle\}$
15: $\quad\quad$ **else if** $\exists\, \mathtt{Var}\langle x, t, _\rangle, \mathtt{Var}\langle x, t', _\rangle \in m$ **where** $t \neq t'$ **then**
16: $\quad\quad\quad \mathcal{U} \leftarrow \mathcal{U} \cup \{\mathtt{Fail}\langle m\rangle\}$
17: $\quad\quad$ **else**
18: $\quad\quad\quad \theta \leftarrow [x/t \mid \mathtt{Var}\langle x, t, _\rangle \in m]$ // *get all substitutions from* $m$
19: $\quad\quad\quad \mathcal{U} \leftarrow \mathcal{U} \cup \{\mathtt{Success}\langle \theta, \Sigma_{\mathsf{e}}\rangle\}$
20: **return** $\mathcal{U}$

FIGURE 6.10: The unification algorithm for spatial formulas.

Initially, if the input formula $\Sigma_{\mathsf{h}}$ is non-empty (line 2), UnifySpatial first extracts from $\Sigma_{\mathsf{h}}$ an atomic heap formula $\sigma_{\mathsf{h}}$ (line 3) and finds a set $\mathcal{A}$ of all atomic heap predicates in $\Sigma_{\mathsf{e}}$, which have the same predicate symbol with $\sigma_{\mathsf{h}}$ (line 4) and tries to unify $\sigma_{\mathsf{h}}$ with them (lines 5 – 10). More specifically, for each $\sigma_{\mathsf{e}} \in \mathcal{A}$, the procedure UnifySpatial pairwise

unifies parameters $t_h^1, ..., t_h^n$ of $\sigma_\mathsf{h}$ and $t_e^1, ..., t_e^n$ of $\sigma_\mathsf{e}$ (lines 7 – 8). Here, $\sigma_\mathsf{h}$ and $\sigma_\mathsf{e}$ have the same number of parameters because they are of the same predicate symbol. Each pair of the expressions $t_h^i$ and $t_e^i$ will be unified by the procedure UnifyExpr (line 8). The obtained mapping set $\mathrm{M}$ will be used to unify the remaining spatial formulas $\Sigma'_\mathsf{h}$ and $\Sigma'_\mathsf{e}$, where the final result will be accumulated into a set $\mathcal{U}$ (lines 10, 9).

When the input formula $\Sigma_\mathsf{h}$ is non-empty (line 11), then all spatial atoms of the original IH are unified by the mappings in $\mathrm{M}$. Now, each mapping $m \in \mathrm{M}$ will be evaluated to decide whether it is a successful or a failed mapping. More specifically, a mapping $m$ is considered a failure, if it contains an *non-unifiable constant* $c$ which is mapped to a different expression $t$ such that $c{\neq}t$ (line 13), or an *non-unifiable variable* $x$, which is mapped into two different expressions $t, t'$ such that $t{\neq}t'$ (line 15). The failed mapping $m$ will be updated to the output set $\mathcal{U}$ (lines 14 and 16). If none of the previous conditions satisfies, then $m$ is a successful mapping. Here, the pair of the substitution $\theta$ extracted from $m$ and the spatial formula $\Sigma_\mathsf{e}$ successfully forms a spatial unification $\theta, \Sigma_\mathsf{e}$, which will be accumulated to the output set $\mathcal{U}$ (line 19).

Figure 6.11 presents the auxiliary procedure UnifyExpr that unifies two expressions $t_h$ and $t_e$, given a set of mappings $\mathrm{M}$ discovered from the main unification procedure UnifySpatial. Note that $t_h$ and $t_e$ are respectively parameters of atomic heaps in $\Sigma_\mathsf{h}$ and $\Sigma_\mathsf{e}$. Each mapping $m$ in $\mathrm{M}$ will be simply updated with a single mapping from $t_h$ to $t_e$ when $t_h$ is a constant (lines 3 – 5) or a variable (lines 6 – 8). Otherwise, if $t_h$ is a linear arithmetic expression, each of its variable will be mapped to a sub-expression of $t_e$ (lines 9 – 17). More specifically, both $t_h$ and $t_e$ will be normalized by subtracting them from the base coefficient of $t_h$ (lines 10, 11). By this normalization, $t_h = k_1{\cdot}x_1 + ... + k_n{\cdot}x_n$ with $k_i{\neq}0$. Now, the procedure UnifyExpr can extract a core sub-expression $k{\cdot}x$ from $t_h$, i.e., $k{\cdot}x \in \{k_1{\cdot}x_1, ..., k_n{\cdot}x_n\}$, and iteratively unifies $k{\cdot}x$ with all sub-expressions $t$ of $t_e$ whose coefficients are multiples of $k$ (lines 13 – 17). Note that the *multiple-of-k* condition can be tested by checking whether the great common divisor of all coefficients of $t$ divides $k$ ($\mathsf{CoeffGCD}(t)\ \%\ k = 0$, line 13). Afterward, the procedure UnifyExpr can map $x$ to the quotient $\frac{t}{k}$ of $t$ and $k$ (line 15) and proceeds to unify the remaining expressions $t'_h$ and $t'_e$

and updates the final result to the mapping set $\mathrm{M}'$ (lines 16 – 17).

**Procedure** $\mathsf{UnifyExpr}(t_h, t_e, \mathrm{M})$

**Input:** $t_h, t_e$ are two expressions, $\mathrm{M}$ is a set of substitutions **Output:**

1: $\mathrm{M}' \leftarrow \varnothing$
2: **for each** $m$ **in** $\mathrm{M}$ **do**
3: **if** $\mathsf{IsConstant}(t_h)$ **then**
4: $m' \leftarrow m \cup \{\mathtt{Const}\langle t_h, t_e, \mathsf{pos}(t_h)\rangle\}$
5: $\mathrm{M}' \leftarrow \mathrm{M}' \cup \{m'\}$
6: **else if** $\mathsf{IsVariable}(t_h)$ **then**
7: $m' \leftarrow m \cup \{\mathtt{Var}\langle t_h, t_e, \mathsf{pos}(t_h)\rangle\}$
8: $\mathrm{M}' \leftarrow \mathrm{M}' \cup \{m'\}$
9: **else** // *$t_h, t_e$ are linear expressions*
10: $k_0 \leftarrow \mathsf{GetBaseCoefficient}(t_h)$
11: **if** $k_0 \neq 0$ **then** $t_h,\ t_e \leftarrow (t_h - k_0),\ (t_e - k_0)$
12: $k{\cdot}x,\ t'_h \leftarrow \mathsf{ExtractCoreExpr}(t_h)$ // *$t_h = k{\cdot}x + t'_h,\ x \notin t'_h$*
13: $\mathcal{T} \leftarrow \{t \mid t \in \mathsf{SubExprs}(t_e) \wedge \mathsf{CoeffGCD}(t)\ \%\ k = 0\}$
14: **for each** $t$ **in** $\mathcal{T}$ **do**
15: $m' \leftarrow m \cup \{\mathtt{Var}\langle x,\ \frac{t}{k},\ \mathsf{pos}(t_h)\rangle\}$ // *unify $k{\cdot}x$ with $t$*
16: $t'_e \leftarrow t_e - t$
17: $\mathrm{M}' \leftarrow \mathrm{M}' \cup \mathsf{UnifyExpr}(t'_h, t'_e, \{m'\})$
18: **return** $\mathrm{M}'$

FIGURE 6.11: The unification algorithm for expressions.

### 6.5.2 The Failure of the Induction Hypothesis's Pure Unification

Now, suppose that the two spatial formulas $\Sigma_3 * \mathsf{P}(\vec{v})$ and $\Sigma_1 * \mathsf{P}(\vec{u})$ of $H$'s and $E$'s antecedents can be unified by a unification $\theta, \Sigma$. Then, the rule IH also needs to prove a side condition $\Pi_1 \rightarrow \Pi_3\theta$, where $\Pi_1, \Pi_3$ is respectively pure formulas of $E$'s and $H$'s antecedents. If this condition holds, then $\theta$ is called the *pure unification* (Definition 6.7) that maps the pure formula $\Pi_3$ in $H$'s antecedent into the pure formula $\Pi_1$ in $E$'s antecedent.

**Definition 6.7 (Pure Unification)** *Given two pure formulas $\Pi_\mathsf{h}, \Pi_\mathsf{e}$. The substitution $\theta$ is called a* pure unification *that maps $\Pi_\mathsf{h}$ into a substructure of $\Pi_\mathsf{e}$ iff the implication $\Pi_\mathsf{e} \rightarrow \Pi_\mathsf{h}\theta$ holds.*

#### 6.5.2.1 The Non-Unifiability of Pure Formula

If the condition $\Pi_1 \rightarrow \Pi_3\theta$ does not hold, then the main reason of this failure is that $\Pi_3\theta$ contains a *sub-formula* $\Pi$ which cannot be proved by $\Pi_1$. Let $\Pi'$ be a sub-formula $\Pi_3$, which is transformed into $\Pi$ of $\Pi_3\theta$, by the substitution $\theta$, that is, $\Pi'\theta \equiv \Pi$. Then $\Pi'$ is called the *non-unifiable pure formula* of $\Pi_3$, since it is mapped by $\theta$ to a formula $\Pi$, which is unprovable by $\Pi_1$. We formally define the *non-unifiable pure formula* in Definition 6.8.

**Definition 6.8 (Non-unifiable pure formula)** *Given two pure formulas* $\Pi_{\mathsf{e}}, \Pi_{\mathsf{h}}$ *and a substitution* $\theta$*. If the implication* $\Pi_{\mathsf{e}} \rightarrow \Pi_{\mathsf{h}}\theta$ *is* invalid*, then* $\Pi_{\mathsf{u}}$ *is called an non-unifiable pure formula of* $\Pi_{\mathsf{h}}$ *iff* $\Pi_{\mathsf{e}} \rightarrow \Pi_{\mathsf{u}}\theta$ *is* invalid *and there exists* $\Pi$ *such that* $\Pi_{\mathsf{h}} \equiv \Pi_{\mathsf{u}} \wedge \Pi$ *and* $\Pi_{\mathsf{e}} \rightarrow \Pi\theta$ *holds.*

Given that $\theta$ cannot unify the two pure formulas $\Pi_3$ and $\Pi_1$ in $H$'s and $E$'s antecedents, we can either *weaken* the IH $H$ by removing the non-unifiable pure formula $\Pi'$ from $\Pi_3$ to derive a new IH, or *perform a case analysis* on the unproved formula $\Pi$ to derive new sub-goal entailments from $E$. Both the two solutions can increase the success chance of unifying the pure formulas of the (new) IH and the (sub)goal entailments.

For example, in Section 6.2.1, when unifying two pure formulas $k{\geq}100$ and $n{\geq}100$ of $H_1$'s and $E_1'$'s antecedents by the substitution $\theta_1 \triangleq [u/a, y/b, z/c, n{-}1/k]$, the pure formula $k{\geq}100$ is *non-unifiable*. This is because $n{\geq}100$ cannot prove that $n{-}1{\geq}100$, which is obtained by applying $\theta_1$ on $k{\geq}100$.

$$\begin{array}{lcl} H_1 & \triangleq & \mathsf{ls}(a,b,k) * b{\mapsto}c \wedge k{\geq}100 \vdash \mathsf{ls}(a,c,k{+}1) \\ E_1' & \triangleq & x{\mapsto}u * \mathsf{ls}(u,y,n{-}1) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x,z,n{+}1) \end{array}$$

Here, the non-unifiable pure formula $k{\geq}100$ of $H_1$ corresponds to $n{\geq}100$ in $E_1$, since $H_1$ is obtained from $E_1$ by a fresh renaming of all variables. As illustrated in Section 6.2.1, we can weaken the antecedent of $E_1$ by removing $n{\geq}100$ to derive the entailment $\overline{E}_1$. This entailment $\overline{E}_1$ can be easily proved valid by induction. It follows that the original goal entailment $E_1$ can also be concluded valid.

$$\begin{array}{lcl} E_1 & \triangleq & \mathsf{ls}(x,y,n) * y{\mapsto}z \wedge n{\geq}100 \vdash \mathsf{ls}(x,z,n{+}1) \\ \overline{E}_1 & \triangleq & \mathsf{ls}(x,y,n) * y{\mapsto}z \vdash \mathsf{ls}(x,z,n{+}1) \end{array}$$

For another example, in Section 6.2.3, when unifying two pure formulas $a{\neq}b$ and $x{\neq}y \wedge n{\geq}1$ of $H_3$ and $E'_3$ by the substitution $\theta_3 \triangleq [x/a, u/b, z/c, n/k]$, the formula $a{\neq}b$ is *non-unifiable*. This is because when applying $\theta_3$ on $a{\neq}b$ of $H_3$, we obtain $x{\neq}z$, which cannot be proved by $x{\neq}y \wedge n{\geq}1$ in $E'_3$.

$$\begin{array}{lcl} H_3 & \triangleq & \mathsf{lsrev}(a,b,k) * a{\mapsto}c \wedge a{\neq}b \vdash \mathsf{false} \\ E'_3 & \triangleq & \mathsf{lsrev}(x,u,n) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \vdash \mathsf{false} \end{array}$$

Different from the previous example, here we can perform a case analysis on the antecedent of $E'_3$ by the unproved formula $x{\neq}z$ to derive two entailments $E'_{31}$ and $E'_{32}$. As illustrated in Section 6.2.3, the IH $H_3$ now can be used to prove $E'_{31}$, whereas $E'_{32}$ can be prove normally using standard separation logic inference rules.

$$\begin{array}{lcl} E'_{31} & \triangleq & \mathsf{lsrev}(x,u,n) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \wedge x{\neq}u \vdash \mathsf{false} \\ E'_{32} & \triangleq & \mathsf{lsrev}(x,u,n) * u{\mapsto}y * x{\mapsto}z \wedge x{\neq}y \wedge n{\geq}1 \wedge x{=}u \vdash \mathsf{false} \end{array}$$

In the next section, we present a unification algorithm that can examine if a given substitution can successfully unify two pure formulas, and can identify the *non-unifiable pure formula* when the unification fails. This algorithm will be utilized later in Section 6.6 to weaken an IH or to perform a case analysis on the unproved formula to derive new sub-goal entailments.

#### 6.5.2.2 The Unification Algorithm for Pure Formulas

Figure 6.12 presents the unification algorithm UnifyPure. Its inputs include two formulas $\Pi_{\mathsf{e}}$ and $\Pi_{\mathsf{h}}$, which are respectively pure formulas of an IH's and a goal entailment's antecedents, and a substitution $\theta$. The procedure UnifyPure returns `Success` if $\theta$ can unify $\Pi_{\mathsf{h}}$ into $\Pi_{\mathsf{e}}$. Otherwise, it outputs `Fail`$\langle\Pi_{\mathsf{u}}\rangle$ to indicate that these two formulas cannot be unified, and $\Pi_{\mathsf{u}}$ is the non-unifiable pure formula of $\Pi_{\mathsf{h}}$.

Initially, the procedure UnifyPure invokes an off-the-self prover like Z3 [Moura et al., 2008] to prove the implication $\Pi_{\mathsf{e}} \rightarrow \Pi_{\mathsf{h}}\theta$ (line 1). It returns `Success` if the implication holds (line 2). Otherwise, it simply transforms $\Pi_{\mathsf{h}}\theta$ into a CNF form (line 3), and iteratively examines whether each sub-formula $\Pi_i$ of the CNF-transformed $\Pi_{\mathsf{h}}\theta$ is non-unifiable.

More specifically, if the implication $\Pi_e \rightarrow \Pi_i$ is invalid (line 6), UnifyPure updates $\Pi_i$ into an accumulating variable $\Pi_u$, which captures the final non-unifiable pure formula (line 7). For optimization purpose, only $\Pi_i$, which is not subsumed by the existing value of $\Pi_u$, is collected during the iteration (line 6). Finally, UnifyPure returns $\texttt{Fail}\langle\Pi_u\rangle$ to indicate an unsuccessful unification, where $\Pi_u$ is the non-unifiable pure formula (line 8).

**Procedure** UnifyPure($\Pi_h, \Pi_e, \theta$)

**Input:** $\Pi_e, \Pi_h$ are two pure formulas, $\theta$ is a substitution

**Output:** `Success` or $\texttt{Fail}\langle\Pi_u\rangle$ ($\Pi_u$ is the non-unifiable formula).

1: **if** IsValid($\Pi_e \rightarrow \Pi_h\theta$) **then**
2: **return** `Success`
3: $\Pi_1 \wedge ... \wedge \Pi_n \leftarrow$ CNF($\Pi_h\theta$) // *transform* $\Pi_h\theta$ *to CNF form*
4: $\Pi_u \leftarrow$ true
5: **for each** $i$ **in** $1, ..., n$ **do**
6: **if** IsInvalid($\Pi_e \rightarrow \Pi_i$) **and** IsInvalid($\Pi_u \rightarrow \Pi_i$) **then**
7: $\Pi_u \leftarrow \Pi_u \wedge \Pi_i$
8: **return** $\texttt{Fail}\langle\Pi_u\rangle$

FIGURE 6.12: The unification algorithm for pure formulas.

## 6.6 The Failure Learning Framework

We are now ready to present our learning-from-failure framework. It includes a set of new inference rules in Section 6.6.1 and a proof search procedure ProveLFF in Section 6.6.2. More specifically, when proving a goal entailment, the procedure ProveLFF can find not only the applicable inference rules from all logical and induction inference rules in Section 6.4, but also the non-unifiable constants, variables, or pure formulas when it fails to apply an IH. In the later case, ProveLFF will exploit these non-unifiable expressions and formulas to find the applicable inference rules from all rules in Section 6.6.1 to derive new sub-goal entailments or generalize the original goal corresponding to the inapplicable IH.

### 6.6.1 Learning from Failure Rules

Figure 6.13 presents our new inference rules, which can exploit the non-unifiable constants, variables, or pure formulas in the failures of the IH applications. They include:

- Generalization rules $G_{\mathrm{v}}, G_{\mathrm{c}}$: This rule generalizes an non-unifiable variable ($G_{\mathrm{v}}$) or an non-unifiable constant ($G_{\mathrm{c}}$) by a fresh variable to introduce a new entailment.
- Weakening rule $W_{\pi}$: This rule weakens the antecedent of an entailment by removing an non-unifiable pure formula to derive a new entailment.
- Case analysis rule CA: this rule performing a case analysis on the non-unifiable pure formula to derive new sub-goal entailments.

$$G_{\mathrm{v}}\ \frac{\mathcal{H},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_1, F_2) \qquad G_{\mathrm{c}}\ \frac{\mathcal{H},\ F_1[u/c] \vdash F_2}{\mathcal{H},\ F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_1, F_2)$$

$$W_{\pi}\ \frac{\mathcal{H},\ F_1 \vdash F_2}{\mathcal{H},\ F_1 \wedge \Pi \vdash F_2} \qquad \mathsf{CA}\ \frac{\mathcal{H},\ F_1 \wedge \Pi \vdash F_2 \qquad \mathcal{H},\ F_1 \wedge \neg\Pi \vdash F_2}{\mathcal{H},\ F_1 \vdash F_2}$$

(where $x, c$, and $\Pi$ in $G_{\mathrm{v}}, G_{\mathrm{c}}$, and $W_{\pi}$, CA are the non-unifiable variable, constant, and pure formula, discovered by the proof system.)

FIGURE 6.13: Learning from failure rules.

### 6.6.2 The Learning From Failure Proof Search Procedure

Figure 6.14 presents the proof search procedure ProveLFF, which has the ability to learn from the failure of the IH application. Similar to the standard proof search in Section 6.4, this procedure takes the input of an IH set $\mathcal{H}$, a goal entailment $F_1 \vdash F_2$ and returns two values `Valid` or `Unkn` to indicate whether the entailment $F_1 \vdash F_2$ can be proved. In addition, ProveLFF can return also a special output `ValidAncestor`$\langle G_1 \vdash G_2 \rangle$ to indicate that the *ancestor* entailment $G_1 \vdash G_2$ of $F_1 \vdash F_2$ is valid. Here $F_1 \vdash F_2$ is derived from $G_1 \vdash G_2$ during the proof derivation.

**Procedure** ProveLFF($\mathcal{H}, F_1 \vdash F_2$)

**Input:** $\mathcal{H}$ a set of IHs, $F_1 \vdash F_2$ is a goal entailment.

**Output:** The validity result of $F_1 \vdash F_2$ (`Valid`, `Unkn`) or of an ancestor entailment $G_1 \vdash G_2$ (`ValidAncestor`$\langle G_1 \vdash G_2 \rangle$).

1: $\mathcal{R} \leftarrow$ FindRulesLFF($\mathcal{H}, F_1 \vdash F_2$)
2: **for each** $R$ **in** $\mathcal{R}$ **do**
3: **if** $R \in \{\vdash_{\Pi}, \bot^2_{\mathrm{L}}, \bot^1_{\mathrm{L}}\}$ **then return** `Valid` // *axiom rule*
4: **else if** $R \in \{W_\pi, G_{\mathrm{v}}, G_{\mathrm{c}}\}$ **then** // *weakening or generalization*
5: $G_1 \vdash G_2 \leftarrow$ GoalEntailment($R$)
6: $\mathcal{H}', G'_1 \vdash G'_2 \leftarrow$ Premise($R$)
7: **if** ProveLFF($\mathcal{H}', G'_1 \vdash G'_2$) $=$ `Valid` **then**
8: **return** `ValidAncestor`$\langle G_1 \vdash G_2 \rangle$
9: **else** // *other inference rules*
10: $(\mathcal{H}_i, F_{1i} \vdash F_{2i})_{i=1\ldots n} \leftarrow$ Premises($R$)
11: $(res_i)_{i=1\ldots n} \leftarrow$ ProveLFF($\mathcal{H}_i, F_{1i} \vdash F_{2i})_{i=1\ldots n}$
12: **if** $\forall i{=}1\ldots n.\ (res_i = \texttt{Valid})$ **then** // *all sub-goals are proved*
13: **return** `Valid`
14: **else if** $\exists i{=}1\ldots n.\ (res_i = \texttt{ValidAncestor}\langle G_1 \vdash G_2 \rangle)$ **then**
15: **if** $(F_1 \vdash F_2) = (G_1 \vdash G_2)$ **then return** `Valid`
16: **else return** `ValidAncestor`$\langle G_1 \vdash G_2 \rangle$ // *backjump*
17: **return** `Unkn` // *all rules fail to prove* $F_1 \vdash F_2$

FIGURE 6.14: The learning-from-failure proof search procedure

Initially, ProveLFF invokes the procedure FindRulesLFF to find a set $\mathcal{R}$ of inference rules which can be applied to the goal entailment (line 1). They include not only the standard logical and induction rules (Figures 6.7 and 6.8), but also the inference rules discovered by learning from the failure of the IH application (Figure 6.13). We will elaborate later the procedure FindRulesLFF. Afterward, the procedure ProveLFF subsequently applies each inference rule $R$ in $\mathcal{R}$ to prove $F_1 \vdash F_2$, or to prove an ancestor entailment $G_1 \vdash G_2$ of $F_1 \vdash F_2$, when $R$ is a generalization $(G_{\mathrm{c}}, G_{\mathrm{v}})$ or a weakening rule $(W_\pi)$. There are three scenarios as follows:

– If $R$ is an axiom rule $(\vdash_{\Pi}, \bot^2_{\mathrm{L}}, \bot^1_{\mathrm{L}})$, ProveLFF immediately returns a `Valid` result to indicate that the entailment $F_1 \vdash F_2$ is valid (line 3).

- If $R$ is a weakening or a generalization rule $(G_{\mathrm{v}}, G_{\mathrm{c}}, W_{\pi})$, which applies on an ancestor entailment $G_1 \vdash G_2$ of $F_1 \vdash F_2$ (line 4), ProveLFF first searches for a proof of the weakened or generalized entailment $G'_1 \vdash G'_2$. If this entailment can be proved valid (line 7), ProveLFF returns a value `ValidAncestor`$\langle G_1 \vdash G_2 \rangle$ to indicate that the ancestor entailment $G_1 \vdash G_2$ of $F_1 \vdash F_2$ is valid.
- Otherwise, ProveLFF continues to prove the sub-goal entailments of $F_1 \vdash F_2$. Here, if ProveLFF returns `Valid` when proving all these sub-goals, then $F_1 \vdash F_2$ will be concluded valid (lines 12, 13). If ProveLFF returns `ValidAncestor`$\langle G_1 \vdash G_2 \rangle$ when proving one of this sub-goal, then the ancestor $G_1 \vdash G_2$ of this sub-goal entailment is valid. Since $F_1 \vdash F_2$ is also the parent of this sub-goal, $G_1 \vdash G_2$ can be either the entailment $F_1 \vdash F_2$ or an ancestor entailment of $F_1 \vdash F_2$. In the first scenario, ProveLFF can easily conclude that $F_1 \vdash F_2$ is valid, since the validity of $G_1 \vdash G_2$ is confirmed earlier (line 15). In the second scenario, ProveLFF returns `ValidAncestor`$\langle G_1 \vdash G_2 \rangle$ to indicate that $G_1 \vdash G_2$ is the ancestor entailment of $F_1 \vdash F_2$, and it is also valid (line 16). These two scenarios can be repeatedly utilized by ProveLFF to perform a *backjump* to the place in the derivation proof tree where the ancestor entailment $G_1 \vdash G_2$ is derived.

Finally, ProveLFF returns an `Unkn` result, if it cannot apply any rules in $\mathcal{R}$ to prove the goal entailment $F_1 \vdash F_2$ or any of its ancestor entailments.

Figure 6.15 presents the procedure FindRulesLFF which finds possible inference rules to apply to the goal entailment $F_1 \vdash F_2$. More specifically, FindRulesLFF first finds a set of standard inference rules whose premises can be syntactically derived from the goal entailment $F_1 \vdash F_2$ (line 1). They include the logical rules in Figure 6.7 and the induction rule ID in Figure 6.8. Afterward, FindRulesLFF iteratively examines each IH $H \triangleq \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4$ in the set $\mathcal{H}$ to discover suitable IH application rule or failure learning rules (lines 2 – 19). There are three scenarios as follows:

- If both the spatial and the pure formulas of the IH $H$'s antecedents can be unified with those in the goal entailment, FindRulesLFF returns an IH application rule (IH) (line 10).
- If the spatial unification is successful, but the pure unification is not, FindRulesLFF

will identify the non-unifiable pure formula $\Pi_{\mathsf{u}}$ (line 6) and returns a case analysis rule (CA) and a weakening rule ($W_\pi$).

– If the spatial unification is unsuccessful, FindRulesLFF will identify the non-unifiable constant or variable (lines 13, 18) and introduce the corresponding generalization rule $G_{\mathrm{c}}$, $G_{\mathrm{v}}$ (lines 14, 19). Note that to make the proof search more practical, we consider only the unsuccessful spatial unification which has one non-unifiable constant or one non-unifiable variable. (lines 12 and 15).

Finally, FindRulesLFF will return a set of all discovered inference rules to the main proof search procedure ProveLFF (line 20).

**Procedure** FindRulesLFF$(\mathcal{H}, F_1 \vdash F_2)$

**Input:** $\mathcal{H}$ is an IH set, $F_1 \vdash F_2$ is the goal entailment.

**Output:** a set of inference rules which can be applied to $F_1 \vdash F_2$.

1: $\mathcal{R} \leftarrow$ FindStandardRules$(\mathcal{H}, F_1 \vdash F_2)$ *// not using IHs*
2: **for each** $\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4$ **in** $\mathcal{H}$ **do** *// using IHs*
3: **if** $F_1 \equiv \Sigma_1 \wedge \Pi_1$ **and** $\exists \mathsf{P}(\vec{u}) \in \Sigma_1$ **where** $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v})$ **then**
4: $\mathcal{U} \leftarrow$ UnifySpatial$(\Sigma_3 * \mathsf{P}(\vec{v}), \Sigma_1, \varnothing)$
5: **for each** $\mathtt{Success}\langle\theta, \Sigma\rangle$ **in** $\mathcal{U}$ **where** $\mathsf{P}(\vec{u})\theta \cong \mathsf{P}(\vec{v})$ **do**
6: **if** UnifyPure$(\Pi_3, \Pi_1, \theta) = \mathtt{Fail}\langle\Pi_{\mathsf{u}}\rangle$ **then**
7: $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleCA$(\mathcal{H}, F_1 \vdash F_2, \Pi_{\mathsf{u}})\}$
8: $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleW$_\pi(\mathcal{H}, \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4, \Pi_{\mathsf{u}})\}$
9: **else**
10: $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleIH$(\mathcal{H}, F_1 \vdash F_2, \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4, \theta, \Sigma)\}$
11: **for each** $\mathtt{Fail}\langle m\rangle$ **in** $\mathcal{U}$ **do**
12: **if** HasOnlyOneNonUnifiableConst$(m)$ **then**
13: $c, pos \leftarrow$ GetNonUnifiableConstAndPosition$(m)$
14: $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleG$_{\mathrm{c}}(\mathcal{H}, \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4, c, pos)\}$
15: **if** HasOnlyOneNonUnifiableVar$(m)$ **then**
16: $x \leftarrow$ GetNonUnifiableVar$(m)$
17: **for each** $t$ **where** $\mathtt{Var}\langle x, t, _\rangle \in m$ **do**
18: $\rho \leftarrow \{pos \mid \mathtt{Var}\langle x, t, pos\rangle \in m\}$
19: $\mathcal{R} \leftarrow \mathcal{R} \cup \{$RuleG$_{\mathrm{v}}(\mathcal{H}, \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4, x, \rho)\}$
20: **return** $\mathcal{R}$ *// all discovered rules*

FIGURE 6.15: Find inference rules

### 6.6.3 Soundness

We claim that our learning from failure framework is sound, that is, if the proof search procedure ProveLFF is able to prove an entailment, then this entailment is semantically valid. We formally restate this soundness property in Theorem 6.3.

**Theorem 6.3 (Soundness)** *Given an entailment $E$, if the proof search procedure* ProveLFF *returns* `Valid` *when proving $E$, then $E$ is semantically valid.*

To prove the above soundness theorem, we need to prove that all inference rules in Figures 6.7, 6.8 and 6.13 are sound, and the proof search procedure implements appropriately the structural induction principle. We present the detailed proof in Appendix C.

## 6.7 Experiments

We have integrated the proposed *learning-from-failure* framework into a prototype prover, named **Songbird**$_{\mathbf{LFF}}$, which is built on top of our existing prover **Songbird**, its name stands for "**Songbird** + **F**ailure **L**earning. We have conducted two experiments to evaluate its ability to prove the entailments' validity and the formulas' unsatisfiability. The experiments were organized on a computer with CPU Intel® Core™ i7-6700 (3.4GHz), RAM 16GB, and OS Ubuntu 14.04 LTS. In this environment, we evaluated the efficiency of **Songbird**$_{\mathbf{LFF}}$ against (available) state-of-the-art separation logic provers: **Slide** [Iosif et al., 2013], **Sleek** [Chin et al., 2012], **Spen** [Enea et al., 2014], **Cyclist** [Brotherston et al., 2011a], **SLsat** [Brotherston et al., 2014a] (for proving unsatisfiability only), and **Songbird** with mutual induction (**Songbird**$_{\mathbf{MI}}$) [Ta et al., 2016] and lemma synthesis (**Songbird**$_{\mathbf{LS}}$) [Ta et al., 2018]. In both proving tasks, each prover must finish a job in 120 seconds. The artifact of **Songbird**$_{\mathbf{LFF}}$ is available online at:

`http://songbird-prover.github.io/learning-from-failure/` .

### 6.7.1 Task 1: Proving Validity of Entailments

In this task, we evaluate the aforementioned provers (except **SLsat**) on two collections of literature benchmarks: (1) sll_entl and slrd_entl from the Separation Logic Competition SL-COMP 2014 [Sighireanu et al., 2016] and (2) slrd_ind, slrd_lm from our previous works [Ta et al., 2016; Ta et al., 2018]. The experimental results are presented in Tables 6.1 and 6.2. Entailments in the benchmark sll_entl of singly linked lists is categorized by their original sub-benchmarks. In the other benchmarks, with the slrd prefix, entailment are classified based on the shapes of data structures and numerical properties.

| **Benchmark** | | | **Proved Entailments** | | | | | | | **Average Time (s)** | | | | | | | **Songbird$_{\text{LFF}}$ Stat.** | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Category | #Ent | Sld | Slk | Spn | Ccl | Smi | Sls | **Slff** | Sld | Slk | Spn | Ccl | Smi | Sls | **Slff** | CA | $W_\pi$ | $G_v$ | $G_c$ |
| sll_entl | bolognesa | 57 | 0 | 0 | **57** | 0 | **57** | **57** | **57** | – | – | 0.41 | – | 0.79 | 0.79 | 0.49 | 0 | 0 | 0 | 0 |
| | clones | 60 | 0 | **60** | **60** | **60** | **60** | **60** | **60** | – | 0.06 | 0.06 | 0.01 | 0.03 | 0.03 | 0.01 | 0 | 0 | 0 | 0 |
| | smallfoot | 55 | 0 | **55** | **55** | **55** | **55** | **55** | **55** | – | 0.05 | 0.05 | 0.20 | 0.13 | 0.13 | 0.07 | 0 | 0 | 0 | 0 |
| slrd_entl | singly-ll | 64 | 12 | 48 | 3 | 63 | 63 | **64** | **64** | 0.08 | 0.13 | 0.04 | 0.04 | 0.07 | 0.06 | 0.03 | 1 | 1 | 0 | 0 |
| | doubly-ll | 37 | 14 | 17 | 9 | 29 | 25 | **35** | 27 | 2.74 | 0.18 | 0.04 | 3.75 | 0.09 | 4.06 | 0.11 | 0 | 1 | 0 | 1 |
| | nested-ll | 11 | 0 | 5 | **11** | 7 | **11** | **11** | **11** | – | 0.44 | 0.04 | 1.96 | 0.10 | 0.09 | 0.04 | 0 | 0 | 0 | 0 |
| | skip-list | 13 | 0 | 4 | **13** | 5 | **13** | **13** | **13** | – | 0.27 | 0.08 | 0.09 | 0.23 | 0.24 | 0.18 | 0 | 0 | 0 | 0 |
| | tree | 26 | 12 | 14 | 0 | 23 | **24** | **24** | **24** | 9.16 | 0.21 | – | 0.91 | 4.30 | 0.20 | 0.09 | 0 | 0 | 0 | 0 |
| Total | | 323 | 38 | 203 | 208 | 242 | 308 | **319** | 311 | 3.93 | 0.11 | 0.15 | 0.65 | 0.55 | 0.66 | 0.14 | 1 | 2 | 0 | 1 |

(Sld, Slk, Spn, Ccl, Smi, Sls and Slff respectively stands for
**Slide**, **Sleek**, **Spen**, **Cyclist**, **Songbird$_{\text{MI}}$**, **Songbird$_{\text{LS}}$** and **Songbird$_{\text{LFF}}$**)

TABLE 6.1: Evaluation on the benchmarks of SL-COMP 2014.

Tables 6.1 and 6.2 show the number of entailments which can be proved by a prover and the average proving time (in seconds) for an entailment. Note that the numbers in **bold** are the best results, and the total number of entailments are presented in the column #Ent. We also provide additional statistics how many times our prover **Songbird$_{\text{LFF}}$** applies each failure learning rules, such as the generalization rules ($G_v, G_c$), the weakening rule ($W_\pi$) and the case analysis rule (CA).

| Benchmark | | | Proved Entailments | | | | Average Time (s) | | | | $\textbf{Songbird}_{\textbf{LFF}}$ Stat. | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Category | #Ent | Ccl | Smi | Sls | **Slff** | Ccl | Smi | Sls | **Slff** | CA | $W_\pi$ | $G_\mathrm{v}$ | $G_\mathrm{c}$ |
| slrd_ind | ll/ll2 | 24 | **24** | **24** | **24** | 21 | 0.38 | 0.14 | 4.02 | 0.12 | 0 | 0 | 0 | 0 |
| | ll-even/odd | 20 | **20** | **20** | **20** | **20** | 1.35 | 0.11 | 0.18 | 0.05 | 0 | 0 | 0 | 0 |
| | ll-left/right | 20 | **20** | **20** | **20** | **20** | 1.04 | 0.09 | 0.16 | 0.04 | 0 | 0 | 0 | 0 |
| | misc. | 32 | 31 | **32** | **32** | **32** | 5.90 | 0.19 | 0.84 | 0.18 | 0 | 0 | 0 | 0 |
| slrd_lm | ll/rev | 20 | 17 | 18 | **20** | **20** | 23.42 | 3.71 | 10.92 | 0.38 | 0 | 0 | 0 | 2 |
| | ll-even/odd | 17 | **17** | **17** | **17** | **17** | 0.23 | 0.05 | 0.06 | 0.02 | 0 | 0 | 0 | 0 |
| | ll/rev+arith• | 72 | 0 | 11 | **72** | 21 | – | 1.95 | 3.79 | 0.82 | 5 | 5 | 0 | 9 |
| | ll-sorted• | 25 | 0 | 7 | **19** | 17 | – | 0.60 | 1.12 | 0.09 | 1 | 1 | 0 | 0 |
| | dll/rev/null | 22 | **22** | **22** | **22** | **22** | 0.35 | 0.54 | 7.84 | 0.03 | 0 | 0 | 0 | 1 |
| | dll/rev/null+arith• | 94 | 0 | 17 | **94** | 84 | – | 0.61 | 37.20 | 2.12 | 37 | 62 | 0 | 34 |
| | ll/dll-mixed | 5 | **5** | **5** | **5** | **5** | 0.04 | 0.06 | 0.07 | 0.02 | 0 | 0 | 0 | 0 |
| | ll/dll-mixed+arith• | 15 | 0 | 5 | **15** | 14 | – | 0.19 | 23.71 | 2.03 | 4 | 5 | 0 | 4 |
| | tree/tseg | 18 | 7 | 14 | **18** | 13 | 0.13 | 12.81 | 9.07 | 3.91 | 0 | 0 | 0 | 0 |
| | tree/tseg+arith• | 12 | 0 | 3 | **12** | 5 | – | 1.00 | 53.04 | 17.84 | 2 | 9 | 0 | 0 |
| Total | | 396 | 163 | 215 | **390** | 311 | 3.99 | 1.46 | 14.02 | 1.23 | 49 | 82 | 0 | 50 |

(Ccl, Smi, Sls and Slff respectively stands for
**Cyclist**, $\textbf{Songbird}_{\textbf{MI}}$, $\textbf{Songbird}_{\textbf{LS}}$ and $\textbf{Songbird}_{\textbf{LFF}}$)

TABLE 6.2: Evaluation on the synthetic benchmarks slrd_ind and slrd_lm.

In this experiment, $\textbf{Songbird}_{\textbf{LFF}}$ is the second best prover, which is only below the prover $\textbf{Songbird}_{\textbf{LS}}$ with lemma synthesis. However, there are two entailments in the benchmarks slrd_entl and slrd_lm which $\textbf{Songbird}_{\textbf{LS}}$ cannot prove within the 120s timeout while $\textbf{Songbird}_{\textbf{LFF}}$ can. The attempt of $\textbf{Songbird}_{\textbf{LS}}$ to synthesize supporting lemmas for these entailments did not succeed while by applying the right learning-from-failure rules, $\textbf{Songbird}_{\textbf{LFF}}$ can prove them quickly. In fact, $\textbf{Songbird}_{\textbf{LFF}}$ is more efficient than $\textbf{Songbird}_{\textbf{LS}}$, especially in the $\textbf{Songbird}_{\textbf{MI}}$'s collection where $\textbf{Songbird}_{\textbf{LFF}}$'s runtime is averagely 11.4 times faster than $\textbf{Songbird}_{\textbf{LS}}$.

It is worth noting that the standard structural induction proof is sufficient for proving most of the entailments in the SL-COMP benchmarks (sll_entl and slrd_entl) and the $\textbf{Songbird}_{\textbf{MI}}$'s benchmark slrd_ind [Ta et al., 2016]. In these benchmarks, the learning-from-failure rules are utilized for only two entailments in slrd_entl, which $\textbf{Songbird}_{\textbf{MI}}$ cannot

prove. On the other hand, the learning-from-failure technique is applied more frequently in the slrd_lm benchmark, especially when proving entailments with arithmetics.

### 6.7.2 Task 2: Proving Unsatisfiability of Formulas

| **Benchmark** slrd_unsat | | **Proved Formulas** | | | | | **Average Time (s)** | | | | | **Songbird$_{\text{LFF}}$ Stat.** | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Category | #Fml | **Ssa** | Ccl | Smi | Sls | Slff | **Ssa** | Ccl | Smi | Sls | Slff | CA | $W_\pi$ | $G_\text{v}$ | $G_\text{c}$ |
| ll/rev/null | 80 | **80** | 60 | 25 | 54 | 77 | 0.01 | 0.04 | 0.04 | 0.30 | 0.31 | 68 | 5 | 15 | 12 |
| ll/rev/null+arith• | 250 | 0 | 0 | 86 | 121 | **245** | – | – | 5.73 | 3.70 | 1.22 | 272 | 150 | 14 | 33 |
| dll/rev/null | 42 | **42** | 36 | 12 | 34 | **42** | 0.01 | 0.05 | 0.05 | 0.09 | 0.76 | 51 | 8 | 0 | 0 |
| dll/rev/null+arith• | 146 | 0 | 0 | 61 | 97 | **146** | – | – | 0.15 | 1.09 | 1.15 | 168 | 95 | 0 | 0 |
| tree/tseg/rev | 54 | **54** | 33 | 23 | 28 | 46 | 0.01 | 0.05 | 0.04 | 0.37 | 0.91 | 54 | 0 | 13 | 5 |
| tree/tseg/rev+arith• | 260 | 0 | 0 | 178 | 179 | **248** | – | – | 0.07 | 1.00 | 1.38 | 252 | 0 | 39 | 33 |
| Total | 832 | 176 | 129 | 385 | 513 | **804** | 0.01 | 0.04 | 1.34 | 1.48 | 1.13 | 865 | 258 | 81 | 83 |

(**Ssa**, Ccl, Smi, Sls and Slff respectively stands for
**SLsat**, **Cyclist**, **Songbird$_{\text{MI}}$**, **Songbird$_{\text{LS}}$** and **Songbird$_{\text{LFF}}$**)

TABLE 6.3: Evaluation on the synthetic benchmark slrd_unsat.

In the second task, we design a new benchmark slrd_unsat to compare our **Songbird$_{\text{LFF}}$** prover against other provers (including **SLsat** [Brotherston et al., 2014a]) in proving the unsatisfiability of formulas. This benchmark was composed by using similar heap predicates utilized in the previous benchmarks sll_entl, slrd_entl, slrd_ind, slrd_lm. We also indicate entailment categories containing *arithmetic* constraints with •. Except for the prover **SLsat** which can check the satisfiability/unsatisfiability of formulas directly, other provers need to indirectly prove the entailment $F \vdash \text{false}$.

Table 6.3 shows the overall evaluation result in this task. Our prover **Songbird$_{\text{LFF}}$** can prove the unsatisfiability of more than 96.6% of the total formulas (804/832 formulas). This result of **Songbird$_{\text{LFF}}$** is significantly better than others, about 35% better than the second best prover **Songbird$_{\text{LS}}$**'s result and more than 50% better than the third best prover **Songbird$_{\text{MI}}$**'s. Also, its average time for proving a formula is the best among these three variant provers of **Songbird**.

Moreover, **Songbird$_{\mathbf{LFF}}$** achieves the best results in most of the categories, except for the *non-arithmetic* categories of singly linked lists and trees. In these categories, the decision procedure **SLsat** for satisfiability checking can prove that *all* the formulas are unsatisfiable. However, it only considers a decidable fragment of separation logic with *only* equality but *not* arithmetic constraints.

In this experiment, *all* of the learning-from-failure rules (CA, $W_\pi$, $G_\mathrm{v}$, and $G_\mathrm{c}$) are utilized by **Songbird$_{\mathbf{LFF}}$**. Here, the case analysis rule CA is prominently used. In fact, the rule CA is usually a preferred option, since it makes the proofs advance without back-jumping to the original goal entailments/formulas and restarting their proofs.

## 6.8 Related Work

We have discussed in Section 6.2 that existing entailment proving techniques which restrict the syntax and the semantics of inductive heap predicates [Berdine et al., 2004; Berdine et al., 2005b; Piskac et al., 2013; Piskac et al., 2014; Bozga et al., 2010; Cook et al., 2011; Pérez et al., 2011; Pérez et al., 2013; Enea et al., 2014; Enea et al., 2015; Gu et al., 2016; Xu et al., 2017; Iosif et al., 2013; Iosif et al., 2014] or utilize sequent-based proof systems [Nguyen et al., 2007; Nguyen et al., 2008; Chin et al., 2012; Qiu et al., 2013] fail to prove the motivating entailments $E_1, E_2, E_3, E_4$ in Section 6.2 due to their inability to handle linear arithmetic constraints or the unlimited unfolding of inductive heap predicates. Moreover, the fix-point computation approach [Brotherston et al., 2014a] does not support linear arithmetic when checking for unsatisfiability of the motivating formula $F_1, F_2, F_3$ in Section 6.2. Finally the cyclic proof and induction proof approaches [Brotherston et al., 2011a; Chu et al., 2015; Le et al., 2016; Le et al., 2017; Ta et al., 2016] also fail to prove all motivating formulas/entailments $F_1, F_2, F_3, E_1, E_2, E_3, E_4$ since they cannot handle specific constraints in the entailments, which prevent the proof systems from applying the suitable induction hypotheses. In contrast to these techniques, our proposed framework can *learn from the failure* of the IH applications in traditional cyclic-based and induction-based techniques to discover better proof strategies. Consequently, it can prove

all entailments $E_1, E_2, E_3, E_4$, and by that means can conclude about the unsatisfiability of the formulas $F_1, F_2, F_3$.

Our learning-from-failure idea is inspired by a work of Ireland and Bundy [Ireland et al., 1996], which rectifies unsuccessful proofs in first-order inductive theories. These authors proposed a heuristic controlling system called *rippling*, which can annotate internal structures of inductive terms into objects called *wave-fronts*, *wave-holes*, utilizes them to find similar structures between an induction hypothesis and a goal statement. If *rippling* fails to find such similarity, its failure can be analyzed to discover suitable *wave-rules* or *auxiliary lemmas* to guide the proof search. In contrast to this method, we focus on the specific problem of proving separation logic entailment by induction and analyze the failure directly at the induction hypothesis application, but not the heuristic control system rippling as in [Ireland et al., 1996]. Nonetheless, we all share the same goal to improve the induction proofs' completeness.

We now discuss the difference between our learning from failure framework with the lemma synthesis framework in Chapter 5. Firstly, these frameworks can both prove the motivating entailment $E_1$. However, the lemma synthesis framework need to spend more effort to synthesizing an auxiliary lemma like $\mathsf{ls}(a, b, k) * b{\mapsto}c \vdash \mathsf{ls}(a, c, k{+}1)$. It first needs to identify a lemma template like $\mathsf{ls}(x_1, x_2, n_1) * x_3{\mapsto}x_4 \vdash \exists x_5, x_6, n_2.\mathsf{ls}(x_5, x_6, n_2)$. It then exhaustively refines the template's antecedent and consequent, using a method that combines structural induction and constraint solving, to discover the desired lemma. In contrast, our learning form failure framework can quickly prove $E_1$ by learning from the failure of the induction hypothesis application, as illustrated in Section 6.2.1. Secondly, this synthesis framework cannot prove the entailments $E_2, E_3, E_4$. The consequents of these entailments are $\mathsf{false}$, therefore, the framework cannot select any heap predicate symbols to create necessary lemma templates. Consequently, it cannot synthesize any supporting lemmas for proving $E_2, E_3, E_4$. In contrast, all these entailments can be proved by our learning from failure framework, as shown in Sections 6.2.2, 6.2.3, 6.2.4.

## 6.9 Summary

We have presented a novel framework that can systematically *analyze* and *learn* from the failure of an induction proof for separation logic entailments to remedy the unsuccessful proofs. More specifically, our framework is able to identify the main failure reason, whether it occurs when unifying the heap component or the pure component of an IH with a sub-goal entailment. Based on that, it can propose suitable solutions that can weaken or generalize the IH, or can case-analyze the sub-goal entailment to increase the current IH's applicability. In theory, our learning-from-failure can improve the completeness of induction proof, since it not only can maximize the usability of the existing IH but also can introduce more promising IHs. In practice, we have shown by experiment that our learning-from-failure prover can prove many entailments more efficiently than existing provers, and can prove the unsatisfiability of many formulas which cannot be proved by all existing solver.

# Chapter 7

# Future Work

For the future work, we plan to explore various problems related to the entailment proving problem in different separation logic settings. Firstly, we aim to support more expressive fragments of separation logic, such as separation logic with permission constraints, non-linear arithmetic constraints, and second-order constraints like set or multiset. Secondly, we want to investigate the bi-abductive reasoning problem. Finally, we plan to research the entailment proving problem in separation logic with both the separating conjunction ($*$) and the separating implication operator ($-\!\!*$). Some initial ideas are outlined as follows.

## 7.1 Separation Logic with More Expressive Properties

In this thesis, we have presented comprehensive solutions to prove separation logic entailments using proof techniques based on mathematical induction. However, our considered logic fragment currently supports only standard heap predicates with equality and linear arithmetic constraints. In future, we would like to investigate also separation logic fragments which support permission constraints, non-linear arithmetic constraints and second-order constraints like set and multiset.

The separation logic fragment with permission constraints has been actively studied to reason about concurrent data structures in multithreading programs [Bornat et al., 2005].

However, little effort has been spent on proving entailments which contain both inductive heap predicates and permission constraints. Here, we plan to apply mathematical induction to prove these entailments. A challenging question here is how to propose a suitable induction principle to reason about not only the structure of heap predicates or the size of heap model but also the permission constraints of heap predicates?

On the other hand, the extension of our current entailment proving techniques to support more expressive pure properties like non-linear arithmetic constraints or second-order constraints is more straightforward since both the two proposed mutual induction principle and structural induction principle are still applicable. However, we need to consider other technical difficulties, such as:

- In the context of proving pure entailments, how to discharge non-linear arithmetic or second-order constraints to off-the-shelf provers effectively?
- In the context of the lemma synthesis framework, how to solve a constraint set of unknown relations containing such non-linear or second-order constraints?
- In the context of the learning from failure framework, how to implement the unification algorithm on pure formulas which also contain non-linear or second-order constraints?

## 7.2 Bi-Abductive Reasoning in Separation Logic

One of the important problems in program verification using separation logic is how to perform bi-abductive reasoning when proving entailments related to a function call. In particular, this reasoning aims to discover the missing program state that is needed for checking the pre-condition of the callee and allow the analysis to proceed as well as to discover the program state that the caller leaves unchanged [Calcagno et al., 2009].

Ordinarily, given an entailment $F_1 \vdash F_2$, the standard proof techniques aim to answer the question: whether the entailment $F_1 \vdash F_2$ is valid? More intuitively, they need to check if the program states modeled by $F_1$ can also be modeled by $F_2$. This entailment

checking problem is frequently performed when normalizing a program state or proving post-conditions of a function.

However, in the context of proving pre-conditions of a function call, the caller's program states can be different from the states described in the callee's pre-conditions. For example, in Example 1.2 in Chapter 2, when the function `length` recursively invokes itself to compute the length of the linked list pointed by `y`, the pre-condition of this function call is a substructure of the linked list pointed by `x` in the caller's program state. Here, the bi-abductive reasoning technique aims to answer the question: which information will be processed by the callee and which information will remain in the caller? Such information is necessary for local reasoning about the program.

Formally, suppose that the program state of a caller is modeled by $G_1$, and the pre-condition of a callee is modeled by $G_2$. Then the program state which is needed by the callee is called the *AntiFrame*, whereas the program state remains in the caller is called the *Frame*.

Given two formulas $G_1, G_2$, the bi-abductive reasoning problem aims to identify the necessary *AntiFrame* and *Frame* so that the following entailment is valid:

$$G_1 * (AntiFrame) \vdash G_2 * (Frame)$$

In future, we plan to equip the prover **Songbird** with an ability to find the *Frame* and *AntiFrame* formulas and solve the bi-abductive reasoning problem. This feature directly benefits the modular verification of programs.

## 7.3 Separation Logic with Magic Wand

Little research has been conducted in the separation logic fragment which supports both the separating conjunction ($*$) and separating implication ($-\!\!*$), i.e., the magic wand. Also, these works either do not support inductive heap predicates [Lee et al., 2014] or handle only a pre-defined heap predicate modeling linked list [Hou et al., 2015]. In the future,

we would like to investigate further the possibilities of proving entailments in the separation logic fragment containing both the *separating conjunction* operator ($*$), the separation implication ($-\!\!*$) operator and *user-defined inductive heap predicates*. We aim to study the applicability of mathematical induction in reasoning about the recursive structures of heap predicates, given the appearance of the separation implication operator ($-\!\!*$). This problem is challenging and has not been solved by any existing works.

# Chapter 8

# Conclusion

We have presented in this thesis the problem of proving separation logic entailments, theoretical background in separation logic, and a thorough survey of related works on the entailment proof. We have also proposed comprehensive solutions to prove entailments in the fragment of symbolic-heap separation logic with inductive heap predicates and linear arithmetic constraints. They include:

- The mutual induction proof system, which allows entailments derived in an ongoing proof can be used as hypotheses to prove future entailments introduced within the same proof. This proof system can prove entailments more effectively since the applicable hypotheses are not limited to only the induction hypotheses but also other derived entailments.
- The lemma synthesis framework, which can automatically synthesize lemmas to serve as eureka steps to assist in proving entailments. The synthesized lemmas are general and modular enough to support proving sophisticated entailments, which cannot be handled by any existing provers.
- The learning from failure framework, which can systematically analyze the failure of the induction hypothesis applications in an entailment proof to identify the reasons why this proof fail. This information enables the framework to derive suitable weakening, generalization or case-analysis strategies to remedy the entailment proof.

All the above solutions aim to have the same goal of improving the completeness of mathematical induction in proving separation logic entailments. They have been implemented in a prototype prover named **Songbird**, which is available for download at:

http://songbird-prover.github.io

We have experimented **Songbird** on various literature benchmarks from the separation logic competition SL-COMP 2014 and other synthetic benchmarks containing sophisticated entailments. To date, **Songbird** can prove most of the entailments and outperforms all state-of-the-art separation logic provers in all experimented benchmarks. These experimental results demonstrate the efficiency the and the effectiveness of our entailment proving techniques. They also open up more opportunities to apply techniques based on separation logic to verify real-world programs.

# Appendix A

# The soundness of the Mutual Induction Proof System

We prove the soundness of the mutual induction proof system in Chapter 4 by showing the soundness of its inference rules and the proof system as in the followings.

## A.1 Inference Rules

The soundness of the inference rules are presented by showing that if entailments in their premises are valid, and their side conditions are satisfied, then goal entailments in their conclusions are also valid.

**1. Axiom rules $\bot^1_{\mathbf{L}}, \bot^2_{\mathbf{L}}$ and $\vdash_{\Pi}$:**

$$\vdash_{\Pi} \frac{}{\mathcal{H},\, \rho,\, \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2 \quad \bot^1_{\mathrm{L}} \frac{}{\mathcal{H},\, \rho,\, F_1 \wedge \Pi \vdash F_2}\ \Pi \rightarrow \mathsf{false} \quad \bot^2_{\mathrm{L}} \frac{}{\mathcal{H},\, \rho,\, F_1 * u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{w} \vdash F_2}$$

- It is easy to verify that the antecedents of goal entailments in the two rules $\bot^1_{\mathrm{L}}, \bot^2_{\mathrm{L}}$ are unsatisfiable, since they either contain a contradiction ($u \neq u$ in the rule $\bot^1_{\mathrm{L}}$) or contain two data nodes having the same memory address ($u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{w}$ in the rule $\bot^2_{\mathrm{L}}$). Therefore, these entailments are evidently valid.
- When the rule $\vdash_{\Pi}$ is applied, pure provers will be invoked to check the side condition: $\Pi_1 \Rightarrow \Pi_2$. If this side condition holds, then clearly the entailment $\Pi_1 \vdash \Pi_2$ is valid. $\square$

**2. Rule $=_{\mathrm{L}}$ and $=_{\mathrm{R}}$:**

$$=_{\mathrm{L}}\ \frac{\mathcal{H},\ \rho',\ F_1[u/v] \vdash F_2[u/v]}{\mathcal{H},\ \rho,\ F_1 \wedge u{=}v \vdash F_2} \qquad\qquad =_{\mathrm{R}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists\vec{x}.F_2}{\mathcal{H},\ \rho,\ F_1 \vdash \exists\vec{x}.(F_2 \wedge u{=}u)}$$

- The soundness of the rule $=_{\mathrm{R}}$ is evident since the condition $u{=}u$ in the goal entailment $F_1 \vdash \exists\vec{x}.(F_2 \wedge u{=}u)$ is a tautology.
- For the rule $=_{\mathrm{L}}$, consider an arbitrary model $s, h$ of the goal entailment's antecedent. Then $s, h \models F_1 \wedge u{=}v$. It follows that $s(u) = s(v)$, therefore $s, h \models F_1[u/v]$. Since the entailment $F_1[u/v] \vdash F_2[u/v]$ in the rule's premise is valid, it implies $s, h \models F_2[u/v]$. But this also means $s, h \models F_2$, since $s(u) = s(v)$. Therefore, $F_1 \wedge u{=}v \vdash F_2$ is valid. □

**3. Rule $\exists_{\mathrm{L}}$ and $\exists_{\mathrm{R}}$:**

$$\exists_{\mathrm{L}}\ \frac{\mathcal{H},\ \rho',\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \rho,\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2) \qquad\qquad \exists_{\mathrm{R}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash F_2[e/x]}{\mathcal{H},\ \rho,\ F_1 \vdash \exists x.F_2}$$

- To prove the correctness of the rule $\exists_{\mathrm{L}}$, we consider an arbitrary model $s, h$ such that $s, h \models \exists x.F_1$. By semantics of the $\exists$ quantification, there is an integer value $v \in \mathtt{Int}$ such that $s', h \models F_1$, with $s' = [s|x{:}v]$. Then $s'', h \models F_1[u/x]$ with $s''$ is extended from $s'$ such that $s''(u) = s'(x)$. Since $s' = [s|x{:}v]$, it follows that $s'' = [s|u{:}v]$. On the other hand, given that entailment in the rule's premise is valid, then $s'', h \models F_2$. It implies that $[s|u{:}v] \models F_2$. In addition, $u \notin \mathtt{FV}(F_2)$. Therefore $s, h \models F_2$. Since $s, h$ is chosen arbitrarily, it follows that the rule $\exists_{\mathrm{L}}$ is correct.
- The correctness of $\exists_{\mathrm{R}}$ is straight forward. Suppose that $s, h$ is an arbitrary model such that $s, h \models F_1$. Since entailment in the rule's premise is valid, then $s, h \models F_2[e/x]$. It follows that $s, h$ also satisfies $\exists x.F_2$, by simply choosing value $v$ of $x$ such that $v = [\![e]\!]_s$. Since $s, h$ is chosen arbitrarily, it follows that the entailment in the rule's conclusion is valid. Therefore $\exists_{\mathrm{R}}$ is valid. □

**4. Rule $\mathsf{E}_{\mathrm{L}}$ and $\mathsf{E}_{\mathrm{R}}$:**

$$\mathsf{E}_{\mathrm{L}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash F_2}{\mathcal{H},\ \rho,\ F_1 * \mathsf{emp} \vdash F_2} \qquad\qquad \mathsf{E}_{\mathrm{R}}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists\vec{x}.F_2}{\mathcal{H},\ \rho,\ F_1 \vdash \exists\vec{x}.(F_2 * \mathsf{emp})}$$

It is evident that two assertions $F_1 * \mathsf{emp}$ and $F_1$ in the rule $\mathsf{E_L}$ are semantically equivalent. In addition, $F_2 * \mathsf{emp}$ and $F_2$ in the rule $\mathsf{E_R}$ are also semantically equivalent. It follows that both the two rules $\mathsf{E_L}$ and $\mathsf{E_R}$ are correct. □

**5. Rule $*\mapsto$ and $*\mathsf{P}$:**

$$*\mapsto \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists\vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})}{\mathcal{H},\ \rho,\ F_1 * u \overset{\iota}{\mapsto} \vec{v} \vdash \exists\vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})}\ u, \vec{v} \# \vec{x} \qquad *\mathsf{P}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists\vec{x}.(F_2 \wedge \vec{u}{=}\vec{v})}{\mathcal{H},\ \rho,\ F_1 * \mathsf{P}(\vec{u}) \vdash \exists\vec{x}.(F_2 * \mathsf{P}(\vec{v}))}\ \vec{u} \# \vec{x}$$

– We present the soundness proof of the rule $*\mapsto$, the soundness of $*\mathsf{P}$ can proved in a similar way. Consider an arbitrary model $s, h$ such that $s, h \models F_1 * u \overset{\iota}{\mapsto} \vec{v}$. Then, there exists $h_1 \# h_2$ such that $h = h_1 \circ h_2$, and $s, h_1 \models F_1$, and $s, h_2 \models u \overset{\iota}{\mapsto} \vec{v}$. On the one hand, entailment in the rule's premise is valid, it follows that $s, h_1 \models \exists\vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})$. By semantics of $\exists$ quantification, $s', h_1 \models F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w}$, with $s'$ is a model extended from $s$ with integer values of $\vec{x}$. On the other hand, the rule's side condition gives $u \notin \vec{x}$, and $\vec{v} \# \vec{x}$, and $s'$ is extended from $s$ with values of $\vec{x}$, and $s, h_2 \models u \overset{\iota}{\mapsto} \vec{v}$, it follows that $s', h_2 \models u \overset{\iota}{\mapsto} \vec{v}$. Combining these two hands, with the fact that $h_1 \# h_2$ and $h = h_1 \circ h_2$, the following holds: $s', h \models F_2 * u \overset{\iota}{\mapsto} \vec{v} \wedge u{=}t \wedge \vec{v}{=}\vec{w}$. By semantics of equality $(=)$, the following also holds: $s', h \models F_2 * t \overset{\iota}{\mapsto} \vec{w} \wedge u{=}t \wedge \vec{v}{=}\vec{w}$. By weakening this assertion via dropping the condition $u{=}t \wedge \vec{v}{=}\vec{w}$, it is evident that $s', h \models F_2 * t \overset{\iota}{\mapsto} \vec{w}$. Since $s'$ is extended from $s$ with values of $\vec{x}$, it is evident that $s, h \models \exists\vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})$. Recall that $s, h$ is chosen arbitrarily, this implies that the rule $*\mapsto$ is sound. □

**6. Rule $\mathsf{P_R}$:**

$$\mathsf{P_R}\ \frac{\mathcal{H},\ \rho',\ F_1 \vdash \exists\vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))}{\mathcal{H},\ \rho,\ F_1 \vdash \exists\vec{x}.(F_2 * \mathsf{P}(\vec{u}))}\ F_i^{\mathsf{P}}(\vec{u}) \overset{\text{def}}{\Rightarrow} \mathsf{P}(\vec{u})$$

– Consider an arbitrary model $s, h$ such that $s, h \models F_1$. Since entailment in the rule's premise is valid, it follows that $s, h \models \exists\vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))$. In addition, the rule's side condition that $F_i^{\mathsf{P}}(\vec{u})$ is one of the definition cases of $\mathsf{P}(\vec{u})$ clearly implies that $s, h \models \exists\vec{x}.(F_2 * \mathsf{P}(\vec{u}))$. Since $s, h$ is chosen arbitrarily, it follows that entailment in the rule's conclusion is valid. □

**7. Rule** ID**:**

$$\mathsf{ID}\ \frac{\mathcal{H}',\ (\mathsf{ID}) :: \rho,\ F_1 * F_1^{\mathsf{P}}(\vec{u}) \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ (\mathsf{ID}) :: \rho,\ F_1 * F_m^{\mathsf{P}}(\vec{u}) \vdash F_2}{\mathcal{H},\ \rho,\ F_1 * \mathsf{P}(\vec{u}) \vdash F_2}\ \dagger$$

$$\dagger:\ \mathcal{H}' \triangleq \mathcal{H}' \cup \{(F_1 * \mathsf{P}(\vec{u}) \vdash F_2, \mathbf{?})\},\ \text{and}\ \mathsf{P}(\vec{u}) \stackrel{\text{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u})$$

– We show that if all of the entailments $F_1 * F_1^{\mathsf{P}}(\vec{u}) \vdash F_2$,..., $F_1 * F_m^{\mathsf{P}}(\vec{u}) \vdash F_2$ in the rule premise are valid, then so is the entailment $F_1 * \mathsf{P}(\vec{u}) \vdash F_2$ in the conclusion.

Indeed, consider an arbitrary model $s, h$ such that $s, h \models F_1 * \mathsf{P}(\vec{u})$. Side condition of the rule gives that $\mathsf{P}(\vec{u}) \triangleq F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u})$, i.e., $F_1^{\mathsf{P}}(\vec{u})$, ..., $F_m^{\mathsf{P}}(\vec{u})$ are all definition cases of $\mathsf{P}(\vec{u})$. Since $s, h \models F_1 * \mathsf{P}(\vec{u})$, it follows that $s, h \models F_1 * F_i^{\mathsf{P}}(\vec{u})$, for all $i = 1 \ldots m$. On the other hand, $F_1 * F_1^{\mathsf{P}}(\vec{u})$, ..., $F_1 * F_m^{\mathsf{P}}(\vec{u})$ are antecedents of all entailments in this rule's premises, and these entailments have the same consequent $F_2$. Therefore, $s, h \models F_2$. Since $s, h$ is chosen arbitrarily, it follows that entailment in the rule's conclusion is valid. This confirms the soundness of the rule ID. □

**8. Rule** IH**:**

$$\mathsf{IH}\ \frac{\mathcal{H} \cup \{(H, status)\},\ (\mathsf{IH}) :: \rho,\ F_4\theta * \Sigma' \wedge \Pi_1 \vdash F_2}{\mathcal{H} \cup \{(H \triangleq \Sigma_3 \wedge \Pi_3 \vdash F_4, status)\},\ \rho,\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \exists \theta, \Sigma'.(\Sigma_1 \cong \Sigma_3\theta * \Sigma' \wedge \Pi_1 \rightarrow \Pi_3\theta);\ \dagger$$

$$\dagger:\ (status = \checkmark)\ \vee\ \exists\, \iota, u, \vec{v}, \Sigma''.(\Sigma' \cong u \stackrel{\iota}{\mapsto} \vec{v} * \Sigma'')$$
$$\vee\ \exists\, \rho_1, \rho_2.(\rho = \rho_1 @ [(* \mapsto)] @ \rho_2 \wedge (\mathsf{ID}) \notin \rho_1 \wedge (\mathsf{ID}) \in \rho_2)$$

– We show that if two entailments $F_4\theta * \Sigma' \wedge \Pi_1 \vdash F_2$ and $\Sigma_3 \wedge \Pi_3 \vdash F_4$ in the rule's premise are valid, then so is the entailment in the rule's conclusion.

Indeed, the side condition $\Sigma_1 \cong \Sigma_3\theta * \Sigma'$ and $\Pi_1 \Rightarrow \Pi_3\theta$ implies that $\Sigma_1 \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma' \wedge \Pi_3\theta \wedge \Pi_1$ is valid. By applying Reynolds's substitution law [Reynolds, 2008], the hypothesis $\Sigma_3 \wedge \Pi_3 \vdash F_4$ implies that $\Sigma_3\theta \wedge \Pi_3\theta \vdash F_4\theta$ is valid. It follows that the following entailment is also valid: $\Sigma_3\theta * \Sigma' \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma' \wedge \Pi_1$.

We have shown that the two entailments $\Sigma_1 \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma' \wedge \Pi_3\theta \wedge \Pi_1$ and $\Sigma_3\theta * \Sigma' \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma' \wedge \Pi_1$ are valid. In addition, the rule's premise gives that $F_4\theta * \Sigma' \wedge \Pi_1 \vdash F_2$ is valid. It follows that the entailment $\Sigma_1 \wedge \Pi_1 \vdash F_2$ in the rule's conclusion is valid as well. Therefore, the rule IH is correct. □

## A.2 The Proof System

We are now ready to prove the soundness of our proof system, which is stated in Theorem 4.4: *Given an entailment $E$, if the proof search procedure returns* `Valid` *when proving $E$, then $E$ is valid*. Our soundness proof is as follow.

Suppose that during proving the entailment $E$, the proof search procedure derives a proof tree $\mathcal{T}$ of $E$. If the rule IH (*apply hypothesis*) is not used in $\mathcal{T}$, then it is clear that induction is not used in proof of $E$. Then, it is clear that $E$ is valid, since the soundness of our inference rules are proven in the previous section.

If the rule IH is used in $\mathcal{T}$, then statuses of the applied hypotheses, at the moment they are used, can be either *valid* (✓) or *unknown* (?). If statuses of all the applied hypotheses in the proof tree $\mathcal{T}$ are valid (✓), then it is clear that $E$ is valid, due to the correctness of our inference rules. If there exist some applied hypotheses whose statuses are unknown (?), then these hypotheses may participate in a mutual induction proof. We will show that the entailment $E$ is also valid in this case.

Recall that hypotheses applied by the rule IH are either (i) induction hypotheses recorded by the rule ID, or (ii) hypotheses derived by other rules during proof search. Therefore, in the proof tree $\mathcal{T}$, both the induction hypotheses and other hypotheses can participate in a mutual induction proof. We will transform the proof tree $\mathcal{T}$ into a new tree $\mathcal{T}'$ in which the mutual induction proof involves only the induction hypotheses recorded by the rule ID. This can be done by modifying the rule IH to put the used hypothesis, which is not derived by the rule ID, into premises of the rule.

In particular, suppose the (simplified) rule IH which applies a hypothesis $\Sigma_3 \wedge \Pi_3 \vdash F_4$, (not derived by the rule ID), into proving a goal entailment $\Sigma_1 \wedge \Pi_1 \vdash F_2$ as follow:

$$\mathsf{IH}\ \frac{F_4\theta * \Sigma' \vdash F_2}{\Sigma_1 \wedge \Pi_1 \vdash F_2}\ \begin{array}{l}\text{apply hypothesis: } \Sigma_3 \wedge \Pi_3 \vdash F_4, \text{ (which is not derived by rule ID)}\\ \exists\, \theta, \Sigma'.(\Sigma_1 \equiv \Sigma_3\theta * \Sigma' \ \wedge\ \Pi_1 \Rightarrow \Pi_3\theta)\end{array}$$

We modify the rule IH so that the hypothesis $\Sigma_3 \wedge \Pi_3 \vdash F_4$ appears on the premises as follows:

$$\text{modified IH}\ \frac{\Sigma_3 \wedge \Pi_3 \vdash F_4 \qquad F_4\theta * \Sigma' \vdash F_2}{\Sigma_1 \wedge \Pi_1 \vdash F_2}\ \begin{array}{l}\Sigma_3 \wedge \Pi_3 \vdash F_4 \text{ is not derived by rule ID} \\ \exists\, \theta, \Sigma'.(\Sigma_1 \equiv \Sigma_3\theta * \Sigma' \ \wedge\ \Pi_1 \Rightarrow \Pi_3\theta)\end{array}$$

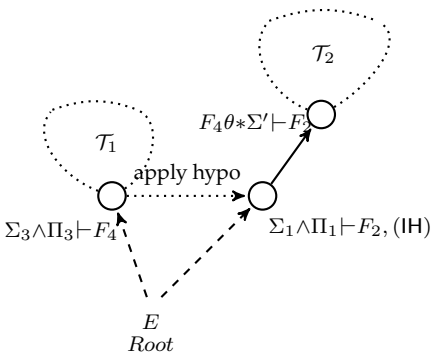



FIGURE A.1: Original proof tree $\mathcal{T}$

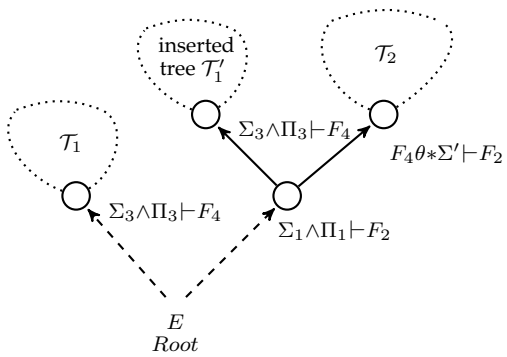



FIGURE A.2: Transformed tree $\mathcal{T}'$

By modifying the rule IH, we can transform the proof tree $\mathcal{T}$ into a proof tree $\mathcal{T}'$, where every node of the tree $\mathcal{T}$ which applies a hypothesis (not derived by the rule ID) to prove an entailment is replaced by a new node that contains not only the target entailment but also full proof tree of the applied hypothesis. Since this transformation is performed only on hypotheses not derived by the rule ID, it follows that in the new proof tree $\mathcal{T}'$, only induction hypotheses recorded by the rule ID participating into the mutual induction proof. We have shown earlier in the discussion about the rule IH that its side condition about model decreasing maps to the well-founded relation of our mutual induction principle. Therefore, the mutual induction principle is ensured by this new proof tree. This implies that all induction hypotheses (recorded by the rule ID) are valid. Therefore, the original entailment is also valid. □

# Appendix B

# The soundness of the Lemma Synthesis Framework

## B.1 Inference Rules

We prove the soundness of inference rules in Section 5.4 by showing that if entailments in their premises are valid, and their side conditions, if any, are satisfied, then goal entailments in their conclusions are also valid.

**1. Rules** $\vdash_{\Pi}, \bperp^1_{\mathbf{L}}, \bperp^2_{\mathbf{L}}$

$$\vdash_{\Pi} \frac{}{\mathcal{H},\, \mathcal{L},\, \Pi_1 \vdash \Pi_2}\, \Pi_1 \rightarrow \Pi_2 \quad \bperp^1_{\mathrm{L}} \frac{}{\mathcal{H},\, \mathcal{L},\, F_1 * u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{t} \vdash F_2} \quad \bperp^2_{\mathrm{L}} \frac{}{\mathcal{H},\, \mathcal{L},\, F_1 \wedge \Pi_1 \vdash F_2}\, \Pi_1 \rightarrow \mathsf{false}$$

- When the rule $\vdash_{\Pi}$ is applied, SMT solvers or theorem provers, such as Z3 [Moura et al., 2008] or Omega Calculator [Pugh, 1991], can be invoked to check the side condition: $\Pi_1 \rightarrow \Pi_2$. If this side condition holds, then clearly the entailment $\Pi_1 \vdash \Pi_2$ is valid.
- For the two rules $\bperp^1_{\mathrm{L}}, \bperp^2_{\mathrm{L}}$, it is easy to verify their goal entailments' antecedents are unsatisfiable since they either contain an overlapped singleton heap ($u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{t}$ in the rule $\bperp^1_{\mathrm{L}}$), or a contradiction ($\Pi_1 \rightarrow \mathsf{false}$ in the rule $\bperp^1_{\mathrm{L}}$). Therefore, these entailments are evidently valid.

**2. Rules $=_{\mathbf{L}}, \exists_{\mathbf{L}}, \exists_{\mathbf{R}}$**

$$=_{\mathrm{L}} \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/v] \vdash F_2[u/v]}{\mathcal{H},\ \mathcal{L},\ F_1 \wedge u{=}v \vdash F_2} \quad \exists_{\mathrm{L}} \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \exists x.F_1 \vdash F_2}\, u \notin \mathtt{FV}(F_2) \quad \exists_{\mathrm{R}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.F_2[u/v]}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}, v.(F_2 \wedge u{=}v)}$$

- For the rule $=_{\mathrm{L}}$, consider an arbitrary model $s, h$ of the goal entailment's antecedent. Then $s, h \models F_1 \wedge u{=}v$. It follows that $s(u) = s(v)$, therefore $s, h \models F_1[u/v]$. Since the entailment $F_1[u/v] \vdash F_2[u/v]$ in the rule's premise is valid, it implies $s, h \models F_2[u/v]$. This means $s, h \models F_2$, due to $s(u) = s(v)$. Therefore, $F_1 \wedge u{=}v \vdash F_2$ is valid.
- To prove the correctness of the rule $\exists_{\mathrm{L}}$, we consider an arbitrary model $s, h$ such that $s, h \models \exists x.F_1$. By semantics of the $\exists$ quantification, there is an integer value $v \in \mathtt{Int}$ such that $s', h \models F_1$, with $s' = [s|x{:}v]$. Then $s'', h \models F_1[u/x]$ with $s''$ is extended from $s'$ such that $s''(u) = s'(x)$. Since $s' = [s|x{:}v]$, it follows that $s'' = [s|u{:}v]$. On the other hand, given that entailment in the rule's premise is valid, then $s'', h \models F_2$. It implies that $[s|u{:}v] \models F_2$. In addition, $u \notin \mathtt{FV}(F_2)$. Therefore $s, h \models F_2$. Since $s, h$ is chosen arbitrarily, it follows that the rule $\exists_{\mathrm{L}}$ is correct.
- The correctness of $\exists_{\mathrm{R}}$ is straight forward. Suppose that $s, h$ is an arbitrary model such that $s, h \models F_1$. Since entailment in the rule's premise is valid, then $s, h \models F_2[e/x]$. It follows that $s, h$ also satisfies $\exists x.F_2$, by simply choosing value $v$ of $x$ such that $v = [\![e]\!]_s$. Since $s, h$ is chosen arbitrarily, it follows that the entailment in the rule's conclusion is valid. Therefore $\exists_{\mathrm{R}}$ is valid.

**3. Rules $\mathsf{E}_{\mathbf{L}}, \mathsf{E}_{\mathbf{R}}$**

$$\mathsf{E}_{\mathrm{L}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ F_1 * \mathsf{emp} \vdash F_2} \qquad \mathsf{E}_{\mathrm{R}} \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.F_2}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{emp})}$$

- It is evident that two formulas $F_1 * \mathsf{emp}$ and $F_1$ in the rule $\mathsf{E}_{\mathrm{L}}$ are semantically equivalent. In addition, $F_2 * \mathsf{emp}$ and $F_2$ in the rule $\mathsf{E}_{\mathrm{R}}$ are also semantically equivalent. It follows that both the two rules $\mathsf{E}_{\mathrm{L}}$ and $\mathsf{E}_{\mathrm{R}}$ are correct.

**4. Rule** CA

$$\mathsf{CA}\,\frac{\mathcal{H},\,\mathcal{L},\,F_1 \wedge \Pi \vdash F_2 \qquad \mathcal{H},\,\mathcal{L},\,F_1 \wedge \neg\Pi \vdash F_2}{\mathcal{H},\,\mathcal{L},\,F_1 \vdash F_2}$$

– The rule's premises provide that $F_1 \wedge \Pi \vdash F_2$ and $F_1 \wedge \neg\Pi \vdash F_2$ are valid. For an arbitrary model $s, h$ such that $s, h \models F_1$, we consider two cases as follows.

(1) $s, h \models F_1 \wedge \Pi$. Since $F_1 \wedge \Pi \vdash F_2$ is valid, it follows that $s, h \models F_2$.

(2) $s, h \not\models F_1 \wedge \Pi$. Since $s, h \models F_1$, it follows that $s, h \not\models \Pi$. Consequently, $s, h \models \neg\Pi$, by the semantics of separation logic formulas. We have shown that $s, h \models F_1$ and $s, h \models \neg\Pi$, it implies that $s, h \models F_1 \wedge \neg\Pi$. Since $F_1 \wedge \neg\Pi \vdash F_2$ is valid, it follows that $s, h \models F_2$.

In both the above cases, we show that for an arbitrary model $s, h$, if $s, h \models F_1$, then $s, h \models F_2$. Therefore, the entailment $F_1 \vdash F_2$ in the conclusion of the rule CA is valid.

**5. Rules** $*\!\!\mapsto$, $*\mathsf{P}$

$$*\!\!\mapsto\,\frac{\mathcal{H},\,\mathcal{L},\,F_1 \vdash \exists\vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})}{\mathcal{H},\,\mathcal{L},\,F_1 * u \overset{\iota}{\mapsto} \vec{v} \vdash \exists\vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})}\; u, \vec{v} \# \vec{x} \qquad *\mathsf{P}\,\frac{\mathcal{H},\,\mathcal{L},\,F_1 \vdash \exists\vec{x}.(F_2 \wedge \vec{u}{=}\vec{v})}{\mathcal{H},\,\mathcal{L},\,F_1 * \mathsf{P}(\vec{u}) \vdash \exists\vec{x}.(F_2 * \mathsf{P}(\vec{v}))}\; \vec{u} \# \vec{x}$$

– To prove the rule $*\!\!\mapsto$, we consider an arbitrary model $s, h$ such that $s, h \models F_1 * u \overset{\iota}{\mapsto} \vec{v}$. Then, there exists $h_1 \# h_2$ such that $h = h_1 \circ h_2$, and $s, h_1 \models F_1$, and $s, h_2 \models u \overset{\iota}{\mapsto} \vec{v}$. On the one hand, entailment in the rule's premise is valid, it follows that $s, h_1 \models \exists\vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})$. By semantics of $\exists$ quantification, $s', h_1 \models F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w}$, with $s'$ is a model extended from $s$ with integer values of $\vec{x}$. On the other hand, the rule's side condition gives $u \notin \vec{x}$, and $\vec{v} \# \vec{x}$, and $s'$ is extended from $s$ with values of $\vec{x}$, and $s, h_2 \models u \overset{\iota}{\mapsto} \vec{v}$, it follows that $s', h_2 \models u \overset{\iota}{\mapsto} \vec{v}$. Combining these two hands, with the fact that $h_1 \# h_2$ and $h = h_1 \circ h_2$, the following holds: $s', h \models F_2 * u \overset{\iota}{\mapsto} \vec{v} \wedge u{=}t \wedge \vec{v}{=}\vec{w}$. By semantics of equality $(=)$, the following also holds: $s', h \models F_2 * t \overset{\iota}{\mapsto} \vec{w} \wedge u{=}t \wedge \vec{v}{=}\vec{w}$. By weakening this formula via dropping the condition $u{=}t \wedge \vec{v}{=}\vec{w}$, it is evident that $s', h \models F_2 * t \overset{\iota}{\mapsto} \vec{w}$. Since $s'$ is extended from $s$ with values of $\vec{x}$, it is evident that $s, h \models \exists\vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})$. Recall that $s, h$ is chosen arbitrarily, this implies that the rule $*\!\!\mapsto$ is sound.

– The soundness of $*\mathsf{P}$ can be proved in a similar way with the rule $*\mapsto$ above.

**6. Rule $\mathsf{P_R}$**

$$\mathsf{P_R}\,\frac{\mathcal{H},\,\mathcal{L},\,F_1 \vdash \exists\vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))}{\mathcal{H},\,\mathcal{L},\,F_1 \vdash \exists\vec{x}.(F_2 * \mathsf{P}(\vec{u}))}\;F_i^{\mathsf{P}}(\vec{u}) \overset{\text{def}}{\Rightarrow} \mathsf{P}(\vec{u})$$

– Consider an arbitrary model $s, h$ such that $s, h \models F_1$. Since entailment in the rule's premise is valid, it follows that $s, h \models \exists\vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))$. In addition, the rule's side condition that $F_i^{\mathsf{P}}(\vec{u})$ is one of the definition cases of $\mathsf{P}(\vec{u})$ clearly implies that $s, h \models \exists\vec{x}.(F_2 * \mathsf{P}(\vec{u}))$. Since $s, h$ is chosen arbitrarily, it follows that entailment in the rule's conclusion is valid.

**7. Rule $\mathsf{ID}$**

$$\mathsf{ID}\,\frac{\mathcal{H}',\,\mathcal{L},\,\Sigma_1 * F_1^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\,\mathcal{L},\,\Sigma_1 * F_m^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\,\mathcal{L},\,\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\;\dagger$$

$\dagger : \mathsf{P}(\vec{u}) \overset{\text{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u})$ and $\mathcal{H}' \triangleq \mathcal{H} \cup \{\, H \,\}$,
where $H$ is obtained by freshly renaming all variables of $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$

– We show that if all of the entailments $F_1 * F_1^{\mathsf{P}}(\vec{u}) \vdash F_2, \ldots, F_1 * F_m^{\mathsf{P}}(\vec{u}) \vdash F_2$ in the rule premise are valid, then so is the entailment $F_1 * \mathsf{P}(\vec{u}) \vdash F_2$ in the conclusion.

Indeed, consider an arbitrary model $s, h$ such that $s, h \models F_1 * \mathsf{P}(\vec{u})$. The side condition of the rule gives that $\mathsf{P}(\vec{u}) \triangleq F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u})$, i.e., $F_1^{\mathsf{P}}(\vec{u}), \ldots, F_m^{\mathsf{P}}(\vec{u})$ are all definition cases of $\mathsf{P}(\vec{u})$. Since $s, h \models F_1 * \mathsf{P}(\vec{u})$, it follows that $s, h \models F_1 * F_i^{\mathsf{P}}(\vec{u})$, for all $i = 1 \ldots m$. On the other hand, $F_1 * F_1^{\mathsf{P}}(\vec{u}), \ldots, F_1 * F_m^{\mathsf{P}}(\vec{u})$ are antecedents of all entailments in this rule's premises, and these entailments have the same consequent $F_2$. Therefore, $s, h \models F_2$. Since $s, h$ is chosen arbitrarily, it follows that entailment in the rule's conclusion is valid. This confirms the soundness of the rule $\mathsf{ID}$.

**8. Rule $\mathsf{IH}$**

$$\mathsf{IH}\,\frac{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\,\mathcal{L},\,F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\,\mathcal{L},\,\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\;\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v});\dagger$$

$\dagger : \exists\theta, \Sigma.(\Sigma_1 * \mathsf{P}(\vec{u}) \;\cong\; \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta)$

– The rule's side conditions $\Sigma_1 * \mathsf{P}(\vec{u}) \cong \Sigma_3\theta * \mathsf{P}'(\vec{v})\theta * \Sigma$ and $\Pi_1 \rightarrow \Pi_3\theta$ imply that the entailment $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash \Sigma_3\theta * \mathsf{P}'(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1$ is valid.

  By applying Theorem 2.2, the induction hypothesis $\Sigma_3 * \mathsf{P}'(\vec{v}) \wedge \Pi_3 \vdash F_4$ implies that $\Sigma_3\theta * \mathsf{P}'(\vec{v})\theta \wedge \Pi_3\theta \vdash F_4\theta$ is valid. It follows that the following entailment is also valid: $\Sigma_3\theta * \mathsf{P}'(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma \wedge \Pi_1$.

  We have shown that the two entailments $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash \Sigma_3\theta * \mathsf{P}'(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1$ and $\Sigma_3\theta * \mathsf{P}'(\vec{v})\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma \wedge \Pi_1$ are valid. In addition, the rule's premise gives that $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$ is valid. It follows that the entailment $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$ in the rule's conclusion is valid as well. Therefore, the rule IH is correct.

**9. Rules** $\mathsf{LM}_{\mathrm{L}}, \mathsf{LM}_{\mathrm{R}}$

$$\mathsf{LM}_{\mathrm{L}}\ \frac{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash F_4\},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash F_4\},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \exists\theta, \Sigma.\ (\Sigma_1 \cong \Sigma_3\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta)$$

$$\mathsf{LM}_{\mathrm{R}}\ \frac{\mathcal{H},\ \mathcal{L} \cup \{F_3 \vdash \exists\vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists\vec{x}.(F_3\theta * \Sigma \wedge \Pi_2)}{\mathcal{H},\ \mathcal{L} \cup \{F_3 \vdash \exists\vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists\vec{x}.(\Sigma_2 \wedge \Pi_2)}\ \exists\theta, \Sigma.(\Sigma_4\theta * \Sigma \cong \Sigma_2) \wedge (\vec{w}\theta \subseteq \vec{x})$$

– The soundness of the lemma application rule $\mathsf{LM}_{\mathrm{L}}$ can be proved similarly to IH. Specifically, this rule's side conditions $\Sigma_1 \cong \Sigma_3\theta * \Sigma$ and $\Pi_1 \rightarrow \Pi_3\theta$ imply that the entailment $\Sigma_1 \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1$ is valid. On the other hand, by applying Theorem 2.2, the lemma $\Sigma_3 \wedge \Pi_3 \vdash F_4$ implies that the entailment $\Sigma_3\theta \wedge \Pi_3\theta \vdash F_4\theta$ is valid. It follows that the following entailment is also valid: $\Sigma_3\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma \wedge \Pi_1$. We have shown that both the two entailments $\Sigma_1 \wedge \Pi_1 \vdash \Sigma_3\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1$ and $\Sigma_3\theta * \Sigma \wedge \Pi_3\theta \wedge \Pi_1 \vdash F_4\theta * \Sigma \wedge \Pi_1$ are valid. In addition, this rule's premise provides that the entailment $F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2$ is also valid. Consequently, the goal entailment $\Sigma_1 \wedge \Pi_1 \vdash F_2$ is valid as well. Therefore, the soundness of the rule $\mathsf{LM}_{\mathrm{L}}$ is claimed.

– For the rule $\mathsf{LM}_{\mathrm{R}}$, the lemma $F_3 \vdash \exists\vec{w}.(\Sigma_4 \wedge \Pi_4)$ implies that $F_3 \vdash \exists\vec{w}.\Sigma_4$ is valid, therefore, $F_3\theta \vdash \exists\vec{w}\theta.\Sigma_4\theta$ is also valid by Theorem 2.2. It follows that $F_3\theta * \Sigma \wedge \Pi_2 \vdash \exists\vec{w}\theta.(\Sigma_4\theta * \Sigma \wedge \Pi_2)$ is also valid. In addition, this rule's side condition provides that $\vec{w}\theta \subseteq \vec{x}$, therefore, $F_3\theta * \Sigma \wedge \Pi_2 \vdash \exists\vec{x}.(\Sigma_4\theta * \Sigma \wedge \Pi_2)$ is valid. Furthermore, the rule's premise provides that the entailment $F_1 \vdash \exists\vec{x}.(F_3\theta * \Sigma \wedge \Pi_2)$ is valid. Hence, by

Theorem 2.1, the entailment $F_1 \vdash \exists\vec{x}.(\Sigma_4\theta * \Sigma \wedge \Pi_2)$ is valid. On the other hand, the rule's side condition $\Sigma_4\theta * \Sigma \cong \Sigma_2$ implies that the entailment $\Sigma_4\theta * \Sigma \wedge \Pi_2 \vdash \Sigma_2 \wedge \Pi_2$ is valid. We have shown that both the two entailments $F_1 \vdash \exists\vec{x}.(\Sigma_4\theta * \Sigma \wedge \Pi_2)$ and $\Sigma_4\theta * \Sigma \wedge \Pi_2 \vdash \Sigma_2 \wedge \Pi_2$ are valid, it follows by Theorem 2.1 again that the goal entailment $F_1 \vdash \exists\vec{x}.(\Sigma_2 \wedge \Pi_2)$ is also valid. The correctness of the rule $\mathsf{LM}_{\mathrm{R}}$, therefore, is claimed.

## B.2 Synthesis Rules

We now present the soundness proof the synthesis rules, presented in Figure 5.9, by showing that if all *assumptions* and entailments in their premises are valid, and their side conditions, if any, are satisfied, then goal entailments in their conclusions are also valid.

**1. Rules $\mathsf{U}^1_\Pi, \mathsf{U}^2_\Pi$**

$$\mathsf{U}^1_\Pi \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_2\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2} \qquad \mathsf{U}^2_\Pi \frac{\mathcal{A} \triangleq \{\Pi_1 \rightarrow \exists\vec{w}.(\Pi_2 \wedge U(\vec{x}))\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \vdash \exists\vec{w}.(\Pi_2 \wedge U(\vec{x}))}$$

- Consider an arbitrary model $s, h$ such that $s, h \models \Pi_1 \wedge U(\vec{x})$. Since the unknown assumption $\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_2$ is assumed valid, then it follows from the semantics of $\rightarrow$ that $s, h \models \Pi_2$. Since $s, h$ is chosen arbitrarily, it follows that the goal entailment $\Pi_1 \wedge U(\vec{x}) \vdash \Pi_2$ is also valid.
- The proof of the rule $\mathsf{U}^2_\Pi$ is similar to the proof of $\mathsf{U}^1_\Pi$.

**2. Rules $\mathsf{U}^1_\Sigma, \mathsf{U}^2_\Sigma$**

$$\mathsf{U}^1_\Sigma \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2}\ \Sigma_1 \neq \mathsf{emp} \qquad \mathsf{U}^2_\Sigma \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \wedge U(\vec{x}) \vdash \exists\vec{w}.(\Sigma_2 \wedge \Pi_2)}\ \Sigma_2 \neq \mathsf{emp}$$

- If the unknown assumption $\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}$ in the premise of the rule $\mathsf{U}^1_\Sigma$ is valid, then it is evident that $\Pi_1 \wedge U(\vec{x}) \equiv \mathsf{false}$ since only false can prove false. It follows that $\Sigma_1 \wedge \Pi_1 \wedge U(\vec{x}) \equiv \mathsf{false}$. Consequently, the goal entailment $\Sigma_1 \wedge \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2$ is valid since its antecedent is a contradiction.

– The soundness proof the rule $\mathsf{U}^2_\Sigma$ is similar where the assumption $\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}$ in the rule's premise also implies that the goal entailment's antecedent is a contradiction ($\Pi_1 \wedge U(\vec{x}) \equiv \mathsf{false}$)

**3. Rule $\mathsf{U}_{\mathbf{IH}}$**

$$\mathsf{U}_{\mathrm{IH}}\ \frac{\mathcal{H}',\ \mathcal{L},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2 \qquad \mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow (\Pi_3 \wedge U(\vec{y}))\theta\}}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash F_4\},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \wedge U(\vec{x}) \vdash F_2}\ \dagger$$

$$\dagger:\ \mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v});\ \exists\theta, \Sigma.\ (\Sigma_1 * \mathsf{P}(\vec{u}) \cong \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma)$$
$$\text{and } \mathcal{H}' = \mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash F_4\}$$

– Since the rule $\mathsf{U}_{\mathrm{IH}}$ applies an induction hypothesis, which contains an unknown relation, to prove another unknown entailment, the soundness of this rule can be proved in a similar manner to the normal induction hypothesis application rule IH. Note that in the proof of $\mathsf{U}_{\mathrm{IH}}$, although the side condition $\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_3\theta \wedge U(\vec{y})\theta$ contains unknown relations $U(\vec{x})$ and $U(\vec{y})\theta$, validity of this side condition is clearly implied by the assumption $\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_3\theta \wedge U(\vec{y})\theta\}$ in the rule's premises.

## B.3 Proofs of Propositions 5.1, 5.2, 5.3

**Proposition 5.1 (Proof of an unknown entailment)** *Given an unknown entailment $E$. If the procedure* Prove *returns* Valid$\langle_\rangle$ *and generates an assumption set $\mathcal{A}$ when proving $E$ in the lemma-synthesis-disabled mode* (NoSyn)*, using an empty induction hypothesis set* ($\mathcal{H}{=}\varnothing$) *and a valid lemma set $\mathcal{L}$, then $E$ is semantically valid, given that all assumptions in $\mathcal{A}$ are valid.*

**Proposition 5.2 (Proof of a normal entailment when the lemma synthesis is disabled)** *Given a normal entailment $E$ which does not contain any unknown relation. If the procedure* Prove *returns* Valid$\langle_\rangle$ *when proving $E$ in the lemma-synthesis-disabled mode* (NoSyn)*, using an empty induction hypothesis set* ($\mathcal{H}{=}\varnothing$) *and a valid lemma set $\mathcal{L}$ as its inputs, then $E$ is semantically valid.*

**Proposition 5.3 (Proof of a normal entailment when the lemma synthesis is enabled)** *Given a normal entailment $E$ which does not contain any unknown relation. If the procedure*

Prove *returns* Valid$\langle_\rangle$ *and synthesizes a set of lemmas* $\mathcal{L}_{\text{syn}}$ *when proving* $E$ *in the lemma-synthesis-enabled mode* (SynLM)*, using an empty induction hypothesis set* ($\mathcal{H}{=}\varnothing$) *and a valid lemma set* $\mathcal{L}$ *as its inputs, then the entailment* $E$ *and all lemmas in* $\mathcal{L}_{\text{syn}}$ *are semantically valid.*

We have argued in Sections B.1 and B.2 that all of our inference rules and synthesis rules are sound. In addition, our proof system, *without the lemma synthesis component*, is clearly an instance of Noetherian induction where the substructural relation is proved to be well-founded (Theorem 5.1) and is utilized by the induction hypothesis application rule IH. Therefore, the soundness of the proof system, when the lemma synthesis is disabled (Prove is invoked with $mode$ = NoSyn), is guaranteed by the Noetherian induction principle and the soundness of all inference and synthesis rules. Now, we consider each proposition as follows.

- In Proposition 5.1, the proof system is invoked in the lemma-synthesis-disabled mode (NoSyn). Based on our argument above about the proof system's soundness in this mode, it is clear that if Prove returns Valid$\langle_\rangle$ and generates a set of assumptions $\mathcal{A}$ when proving the unknown entailment $E$, then $E$ is semantically valid given that all unknown assumptions in $\mathcal{A}$ are valid.

- In Proposition 5.2, the proof system is also invoked in the lemma-synthesis-disabled mode (NoSyn). Since the goal entailment does not contain any unknown relation, there is no unknown assumption introduced while proving $E$. Hence, if Prove returns Valid$\langle_\rangle$, then $E$ is semantically valid.

- In Proposition 5.3, the proof system can synthesize a set of new lemma $\mathcal{L}_{\text{syn}}$ when proving $E$. Hence, we rely on the soundness of the lemma synthesis framework (Section 5.5) to argue about the soundness of the proof system. Details about the lemma synthesis's soundness are presented in Theorem 5.4. Now, suppose that the lemma synthesis is already proved sound, then all lemmas synthesized in $\mathcal{L}_{\text{syn}}$ are semantically valid. Following that, we can separate the proof derivation of $E$ into multiple phases, in which the auxiliary phases are used to synthesize all lemmas in $\mathcal{L}_{\text{syn}}$, and the main phase is to prove $E$, using both the input lemmas in $\mathcal{L}$ or the newly synthesized lemmas in $\mathcal{L}_{\text{syn}}$.

Certainly, this main phase reflects the structural induction aspect of our proof system, without synthesizing lemmas. In addition, all lemmas given in $\mathcal{L}$ and $\mathcal{L}_{\mathrm{syn}}$ are valid. Therefore, if Prove returns Valid$\langle_\rangle$ when proving $E$, then $E$ is semantically valid. $\square$

# Appendix C

# Soundness of the Failure Learning Framework

We will present in this appendix the soundness of our learning-from-failure proof system. We first prove the soundness of all inference rules and use that to argue about the soundness of the entire proof system.

## C.1 Logical and Induction Rules

Soundness of logical and induction rules $\vdash_{\Pi}$, $\bot^1_{\mathrm{L}}$, $\bot^2_{\mathrm{L}}$, $=_{\mathrm{L}}$, $\exists_{\mathrm{L}}$, $\exists_{\mathrm{R}}$, $\mathsf{E}_{\mathrm{L}}$, $\mathsf{E}_{\mathrm{R}}$, $*\!\!\mapsto$, $*\mathsf{P}$, $\mathsf{P}_{\mathrm{R}}$, $\mathsf{ID}$, $\mathsf{IH}$ in Figures 6.7 and 6.8 can be proved similar to the proofs of the inference rules in Section B.1, Appendix B. In the following, we prove the soundness of the rule $\mathsf{P}_{\mathrm{L}}$, which has not been mentioned in the Appendix B.

**1. Rule $\mathsf{P}_{\mathrm{L}}$**

$$\mathsf{P}_{\mathrm{L}}\,\frac{\mathcal{H},\ F_1 * \mathsf{P}(\vec{u}) \vdash F_2}{\mathcal{H},\ F_1 * \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \Sigma_1 \wedge \Pi_1 \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u})$$

– Consider an arbitrary model $s, h$ such that $s, h \models F_1 * \Sigma_1 \wedge \Pi_1$. The side condition of this rule provides that $\Sigma_1 \wedge \Pi_1 \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u})$. It follows that $s, h \models F_1 * \mathsf{P}(\vec{u})$. Since the

entailment $F_1 * \mathsf{P}(\vec{u})$ in this rule's premise is valid, it follows that $s, h \models F_2$. Recall that $s, h$ is chosen arbitrarily. It follows that the entailment $F_1 * \Sigma_1 \wedge \Pi_1 \vdash F_2$ is valid. □

## C.2 Failure Learning Rules

**1. Rule $W_\pi$**

$$W_\pi \ \frac{\mathcal{H},\ F_1 \vdash F_2}{\mathcal{H},\ F_1 \wedge \Pi \vdash F_2}$$

– Consider an arbitrary model $s, h$ such that $s, h \models F_1 \wedge \Pi$. It follows that $s, h \models F_1$. In addition, this rule's premise provides that the entailment $F_1 \vdash F_2$ is valid. Therefore, $s, h \models F_2$. We have shown that for an arbitrary model $s, h$, if $s, h \models F_1 \wedge \Pi$, then $s, h \models F_2$. Therefore, the entailment $F_1 \vdash F_2$ in the rule $W_\pi$'s conclusion is valid.

**2. Rules $G_\mathrm{v}$ and $G_\mathrm{c}$**

$$G_\mathrm{v} \ \frac{\mathcal{H},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ F_1 \vdash F_2} \ u \notin \mathtt{FV}(F_1, F_2) \qquad\qquad G_\mathrm{c} \ \frac{\mathcal{H},\ F_1[u/c] \vdash F_2}{\mathcal{H},\ F_1 \vdash F_2} \ u \notin \mathtt{FV}(F_1, F_2)$$

– We first show the correctness of the rule $G_\mathrm{v}$. Consider an arbitrary model $s, h$ such that $s, h \models F_1$. Let $s' = [s \,|\, u{:}s(x)]$. Since $u \notin \mathtt{FV}(F_1)$, it is evident that $s', h \models F_1[u/x]$. In addition, this rule's premise provides that the entailment $F_1[u/] \vdash F_2$ is valid. Therefore, $s', h \models F_2$. Since $u \notin \mathtt{FV}(F_2)$, it follows that $s, h \models F_2$. We have shown that for an arbitrary model $s, h$, if $s, h \models F_1 \wedge \Pi$, then $s, h \models F_2$. Therefore, the entailment $F_1 \vdash F_2$ in the rule $G_\mathrm{v}$'s conclusion is valid.

– The correctness of the rule $G_\mathrm{c}$ can be proved in a similar manner.

**3. Rule $\mathsf{CA}$**

$$\mathsf{CA} \ \frac{\mathcal{H},\ F_1 \wedge \Pi \vdash F_2 \qquad \mathcal{H},\ F_1 \wedge \neg\Pi \vdash F_2}{\mathcal{H},\ F_1 \vdash F_2}$$

– The rule's premises provide that both the two entailments $F_1 \wedge \Pi \vdash F_2$ and $F_1 \wedge \neg\Pi \vdash F_2$ are valid. For an arbitrary model $s, h$ such that $s, h \models F_1$, we consider two cases as follows.

+ $s, h \models F_1 \wedge \Pi$. Since $F_1 \wedge \Pi \vdash F_2$ is valid, it follows that $s, h \models F_2$.
+ $s, h \not\models F_1 \wedge \Pi$. Since $s, h \models F_1$, it follows that $s, h \not\models \Pi$. Consequently, $s, h \models \neg\Pi$, by the semantics of separation logic formulas. We have shown that $s, h \models F_1$ and $s, h \models \neg\Pi$, it implies that $s, h \models F_1 \wedge \neg\Pi$. Since $F_1 \wedge \neg\Pi \vdash F_2$ is valid, it follows that $s, h \models F_2$.

In both the above cases, we show that for an arbitrary model $s, h$, if $s, h \models F_1$, then $s, h \models F_2$. Therefore, the entailment $F_1 \vdash F_2$ in the conclusion of the rule CA is valid.

## C.3 The Proof System

We are now ready to show the soundness of our learning-from-failure proof system, which is state earlier in Theorem 6.3.

Recall that our proof system is an instance of Noetherian induction where the substructural relation is proved to be well-founded (Theorem 6.1) and is utilized by the induction hypothesis application rule IH. Since all inference rules are proven sound, therefore, soundness of our induction proof system is guaranteed by the Noetherian induction principle. Consequently, if the proof system returns `Valid` when proving the entailment $E$, then $E$ is indeed valid.

# Bibliography

Antonopoulos, Timos, Nikos Gorogiannis, Christoph Haase, Max I. Kanovich, and Joël Ouaknine (2014). "Foundations for Decision Problems in Separation Logic with General Inductive Predicates". In: *Foundations of Software Science and Computation Structures (FOSSACS)*, pp. 411–425.

Berdine, Josh, Cristiano Calcagno, and Peter W. O'Hearn (2004). "A Decidable Fragment of Separation Logic". In: *International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, pp. 97–109.

Berdine, Josh, Cristiano Calcagno, and Peter W. O'Hearn (2005a). "Smallfoot: Modular Automatic Assertion Checking with Separation Logic". In: *International Symposium on Formal Methods for Components and Objects (FMCO)*, pp. 115–137.

Berdine, Josh, Cristiano Calcagno, and Peter W. O'Hearn (2005b). "Symbolic Execution with Separation Logic". In: *Asian Symposium on Programming Languages and Systems (APLAS)*, pp. 52–68.

Berdine, Josh, Byron Cook, and Samin Ishtiaq (2011). "SLAyer: Memory Safety for Systems-Level Code". In: *International Conference on Computer Aided Verification (CAV)*, pp. 178–183.

Bornat, Richard, Cristiano Calcagno, Peter W. O'Hearn, and Matthew J. Parkinson (2005). "Permission accounting in separation logic". In: *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*, pp. 259–270.

Bozga, Marius, Radu Iosif, and Swann Perarnau (2008). "Quantitative Separation Logic and Programs with Lists". In: *International Joint Conference on Automated Reasoning (IJCAR)*, pp. 34–49.

Bozga, Marius, Radu Iosif, and Swann Perarnau (2010). "Quantitative Separation Logic and Programs with Lists". In: *Journal of Automated Reasoning (JAR)* 45.2, pp. 131–156.

Brotherston, James and Max I. Kanovich (2010). "Undecidability of Propositional Separation Logic and Its Neighbours". In: *Symposium on Logic in Computer Science (LICS)*, pp. 130–139.

Brotherston, James, Dino Distefano, and Rasmus Lerchedahl Petersen (2011a). "Automated Cyclic Entailment Proofs in Separation Logic". In: *International Conference on Automated Deduction (CADE)*, pp. 131–146.

Brotherston, James and Alex Simpson (2011b). "Sequent calculi for induction and infinite descent". In: *J. Log. Comput.* 21.6, pp. 1177–1216.

Brotherston, James, Nikos Gorogiannis, and Rasmus Lerchedahl Petersen (2012). "A Generic Cyclic Theorem Prover". In: *Asian Symposium on Programming Languages and Systems (APLAS)*, pp. 350–367.

Brotherston, James, Carsten Fuhs, Juan A. Navarro Pérez, and Nikos Gorogiannis (2014a). "A decision procedure for satisfiability in Separation Logic with inductive predicates". In: *Joint Meeting of International Conference on Computer Science Logic and Symposium on Logic in Computer Science, CSL-LICS*, 25:1–25:10.

Brotherston, James and Nikos Gorogiannis (2014b). "Cyclic Abduction of Inductively Defined Safety and Termination Preconditions". In: *International Static Analysis Symposium (SAS)*, pp. 68–84.

Brotherston, James, Nikos Gorogiannis, Max I. Kanovich, and Reuben Rowe (2016). "Model checking for Symbolic-Heap Separation Logic with inductive predicates". In: *Symposium on Principles of Programming Languages (POPL)*, pp. 84–96.

Bundy, Alan (2001). "The Automation of Proof by Mathematical Induction". In: *Handbook of Automated Reasoning (in 2 volumes)*, pp. 845–911.

Calcagno, Cristiano, Hongseok Yang, and Peter W. O'Hearn (2001). "Computability and Complexity Results for a Spatial Assertion Language for Data Structures". In: *Asian Symposium on Programming Languages and Systems (APLAS)*, pp. 289–300.

Calcagno, Cristiano, Dino Distefano, Peter W. O'Hearn, and Hongseok Yang (2009). "Compositional shape analysis by means of bi-abduction". In: *Symposium on Principles of Programming Languages (POPL)*, pp. 289–300.

Calcagno, Cristiano et al. (2015). "Moving Fast with Software Verification". In: *NASA International Symposium on Formal Methods (NFM)*, pp. 3–11.

Chin, Wei-Ngan, Cristina David, Huu Hai Nguyen, and Shengchao Qin (2012). "Automated verification of shape, size and bag properties via user-defined predicates in Separation Logic". In: *Science of Computer Programming (SCP)* 77.9, pp. 1006–1036.

Chu, Duc-Hiep, Joxan Jaffar, and Minh-Thai Trinh (2015). "Automatic induction proofs of data-structures in imperative programs". In: *Conference on Programming Language Design and Implementation (PLDI)*, pp. 457–466.

Colón, Michael, Sriram Sankaranarayanan, and Henny Sipma (2003). "Linear Invariant Generation Using Non-linear Constraint Solving". In: *International Conference on Computer Aided Verification (CAV)*, pp. 420–432.

Cook, Byron, Christoph Haase, Joël Ouaknine, Matthew J. Parkinson, and James Worrell (2011). "Tractable Reasoning in a Fragment of Separation Logic". In: *International Conference on Concurrency Theory (CONCUR)*, pp. 235–249.

Cousot, Patrick and Radhia Cousot (2010). "A gentle introduction to formal verification of computer systems by abstract interpretation". In: *Logics and Languages for Reliability and Security*, pp. 1–29.

Enea, Constantin, Ondrej Lengál, Mihaela Sighireanu, and Tomás Vojnar (2014). "Compositional Entailment Checking for a Fragment of Separation Logic". In: *Asian Symposium on Programming Languages and Systems (APLAS)*, pp. 314–333.

Enea, Constantin, Mihaela Sighireanu, and Zhilin Wu (2015). "On Automated Lemma Generation for Separation Logic with Inductive Definitions". In: *International Symposium on Automated Technology for Verification and Analysis (ATVA)*, pp. 80–96.

European Space Agency (1996). "Ariane 501 Inquiry Board Report". In: European Space Agency. Retrieved on March 16th, 2017. URL: http://esamultimedia.esa.int/docs/esa-x-1819eng.pdf.

Gu, Xincai, Taolue Chen, and Zhilin Wu (2016). "A Complete Decision Procedure for Linearly Compositional Separation Logic with Data Constraints". In: *Automated Reasoning - 8th International Joint Conference, IJCAR 2016, Coimbra, Portugal, June 27 - July 2, 2016, Proceedings*, pp. 532–549.

Haase, Christoph, Samin Ishtiaq, Joël Ouaknine, and Matthew J. Parkinson (2013). "SeLoger: A Tool for Graph-Based Reasoning in Separation Logic". In: *International Conference on Computer Aided Verification (CAV)*, pp. 790–795.

Harrison, John (2009). *Handbook of Practical Logic and Automated Reasoning*. 1st. New York, NY, USA: Cambridge University Press. ISBN: 0521899575, 9780521899574.

Hoare, C. A. R. (1969). "An Axiomatic Basis for Computer Programming". In: *Commun. ACM* 12.10, pp. 576–580.

Hou, Zhe, Rajeev Goré, and Alwen Tiu (2015). "Automated Theorem Proving for Assertions in Separation Logic with All Connectives". In: *International Conference on Automated Deduction (CADE)*, pp. 501–516.

Iosif, Radu, Adam Rogalewicz, and Jiri Simácek (2013). "The Tree Width of Separation Logic with Recursive Definitions". In: *International Conference on Automated Deduction (CADE)*, pp. 21–38.

Iosif, Radu, Adam Rogalewicz, and Tomás Vojnar (2014). "Deciding Entailments in Inductive Separation Logic with Tree Automata". In: *International Symposium on Automated Technology for Verification and Analysis (ATVA)*, pp. 201–218.

Ireland, Andrew and Alan Bundy (1996). "Productive Use of Failure in Inductive Proof". In: *Journal of Automated Reasoning (JAR)* 16.1-2, pp. 79–111.

Le, Quang Loc, Jun Sun, and Wei-Ngan Chin (2016). "Satisfiability Modulo Heap-Based Programs". In: *International Conference on Computer Aided Verification (CAV)*, pp. 382–404.

Le, Quang Loc, Makoto Tatsuta, Jun Sun, and Wei-Ngan Chin (2017). "A Decidable Fragment in Separation Logic with Inductive Predicates and Arithmetic". In: *International Conference on Computer Aided Verification (CAV)*, pp. 495–517.

Lee, Wonyeol and Sungwoo Park (2014). "A proof system for separation logic with magic wand". In: *Symposium on Principles of Programming Languages (POPL)*, pp. 477–490.

Madhusudan, Parthasarathy, Xiaokang Qiu, and Andrei Stefanescu (2012). "Recursive proofs for inductive tree data-structures". In: *Symposium on Principles of Programming Languages (POPL)*, pp. 123–136.

Mashable (2014). "Facebook Changes Its "Move Fast and Break Things" Motto". In: Mashable News. Retrieved on 2017, March 16th. URL: http://mashable.com/2014/04/30/facebooks-new-mantra-move-fast-with-stability.

Moura, Leonardo Mendonça De and Nikolaj Bjørner (2008). "Z3: An Efficient SMT Solver". In: *International Conference on Tools and Algorithms for Construction and Analysis of Systems (TACAS)*, pp. 337–340.

Nguyen, Huu Hai, Cristina David, Shengchao Qin, and Wei-Ngan Chin (2007). "Automated Verification of Shape and Size Properties Via Separation Logic". In: *Int. Conf. on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pp. 251–266.

Nguyen, Huu Hai and Wei-Ngan Chin (2008). "Enhancing Program Verification with Lemmas". In: *International Conference on Computer Aided Verification (CAV)*, pp. 355–369.

O'Hearn, Peter W., John C. Reynolds, and Hongseok Yang (2001). "Local Reasoning about Programs that Alter Data Structures". In: *International Conference on Computer Science Logic (CSL)*, pp. 1–19.

Pérez, Juan Antonio Navarro and Andrey Rybalchenko (2011). "Separation Logic + Superposition Calculus = Heap Theorem Prover". In: *Conference on Programming Language Design and Implementation (PLDI)*, pp. 556–566.

Pérez, Juan Antonio Navarro and Andrey Rybalchenko (2013). "Separation Logic Modulo Theories". In: *Asian Symposium on Programming Languages and Systems (APLAS)*, pp. 90–106.

Piskac, Ruzica, Thomas Wies, and Damien Zufferey (2013). "Automating Separation Logic Using SMT". In: *International Conference on Computer Aided Verification (CAV)*, pp. 773–789.

Piskac, Ruzica, Thomas Wies, and Damien Zufferey (2014). "Automating Separation Logic with Trees and Data". In: *International Conference on Computer Aided Verification (CAV)*, pp. 711–728.

Pugh, William (1991). "The Omega Test: a fast and practical integer programming algorithm for dependence analysis". In: *Proceedings Supercomputing '91, Albuquerque, NM, USA, November 18-22, 1991*, pp. 4–13.

Qiu, Xiaokang, Pranav Garg, Andrei Stefanescu, and Parthasarathy Madhusudan (2013). "Natural proofs for structure, data, and separation". In: *Conference on Programming Language Design and Implementation (PLDI)*, pp. 231–242.

Research Triangle Institute (2002). "The Economic Impacts of Inadequate Infrastructure for Software Testing". In: National Institute of Standards & Technology, U.S. Department of Commerce. Retrieved on March 16th, 2017.

Reynolds, John C. (2000). "Intuitionistic Reasoning about Shared Mutable Data Structure". In: *Millennial Perspectives in Computer Science*. Palgrave, pp. 303–321.

Reynolds, John C. (2008). "An Introduction to Separation Logic". In: Lecture Notes for the PhD Fall School on Logics and Semantics of State, Copenhagen 2008. Retrieved on 2017, March 16th. URL: http://www.cs.cmu.edu/~jcr/copenhagen08.pdf.

Schrijver, Alexander (1986). *Theory of Linear and Integer Programming*. New York, NY, USA: John Wiley & Sons, Inc. ISBN: 0-471-90854-1.

Sighireanu, Mihaela and David R. Cok (2016). "Report on SL-COMP 2014". In: *Journal on Satisfiability, Boolean Modeling and Computation (JSAT)* 9, pp. 173–186.

Ta, Quang-Trung, Ton Chanh Le, Siau-Cheng Khoo, and Wei-Ngan Chin (2016). "Automated Mutual Explicit Induction Proof in Separation Logic". In: *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, pp. 659–676.

Ta, Quang-Trung, Ton Chanh Le, Siau-Cheng Khoo, and Wei-Ngan Chin (2018). "Automated Lemma Synthesis in Symbolic-Heap Separation Logic". In: *Proceedings of the ACM on Programming Languages (PACMPL)* 2.POPL, 9:1–9:29.

United State General Accounting Office (1992). "Software Problem Led to System Failure at Dhahran, Saudi Arabia". In: United State General Accounting Office. Retrieved on March 16th, 2017. URL: http://www.gao.gov/assets/220/215614.pdf.

Whitehead, Alfred North and Bertrand Russell (1912). *Principia Mathematica*. Principia Mathematica. University Press.

World Bank (2017). "World Bank Open Data". In: World Bank. Retrieved on March 16th, 2017. URL: http://databank.worldbank.org.

Xu, Zhaowei, Taolue Chen, and Zhilin Wu (2017). "Satisfiability of Compositional Separation Logic with Tree Predicates and Data Constraints". In: *International Conference on Automated Deduction (CADE)*, pp. 509–527.